



विद्याविनियोगाद्विकासः

Product Mindset

IIMA Product Management Prep Book

**EDITION
2026-27**





ACKNOWLEDGEMENT

There has been a steep increase in the number of firms offering product management roles in the recent past. This is not only maneuvered by an increase in the number of firms visiting the graduate students, but also by expansion in their scope to associate product manager roles for undergrads. Despite this, there exists a gap in terms of comprehensive preparation material suited for the Indian audience. Filling this gap was an uphill task, which couldn't have been possible without the stakeholders who were a part of the journey at different points in time.

First and foremost, we extend our deepest gratitude to the entire **ProdMan Club (2026-27)** and the team of collaborators. They dedicated countless hours to the rigorous process of ideating, curating, and refining the content within these pages. We are also incredibly thankful to the wider student and alumni community, whose valuable efforts in the past have greatly enriched this work.

Thanks are also due to the **Design team** and the **Publications team** for bringing the content to its present shape.

Finally, we thank you, **the reader**. By engaging with this material, you become a vital part of this initiative. We hope you find it valuable, and we welcome any feedback that will help us strive for continuous improvement.



ACKNOWLEDGEMENT



Furthermore, we would like to individually thank the people who made this possible:

Publications Team: Abhishek Aditya, Dhanshree Meena, Meghna, Kiran Ambokar, Shalini Sheetal, Shounak Ray

Design Team: Gonapa Sai Uddiep, Harsh Badhai, Jai Kumar, Keerat Kaur, Piyusha Patil, Sarthak Kumar

Last, but not least, our coordinator - **Subhash S**, for spearheading the effort from start to finish and making this primer a reality not just for the present, but for the future as well.

**The ProDMan Club
IIM Ahmedabad**

Table of Contents

PM Concepts	Page
Product Design	9
PLC	11
User Personas	13
Jobs-to-be-Done	15
Customer Journey Mapping	63
User Stories	17
UX Design	18
PRD, MVP	19
CIRCLES, HOOK Model	23
Feature Prioritization	26
Rice	28
Value Effort	31
Kano Model	34
MoSCoW	37
ICE	39
Metrics	41
HEART, AARRR	43
North Star Metric	47
Counter Metrics and Guardrails	49

PM Concepts	Page
Segmentation	50
Testing	52
A/B Testing, Evals	54
Precision, Recall and Thresholds	57
AI Guardrails and Human-in-the-loop	59
Cost-per-task	61
Kano Model, MoSCoW, ICE	34
Tech Concepts	Page
Agile Methodology	70
User Experience Design	75
Machine Learning, AI, AR, VR	82
Cloud Computing, Blockchain, Internet of Things	96
Economics of Platforms	111
LLMS & Prompt Engineering	115
Customer Funnel and Brand Lift	121
MCP, RAG, Agentic AI	125
Low Code Platforms	134
Building Trust in Products	137
System Design	139

Table of Contents

Know Your Sector	Page
Quick Commerce	146
FinTech	150
Travel Tech	156
Food Tech	160
E-Commerce	164
OTT	168
Ride Hailing Tech	172
SaaS	176
DevTools	181
HealthTech	186
EdTech	191
AI	196
Tackling Cases	Page
Guesstimate Tips	202
Product Design: Do's & Don'ts	205
Intro to Go To Market Strategies	212
Intro to Product Metrics	223
Intro to Pricing	227
Intro to RCA	231

Practice Cases	Page
Guesstimates	234
Instagram Daily Uploads in India	235
YouTube Daily Revenue	236
Zoom Daily Server Usage	237
Food Delivery App Users	239
Dark stores for 10-min coverage of Ahmedabad	240
New Product Design	242
Office Space Sharing	243
Increase Voter Turnout	245
Disney Theme Park Experience	247
Startup Growth Hub by LinkedIn	249
Ola for Schoolkids	250
Airline Entertainment System	252
Podcast App at Spotify	254
Ride Hailing App	256
Health and Fitness App	258
Internal Knowledge Repository	260
Book Recommendation App	262
Samsung Galaxy Buds Experience	264

Table of Contents

Practice Cases	Page
New Product Design	242
AI Customer-Support Copilot	267
Quick Commerce Vendor Mgmt.	269
Bicycle Renting App	272
Antique Marketplace	274
GTM Strategies	277
Social Fitness Product	278
Table Reservation	280
Launching an AI Sales Assistant	283
Tata Group Loyalty Program	285
Metrics	287
Instagram Creator Marketplace	288
Facebook Marketplace	290
Big Basket	291
Measuring an AI Feature	293
Promotion in Google Pay	294
WhatsApp In-App Video Playing	295

Practice Cases	Page
Pricing	296
Amazon Prime Pricing Projection	297
JioSaavn Family Plan	298
Netflix Pricing Strategy	299
Vendor Price Negotiation	300
RCA	302
Drop in Engagement	303
Low Adoption Rate	305
Spike in Ride Cancellation	306
Decline in Subscriptions	307
Drop in Daily Active Users	308
Decline in Cart Additions	309
Low CRM Adoption	311
Users Abandoning Our AI Chatbot	313
App Improvement	315
Facebook Reactions	316
BookMyShow User Engagement	317

What Is Product Management

Throughout business history, innovators have kindled ideas that solve real problems. They built them, tested them, sometimes failed, and iterated with one goal: to create and deliver meaningful solutions. Today, that end-to-end process is embodied in the role of the Product Manager.

A common definition is that a **product manager is the CEO of a product**. PMs shoulder broad responsibility for vision, execution, and outcomes. They **guide teams to discover, develop, and ship the right product**, while bridging gaps among stakeholders.

What does a PM do?

At its heart, product management is a **strategic function at the intersection of business, technology, and user experience**. A few key traits of a PM include:

- **Customer empathy:** Uncovering both explicit and latent needs is essential. The challenge lies in detecting unstated problems, going beyond what customers say to what they actually require. PMs listen to customers to define problems but take ownership in shaping the solution.
- **Solution Ownership:** PMs decide *how to solve a problem*. The quality of the solution, and its fit with customer context, sets apart great PMs from average ones.
- **Prioritization of Decisions:** A PM constantly evaluates competing ideas. By weighing metrics like feasibility, impact, effort, and business value, PMs ensure teams focus on what matters most so that faster delivery of critical features is achieved while preventing wasted effort.
- **Excellent Communication:** PMs must adapt their message for different audiences, executives, engineers, designers, or marketers, using the right vocabulary and context to align everyone toward the same product vision.

Key Responsibilities of a Product Manager

While responsibilities vary by company size, industry, and maturity, your journey as a PM generally include:

- **Market Research & Insight:** Conducting customer and competitive analysis to guide decisions
- **Vision & Strategy:** Crafting a clear product vision linked to business goals and translating it into a roadmap
- **Idea Generation & Prioritization:** Conducting meetings to address customer needs and prioritizing features using criteria like impact, feasibility, and value
- **Design & Development Collaboration:** Shaping wireframes, writing user stories, facilitating development with engineering teams, and supporting testing and iteration
- **Go-to-Market & Launch:** Coordinating marketing strategy, aligning with launch timelines, and ensuring support readiness
- **Ownership of Success Metrics:** Tracking performance via customer satisfaction, adoption, retention, revenue etc.

Performance measurement of PMs

Your performance as a PM will be measured using activities associated with product success, customers, and the overall process of product design and launch.

KPIs commonly used are:

- Monthly Revenue - Month-wise revenue generated by the product
- Delivery-On-Time - Delivery of product or new feature on time
- Product Quality - Number of deviation from standard quality practices
- Customer Counts - Increase in customer over a specified duration
- Stickiness Ratio - Daily active user/ Monthly active user (DAU/DMU)
- Customer Retention - Number of customers staying with the company over a specified period
- Customer Satisfaction Score Measures overall satisfaction of customers

01

PM Concepts

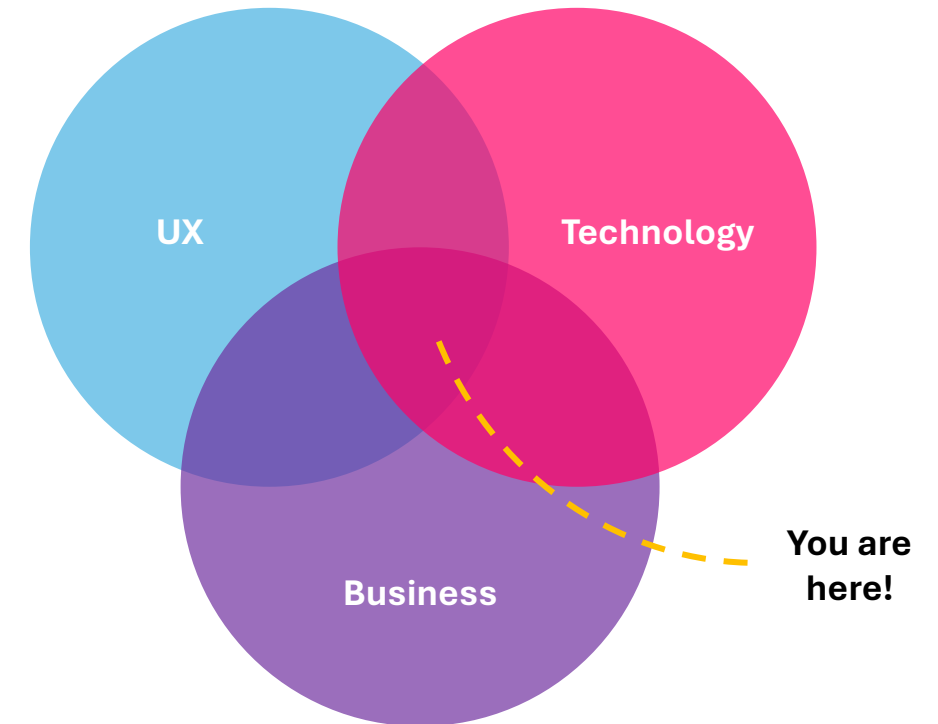
Why Product Design Matters for PMs

A Product Manager bridges customer needs, business goals, and technical delivery. You're not just deciding what to build, you're making sure it solves the right problem in a way that is valuable, usable, and feasible.

Good product design ensures:

- The **right problem** is solved (not just what's easy to build)
- Solutions are **feasible** for engineering and **viable** for the business
- Users can achieve their goals **smoothly and intuitively**

As a PM, you don't need to be the lead designer but must guide design decisions, so the product delivers value consistently.



1. **Start with the user:** Deeply understand their pain points, workflows, and environment.
2. **Collaborate early:** Involve design and engineering in ideation early, not just delivery.
3. **Prioritize trade-offs:** Every design choice impacts cost, time, and user experience.
4. **Iterate fast:** Get early versions into users' hands, test, and refine. Use prototypes, MVPs, and metrics to validate before scaling.

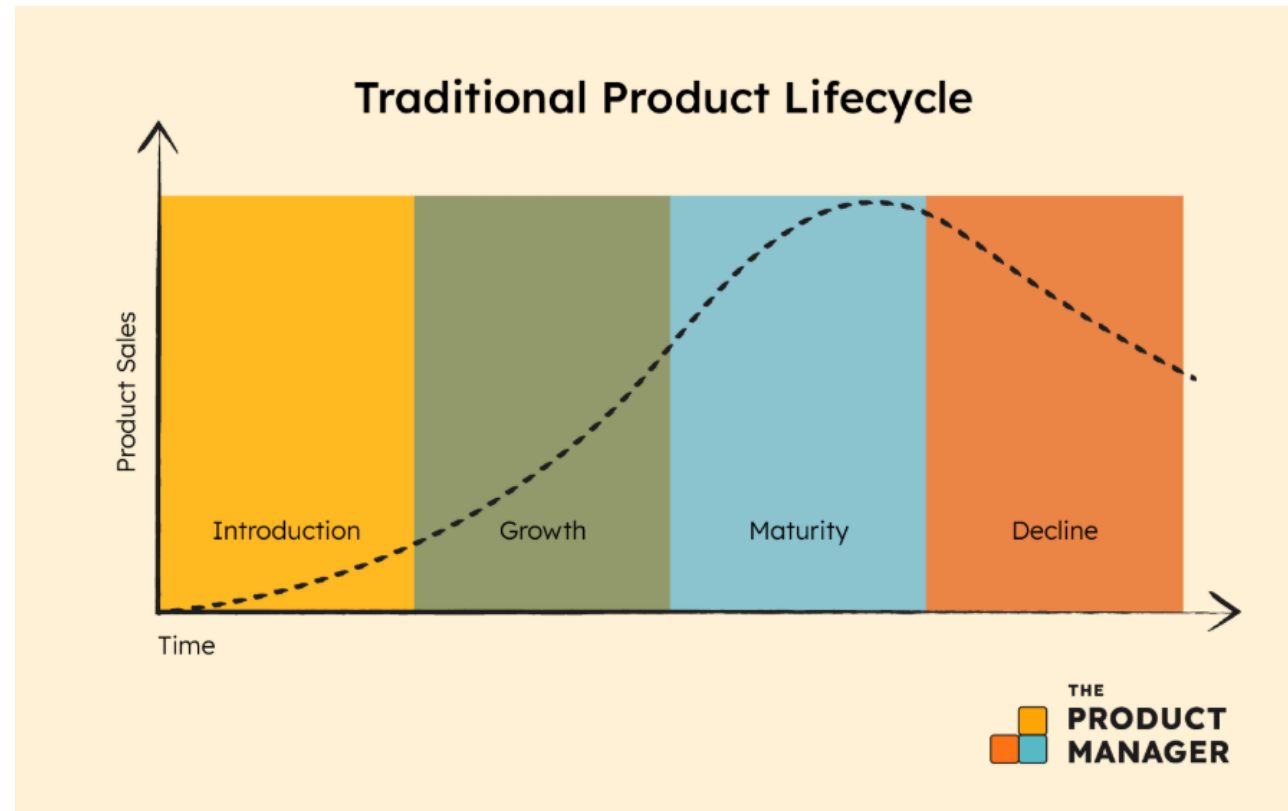
What it is:

The stages a product goes through in the market: Introduction → Growth → Maturity → Decline.

1. **Introduction:** You've just launched. Focus is on **adoption** and finding your first true fans (early adopters). You're validating your core hypothesis with a real-world audience.
2. **Growth:** You have product-market fit! Focus shifts to **scaling**. You're rapidly adding features, entering new markets, and acquiring users quickly to build a defensible market position.
3. **Maturity: Growth is slowing down.** Focus is now on **optimization and retention**. You're fending off competitors, increasing efficiency, and finding new ways to get more value from your existing user base.
4. **Decline:** The market is shrinking, or a superior technology has emerged. Focus is on **profitability**. You may be "sunsetting" the product, minimizing costs, or milking the remaining cash flow before retiring it.

Why PMs should care:

- Different stages need different strategies (e.g., heavy marketing in Introduction, cost optimization in Maturity).
- Helps decide whether to invest, sustain, pivot, or sunset a product.



Reference: <https://www.investopedia.com/terms/p/product-life-cycle.asp>

Example:

Instagram Reels in 2020 was in Introduction with rapid feature pushes and heavy promotion. In 2025, it's in Maturity, focusing on refining UX and increasing ad monetization.

What it is:

Fictional, research-backed representations of your target customers, including demographics, product usage behaviors, goals, and challenges. This helps in achieving a product-market fit. It is recommended to come up with around 3-4 user personas.

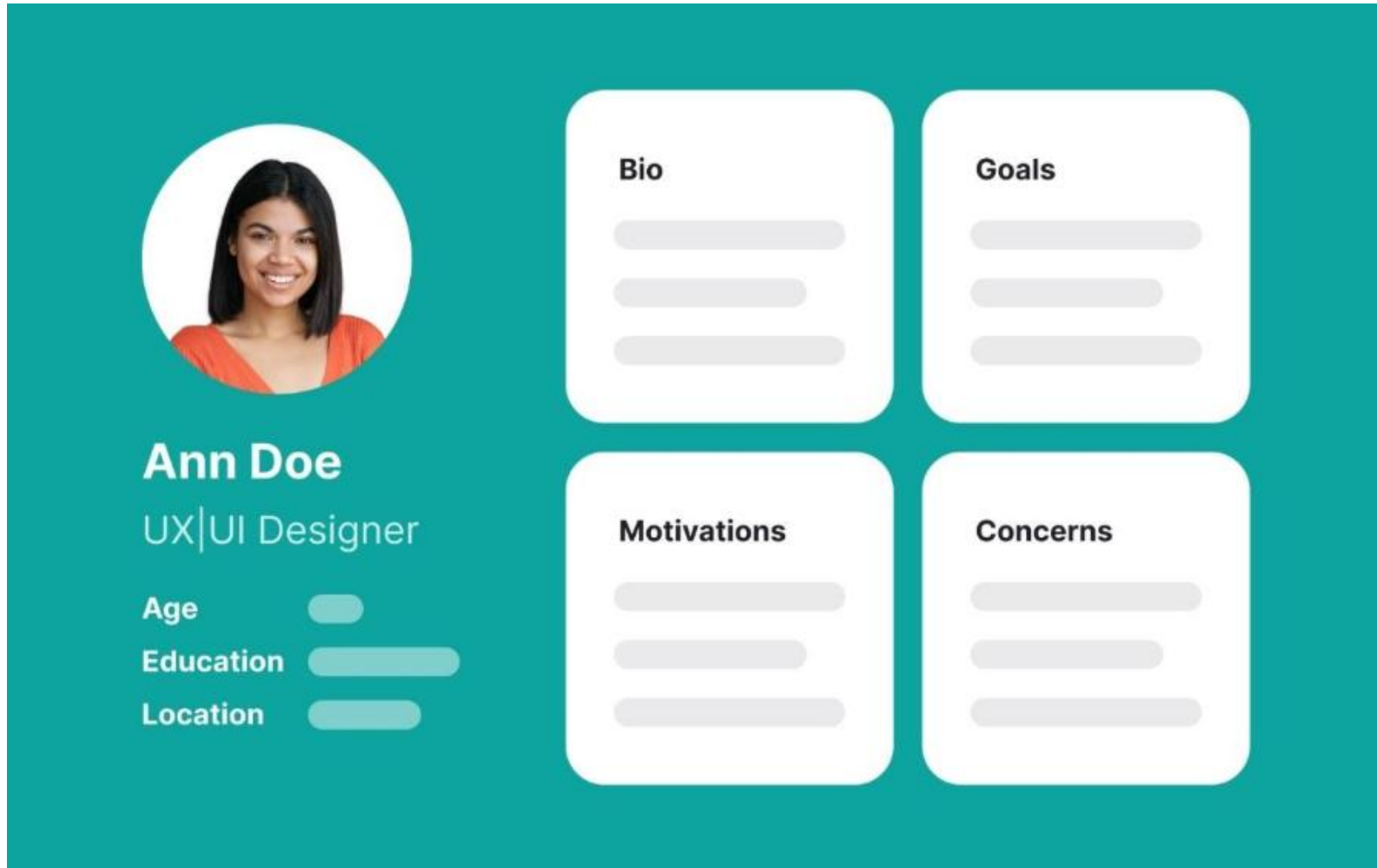
Why PMs should care:

- Personas anchor design choices in real user needs rather than internal opinions.
- They help align the team (designers, engineers, marketing) on who you're building for (even prioritize some personas over others)
- In roadmap discussions, you can use personas to defend priorities ("Feature X matters because Persona A struggles with Y problem.")

Example:

For a budgeting app, your primary persona might be "Ravi, 28, urban professional, wants to track expenses easily on mobile, hates manual entry." That means you might prioritize **bank sync** over advanced analytics.

Reference: <https://app.uxcel.com/courses/design-foundations/ux-deliverables-002/user-persona-7788>

A user persona card for Ann Doe, a UX/UI Designer. The card has a teal background and contains a circular profile picture of a woman with dark hair wearing an orange top. Below the picture, the name 'Ann Doe' and title 'UX|UI Designer' are displayed. To the left of the card, there are three horizontal bars representing 'Age', 'Education', and 'Location'. To the right, there are four white rounded rectangular boxes labeled 'Bio', 'Goals', 'Motivations', and 'Concerns', each containing three horizontal bars for text input.



Ann Doe
UX|UI Designer

Age 

Education 

Location 

Bio



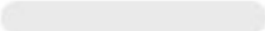


Goals



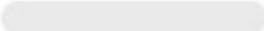


Motivations





Concerns





What is JTBD?

- A framework for understanding the progress a customer is trying to make in a particular situation.
- Customers do not buy products for their features. They hire a product, a workaround, or even inaction to move from where they are to where they want to be. The same product gets hired for different jobs, a delivery app might be hired to save time after a late class, to avoid cooking, or to coordinate a group meal.
- **Job statement:** When [situation], I want to [make progress], so I can [achieve the desired outcome]

Dimension	What it captures
Functional	The practical outcome they want.
Emotional	How they want to feel during and after.
Social	How they want to be seen by others.
Circumstance	The situation and trigger that make the job matter right now.

Why PMs should care

It moves the conversation from requested features to underlying motivation, and reveals substitutes outside the obvious category - including doing nothing. That makes prioritisation stick to durable outcomes rather than passing trends, and it explains why someone should switch from what they use today.

Personas explain who the customer is. JTBD explains why that customer seeks a solution in a specific situation.

The four forces that decide a switch

Force	What it represents	Question for the PM
Push	Dissatisfaction with the current situation.	What made the old way no longer acceptable?
Pull	Attraction toward a new outcome or solution.	What promised a better experience?
Anxiety	Concerns about adopting the new solution.	What could prevent them from switching?
Habit	Comfort, routines and switching costs.	Why might they stay with the old way?
Push and Pull move the customer toward you. Anxiety and Habit hold them back. A product that only strengthens Pull often fails, because the real blocker was Anxiety		

- **Example:** An IIMA student finishes an evening class after the hostel mess has closed. Job: when evening class ends after the mess closes, I want a dependable, filling meal with minimal effort, so I can eat quickly without overspending or losing study time.

Element	Finding
Circumstance	Class ends late. The mess is closed. Time, energy and budget are constrained.
Functional progress	A filling meal within roughly 30 minutes at an acceptable total cost.
Emotional and social	Reduce uncertainty. Avoid depending on friends or appearing disorganised.
Solutions hired today	Delivery apps, campus canteen, instant noodles, leftovers, asking a friend.
Competition insight: the relevant competitors are not only other delivery apps. Instant noodles, the campus canteen or a friend may all be hired for the same job.	

What it is:

Short, structured statements of a feature from a user's perspective:

“As a **[user type]**, I want **[goal]** so that **[benefit]**.”

Why PMs should care:

- Keeps requirements user-focused rather than tech-focused.
- Makes it easier for developers to understand the context and purpose of a feature.
- Helps you slice work into smaller, deliverable pieces for faster iteration.

Example:

“As Ravi, I want to scan receipts so that I don't have to type expenses manually.” This drives a feature decision to add an OCR-powered upload option.

What it is:

User Experience (UX) is how a user interacts with and feels about your product and its ease of use, clarity, flow, and satisfaction.

Why PMs should care:

- Poor UX kills adoption even if the feature is powerful.
- PMs must advocate for usability testing, accessibility, and intuitive workflows.
- Good UX can reduce customer support costs and increase adoption and retention.

Example:

In a food delivery app, if users drop off at the address entry, simplifying it with “use current location” can improve order completion rates.

What it is:

A detailed, formal document that specifies how customers will interact with a product, the functionality needed to fulfil all use cases, and the requirements for engineering and other teams. It often includes wireframes, user stories, and technical constraints. The PRD serves as a central reference point for engineering, quality assurance, customer service, sales, and product marketing, ensuring everyone understands what is to be built and why.

Key Sections:

- **Problem Statement:** The customer pain points the product aims to solve, backed by market research or customer discovery.
- **Goals/Objectives:** Desired user and business outcomes, aligned with the product vision and company strategy.
- **User Personas & Stories:** Detailed personas and narratives describing how representative users will accomplish key tasks.
- **Features & Scope:** Clear definition of in-scope and out-of-scope features for the release, with prioritization.
- **Requirements & Use Cases:** Comprehensive description of all the ways the product will be used, including functional and non-functional requirements.
- **Wireframes/Prototypes:** Visual layouts or interactive models to guide engineering and design.
- **Success Metrics:** Quantitative and qualitative measures of success (e.g., adoption rates, engagement metrics, conversion rates).

Why PMs should care:

- Provides necessary detail for engineering to build without ambiguity, reducing back-and-forth.
- Acts as the single source of truth for all stakeholders, minimizing misalignment.
- Enables cross-functional teams to translate product goals into actionable work plans.
- Reduces the risk of scope creep by clarifying priorities and constraints upfront.

Example:

Before launching a “Dark Mode” feature:

- **Problem:** Users experience eye strain in low-light conditions.
- **Personas & Story:** “Melody, a working mother in her 40s, uses the app late at night after putting her kids to bed. She finds the bright interface uncomfortable and wants a softer display option.”
- **Scope:** Apply dark mode to all core screens; exclude legacy admin dashboard in v1.
- **Requirements:** Toggle switch in settings, persist mode preference, ensure color contrast meets accessibility standards.
- **Success Metric:** $\geq 40\%$ of nightly active users enable dark mode within one month of release.

What it is:

An MVP is the most misunderstood concept in product management. It is **not** a half-finished, buggy product. It is the version of a new product that allows you to collect the **maximum amount of validated learning about your customers with the least amount of effort**.

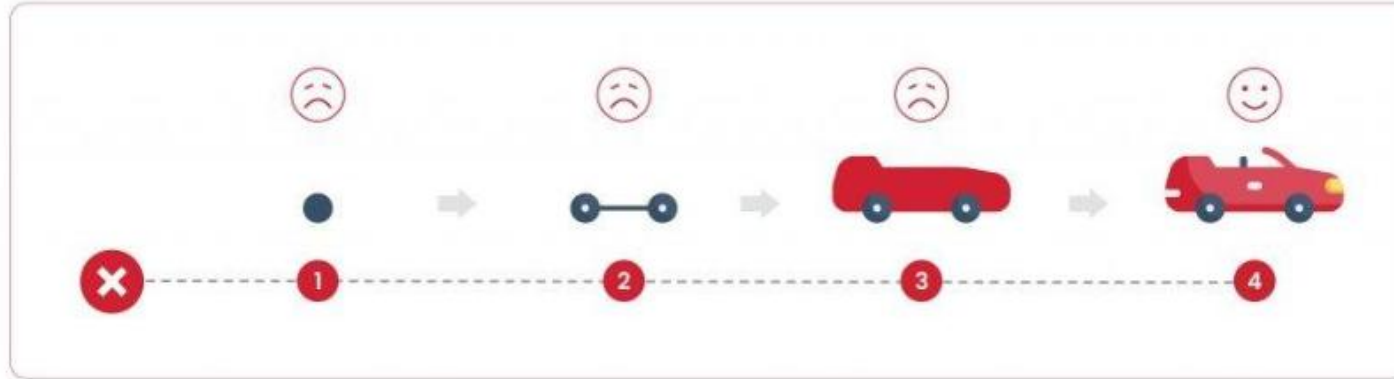
Why PMs should care:

- Prevents overbuilding and wasting resources.
- Lets you test assumptions early and pivot quickly if needed.
- Gives real-world feedback to inform future iterations.

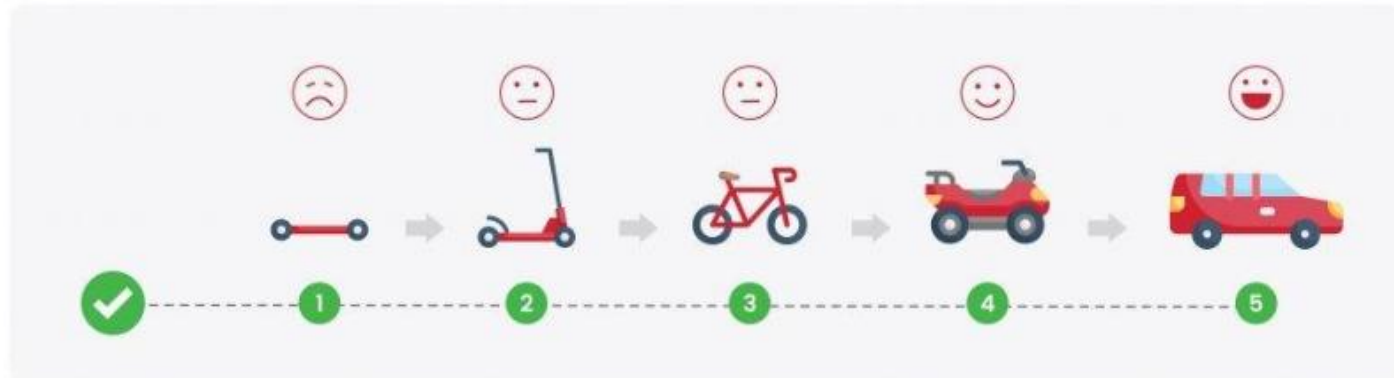
Example:

Instead of building a complete AI résumé review platform, launch a simple “upload PDF, get instant score” tool to see if job seekers are interested.

Reference: <https://productschool.com/resources/glossary/mvp-minimum-viable-product>



MVP = MINIMUM VIABLE PRODUCT



What it is:

A problem-solving framework for design questions

- **Comprehend the Situation:** Ask clarifying questions to understand the goal.
- **Identify the Customer:** Define your target user personas.
- **Report the Customer's Needs:** State their problems and use cases (via user stories).
- **Cut Through Prioritization:** Prioritize which use case to solve first.
- **List Solutions:** Brainstorm multiple potential solutions.
- **Evaluate Trade-offs:** Analyze the pros and cons of your proposed solutions.
- **Summarize Your Recommendation:** State your final recommendation

Why PMs should care:

- Useful in structured thinking during design discussions.
- Helps avoid jumping to solutions without fully understanding the problem.

What it is:

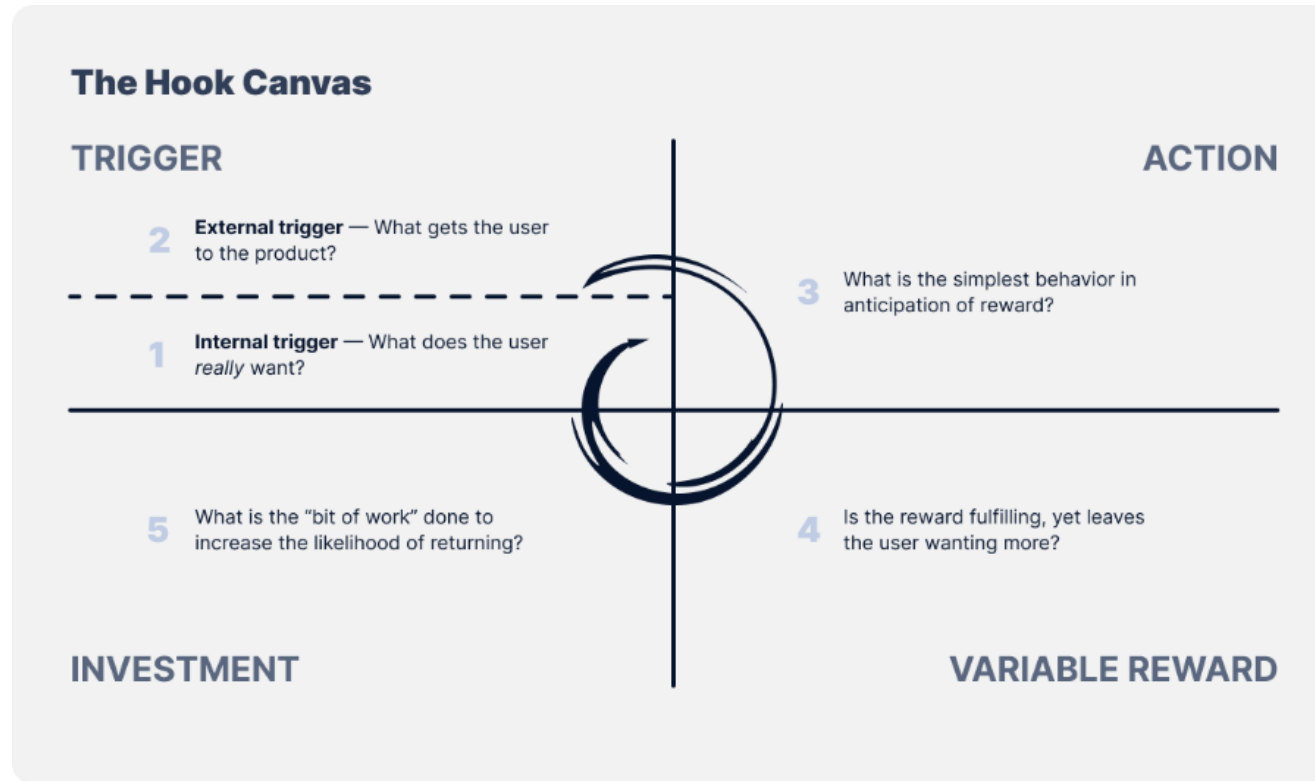
A behavioral design loop for building habit-forming products:

- **Trigger:** The cue that initiates a behavior. This can be external (like a notification) or internal (like the feeling of boredom).
- **Action:** The simplest action a user can take in anticipation of a reward (e.g., scrolling a feed, clicking a play button).
- **Variable Reward:** The reward that satisfies the user's need but is delivered unpredictably. This is what creates craving (e.g., the thrill of seeing what's next in your social feed).
- **Investment:** A small bit of work the user puts into the product, which loads the next trigger and increases the likelihood of them returning (e.g., following a user, creating a playlist, sending a message). This work could bring returns from the user (e.g., user spending money/inviting other people to the platform/providing data, etc.)

Why PMs should care:

- Helps create products users naturally return to.
- Encourages thinking about long-term engagement rather than just acquisition.

Reference: <https://artkai.io/blog/the-hooked-model>



Example: Duolingo

- Trigger: Daily push notification.
- Action: Complete a lesson.
- Variable Reward: Fun streak animations.
- Investment: User progresses in a skill tree, making them return tomorrow.

As a PM, you will *always* have more feature requests than you can build. Your job is to decide **what to work on first**, balancing:

- **User Value:** How much pain does this solve? How much delight does it create?
- **Business Value:** Does this drive revenue, increase market share, improve retention, or reduce costs?
- **Strategic Fit:** Does this feature move us closer to our long-term product vision?
- **Effort & Feasibility:** What will it take to build this, and do we have the resources and dependencies in place?

Poor prioritization wastes engineering effort; delays impact and frustrates stakeholders.
Good prioritization **maximizes ROI** and keeps the team focused.

Start with Your North Star: Always begin with your company's strategic goals and your product's vision. If a feature doesn't align with these, it should be heavily questioned, no matter how exciting it seems.

Gather & Centralize Inputs: Create a single repository for all ideas. This includes user feedback from support tickets, requests from the sales team, insights from market research, new technology opportunities, and ideas from your internal team.

Choose the Right Framework: No single framework is perfect for all situations. Select a tool that fits the stage of your product, the data you have available, and the decision you need to make (more on this below).

Socialize & Align: Prioritization is not a solo activity done in a dark room. Share your framework and reasoning with key stakeholders (engineering leads, design, marketing, leadership). This transparency builds trust and alignment, even when you have to say "no".

Iterate and Revisit: Priorities are not set in stone. The market changes, you get new data, a competitor makes a move. Revisit and adjust your roadmap regularly, typically every quarter.

[Refer to the Metrics section to understand what a **North Star** is.]

RICE

What it is:

A prioritization framework that scores ideas using: **(Reach × Impact × Confidence) / Effort**

- **Reach:** How many users will this feature affect over a specific period?
 - e.g., 5,000 users/month
- **Impact:** How much will this move the needle for those users?
 - **Use a scale:**
 - 3 for massive impact,
 - 2 for high,
 - 1 for medium,
 - 0.5 for low.
- **Confidence:** How sure are you about your Reach and Impact scores?
 - (100% for high confidence, 80% for medium, 50% for low).
 - This is crucial for de-biasing.
- **Effort:** How many "person-months" will this take from the entire team (product, design, engineering)?

Why PMs should care:

- Gives a structured quantitative way to compare ideas.
- Reduces bias towards “loudest voice in the room”.

Example:

You have two ideas:

- Add “One-Click Checkout” (Reach: 50K users, Impact: High, Confidence: 80%, Effort: 3 months).
- Add “Gift Wrapping Option” (Reach: 10K, Impact: Medium, Confidence: 90%, Effort: 1 month).

The RICE score will likely show that “One-Click Checkout” has a higher impact overall, even though it’s harder to build.

Optional note :

RICE is most effective when comparing initiatives within the same product or portfolio.

Reference: <https://whatfix.com/blog/rice-scoring-model/>

R.I.C.E Prioritizations Method

Priority	Project	Reach	Impact	Confidence	Effort	R.I.C.E Score
1	Biometric Authentication	81		 75%		152
2	Stripe Pay integration	70		 50%		35
3	Shopify e-commerce cart	58		 50%		29
4	API Dashboard	43		 50%		14



What it is:

A 2×2 chart:

- **High Value / Low Effort** → Quick Wins, they provide immediate impact and build momentum.
- **High Value / High Effort** → Major Projects, these are your strategic initiatives. They need to be broken down into smaller phases and planned carefully
- **Low Value / Low Effort** → Fill-ins, only do these if you have free time, which you rarely will. They don't move the needle
- **Low Value / High Effort** → Avoid, at all costs.

What brings “Value”?

- **User Value (Customer Impact):** Improves satisfaction, solves pain points, or creates delight for users.
- **Business Value (Commercial Impact):** Drives revenue, accelerates acquisition and growth, boosts retention, or reduces costs.
- **Strategic Value (Future Impact):** Strengthens competitive advantage, supports long-term goals, and aligns with the company's vision.

What demands “Effort”?

- **Development Effort:** Time and complexity required for design, coding, and testing.
- **Resource Cost:** Financial investment, team bandwidth, and opportunity cost.
- **Risk & Uncertainty:** Technical challenges, dependency bottlenecks, or potential market failure.

Why PMs should care:

- A simple visual tool for prioritizing initiatives.
- Useful for early-stage or small-team prioritization.

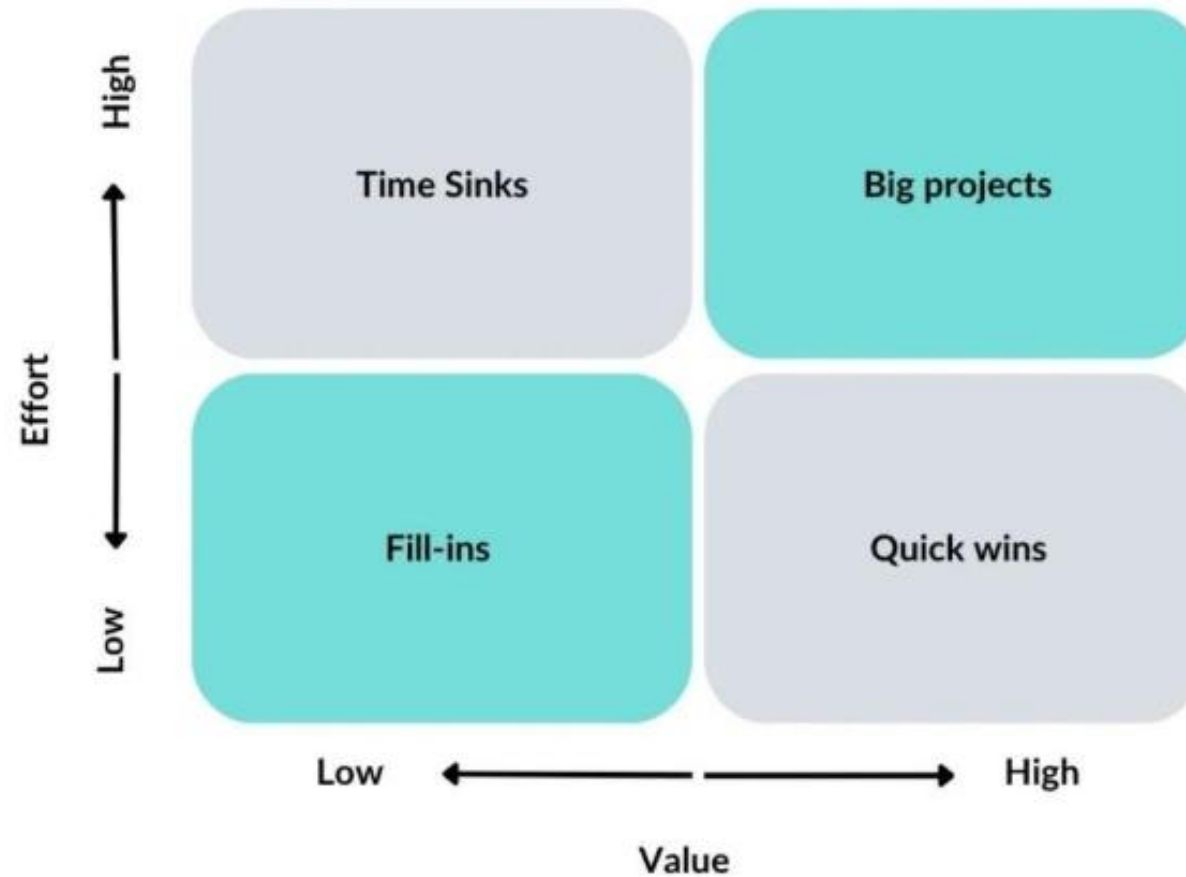
Example:

A “Search Bar Improvement” that boosts retention but takes 2 days to code? → Quick Win → Build now.

A “New AI Recommendation Engine” that takes 6 months with unclear ROI? → Major Project → Defer until there is stronger evidence of value.

Reference: <https://www.userxd.co/post/how-to-understand-value-vs-effort-in-ux-a-comprehensive-guide>

The Value vs. Effort Matrix



What it is:

The Kano Model shifts your thinking from "what features should we build?" to "how will this feature affect user satisfaction?" It classifies features into:

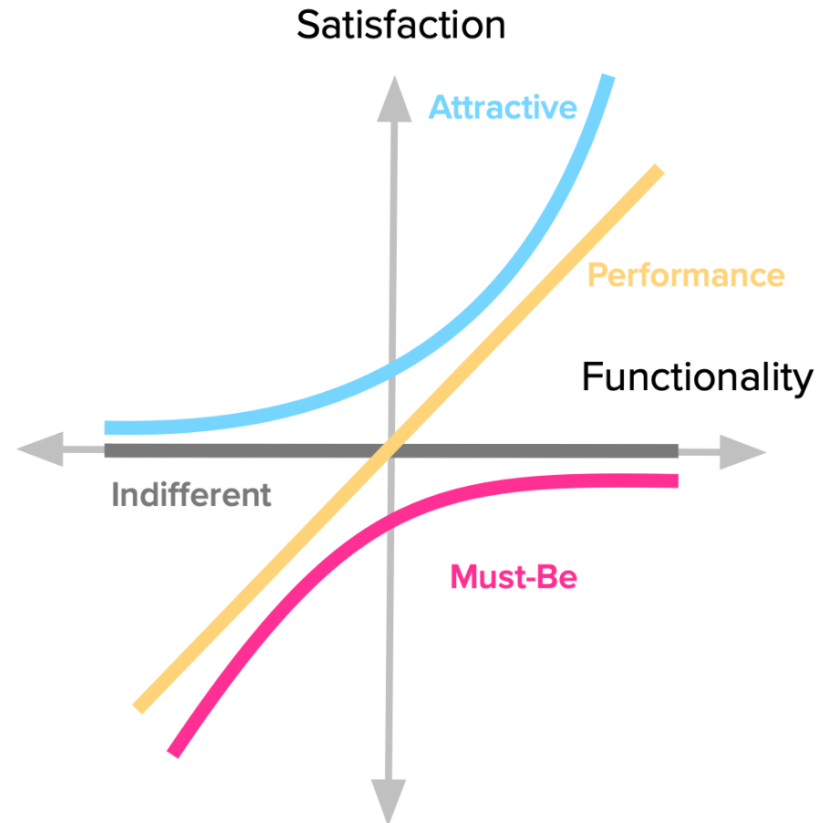
- **Must-haves (Basic Expectations):** Users expect these features. Their absence causes dissatisfaction, but their presence does not increase satisfaction.
 - e.g., A hotel room must have a bed).
- **Performance Features (Satisfiers):** User satisfaction increases as the quality or quantity of these features improves.
 - (e.g., Faster Wi-Fi, more storage space).
- **Delighters (Exciters):** Unexpected features that create delight. Their absence doesn't cause dissatisfaction, but their presence can create loyal fans.
 - (e.g., a free welcome gift in your hotel room).
- **Indifferent:** Features that have little or no impact on customer satisfaction.



Why PMs should care:

- Helps prioritize investments by distinguishing between expected features and delight factors.
- Keeps user satisfaction at the center of product decisions, not just business metrics.
- Today's Delighters often become tomorrow's Must-haves as user expectations evolve.

Reference: [https://www.qualtrics.com/en-au/experience-management/research/kano-analysis/#:~:text=The%20Kano%20Analysis%20model%20\(pronounced,can%20be%20measured%20and%20explored.](https://www.qualtrics.com/en-au/experience-management/research/kano-analysis/#:~:text=The%20Kano%20Analysis%20model%20(pronounced,can%20be%20measured%20and%20explored.)



Example:

For a hotel booking app:

Must-have: Accurate room availability

Performance: Lower prices, better filters

Delighter: Free late checkout for loyal customers

Reference: <https://foldingburritos.com/blog/kano-model/>

What it is:

MoSCoW is less about prioritizing a backlog and more about defining what must be included in a specific release or MVP.

- **Must Have:** Non-negotiable features for the release to be viable. If even one of these is missing, the launch is a failure.
- **Should Have:** Important but not vital. The release is still valuable without them, but they would be painful to leave out.
- **Could Have:** Desirable but not necessary. These are nice-to-haves that will be included only if time and resources permit.
- **Won't Have (This Time):** Explicitly out of scope for the current release.

Why PMs should care:

- Enables structured release planning by distinguishing essential features from optional ones.
- Helps manage stakeholder expectations when resources are limited.

The MoSCoW method of prioritisation

BiteSize Learning



Must have

Absolutely essential elements that must be included at any cost.



Should have

Important elements that should only be omitted with careful consideration.



Could have

Desirable elements that would ideally be included if resources allow.



Won't have

Elements that are out-of-scope, unfeasible or actively counterproductive.

Example:

For a conference app MVP:

- Must Have: Agenda & Speaker Info
- Should Have: In-app Q&A
- Could Have: AR Navigation
- Won't Have: Virtual Reality Tour (this year)

Reference: <https://www.bitesizelearning.co.uk/resources/moscow-prioritisation-model>

What it is:

Score = **Impact** × **Confidence** × **Ease** (simpler cousin of RICE, without Reach).

- **Impact:** How much value will this create for users or the business? (Scale of 1-10)
- **Confidence:** How confident are you in your estimates? (Scale of 1-10)
- **Ease:** How easy is this to build? (Scale of 1-10, where 10 is very easy)

Why PMs should care:

- Fast to apply for backlog grooming.
- Ideal when you don't have detailed user reach data yet.
- However, ICE can bias you towards features that deeply impact a small number of users.

Example:

You're deciding between "Dark Mode" vs "Custom Themes" in a beta app:

- Dark Mode: Impact 8, Confidence 9, Ease 7 → Score = 504
- Custom Themes: Impact 5, Confidence 7, Ease 5 → Score = 175
- → Build Dark Mode first.

Example ICE Prioritization

Your Growth Experiments:	Impact	×	Confidence	×	Ease	=	Total Score
Redesign homepage hero	8		7		5		280
Add exit-intent popup on Blog	6		8		9		432
Launch referral program test	9		6		4		216

Reference: <https://growwithward.com/ice-prioritization-framework/>

Without metrics, you're flying blind. PMs need **measurable indicators** to:

- **Objective Decision-Making:** Replace gut feelings and the "loudest voice in the room" with evidence-backed choices.
- **Diagnosing Problems:** Identify precisely where your product is succeeding or failing, so you can focus your efforts where they matter most.
- **Driving Alignment:** Give your team and stakeholders a shared definition of success and a straightforward way to track progress toward it.
- **Telling a Story:** Use data to craft a compelling narrative about your product's journey, its challenges, and its victories.

A PM's success is often judged by impact, and **impact** is demonstrated through **clear, relevant metrics**, i.e. tracking the *right things*.

1. **Start from Your Objective:** What is the single most important goal for your product right now? Is it acquiring new users? Increasing engagement? Driving revenue? Your primary objective dictates your primary metric.
2. **Build a Metric Tree:** Don't just pick a single metric. Map out how your primary goal (the North Star) breaks down into smaller, actionable drivers. This helps your team see how their daily work contributes to the bigger picture.
3. **Choose Leading and Lagging Indicators:**
 - a. **Lagging Indicators** confirm long-term success (e.g., monthly revenue, customer retention). They are easy to measure but hard to influence directly.
 - b. **Leading Indicators** predict future success (e.g., daily active users, number of items added to a cart). They show early signs of progress and are more actionable.
4. **Identify Counter-Metrics:** For every metric you try to improve, you must track a counter-metric to ensure you're not causing harm elsewhere. If you're trying to increase ad clicks (your goal), you must also monitor session time or user complaints (your counter-metric) to make sure you're not just annoying users.
5. **Avoid Vanity Metrics:** A vanity metric is a number that looks good on paper but doesn't actually help you make decisions (e.g., total number of sign-ups since launch). Focus on actionable metrics that change your behavior.

HEART Framework

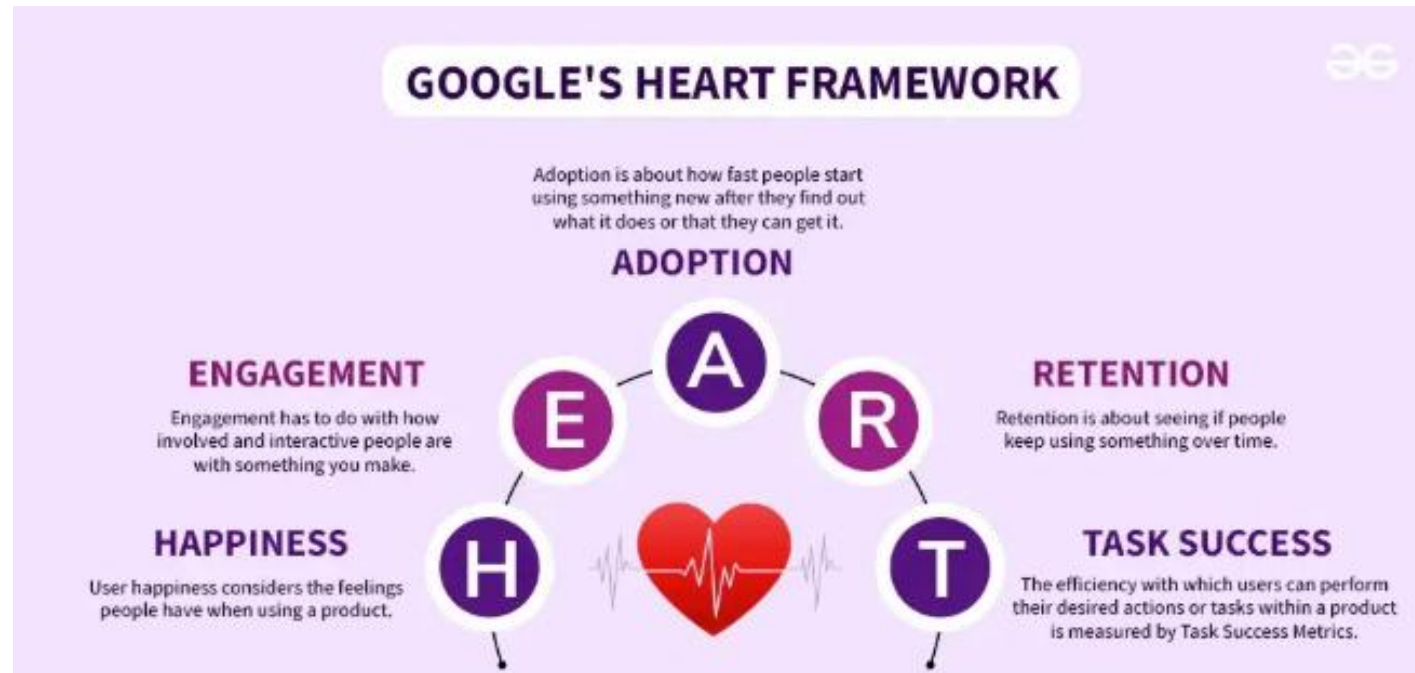
What it is:

A UX measurement framework developed by Google that focuses on five dimensions of user experience:

- **Happiness:** How satisfied are users with your product?
 - (Measured via surveys like NPS, CSAT, or app store ratings).
- **Engagement:** How often and how deeply do users interact with your product?
 - (e.g., sessions per user per week, photos uploaded per day).
- **Adoption:** How many new users are trying your product or a specific feature?
 - (e.g., new accounts created, % of users who use a new feature in its first 7 days).
- **Retention:** What percentage of users are returning over time?
 - (e.g., daily/monthly active users, churn rate).
- **Task Success:** Can users accomplish their goals quickly and easily?
 - (e.g., time to complete a key task, success rate of a search query).

Why PMs should care:

- Gives a holistic view of product experience beyond pure numbers. Helps understand the "why" behind the numbers.
- Helps balance short-term engagement with long-term satisfaction.



Example:

For a project management app:

- Happiness: NPS = +40
- Engagement: Avg. 3 logins/week
- Adoption: +15% new sign-ups MoM
- Retention: 70% of teams are active after 3 months
- Task Success: 90% of tasks created are completed within deadlines

If Retention is dropping while Happiness is high, maybe users like the app but don't find it essential daily.

Reference: <https://www.geeksforgeeks.org/websites-apps/googles-heart-framework-for-measuring-ux/>

What it is:

A funnel-based model to track growth:

- **Acquisition:** How do users find you? (e.g., visitors from organic search, cost per install from ads).
- **Activation:** Do users have a great first experience (their "aha!" moment)? (e.g., % of users who complete onboarding, % who use a key feature in their first session).
- **Retention:** Are users returning? (e.g., 30-day active user rate, churn rate).
- **Referral:** Are users telling others about your product? (e.g., viral coefficient, number of invites sent).
- **Revenue:** Are you making money? (e.g., customer lifetime value (LTV), average revenue per user (ARPU)).

Why PMs should care:

- Perfect for mapping the user journey, especially new products, startups, etc.
- Pinpoints where the funnel is leaking.

Validated learning with pirate metrics (AARRR)



Example:

For a music streaming app:

- Acquisition: 50K installs from ads last month
- Activation: 60% play a song in the first session
- Retention: 30% listen weekly after 1 month
- Referral: 5% invite a friend
- Revenue: ₹150 average monthly subscription per paying user

If Retention is weak, you focus on playlists, personalized recommendations, and offline mode.

Reference: <https://posthog.com/product-engineers/aarr-r-pirate-funnel>

What it is:

A single, guiding metric that reflects the core value you deliver to users.

The Litmus Test for a Good NSM:

- It expresses user value.
- It represents your product strategy and measures progress.
- It's a leading indicator of revenue.

Why PMs should care:

- Creates alignment across teams, i.e., your entire company.
- Prevents teams from chasing random metrics that don't drive long-term value.

Example:

Airbnb's NSM:

Nights Booked. Because it reflects both guest satisfaction (they're booking stays) and host success (earning income).

Duolingo's NSM:

Daily Active Users (DAU) who complete at least one lesson.

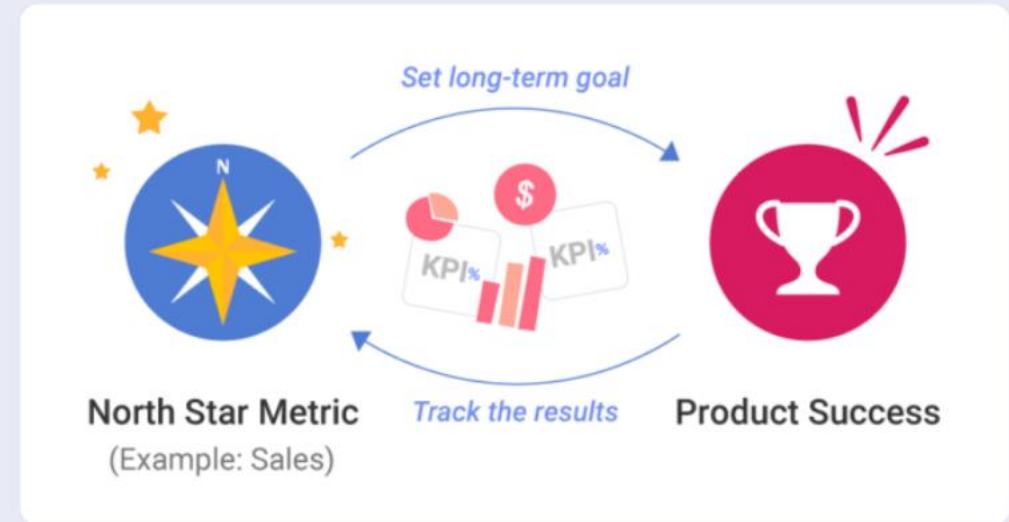
Just tracking DAU would be a vanity metric. By adding the qualifier "complete at least one lesson," they ensure they measure engaged learning, which is the core of their value proposition.

Uber's NSM:

Rides Completed. Because it reflects value for both sides, i.e. the rider and the driver. Further, it acts as a leading indicator of revenue.

Reference: <https://www.slingshotapp.io/blog/north-star-metric>

North Star Metric



Metrics you watch to make sure that improving your main metric has not broken something else.

Term	What it means
Primary metric	The number you are trying to move.
Counter metric	The number that would get worse if you moved the primary one the wrong way.
Guardrail	A counter metric with a limit attached. Cross the limit and you stop, roll back, or do not launch.

The two terms are often used interchangeably. The useful distinction is that a guardrail has a threshold and a consequence. A counter metric you only watch is just another chart.

Example:

A team wants to lift weekly active users on a food delivery app, and proposes sending more push notifications.

Type	Metric	Why
Primary	Weekly active users	The stated goal.
Input	Notification open rate	Tells you whether the message works at all.
Counter	Notification opt-out rate	Rises if users find the volume intrusive.
Guardrail	Uninstall rate must not rise above baseline + 0.2pp	Cross it and the campaign stops.

WAU rises 4%. Opt-out rate goes from 3% to 9%. Uninstalls rise 0.4 percentage points.

Without the guardrail this ships as a win. With it, you can see you borrowed short-term activity against long-term users and you have a pre-agreed reason to stop.

Why PMs should care:

- Every metric can be gamed. Push hard enough on one number and a team will find a way to move it that damages the product. The counter metric is the number that catches that.
- It also protects the launch decision. A threshold agreed before the results arrive gives you a stated reason to say no - which is much harder to argue after you have seen a 4% lift. And naming your own counter metric signals you understand your feature can do harm, which is the fastest way to sound senior in a metrics round.

To find one, ask: if someone wanted to move my primary metric without helping the user, how would they do it?

Primary metric	How it could be gamed	Counter metric
Time spent in app	Slow the interface, bury content	Task completion rate
Notifications sent	Send more, to everyone	Opt-out rate; uninstall rate
Ad revenue per session	More ads per screen	Session length; return rate
Listings removed by moderation	Remove aggressively	Legitimate sellers banned; appeal rate
Orders per user	Deep discounting	Contribution margin per order
Support tickets closed	Close without resolving	Reopen rate

What it is:

Splitting a population into groups that behave differently, so you can see what an average hides. A metric that looks flat overall is often two groups moving in opposite directions.

Dimension	Examples	Reveals
User lifecycle	New, returning, dormant, churned	Whether the problem is acquisition or retention
Tenure	First week, first month, over a year	Onboarding gaps versus long-term habit
Platform	iOS, Android, web, OS version	A release or device-specific bug
Geography	City, tier, country, language	Local supply, pricing or regulatory causes
Behaviour	Power user, casual, single-feature	Which group actually drives the metric
Acquisition source	Organic, paid, referral, partner	Whether a channel brings poor-fit users
Version	App build, model version, experiment arm	Whether a change you shipped caused it

For AI features, model version belongs in this list. It is the dimension most teams forget to cut by.

Why PMs should care:

- A drop concentrated in one segment points to a local cause. A drop spread evenly points to something systemic. Knowing which one you have decides where you look next, and stops you rebuilding a flow when the real issue was one OS version.
- Segmentation also protects you from mix shift. A metric can fall overall while rising in every segment, because the mix of users changed, more traffic arriving from the group that converts worse. If you only look at the total, you can end up "fixing" a product that is getting better.

Example:

Daily active users are down 8% over two weeks. Rather than hypothesising, cut the data twice.

Cut 1 (by platform):

Segment	Share of users	Change in DAU
iOS	50%	-1%
Android	50%	-15%
Overall	100%	-8%

The problem is not the product. It is Android.

Cut 2 (Android, by app version):

Segment	Share of users	Change in DAU
iOS	50%	-1%
Android	50%	-15%
Overall	100%	-8%

Two cuts have taken you from "engagement is down" to "the 8.3 Android release broke something". The fix is a rollback and a patch, not a product redesign and you would never have found it in the overall number, where a 40% collapse in one build shows up as 8%.

Even the best product ideas are just hypotheses until proven in the real world.

Testing helps PMs:

- **De-risk Decisions:** Would you bet a million dollars on a coin flip? Building a feature without testing is the business equivalent. Testing allows you to validate your assumptions on a small scale before committing significant engineering resources.
- **Allocate Capital Intelligently:** Your team's time is your company's capital. Testing ensures you invest in features that demonstrably improve the user experience and drive business metrics.
- **End "Opinion-Based" Debates:** Testing provides an objective referee in arguments about design or functionality. Instead of debating which button color is better, you can say, "Let's test it and let the data decide." This builds a culture of evidence over authority.
- **Learn Faster:** Every test, whether it succeeds or fails, provides a valuable insight into your users' behavior. The faster you learn, the faster you can iterate and outpace the competition

- **Formulate a Crystal-Clear Hypothesis:** This is the most critical step. A strong hypothesis is specific, testable, and connects an action to an expected outcome.
 - Weak Hypothesis: "Changing the button will improve clicks."
 - Strong Hypothesis: "If we change the primary call-to-action button on the homepage from blue to orange (the change), we will increase the click-through rate (the metric) by 10% (the expected impact) because the new color has higher contrast and creates more urgency."
- **Define Your Success Metric (and Counter-Metric):**
 - **Primary Metric:** Choose the one key metric that will determine if the test is a success (e.g., conversion rate, user retention).
 - **Counter-Metric:** Identify a metric that could be negatively affected. If you're testing a change to increase ad clicks, your counter-metric might be session duration or user block rate to ensure you're not just annoying your users.
- **Ensure Statistical Significance:** You need to run the test long enough and on a large enough group of users to be confident that your results aren't due to random chance. Use a sample size calculator to determine this beforehand.
- **Analyze and Act:** Once the test concludes, analyze the results. There are three possible outcomes:
 - **Success:** Your hypothesis was correct. Roll out the change to 100% of users.
 - **Failure:** The change had no effect or an adverse effect. Scrap the idea and, most importantly, document what you learned from the failure.
 - **Inconclusive/Interesting:** The primary metric didn't move, but a secondary metric did. This is an opportunity to dig deeper and form a new hypothesis.

What it is:

A controlled experiment where you compare **two versions** (A = control, B = variation) to see which performs better on a specific metric.

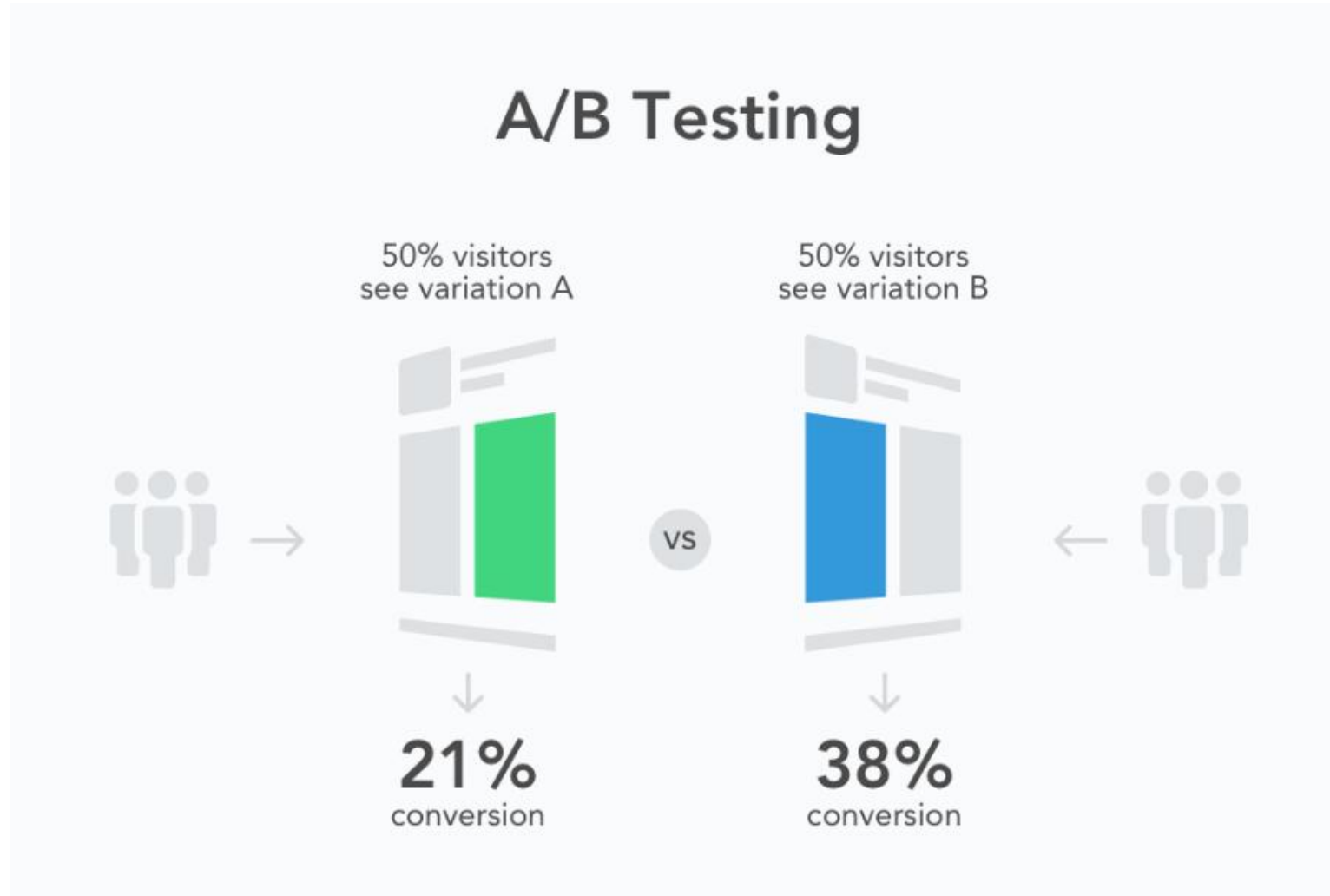
Why PMs should care:

- Avoids “design by opinion” and lets data decide.
- Can be used for UI changes, pricing models, onboarding flows, or feature variations, but just one test variable at a time.
- Minimizes risk by testing on a subset of users before a full rollout.

Example:

You want to improve checkout completion on your e-commerce site.

- **Hypothesis:** A “Buy Now” button in the cart will reduce drop-offs by 10%.
- **Test:**
 - Group A (Control): Regular checkout process.
 - Group B (Variant): Cart page with “Buy Now” button.
- **Result:** Group B has a 14% higher completion rate → Roll it out to all users.



Reference: <https://devopedia.org/a-b-testing>

What it is:

An eval is a test for an AI system. You give it an input, apply grading logic to its output, and measure success. It is how you define "good enough" for a system whose exact behaviour you cannot write down in advance.

Two types, and you need both:

- Offline evals run against a fixed, labelled set during development. They answer: is the model good?
- Online metrics are measured in production with real users. They answer: are users better off?

Where the test cases come from:

- Build the set from real failures - your support queue and logged sessions, not imagined scenarios.
- You do not need thousands. Anthropic's guidance is that 20 to 50 tasks drawn from real failures is a strong start.
- Every new failure you find gets added, so the set grows into a safety net over time.

Who grades the output:

- Humans - accurate but slow. A few hundred labelled examples goes a long way.
- A model acting as judge - cheap and scalable, but it needs a clear rubric, an option to answer "unknown" so it does not invent a verdict, and a periodic check against human raters.

Why PMs should care:

- The eval set is the closest thing an AI feature has to a PRD. It is where the quality bar actually lives.
- Without one, you cannot tell a model improvement from a model regression.
- No eval set, no launch. If you cannot say what score would stop a launch, you do not have a bar.
- Gate the launch on a quality metric, never on an engagement metric.

Example:

Before shipping a new model for a support chatbot, you run 300 labelled conversations covering your top ten intents. The bar is set in advance: hallucination rate under 2%, and no drop against the current model on any single intent. The new model scores better overall but 8% worse on refunds. You do not ship it for refunds. Those failed conversations then join the eval set, so the same regression cannot slip through again.

Two ways of measuring how well a model makes a call, and the dial that sets the balance between them.

Every call a model makes lands in one of four outcomes. The two metrics below are built from these, and the abbreviations are used throughout:

- True Positive (TP) - the model flagged it, and it really was a problem.
- False Positive (FP) - the model flagged it, but it was fine. A false alarm.
- False Negative (FN) - the model let it through, but it was a problem. A miss.
- True Negative (TN) - the model let it through, and it was fine.

The two metrics:

- **Precision = $TP / (TP + FP)$**

Of everything the model flagged, how much was actually right.

Flag 100 listings as fake and 80 truly are → precision = 80%.

- **Recall = $TP / (TP + FN)$.**

Of everything that was actually wrong, how much did the model catch.

200 fake listings exist and the model catches 80 → recall = 40%.

The threshold is the confidence cut-off above which the model acts.

- Raise it and you get fewer false positives but more misses.
- Lower it and you catch more but flag more wrongly.
- Precision and recall move in opposite directions. Improving one worsens the other.

Calibration: A confidence score of 0.9 only means "90% likely correct" if the model is calibrated. Many are not. Check before you trust the number.

Why PMs should care:

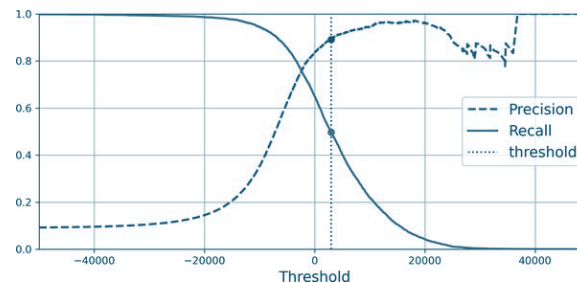
- The engineer can set the threshold anywhere. Only the PM can say which mistake hurts the user more. That makes it a product decision, not an ML decision.
- Optimise recall when false negatives cost more than false positives. Optimise precision when it is critical that positive calls are right.
- "Improve accuracy" tells the team nothing. "Protect precision, we cannot ban genuine sellers" tells them exactly what to build.
- Moving the threshold is the cheapest lever you have. It needs no retraining.

Example:

Two products, opposite answers from the same maths.

- Seller moderation on a marketplace: a false positive bans a real seller and takes away their income. A false negative leaves one fake listing up a day longer. → Protect precision.
- Fraud detection on payments: a false positive blocks one genuine transaction and the user retries. A false negative loses real money. → Protect recall.

Same model, same maths, opposite calls. The difference is what the mistake costs the user.



Reference: Google Machine Learning Crash Course, "Classification: Accuracy, recall, precision and related metrics" - developers.google.com/machine-learning/crash-course/classification/accuracy-precision-recall.

What it is:

The controls that decide what an AI system may send to a user, and where a person must step in.

Guardrails - stopping confident-but-wrong output:

- Grounding: the model may only answer from retrieved sources, not from memory.
- Citations: show the source beside the claim so the user can check it.
- Abstention: if nothing relevant is retrieved, return "no answer" rather than guessing.
- Topic gates: route regulated or high-risk topics - legal, medical, financial - to a human, whatever the confidence score says.
- Hallucination rate: the share of outputs containing an unsupported claim, measured on a labelled sample. Never inferred from complaint volume.

Human-in-the-loop - how much the machine does alone:

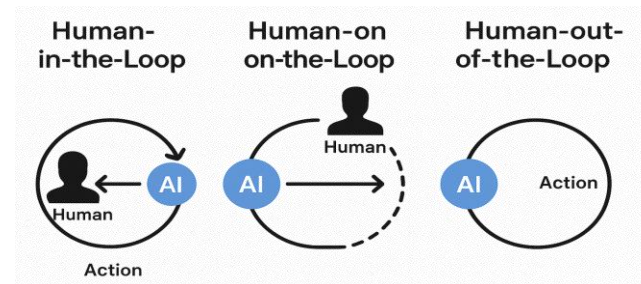
- The spectrum: fully automated → human approves → human assisted → human only.
- Place the gate two ways: by confidence (low scores go to a human) and by risk (sensitive actions go to a human regardless of score).
- Reversibility is the deciding variable. Automate what can be undone. Gate what cannot.
- Automation bias: when the model is usually right, people stop checking and start rubber-stamping. Design against it.

Why PMs should care:

- Confident-but-wrong destroys trust fastest. Users forgive "I don't know". They do not forgive being misled.
- Hallucination rate is usually the right launch gate for a generative feature - not adoption, not engagement.
- Guardrails cost you coverage. Fewer answers, but the ones you give can be trusted. That trade-off is yours to make.
- It decides who is accountable when the system is wrong. That question always comes up, and it is a PM question.
- Naming your human-in-the-loop design early and unprompted signals you treat AI as decision support, not an autonomous agent.

Example:

- An AI copilot for customer support agents. Three automation levels, each chosen by reversibility.
- Drafts a customer reply → the agent edits and sends. The agent stays accountable for every message that goes out.
- Auto-fills internal CRM fields → no approval needed. It is internal and easily corrected.
- Never issues a refund on its own → irreversible, and it touches money.
- Same product, same model, three different answers, because the cost of being wrong is different each time.



References:

Google PAIR, "People + AI Guidebook"
Microsoft, "Guidelines for Human-AI Interaction"

What it is:

The full cost of an AI system delivering one completed unit of user value, and not just one API call. Cost-per-resolved-task is the honest version. It counts the retries, the escalations and the failures. Three model calls at ₹2 each, and only 60% of attempts succeed → cost per resolved task = ₹10, not ₹6. Unlike software, this cost is variable. It scales with every use and never falls to zero.

The latency-quality-cost triangle:

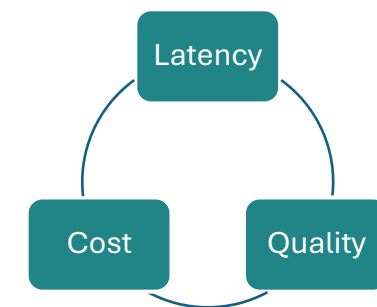
Every AI feature trades three things against each other. You cannot set all three independently.

Corner	What it means, and how you measure it
Quality	How good the output is. Measured by your eval score, not by user opinion.
Latency	How long the user waits. Measure p95, not the average.
Cost	Cost per resolved task, including retries and escalations.

Why the three move together - A bigger model has more parameters. Every word it produces takes more computation. More computation costs more money and takes more time. So pushing quality up drags latency and cost up with it.

The levers:

- Model routing: send simple queries to a cheap model and escalate only when needed.
- Caching: never pay twice for the same answer.
- Scope: a narrower feature costs less per task than an open-ended one.



Why PMs should care:

- Traditional SaaS replicates at near-zero marginal cost. AI does not. Andreessen Horowitz found AI companies often spend a quarter or more of revenue on cloud resources, running gross margins materially below comparable software businesses.
- This breaks per-seat pricing. A flat monthly fee against usage-linked cost means the more successful your feature is, the worse your margin gets.
- Your heaviest users can be your least profitable. Average margin hides this.
- Cost is a product constraint, not a finance problem. It shapes what you build, not just what you charge.
- Asked directly in interviews: "If there is an AI product and a non-AI product, how does your pricing strategy differ?"

Example:

An AI assistant priced at ₹500 per seat per month.

- A light user costs ₹40 a month to serve. A heavy user costs ₹900.
- Average margin looks healthy. The top decile is deeply loss-making.

Every fix is a product decision: route simple queries to a cheaper model, cache repeated answers, cap usage in the base tier, or move from seat pricing to usage-based pricing.

What is a Customer Journey Map?

A customer journey map is a chronological visualization of the process a specific customer goes through to accomplish a goal. It shows the experience from the customer's point of view across stages, channels, and touchpoints rather than limiting the analysis to one screen or feature.

A useful map combines observable behaviour with the customer's thoughts and emotions. It also captures pain points, moments of truth, and opportunities for improvement. The map may describe the current experience or a future experience the product team wants to create.

Key Components

1. **Actor:** The persona or user whose experience is being mapped.
2. **Scenario and goal:** The specific situation and outcome the actor is trying to achieve.
3. **Journey phases:** The high-level stages that organize the experience over time.
4. **Actions and touchpoints:** What the customer does and where interactions occur.
5. **Thoughts and emotions:** What the customer believes, expects, worries about, or feels.
6. **Pain points and opportunities:** Friction to remove and moments where value can be created.

Why it matters for PMs?

1. **End-to-end view:** Prevents teams from optimizing one screen while ignoring the wider experience.
2. **Better prioritization:** Shows where friction is frequent, severe, or damaging to the business.
3. **Cross-functional alignment:** Creates a shared customer narrative for product, design, engineering, operations, marketing, and support.
4. **Clear measurement:** Connects each journey stage with relevant product and operational metrics.

PM Takeaway

A journey map is not merely a process diagram. It explains what the customer is trying to do, how the experience feels, and where the product can intervene.

How to Create a Customer Journey Map

Journey Mapping Process

1. **Choose one actor and scenario.** A student, restaurant partner, and delivery rider experience different journeys and should not be combined into one map.
2. **Set the boundaries.** Define where the journey starts and ends, such as recognizing a need through resolving a post-purchase problem.
3. **Gather evidence.** Use interviews, observation, usability tests, support tickets, reviews, behavioural analytics, and operational data.
4. **Map the phases and actions.** Arrange what the customer does in chronological order across channels and touchpoints.
5. **Add thoughts and emotions.** Capture expectations, questions, confidence, frustration, and delight at each stage.
6. **Identify opportunities and metrics.** Translate the most important pain points into hypotheses and measures of improvement.



Evidence Sources

Source	What it reveals	Example
Customer interviews	Goals, expectations, reasons, and emotions	Why a user abandons checkout
Product analytics	Behaviour, frequency, conversion, and drop-offs	Time spent selecting a restaurant
Support and reviews	Recurring friction and language used by customers	Complaints about late delivery
Observation / usability tests	Unspoken behaviour and workarounds	Switching repeatedly between filters
Operational data	Handoffs, delays, defects, and service constraints	Actual ETA versus promised ETA

Current-State vs Future-State Maps

Map Type	Purpose	When to use
Current State	Documents the experience customers have today.	Diagnosing friction and prioritizing improvements
Future State	Describes the experience the team intends to create.	Aligning teams around a redesigned journey

Practical Rule

Use one actor and one scenario per map. A map that tries to represent everyone usually becomes too generic to guide decisions.

Working Example : Ordering Dinner after a late class

Actor: IIMA student

Scenario: Ordering dinner after an evening class when the hostel mess is closed.

Goal: Receive an affordable, filling meal within roughly 30 minutes, with minimal effort and uncertainty.

Current-State Journey

Stage	Customer Action	Thought / Emotion	Pain Point
Need	Opens the app after class	‘What can I get now?’ Tired	Mess is closed; few known options
Discover	Searches for open and fast options	‘What will arrive quickly?’ Uncertain	Choice overload; unavailable restaurants
Decide and pay	Selects a meal and completes payment	‘Why is the total so high?’ Frustrated	Fees appear late; coupon confusion
Wait	Tracks the order	‘Will this actually arrive on time?’ Anxious	ETA slips; weak delay communication
Receive and resolve	Receives the meal; reports any issue	‘Is everything correct?’ Relieved or disappointed	Missing items; slow support resolution

Opportunities & Metrics

Stage	Potential Improvement	Primary Metric
Need	Late-night shortcut showing open, affordable options	Time to first relevant option
Discover	Rank choices using total cost and realistic ETA	Menu-to-cart conversion
Decide and pay	Show the payable total before checkout	Checkout abandonment
Wait	ETA confidence range and proactive delay alerts	ETA accuracy; support contact rate
Receive and resolve	One-tap issue reporting and faster refund	Defect rate; resolution time

Moment of Truth: The promised delivery time shapes trust throughout the journey. An optimistic ETA may improve conversion initially but damage satisfaction and repeat usage when it is missed.

Turning Journey Insights into Product Decisions

From Map to Roadmap

- 1. **Cluster related pain points.** Combine symptoms that may share a common root cause.
- 2. **Identify moments of truth.** Prioritize interactions that disproportionately affect trust, conversion, or retention.
- 3. **Assess opportunities.** Compare customer frequency and severity with business impact, strategic fit, and implementation effort.
- 4. **Assign ownership.** Clarify whether product, design, engineering, operations, support, or a partner controls the touchpoint.
- 5. **Test and measure.** Convert opportunities into hypotheses, experiments, and stage-specific metrics.

Related Mapping Tools

Tool	Primary Focus	Typical Content	When to Use
Customer Journey Map	Customer’s end-to-end experience	Stages, actions, thoughts, emotions, pain points	Finding and prioritizing experience gaps
User Flow	Steps inside a product interface	Screens, decisions, and navigation paths	Designing a specific task flow
Service Blueprint	Systems supporting the journey	Frontstage, backstage, people, and processes	Fixing operational handoffs and service delivery

Common Mistakes

1. Creating the map from internal assumptions without customer evidence.
2. Combining several personas or scenarios into one generic journey.
3. Mapping only product screens and ignoring offline or cross-channel touchpoints.
4. Listing pain points without owners, priorities, or measurable next steps.
5. Treating the map as a one-time workshop output rather than updating it with new evidence.

PM Interview Takeaway

In a product-design case, first define the actor, goal, and journey boundaries. Map the major stages, identify the most important pain point, and then propose solutions and success metrics. Depth on the critical moment is more valuable than listing features for every stage.

Connection with JTBD

JTBD explains why the customer seeks progress. The journey map shows how the experience unfolds and where that progress is blocked or enabled.

References: [Nielsen Norman Group - Journey Mapping 101](#) | [NN/g - When and How to Create Customer Journey Maps](#)

02

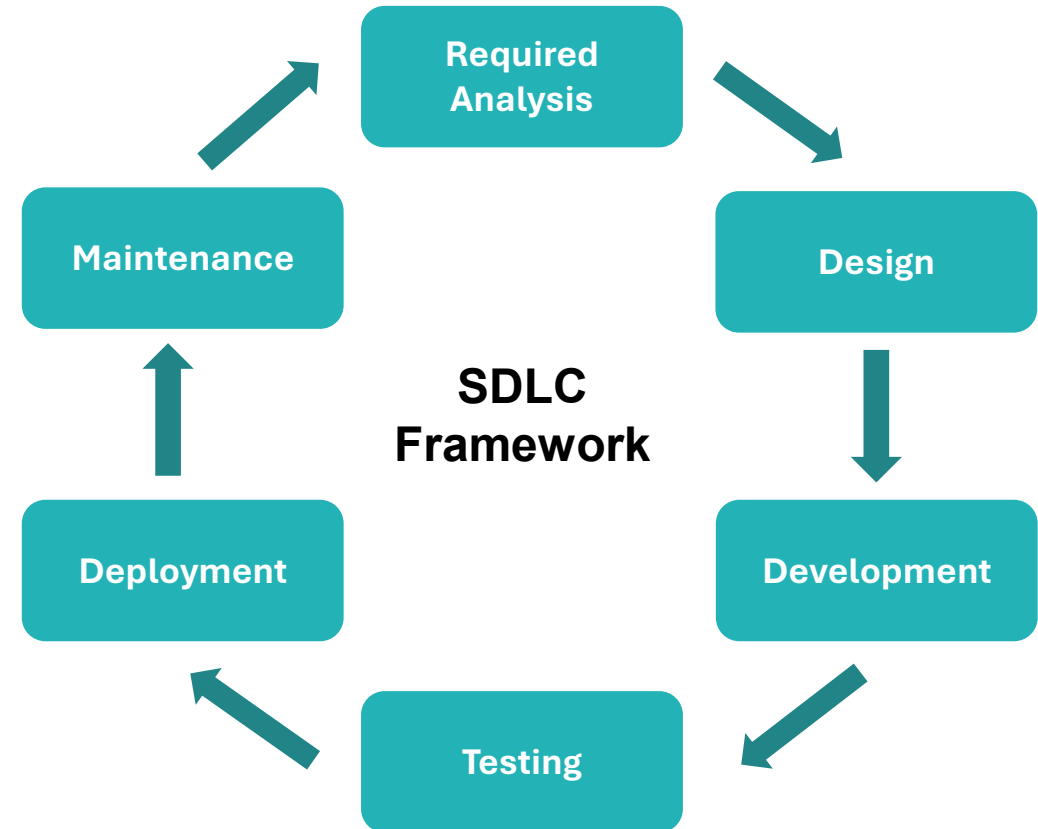
Tech Concepts

SDLC Framework - Why PMs should care

- SDLC keeps the product journey predictable and efficient.
- Understanding each stage helps you time the decisions. You know when to involve which stakeholders, how to manage timelines, and how to prevent costly rework.
- As a PM, you'll often decide when a feature is ready to move to the next phase and balance speed vs. quality.

Example: If you're building a new checkout flow, you lock requirements early so developers aren't guessing mid-coding, which avoids costly rework.

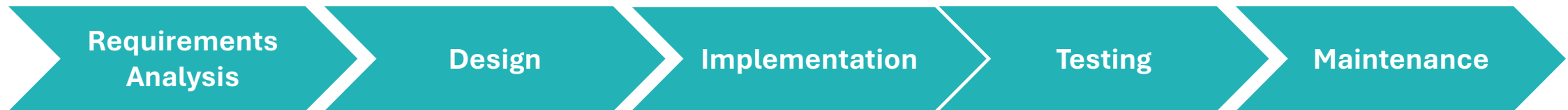
There are two common methodologies for SDLC- **Waterfall** and **Agile**



Waterfall Model

The Waterfall model follows a linear, sequential process where each phase begins only after the previous one is complete, with no overlap. Best suited for small, well-defined projects with minimal expected changes.

Linear flow: Requirement Analysis → Design → Implementation → Testing → Maintenance



Limitations

- **Low adaptability:** Late-stage defects (e.g., in testing) require extensive rework and fallback to previous stages.
- **Delayed delivery:** Sequential phases can lead to late releases, reducing competitive advantage.
- **Defect risk:** Defects are identified during testing and it occurs at the end of SLDC, thus the risk of project failure is concentrated at the end.

Agile

Agile is an iterative, time-boxed approach that delivers software in small increments, enabling rapid adaptation to change. It prioritizes:

- Working software over comprehensive documentation
- Responding to change over following a fixed plan

Core Principles

- Deliver valuable software early and continuously.
- Embrace changing requirements, even late in development, to maintain competitive edge.
- Release working software frequently (every 2-8 weeks) to get rapid feedback and reduce defects.
- Focus on technical excellence and good design to enhance agility.
- Maximize work not done-avoid non-value-adding activities.
- Conduct regular retrospectives for continuous improvement.

Agile Development

Scrum

It is built around **self-organizing, cross-functional teams** working in short, fixed-length cycles called **sprints** (typically 1-4 weeks) while delivering incremental developments.

Roles: Product Owner (defines priorities), Development Team (executes), Scrum Master (facilitates process)

Events: Sprint Planning, Daily Scrum, Sprint Review, Sprint Retrospective.

Kanban

Visualises task on a Kanban Board with columns like “To Do”, “In Progress” and “Completed”.

Facilitates progress tracking, identifies bottlenecks, and adapt to changing requirements.

Ideal for environments with frequently shifting priorities.

Waterfall vs Agile

Aspect	Waterfall	Agile
Process Flow	Linear, sequential phases	Iterative, incremental sprints
Flexibility	Low - changes are costly and disruptive	High - welcomes changes even late in development
Delivery	Single final release after all phases	Frequent, small releases (every 2-8 weeks)
Customer Involvement	Primarily at start and end	Continuous feedback throughout
Risk Management	High risk at end (defects found late)	Risks addressed incrementally
Best Suited For	Small, well-defined projects	Dynamic, evolving requirements
Testing	Performed after implementation is complete	Continuous testing within each sprint
Team Collaboration	Limited between business and developers	Daily collaboration, cross-functional teams
Documentation	Heavy, detailed documentation	Lightweight, just enough for clarity
Examples of Frameworks	N/A (Waterfall is itself a framework)	Scrum, Kanban, XP

User Experience (UX) Design is the process of **designing products that provide meaningful, relevant, and intuitive experiences** to users.

Design Thinking is one of the most widely used approaches for solving user problems and creating user-centered products.

Design Thinking Process

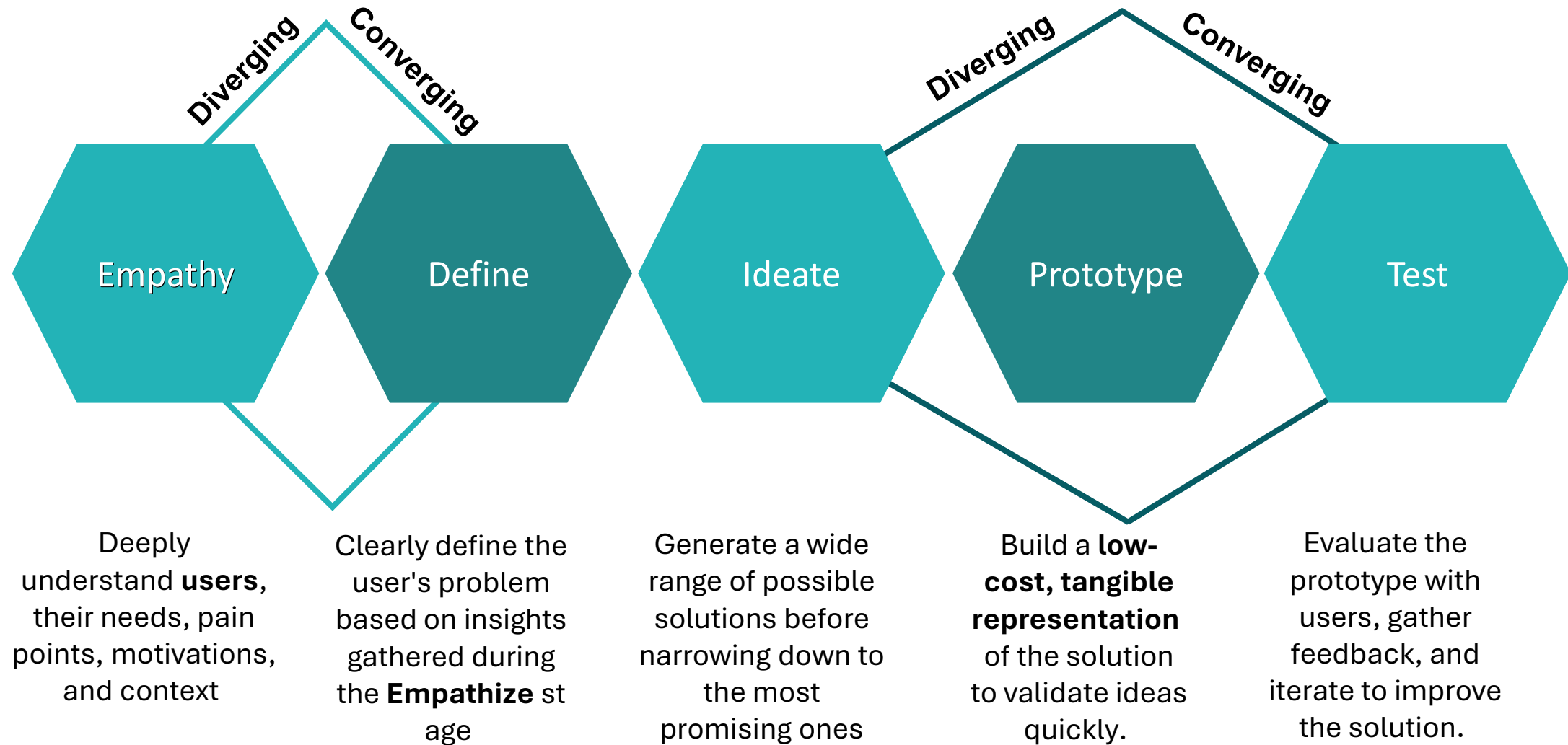
Design Thinking is an **iterative, user-centric problem-solving approach** for creating products that solve real user needs. It focuses on understanding users, defining the right problem, exploring solutions, building prototypes, and validating them through user feedback.

The process is iterative i.e. teams often revisit earlier stages based on user feedback and new learnings.

The five stages of the Design Thinking process are:

1. Empathize
2. Define
3. Ideate
4. Prototype
5. Test

Five Stages (EDIPT)



Step 1: Empathize

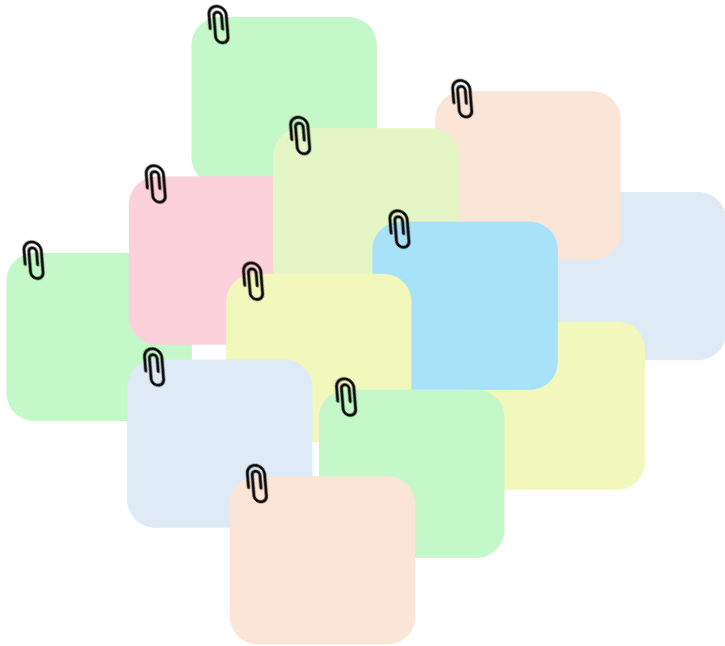
- ❖ Conduct **secondary research** to understand the market, users, and existing solutions.
- ❖ Perform **competitive analysis** to identify opportunities and market gaps.
- ❖ Gather user insights through **interviews, surveys, observations, and usability studies** to understand user needs, pain points, and behaviors.

Step 2: Define

- ❖ Organize research findings using **Affinity Mapping (KJ Method)** to identify common themes and patterns.
- ❖ Synthesize insights to uncover **user pain points, needs, and opportunities**.
- ❖ Formulate a **clear problem statement** from the user's perspective to guide ideation.

The KJ Method and Affinity Mapping

The **KJ Method**, developed by **Jiro Kawakita**, is a structured approach to **Affinity Mapping**. It helps teams organize large amounts of qualitative user research into meaningful themes, making it easier to identify patterns, pain points, and opportunities.



Capture

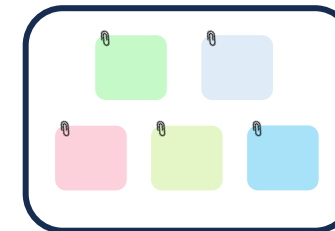
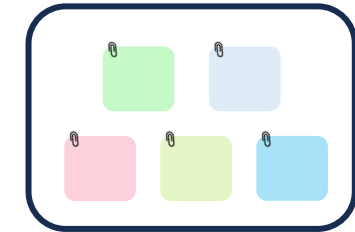
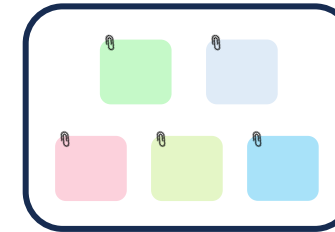
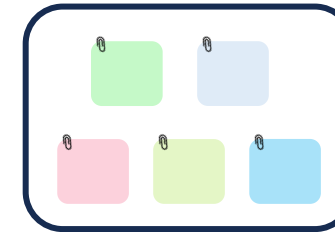
Record observations, user quotes, ideas, or insights on individual sticky notes.

Group

Cluster related notes based on similar themes or patterns.

Label

Assign a descriptive label to each group to summarize the underlying insight.



Step 2: Define

User Persona

A **User Persona** is a fictional representation of an ideal user, created using research and real user insights. It helps teams understand user needs, goals, motivations, and pain points, enabling better product decisions.

A good persona typically includes:

- Demographics
- Goals & motivations
- Pain points
- Behaviors
- Context of use

Types of Personas

- **Primary Persona:** The main target user of the product.
- **Secondary Persona:** Users with additional needs that should also be considered.
- **Negative Persona:** Users for whom the product is **not** intended.

Problem Statement

A problem statement clearly defines the user's challenge from their perspective and guides the team toward solving the right problem.

Template:

[User] needs a way to **[need]** because **[reason/insight]**.

Example:

A busy professional needs a faster checkout process because lengthy payment steps lead to abandoned purchases.

Step 3: Ideate

Generate a wide range of possible solutions without judging ideas initially.

Common techniques include:

- Brainstorming
- Mind Mapping
- Crazy 8s
- SCAMPER

Prioritize ideas based on user value, impact, and implementation effort before selecting the best solution.

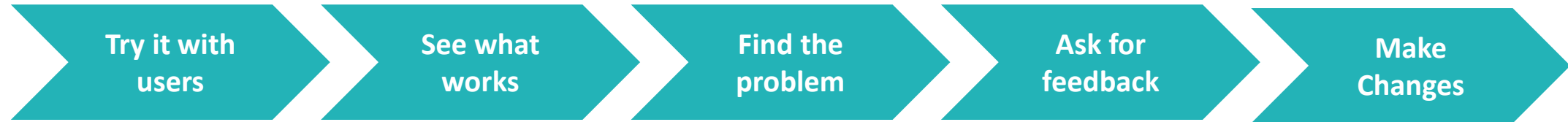
Step 4: Prototype

Create a low-cost representation of the solution to validate assumptions before full-scale development.

Prototype fidelity can range from paper sketches and wireframes to high-fidelity interactive mockups.

Step 5: Test

The aim is to see how people use the prototype and learn what needs to change. You watch, listen, and ask questions while they interact with it.



- Use A/B or multivariate tests when appropriate.
- Conduct heuristic evaluations using Nielsen's 10 usability heuristics (used for testing usability of online products).

Visibility of System Status - Keep users informed with timely feedback (e.g., loading indicators).

Match Between System & Real World - Use familiar terms, icons, and mental models.

User Control & Freedom - If users get into an unwanted scenario by mistake, they should have an emergency exit. Allow undo/redo and easy exits.

Consistency & Standards - Maintain consistent terminology and design patterns. E.g. "add to cart" and "add to basket" should not be used interchangeably.

Error Prevention - Avoid error-prone situations; confirm critical actions.

Recognition Over Recall - Reduce cognitive load with visible options (e.g., autocomplete).

Flexibility & Efficiency - Support both novice and expert users (e.g., shortcuts).

Aesthetic & Minimalist Design - Show only relevant information upfront.

Error Recovery Support - Provide clear, actionable error messages.

Help & Documentation - Offer concise, searchable, task-focused guidance within the interface.

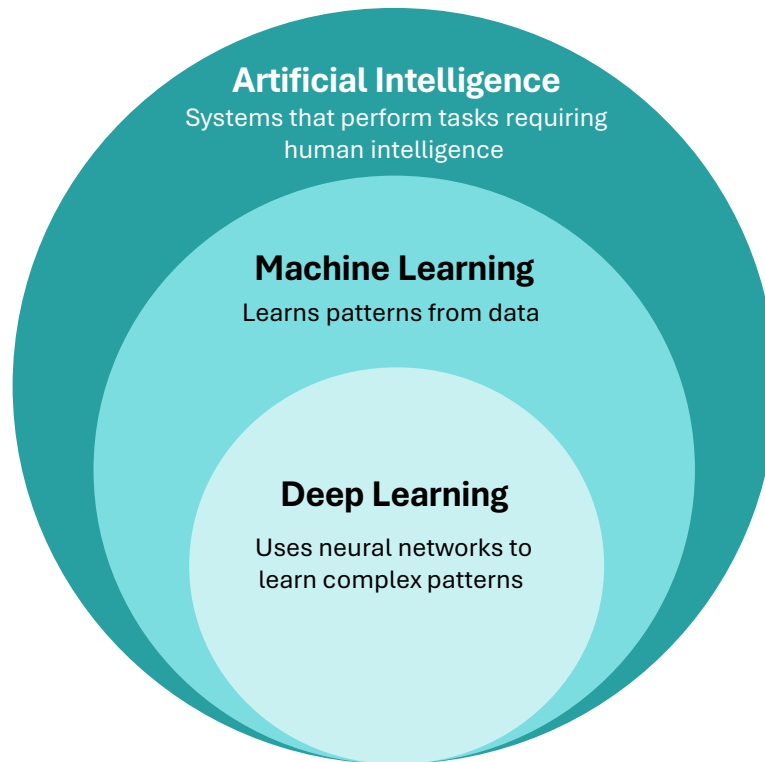
PM Takeaways

Empathy drives better products: Deeply understanding users helps solve the right problems, not just build new features.

Design Thinking is iterative: Product decisions improve through continuous prototyping, testing, and user feedback.

Research reduces uncertainty: User research and validation help minimize assumptions and improve product outcomes.

Balance desirability, feasibility, and viability: Great products meet user needs, are technically feasible, and create business value.

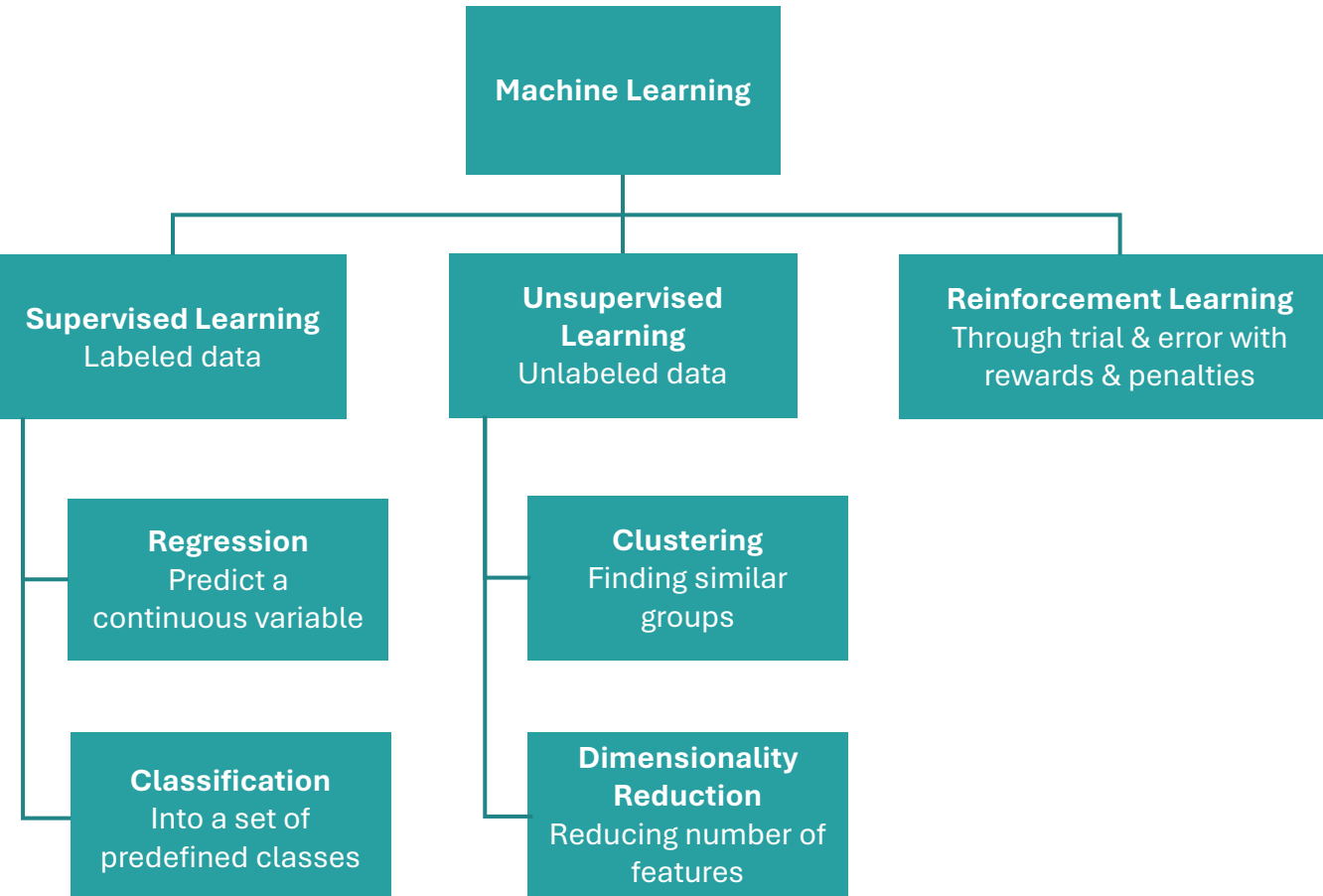


Terminologies

1. **Artificial Intelligence (AI):** Artificial Intelligence (AI) is the field of building systems that can perform tasks that typically require human intelligence, such as **perception, reasoning, learning, and decision-making**.
2. **Machine Learning (ML):** Machine Learning (ML) is a subset of AI where algorithms learn patterns from data to make predictions or decisions **without being explicitly programmed for every scenario**.
3. **Deep Learning (DL):** Deep Learning (DL) is a subset of ML that uses **multi-layered neural networks** to automatically learn complex patterns from large datasets.

Why it matters for PMs

1. AI powers product features such as **personalization, recommendations, search, automation, and intelligent assistants**.
2. Helps build **smarter, data-driven products** that improve user experience and business outcomes.
3. PMs must balance **accuracy, latency, cost, privacy, and fairness** when building AI-powered products.



Types of Machine Learning

1. Supervised Learning: Learns from labelled data.

a) Regression → Predicts continuous values (e.g., house prices, sales forecasting).

b) Classification → Predicts predefined categories (e.g., spam detection, fraud detection, customer churn).

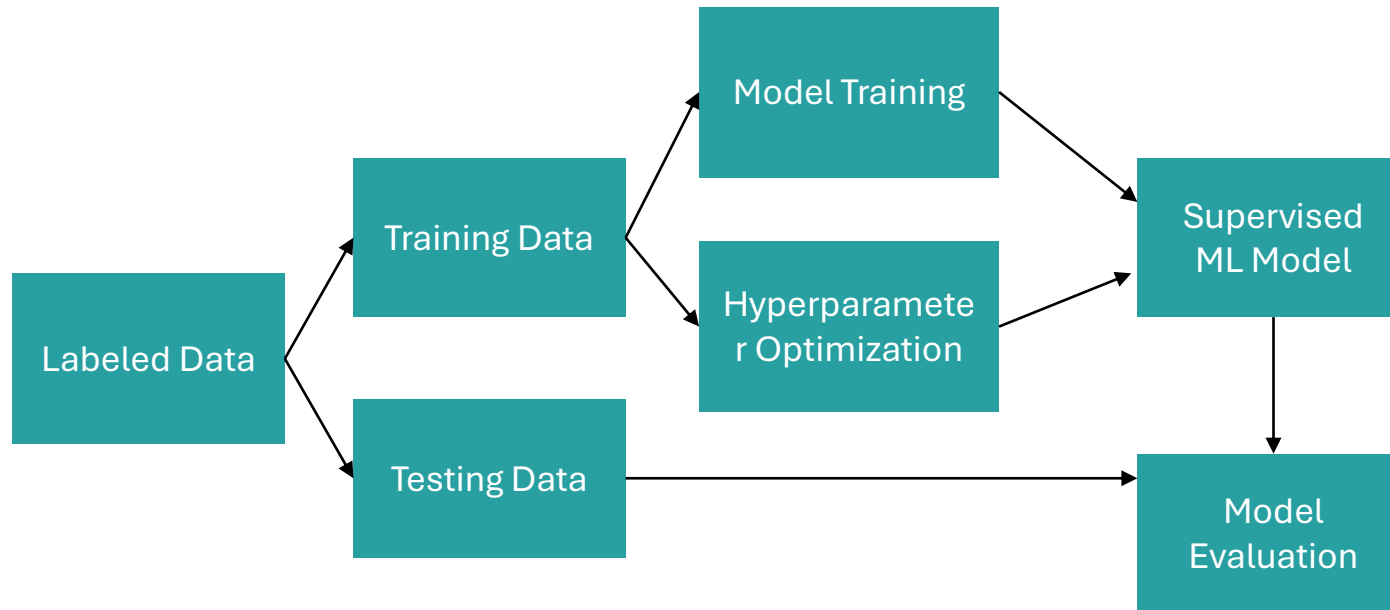
2. Unsupervised Learning: Finds hidden patterns in **unlabelled data**.

a) Clustering → Groups similar data points (e.g., customer segmentation).

b) Dimensionality Reduction → Reduces features while preserving important information (e.g., visualization, feature compression).

Modern foundation models often use self-supervised learning, which learns from unlabelled data by predicting missing or masked information.

3. Reinforcement Learning: Learns through **trial and error** by maximizing rewards over time (e.g., robotics, game AI, recommendation ranking).



Model training basics

1. Labelled data is typically split into:

- **Training Set** → Used to learn patterns and train the model.
- **Validation Set** → Used to tune hyperparameters and compare different models.
- **Test Set** → Used once to evaluate performance on unseen data.

2. After training

The final model is evaluated on the **test set** and, if performance is satisfactory, deployed to make predictions on new data.

PM Takeaway

A successful ML product balances **accuracy, latency, cost, interpretability, and user experience** and not just model performance.

Recommender Systems

Recommender systems help users discover relevant content by predicting what they are likely to enjoy. They power products like **Netflix, Amazon, Spotify, and YouTube**.

1. Content-Based Filtering

Recommends items similar to what the user has previously liked based on item features.

Pros

- Works well for new items
- Easy to explain recommendations

Limitation : May recommend only similar content (limited discovery)

2. Collaborative Filtering

Uses the behaviour and preferences of similar users to recommend items.

Pros

- Can discover unexpected but relevant content
- Does not require item metadata

Limitation : Suffers from the **cold-start problem** for new users or items

3. Hybrid Recommendation

Combines multiple recommendation techniques to improve accuracy and overcome individual limitations.

Examples: Netflix, Amazon, Spotify, YouTube.

Why it matters for PMs

- Personalization improves **engagement, retention, and customer satisfaction**.
- The choice of recommendation approach affects **accuracy, scalability, explainability, and cold-start performance**.
- **Balance personalization with diversity** to avoid repetitive recommendations and improve content discovery.

Neural Networks

A **Neural Network** is a machine learning model made up of interconnected layers of nodes (neurons) that learn complex patterns from data.

Basic Architecture

Input Layer

- Receives input features.

Hidden Layers

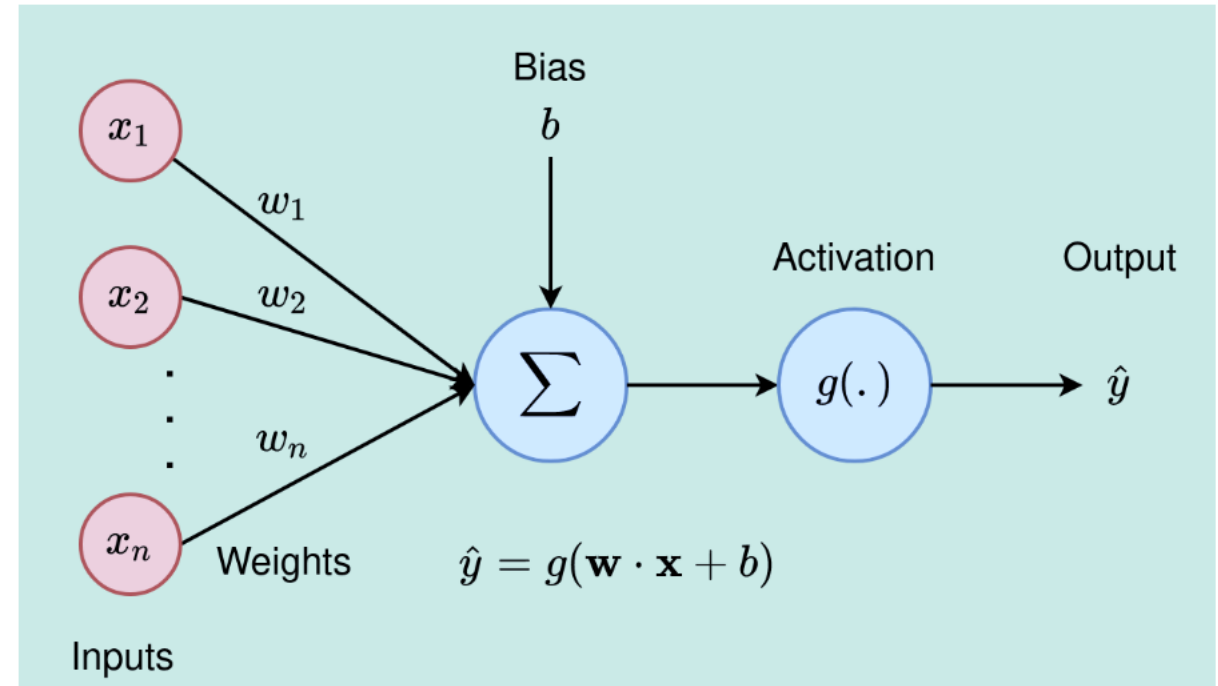
- Process the inputs and learn meaningful patterns.

Output Layer

- Generates the final prediction or classification.

Key Concepts

- **Weights:** Represent the importance of each input.
- **Activation Function:** Determines whether information should be passed to the next layer.
- **Training:** The network adjusts its weights using training data to improve prediction accuracy.



Why it matters for PMs

Neural networks power applications such as:

- Image recognition
- Speech recognition
- Recommendation systems
- Fraud detection
- Large Language Models (LLMs)

Deeper networks can learn more complex patterns but require larger datasets and greater computational resources.

AI/ML - Advantages for PMs

1. **Automation:** Reduces manual effort by automating repetitive tasks.
2. **Personalization:** Delivers tailored recommendations and user experiences.
3. **Better Decision-Making:** Generates data-driven insights for informed decisions.
4. **Scalability:** Processes large volumes of data efficiently.
5. **Continuous Learning:** Models can improve over time with new data.

AI/ML - Disadvantages/Challenges

1. **Data Quality:** Poor or biased data leads to inaccurate predictions.
2. **Bias & Fairness:** Models may unintentionally reinforce biases present in training data.
3. **Privacy & Security:** AI systems must protect sensitive user data and comply with regulations.
4. **Interpretability:** Complex models can be difficult to explain and debug.
5. **Cost & Infrastructure:** Training and deploying AI models can require significant computational resources.
6. **Hallucinations (Generative AI):** Generative AI models may produce incorrect or fabricated outputs that appear convincing.

Algorithm Family	Great For	Ask Your Team	Watch Out For (The PM's Concern)
Linear Models	Establishing a simple baseline, predictable outcomes.	"Is the relationship between our inputs and output likely linear?"	Oversimplification. May miss complex patterns in user behavior.
Tree-Based Models	Explaining why a decision was made (high interpretability).	"How can we visualize the decision rules for the business team?"	Overfitting. The model might learn the training data too well and fail on new data.
Neural Networks	Finding complex, non-linear patterns in massive datasets (e.g., image or speech).	"Do we have enough data and computing power to train this effectively?"	The "Black Box" Problem. It's very difficult to explain why the model made a specific prediction, which is a risk for user trust and debugging.

A Framework for Applying AI/ML

Before building an AI/ML feature, ask these key questions:

1. Product Goal

What user problem are we solving?

Why does this require AI instead of a rule-based solution?

How does this feature support the product goals?

2. Data Readiness

What data do we have, and what data do we need?

Is the data accurate, representative, and unbiased?

Do we have enough data to train a reliable model?

3. MVP & Integration

What is the smallest AI feature that delivers value (MVP)?

How will the AI component integrate with the existing product?

What happens if the AI service is unavailable?

4. Success Metrics

How will success be measured? (e.g., Accuracy, CTR, Conversion, Retention)

What business KPIs should improve?

What is the acceptable trade-off between false positives and false negatives?

5. Monitoring & User Experience

How will users interact with and trust AI-generated outputs?

How will we monitor model performance after launch?

What is the rollback or fallback plan if the model performs poorly?

Generative AI (GenAI)

What is Generative AI?

Generative Artificial Intelligence (GenAI) refers to AI systems that create new content such as text, images, audio, video, or code-based on patterns learned from large datasets. Unlike traditional Machine Learning models, which primarily predict or classify data, Generative AI produces original outputs in response to user prompts.

Types of Generative AI

Type	Description	Popular Products	PM Use Cases
Text Generation	Generates human-like text	ChatGPT, Claude, Gemini	Chatbots, summarization, content creation
Image Generation	Creates images from prompts	Midjourney, DALL·E, Adobe Firefly	Marketing creatives, product mockups
Code Generation	Writes or completes code	GitHub Copilot, Cursor, Amazon Q	Developer productivity, rapid prototyping
Audio Generation	Generates speech or music	ElevenLabs, Suno	Voice assistants, podcasts, accessibility
Video Generation	Creates videos from prompts	Veo, Runway, Pika	Product demos, advertisements, educational content

How is Generative AI Different from Traditional ML?

Traditional Machine Learning	Generative AI
• Predicts or classifies data • Uses structured data • Outputs predictions • Example: Fraud Detection	• Creates new content • Learns from large multimodal datasets • Generates text, images, audio, video, or code • Example: ChatGPT generating an email

Common Applications

- Conversational AI and customer support
- Content creation and summarization
- Personalized recommendations
- Software development assistance
- Marketing and design automation
- Enterprise knowledge assistants

Challenges

- Hallucinations and inaccurate outputs
- Bias in generated content
- Privacy and security concerns
- High computational costs
- Copyright and intellectual property issues

What are AR & VR?

Augmented Reality (AR) overlays digital content onto the real world.

Virtual Reality (VR) immerses users in a completely digital environment.

Feature	Augmented Reality (AR)	Virtual Reality (VR)
Environment	Real world + digital overlays	Fully virtual world
User awareness	User remains aware of surroundings	User is immersed in virtual environment
Hardware	Smartphone, tablet, smart glasses	VR headset + controllers
Goal	Enhance reality	Replace reality
Examples	Pokémon GO, Snapchat Filters, Google Maps Live View	Meta Quest, PlayStation VR, Apple Vision Pro (VR apps)

Why it matters for PMs

- Creates immersive customer experiences.
- Enables visualization before purchase.
- Improves training and remote collaboration.
- Opens new interaction models beyond traditional screens.
- Requires balancing usability, hardware adoption, and development cost.

Applications of AR & VR

Industry	AR Example	VR Example
Retail	IKEA Place furniture preview	Virtual shopping stores
Healthcare	Surgical guidance, vein visualization	Medical training simulations
Education	Interactive textbooks	Virtual classrooms & labs
Manufacturing	Assembly instructions	Safety & equipment training
Navigation	Google Maps Live View	Virtual building walkthroughs
Entertainment	Pokémon GO, Snapchat filters	VR gaming & immersive concerts
Real Estate	Property visualization	Virtual home tours

How AR & VR Work

Augmented Reality (AR)	Virtual Reality (VR)
Visualization: Uses cameras and computer vision to understand the real world and overlay digital objects accurately.	Interaction: Users enter a fully virtual environment using a VR headset and interact through head, hand, or controller tracking.
Instruction: Displays real-time contextual information, labels, or step-by-step guidance directly within the user's field of view.	Interactivity: The virtual environment responds instantly to user actions, enabling natural navigation and object manipulation.
Interaction: Users remain aware of the real world while interacting with digital content through touch, gestures, voice, or smart glasses.	Sensory Feedback: Visual, audio, and haptic feedback create depth, realism, and a strong sense of presence.

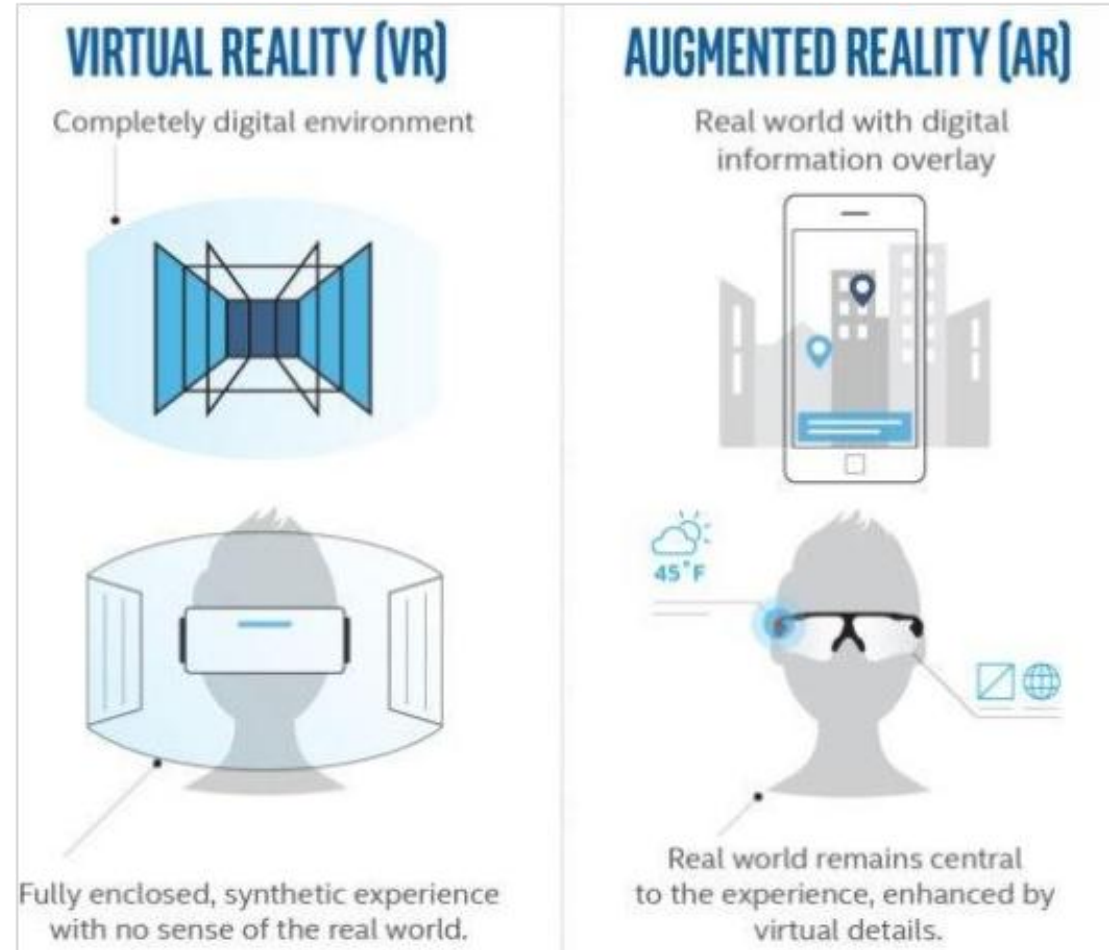
Key Differences

Feature	AR	VR
Goal	Enhance the real world	Replace the real world
Environment	Real + Digital	Fully Virtual
Hardware	Smartphones, Tablets, Smart Glasses	VR Headsets & Controllers
Best For	Navigation, Shopping, Maintenance	Gaming, Training, Simulations

PM Takeaway

- **Choose AR** when users need real-world context with digital assistance.
- **Choose VR** when complete immersion improves learning, collaboration, or entertainment.
- Success depends on **low latency, intuitive interaction, device compatibility, and user comfort.**

Differences Between Augmented Reality & Virtual Reality



PM Decision Framework

When should you build with AR?	When should you build with VR?
1. Users interact with the physical world 2. Context/location matters 3. Need real-time information overlays Examples: • Navigation • Shopping preview • Warehouse picking • Maintenance support	1. Full immersion improves learning 2. Simulations are safer or cheaper 3. Experience is impossible physically Examples: • Pilot training • Surgery practice • Virtual tourism • Gaming

Challenges

AR	VR
Device compatibility	Expensive hardware
Tracking accuracy	Motion sickness
Battery consumption	User fatigue
Lighting/environment issues	Limited adoption
Privacy concerns	Content creation cost

What is Cloud Computing?

Cloud computing is the **on-demand delivery of computing resources** (servers, storage, databases, networking, AI services, etc.) over the internet. Instead of owning infrastructure, organizations pay only for what they use.

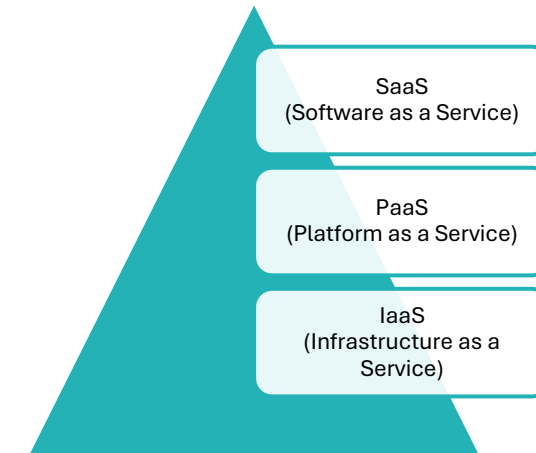
Why it matters

- Lower upfront investment (OpEx vs CapEx)
- Scale infrastructure instantly
- Faster product launches
- Global availability
- Built-in reliability & disaster recovery

PM Takeaway

The higher you go (SaaS → PaaS → IaaS), the **more control you gain**, but the **more engineering responsibility you own**.

Cloud Services Model



Service	What you get	PM Example
IaaS	Infrastructure (VMs, storage, networking)	Hosting your backend on AWS EC2
PaaS	Platform to build & deploy apps	Deploying apps on Heroku or Google App Engine
SaaS	Ready-to-use software	Slack, Salesforce, Google Workspace

Choosing Between IaaS, PaaS & SaaS

	IaaS	PaaS	SaaS
Best for	Custom infrastructure	Fast application development	Business software
Managed by Provider	Hardware	Hardware + Runtime	Everything
PM Focus	Flexibility & scalability	Faster development	User adoption & integrations
Examples	AWS EC2, Azure VM	Heroku, App Engine	Slack, Salesforce
Scenarios	Enterprise custom platform	Startup MVP Quick feature launch	Internal collaboration

Benefits & Challenges of Cloud Computing

Benefits	Challenges
Cost Efficiency - Pay only for the resources you use, reducing large upfront infrastructure investments (CapEx → OpEx).	Vendor Lock-in - Migrating applications and data between cloud providers can be expensive and technically challenging.
Scalability & Elasticity - Instantly scale resources up or down based on user demand without purchasing new hardware.	Security & Compliance - Organizations remain responsible for securing applications and meeting regulations like GDPR or HIPAA.
Faster Time-to-Market - Developers can provision infrastructure in minutes, enabling quicker development and product releases.	Cost Overruns - Poor monitoring or idle resources can lead to unexpectedly high cloud bills.
Global Availability - Deploy applications across multiple regions to reduce latency and improve user experience worldwide.	Service Outages - Cloud providers may experience outages that can temporarily impact product availability.
Reliability & Disaster Recovery - Built-in backups, redundancy, and failover improve business continuity and system reliability.	Data Residency - Some countries require user data to remain within specific geographic regions, adding deployment complexity.

Pricing Models

Pricing Model	How It Works	PM View
Pay-as-You-Go	Pay only for resources used.	Best for variable workloads; flexible but costs can fluctuate.
Subscription	Fixed monthly/annual fee.	Predictable revenue and customer spending.
Tiered Pricing	Multiple plans with increasing features or limits.	Serves different customer segments and drives upgrades.
Freemium	Basic version is free; premium features are paid.	Great for user acquisition; depends on conversion.
Reserved Pricing	Commit to long-term usage for discounts.	Lowers costs for predictable demand.

Deployment Models

Public Cloud

Shared infrastructure, owned by third-party (AWS, Azure), accessed over the internet.

-  Scalable & cost-effective
-  Less control & potential compliance issues



Hybrid Cloud

Mix of private + public cloud; data/apps move between them.

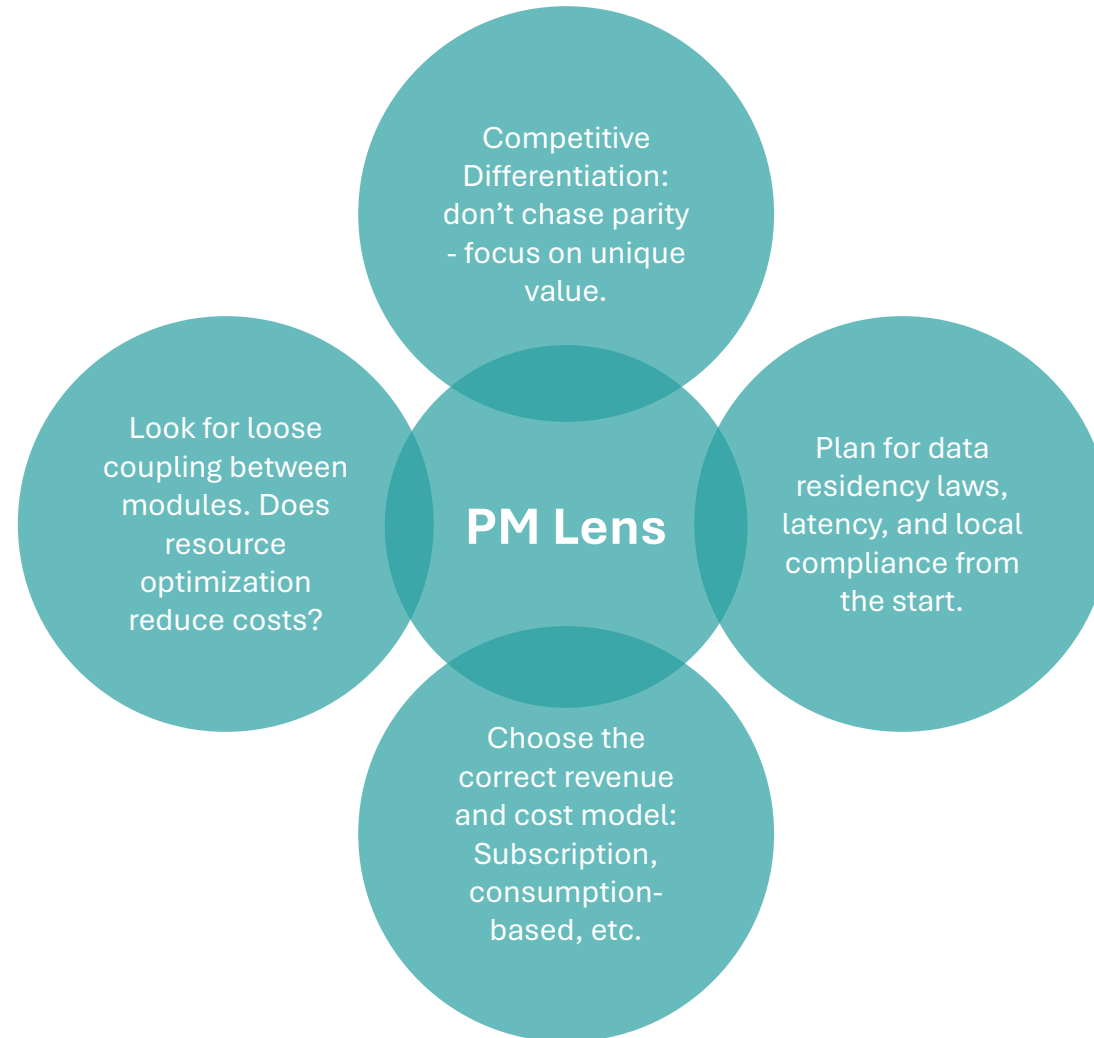
-  Security + scalability
-  More complex to manage



Private Cloud

Exclusive infrastructure for one org, on-prem or hosted.

-  High security & control
-  Expensive & less scalable



Introduction:

A blockchain is a distributed database or ledger shared across a computer network's nodes. They maintain a secure and decentralized record of transactions but are not limited to cryptocurrency uses. In an industry, blockchains can be used to make data industry immutable, i.e., it cannot be altered. It is a form of a mathematical data structure and is programmed to be immune to manipulation.

Properties:

1. **Open** → *Distributed ledger technology* - All network participants can access the distributed ledger. It records transactions only once, eliminating the duplication effort typical of traditional business networks.
2. **Immutable** → *Immutable records* - No participant can tamper with a transaction after it's been recorded in the shared ledger. If a transaction record includes an error, a new transaction must be added to reverse the error, and both transactions are then visible.
3. **Secure** → *Public key cryptography* - It secures transactions and data on the blockchain by leveraging two cryptographic keys: a public key and a private key. The private key holder can authorize transactions, provide security and verify ownership, while the public key allows others to send data to the correct address.

Working:

Each transaction is stored as a “block” containing specific details along with a timestamp to preserve chronological order.



Each block is connected to the previous and subsequent blocks via cryptographic hashes, making alterations virtually impossible without changing all the subsequent blocks.

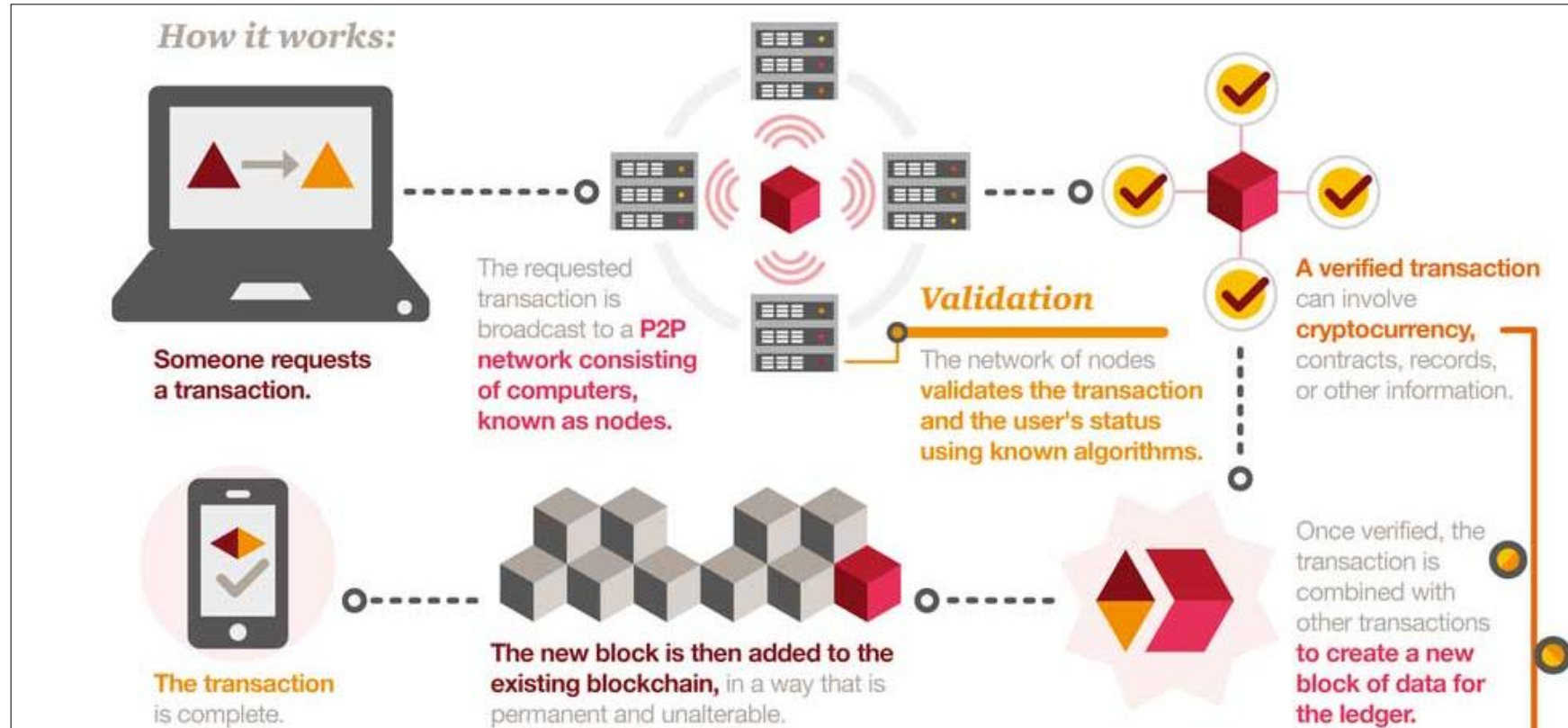


All validated blocks are added to the chain permanently using consensus algorithms like Proof of Work (PoW) or Proof of Stake (PoS), creating a tamper-proof ledger.



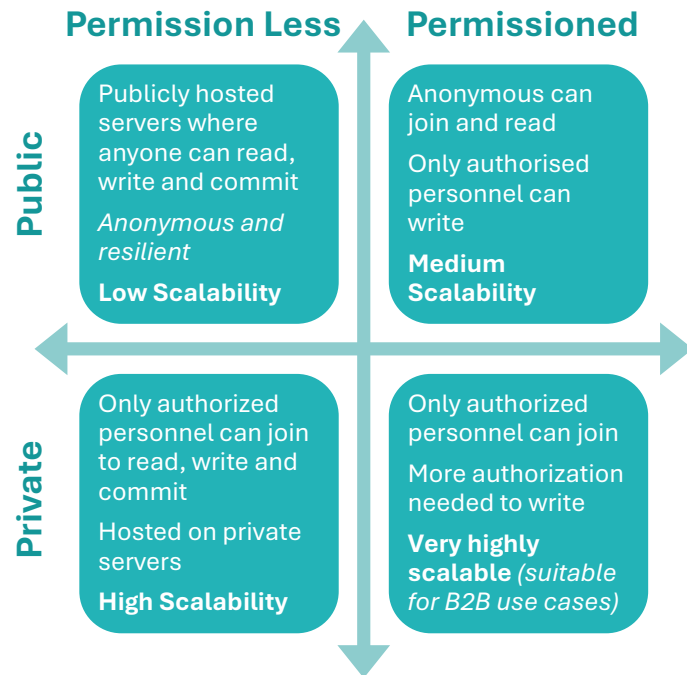
The chain's immutability builds a trusted, transparent record accessible to all network participants, preventing fraud and ensuring accuracy.

Working of Blockchain Technology



Types

1. **Public Blockchain** → A fully open network where anyone can join, view, and validate transactions (e.g., Bitcoin, Ethereum). It offers maximum decentralization but often has slower transaction speeds, higher energy use, and limited privacy.
2. **Private Blockchain** → A decentralized peer-to-peer network controlled by a single organization. The governing entity decides who can join, run the consensus process, and maintain the ledger.



Further types based on permission

1. **Public Permission-less** → Anyone can join the network, read data, and perform transactions without authorization. It operates on public servers and ensures anonymity and decentralization, making it resilient and censorship-resistant. *E.g., Bitcoin, Litecoin and Polkadot.*
2. **Public Permissioned** → Anyone can access and read the blockchain data, but only selected, authorized participants can write or validate transactions. It balances openness with control, offering medium scalability. *E.g., Ripple (XRP Ledger)*
3. **Private Permission-less** → Only authorized individuals can join the network, and all participants can read, write, and commit data freely within the network. Hosted on private servers, it ensures tighter access control and better performance than public systems. *E.g., MultiChain (configured in open-write mode)*
4. **Private Permissioned** → Most restrictive and enterprise-friendly model where only pre-approved participants can join, and even among them, writing rights are further limited to select users. Hosted privately and tightly governed, it ensures maximum control, security, and compliance. *E.g., Hyperledger Fabric and R3 Corda*

Business Value:

1. **Lower risk & compliance costs** → Shared, blockchain-based KYC lets banks verify a customer once and reuse the verification securely across institutions, cutting costs and improving onboarding speed.
For e.g., In Oct 2017, OCBC, HSBC, and MUFG, with Singapore's IMDA, completed Southeast Asia's first blockchain-based KYC POC. KPMG estimated 25-50% cost savings by removing repeated checks and adding a clear audit trail.
2. **Faster & cheaper transactions** → Digitizing trade documents on a blockchain eliminates paper handling and manual checks, speeding up cross-border deals.
For e.g., in May 2018, HSBC and ING settled a soybean shipment from Argentina to Malaysia in 24 hours (vs. 5-10 days), the first live commercial trade finance transaction using blockchain.
3. **Automated, secure contract fulfilment** → Smart contracts execute automatically when conditions are met, reducing human errors and delays.
For e.g., in 2018, Walmart and IBM's Food Trust traced mango shipments in 2.2 seconds (vs. 7 days), improving food safety, recall speed, and transparency.

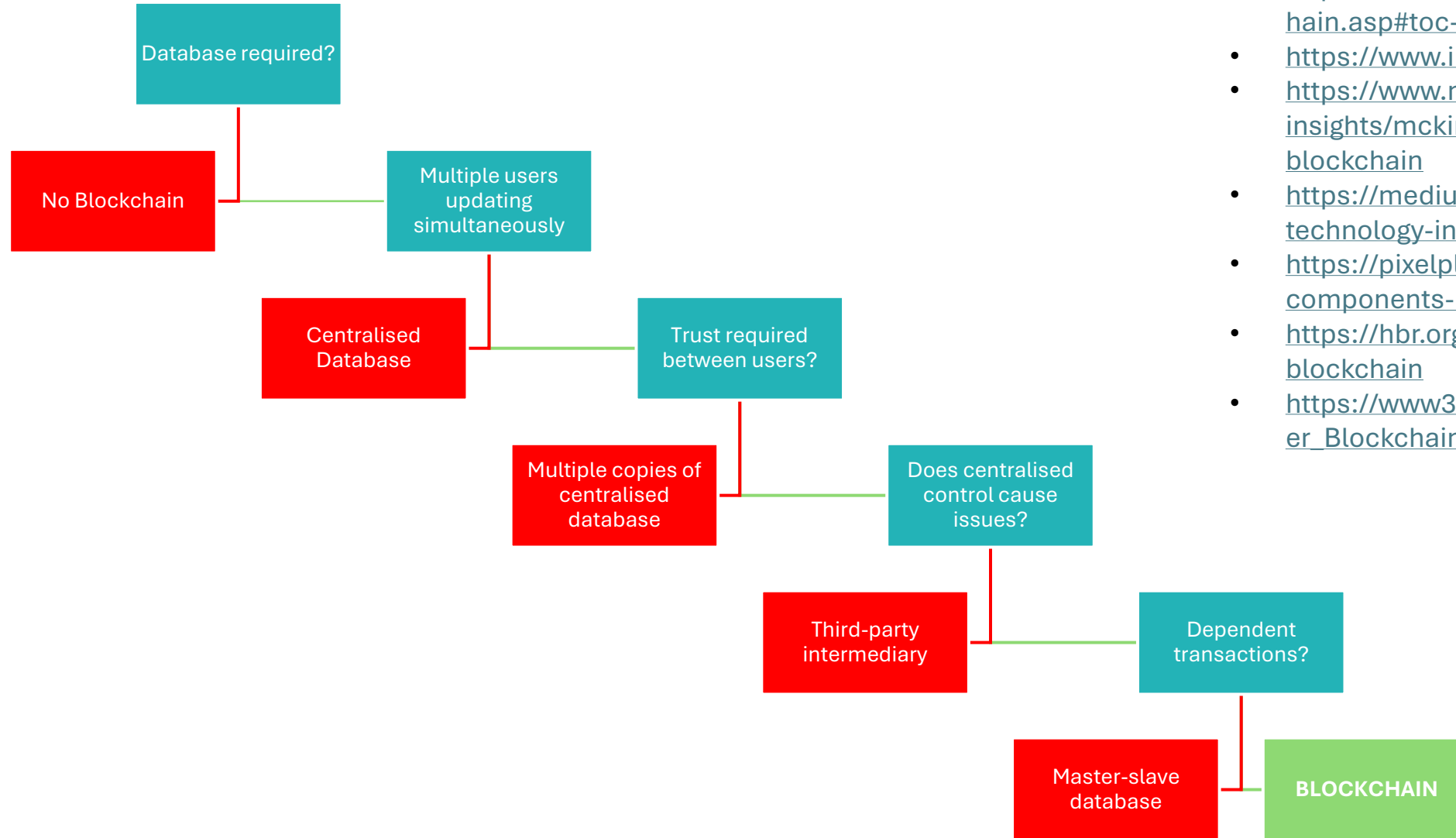
Shortcomings:

1. **High Energy consumption** → Computational power needed for mining results in enormous energy consumption.
2. **User adoption due to complexity** → Wallets, private keys, and smart contracts are complicated for the average user to navigate. Poor UX and technical jargon limit mainstream adoption, especially outside tech-savvy communities.
3. **Regulatory & legal uncertainty** → Blockchain faces inconsistent global regulations that hinder adoption.

Applications:

1. **Cryptocurrency** - Blockchain enables secure, peer-to-peer digital money transfers without banks or intermediaries.
2. **Supply Chain Tracking** - It records every step of a product's journey from source to customer in a tamper-proof ledger.
3. **Smart Contracts** - Can run coded agreements that automatically execute when conditions are met. This removes the need for middlemen and speeds up transactions.
4. **Digital Identity** - It offers individuals and organizations secure, verifiable digital IDs.
5. **Voting Systems** - Blockchain creates transparent, auditable voting records that are hard to alter.

How PMs decide when to use Blockchain



References:

- <https://www.investopedia.com/terms/b/blockchain.asp#toc-benefits-of-blockchains>
- <https://www.ibm.com/think/topics/blockchain>
- <https://www.mckinsey.com/featured-insights/mckinsey-explainers/what-is-blockchain>
- <https://medium.com/@lvooO/blockchain-technology-in-the-food-industry-ddf2d34ccd6>
- <https://pixelplex.io/blog/blockchain-explained-components-platforms-use-cases/>
- <https://hbr.org/2017/01/the-truth-about-blockchain>
- https://www3.weforum.org/docs/48423_Whether_Blockchain_WP.pdf

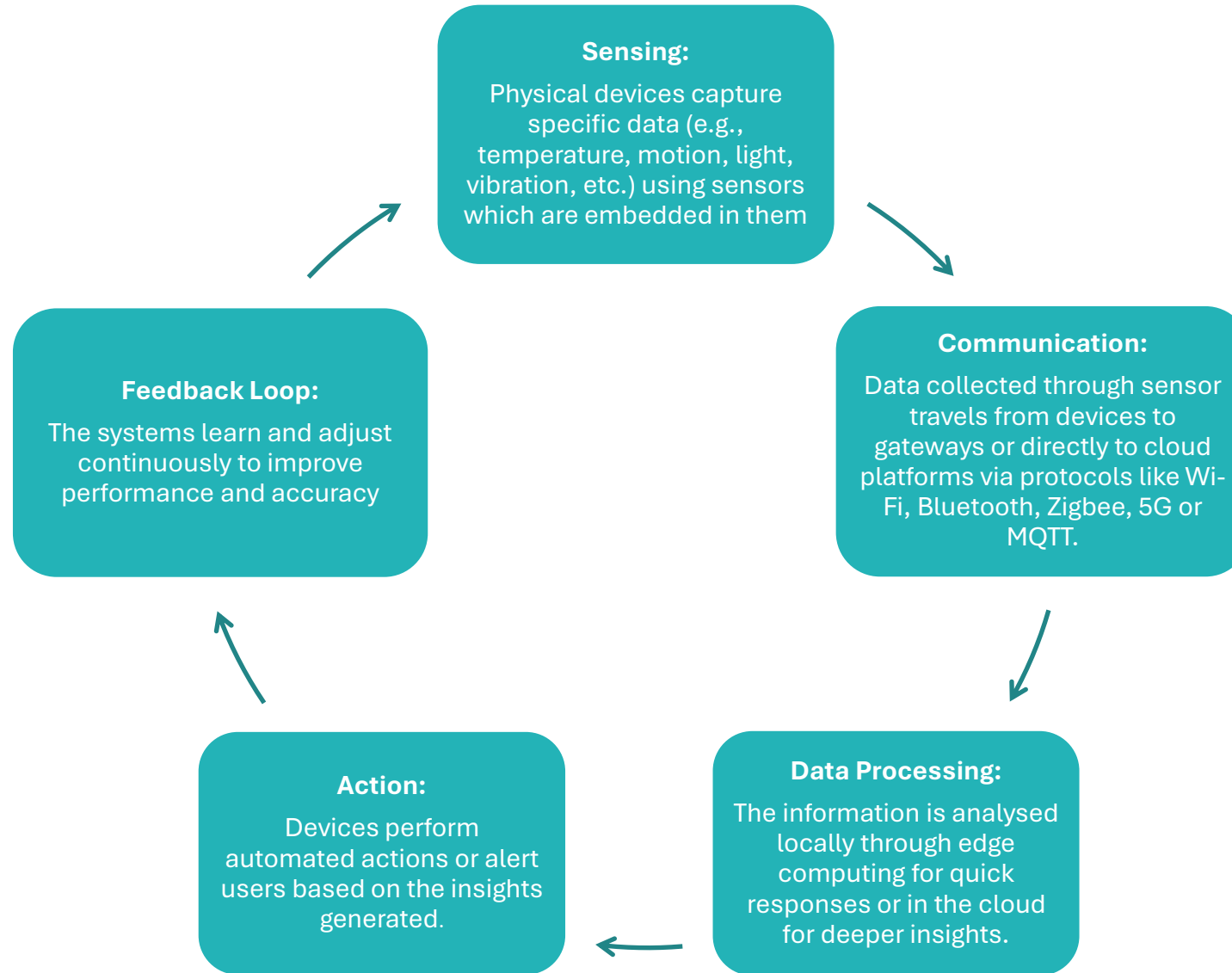
Introduction

1. The Internet of Things (IoT) is a network of physical devices embedded with sensors, software, and connectivity, enabling them to collect, exchange, and act on data. These “smart” devices range from consumer gadgets like fitness trackers to large-scale systems like smart cities, which result in improved efficiency, convenience, and decision-making.
2. IoT will enable PMs to offer personalized experiences through real-time monitoring. It can further be coupled with AI to gather actionable insights and perform predictive analytics, and additional sources of revenue can be unlocked, thus providing a competitive edge.

Properties

1. **Connectivity** → IoT devices use wired or wireless connections to exchange data with other devices or platforms. This typically includes Wi-Fi for home automation, Bluetooth for wearables, Zigbee for low-power smart devices and 5G for high-speed industrial IoT.
2. **Automation** → Devices can execute actions automatically when triggered by certain conditions, reducing manual intervention.
3. **Scalability** → IoT platforms are designed to handle thousands to millions of connected devices simultaneously, with the ability to add more without degrading performance.
4. **Interoperability** → IoT ensures devices from different manufacturers can communicate and work together using standard protocols or APIs.
5. **Real-Time Insights** → IoT allows for continuous data capture and analytics for quick decisions and immediate action.

Working



Types

Type	Consumer IoT	Commercial IoT	Industrial IoT	Infrastructure IoT
Focus	- Convenience - Lifestyle enhancement	- High security standards (sensitive data) - Customer experience	- Predictive maintenance - Safety	- Sustainability - Safety
Sector	- Home - Fitness - Entertainment	- Healthcare - Retail - Hospitality	- Manufacturing - Logistics - Energy - Infrastructure	- Smart cities - Utilities - Transportation
Applications	- Smart homes (voice assistants, smart lighting, connected appliances) - Connected health devices (smart glucose monitors)	- Smart retail (personalized in-store offers) - Asset tracking in warehouses	- Predictive maintenance for machinery - Remote equipment monitoring in manufacturing plants	- Smart grids for energy distribution - Smart water management (leak detection, quality monitoring)

Business Value of IoT

Operational Efficiency

IoT streamlines processes by automating tasks, reducing delays, and optimizing resource allocation. This is especially valuable in industries where downtime is expensive.

For example, Amazon warehouses use IoT-enabled robots to locate and transport items, speed up order fulfilment, and reduce labor costs.

Data-Driven Decision Making

Real-time data collection enables organizations to respond quickly to changing conditions and improve decision quality.

For example, Starbucks uses IoT-connected coffee machines to monitor drink quality and adjust recipes remotely based on performance analytics.

Enhanced Customer Experience

IoT enables personalization, convenience, and faster service, increasing customer loyalty and satisfaction.

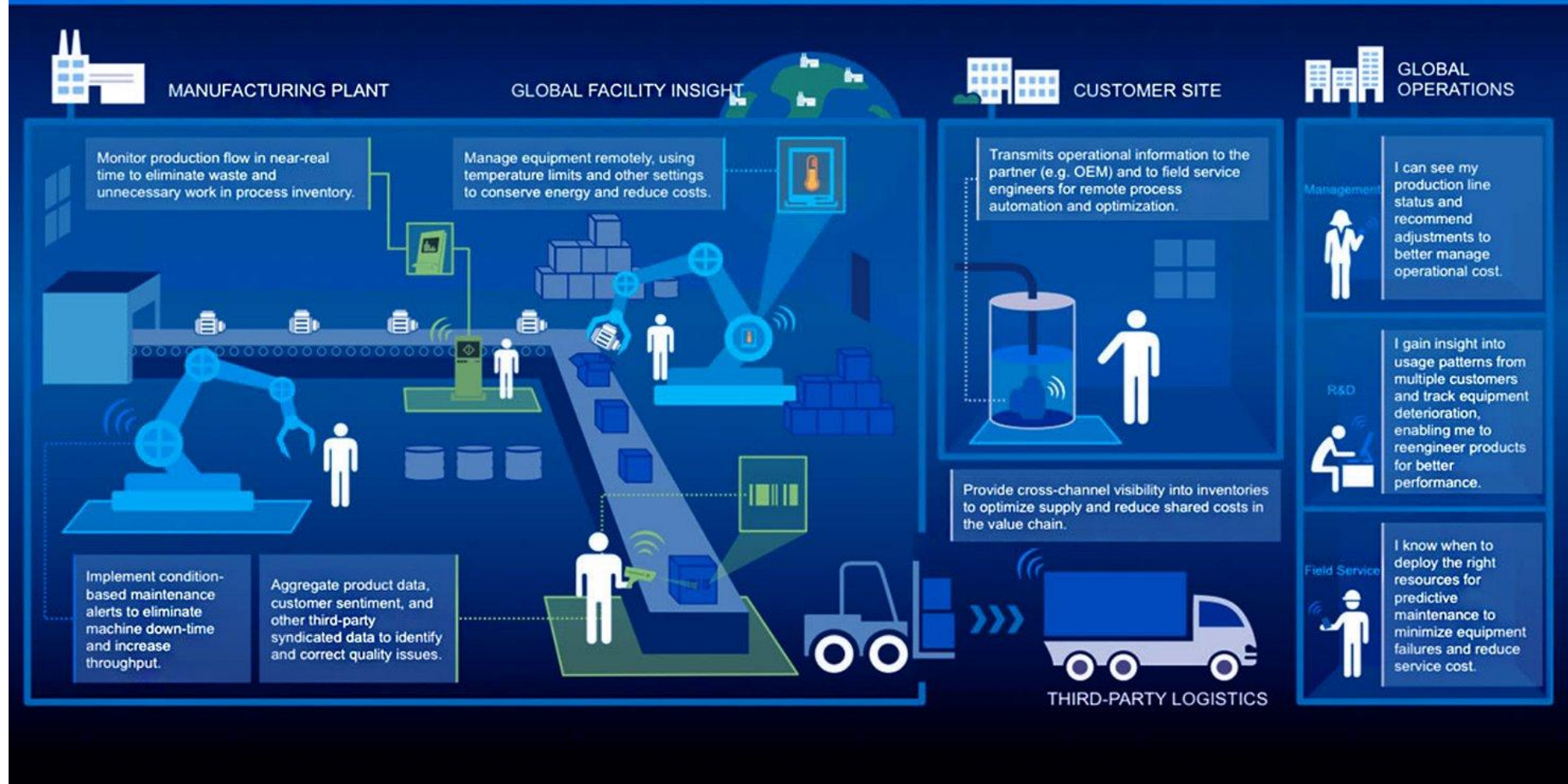
For example, Disney's MagicBand allows park visitors to seamlessly unlock hotel rooms, enter rides, and make cashless payments.

New Revenue Models

IoT provides subscription-based services, pay-per-use models, and data monetization opportunities.

For example, John Deere offers precision farming subscriptions that use IoT data to optimize planting and harvesting schedules.

Internet of Things in Manufacturing



Economic Features of Digital Platforms - Differentiability

Sustainable differentiation lives in the **customer's perception** (not just internal process improvements). For example, if Uber creates a new algorithm that increases cab availability for customers, but the customers do not perceive any significant difference between Ola and Uber, then Uber has not differentiated itself. There are two kinds of differentiation -

- **Horizontal differentiation:** Different tastes/identities (e.g., Reddit communities, niche job boards, gamers vs professionals).
- **Vertical differentiation:** Objectively “better” experience (lower search cost, higher trust/safety, better matches, faster delivery).

Platforms typically win on vertical (quality/governance/trust) and cement their position with horizontal community/identity over time.

Two levers dominate in differentiation -

- **Cost of convergence:** Copying the core product is not difficult in digital platforms. Hence, businesses need to find ways to have a competitive edge over competitors.
- **Switching costs:** For platforms, the aim is to raise the *real* cost of leaving (time, money, convenience, lost benefits) without resorting to dark patterns. Practical levers include:
 - Reputation and history (ratings, reviews) that users don't want to abandon
 - Deep product/workflow integrations (APIs, catalogues, payments, logistics) that would be painful to rebuild elsewhere
 - Loyalty and financial hooks (subscriptions, fast payouts/financing) that accrue over time
 - Social and community graphs (followers, groups, messaging) that users value
 - Personalisation and saved data (preferences, lists, models) that make your experience meaningfully better

Takeaway: Late entrants win by differentiating rather than fighting a pure scale war.

Economic Features of Digital Platforms - Network Effects

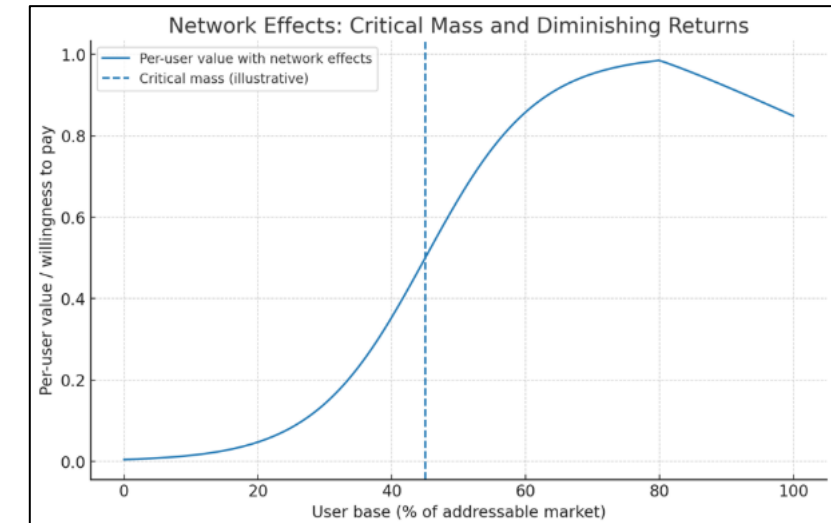
A network effect exists when a user's value increases as more users join. There are various ways in which Network Effects can kick in.

Side of network effects:

- **Same-side:** Value rises as more peers join. For example, Instagram is more useful if your friends are on the platform.
- **Cross-side:** Value rises across sides. More Airbnb hosts incentivise more guests to book on the platform.

Scale of network effects:

- **Local:** Effects confined to geographies/segments. Generally, these are more competitive given the increased investments required to scale the platform across geographies. For example, Uber competes with local rivals in almost every geography it plays in.
- **Global:** Inventory or interactions travel globally. For example, the more hosts there are on Airbnb, the more attractive the platform becomes worldwide.



Network Effects S-curve

Pricing and Competition in Multi-Sided Markets

Two-Sided Price Structure

Platforms set asymmetric prices to internalise cross-side externalities. Typically, they **subsidise the more price-sensitive or network-generating side** (e.g., free usage for consumers) and **monetise the other side** (e.g., advertisers, merchants). The split evolves over the lifecycle-heavy subsidies at launch to cross the cold start, then gradually tapering.

Competition, Multihoming, and Bottlenecks

- **Multihoming:** If users/suppliers easily switch between multiple platforms, rivalry remains active, and mark-ups stay in check.
- **Competitive bottlenecks:** When one side is locked in (exclusivity, high switching costs), the platform with privileged access can capture outsized value. Example here is Apple App Store. Given the heavy lock in of users on the platform, Apple can charge 30% commissions to developers, which is difficult to imagine if there were multiple App stores.
- **Expectations & tipping:** User expectations about future participation can stall launches or accelerate dominance once momentum builds.

Strategic Issues in Platform Management

- **Launching problem:** Overcome cold start via subsidies, exclusive/supply seeding, or a single-player utility that's useful before a network forms. Expect cash burn until critical mass. overcome cold start via subsidies, exclusive/supply seeding, or single-player utility that's useful before a network forms. Expect cash burn until critical mass.
- **Marketplace vs Reseller:** Marketplaces scale breadth and take rates with low capital intensity; reseller/hybrid models offer tighter quality control and margin on selected lines.
- **Platform leakage:** Prevent off-platform transactions by making on-platform strictly safer/better (payments, escrow/insurance, dispute resolution, guarantees).

Demand - Supply, Surplus, and Value Capture

In competitive markets, consumer and producer surplus are split at equilibrium. Platforms frequently deploy **price discrimination** (auctions, personalized ads, tiered fees), enabling them to capture more surplus. Strong network effects also steepen effective demand (higher willingness to pay), increasing total surplus-of which a well-governed, powerful platform can claim a large share.

Conclusion

Digital platforms combine powerful **economies of scale**, **network effects**, and **differentiation** to create self-reinforcing advantages. These forces explain both rapid growth and a tendency towards concentration. Strategy thus revolves around crossing critical mass, choosing sound price structures and governance, and sustaining defensible differentiation-while policy safeguards competition and user welfare without dulling innovation.

LLMs

- ❖ LLM refers to a special type of AI model called a Large Language Model. LLMs, as the name suggests, are trained with extensive text data.
- ❖ With an input of text, they can respond with probabilistically possible next set of words or text based on the statistical patterns it had learnt about the natural language of humans from the training data.

How LLMs *Really* Work: A Probabilistic Approach

- ❖ At its core, a Large Language Model is a sophisticated **probabilistic engine**.
- ❖ When you give an LLM a prompt like *"The first person on the moon was..."*, it doesn't "know" the answer. Instead, it calculates the statistical probability of every word in its vocabulary that could come next.
 - ❖ "Neil" might have a 75% probability, "Buzz" a 15%, and "Mickey" a 0.001%.
- ❖ It chooses a word based on these probabilities and then repeats the process, predicting the next word after "Neil," and so on, to generate a complete response.

Under the Hood: Encoding, Transformers, and Decoding

- To predict the next word, a model needs to understand the input you provide. This happens through a three-step process powered by architecture called the **Transformer**.
- **Encoding: Turning Words into Meaningful Numbers-** Models don't read words, they process numbers. The first step, **encoding**, converts your prompt into a series of numbers called "tokens." Each token represents a word or a piece of a word. These numerical representations (vectors) capture the word's semantic meaning and relationship to other words.
- **The Transformer: Understanding Context-** This is the engine room. The **Transformer architecture**, introduced by Google in 2017, was a revolution because of its "attention mechanism." This allows the model to weigh the importance of different words in the input sequence. When processing the sentence "The robot picked up the heavy box because **it** was strong," the attention mechanism helps the model understand that "**it**" refers to the "robot," not the "box."
- **Decoding: Turning Numbers Back into Words-** After the Transformer processes the encoded input, the **decoding** phase begins. The model generates a new token, a numerical representation of the most probable next word. This token is then converted back into human-readable text. This process repeats, with each newly generated word being fed back into the model to help predict the next one, until the full response is complete.
- This entire architecture is what the "GPT" in models like GPT-4 stands for: **G**enerative **P**re-trained **T**ransformer. It's **G**enerative because it creates new text, is **p**re-trained on a massive dataset, and is built on the **T**ransformer architecture.

Controlling the Output: Key Parameters

- You can influence an LLM's probabilistic choices using specific parameters. While the prompt is your primary tool, the standard technical parameters controlling the model's creativity are Temperature, Top-P and Top-K.
- **Temperature (Creativity vs. Coherence):** Temperature controls the randomness of the output. A **low temperature** (e.g., 0.2) makes the model more deterministic. It will try to choose the word with the highest probability. This is ideal for factual tasks like summarisation or data extraction where you need predictable, coherent answers. On the other hand, a **high temperature** (e.g., 0.9) increases randomness. The model might pick a less likely word, leading to more creative, diverse, or unexpected outputs. This is useful for brainstorming, creative writing, or generating multiple different options.
- **Top-K Sampling:** This parameter instructs the model to create a shortlist of the 'K' most likely next words and then choose only from that list. A **low K** (e.g., K=1) makes the model very safe and deterministic.
- **Top-P (Nucleus Sampling):** Top-P is another method to control randomness. It tells the model to consider only the most probable words that comprise a certain cumulative percentage (the "nucleus").
- For most applications, you'll use a low temperature for predictable outputs. For creative tasks, you might increase it.

The AI Model Landscape: LLMs, SLMs, and Reasoning Models

The term "LLM" is often used as a catch-all, but the landscape is more nuanced. The following table captures the different models that are in use today -

Model Type	Key Characteristics	PM Use Case
LLM	Large, general-purpose text models (GPT-4, Gemini). Versatile but costly	Chatbots, content generation, research assistance
SLMs (Small Language)	Small, efficient, task-specific models	Ticket classification, quick text summarisation
Reasoning Model	Optimized for multi-step problem-solving	Strategic analysis, complex decision support
Multimodal Model	Handle text, images, audio, and video together	Generate UI from sketches, read & summarize reports.
Image Gen Model	Create images from text prompts (DALL·E, Midjourney)	Marketing creatives, product mock-ups
Speech Model	Convert speech ↔ text, add voice to apps	Voice assistants, meeting transcription
Domain Models	Trained for one field (finance, law, medicine)	Contract review, medical advice, market risk
AI Agents	Use multiple models/tools to act autonomously	Automated research, workflow execution

Prompt Engineering

Large language models use text prompts (sometimes with image prompts) to predict the output. **Prompt Engineering** is the practice of strategically designing your input to guide a model toward a specific, high-quality, and relevant output. If an LLM is a brilliant, improvisational actor, the prompt is your script and direction. Your word choice, style and tone, structure, context, training data, model configurations, and the model itself all have a role in how effective your prompt is. Some basic and popular prompt engineering methods are -

Technique	Explanation	Use Case	Example
Zero / One / Few-Shot	Give the model zero, one, or a few examples so it learns the pattern instantly. Zero-shot = just instructions, few-shot = multiple examples.	Classify user feedback into Bug , Feature Request , or UI/UX Issue	"Here are 3 labelled reviews. Now, label these 20 new ones the same way."
Role Prompting	Tell the model to act in a specific role so tone and style match that persona.	Create different outputs for engineers vs. leadership	"Act as a Product Manager. Summarise this idea for the CEO."
Chain-of-Thought (CoT)	Ask the model to think step-by-step before answering, ensuring logical reasoning.	Break down a market entry decision logically	"Should we launch in Southeast Asia? Think step-by-step: market size → competition → costs."
ReAct (Reason + Act)	Alternate between reasoning and using tools (e.g., web search) to get fresh info before final answer.	Gather competitor data and prepare analysis	"Search for current iPhone prices in India, compare with Samsung, then suggest positioning."

Other vital guidelines to craft an effective prompt include

- **Start Simple** - Give a short, explicit instruction first. Add details only if needed.
- **Be Clear** - Say exactly what format, length, and tone you want.
- **Use Positive Language** - Tell the model what to do, not what to avoid.
- **Show Examples** - Give correct, well-formatted samples to copy.
- **Test & Save** - Try your prompt, note what works, and keep a record.

Reference

<https://www.kaggle.com/whitepaper-prompt-engineering>

Introduction

As a product manager, understanding the impact of your marketing efforts on a feature improvement or new product development is crucial. A **brand lift study** is a powerful tool to measure how well your campaign has changed people's **awareness** and **perception** of your product. Awareness improves conversion at the top end of the customer funnel, while perception helps with the bottom end.

A brand lift study helps you measure the effectiveness of your marketing campaigns in improving how people **perceive** and are **aware** of your new feature or product. It's like taking a before-and-after snapshot to see how your campaign has influenced people's opinions and behavior.



Brand Lift Studies probe brand parameters amongst control and exposed audience sets, to measure the impact (delta) of any feature improvement. It is similar to the **RCT (Randomized Controlled Trials) method**.

What questions can you see to get answers to?

Awareness

Are more people aware of the new feature or product after the campaign?

Perception

Do people have a better opinion of the new feature or product?

Intent

Are more people interested in using or buying the new feature or product?

Engagement

Are people more engaged with the new feature or product (e.g., increased usage, positive feedback)?

Benefits of a Brand Lift Study

Understand Impact

Observe how well your marketing efforts for the new feature or product are working.

Make Data Driven Decisions

Use the insights to improve future campaigns and product strategies.

Optimize Budget

Allocate resources more effectively by knowing what works best.

Prove ROI

Demonstrate the return on investment (ROI) of your marketing spend to stakeholders.

How does a Brand Lift Study work?

1

Setting Up the Study

Determine what you want to measure, such as awareness, favorability, or usage intent for the new feature or product. Develop questions to ask people about their awareness and perceptions of the feature or product before and after the campaign.

2

Collecting Data

Split your audience into two groups. One group sees the marketing campaign (exposed group), and the other doesn't (control group). Ask both groups about their awareness and perception of the feature or product before the campaign starts.

3

Running the Campaign

Promote the new feature or product as planned.

4

Post-Campaign Survey

After the campaign ends, ask both groups the same questions again to see if there are any changes in their awareness and perception.

5

Analyzing Results

Look at the differences in responses between the control and exposed groups before and after the campaign. Calculate the "lift," which is the improvement in awareness, perception, or intent caused by the campaign. For example, if 20% of people in the control group knew about the feature before the campaign and 40% in the exposed group knew about it after, the brand lift in awareness would be 20%.

Real-world problem statements



YouTube

*We're rolling out YouTube Shorts globally. Are our campaigns making people aware that Shorts exists as a competitor to TikTok, and do they see it as fun and engaging enough to use? They would track **awareness & perception** here.*



WhatsApp

*We introduced disappearing messages and new encryption features. Do users feel WhatsApp is safer after our awareness campaign, and are they more likely to trust us with private conversations? This would focus on **perception lift (safety, trust)** and **intent** (are people using WhatsApp more for sensitive chats?), especially after Facebook's history with privacy concerns.*



AirBnB

We introduced 'I'm Flexible' search to help users explore new destinations and dates. Did our ads and in-app nudges make travelers aware of this tool, and has it changed their perception of Airbnb from being just a booking site to a discovery platform? This tracks engagement and awareness.

What is RAG?

Retrieval Augmented Generation (RAG) is a feature in modern LLMs that allows them to fetch updated information on questions asked by the users. It allows LLM models to access external knowledge to update its output, rather than being dependent only on the knowledge present in its training data. This has made models more useful for general QnA, as LLMs before RAG (like the initial versions of ChatGPT) were restricted in their knowledge base to the timestamp of its training data, which was till 2021.

What are these external sources?

External sources of data may be results generated by search engines or 3rd party knowledge graphs. They also may be updated company documents for internal LLMs used for reporting or summarization purposes.

Applications

Search Queries: For search queries on LLMs like ChatGPT related to very recent instances, RAG may be used to search for updated web pages covering the instance and using their knowledge to improve the LLM output

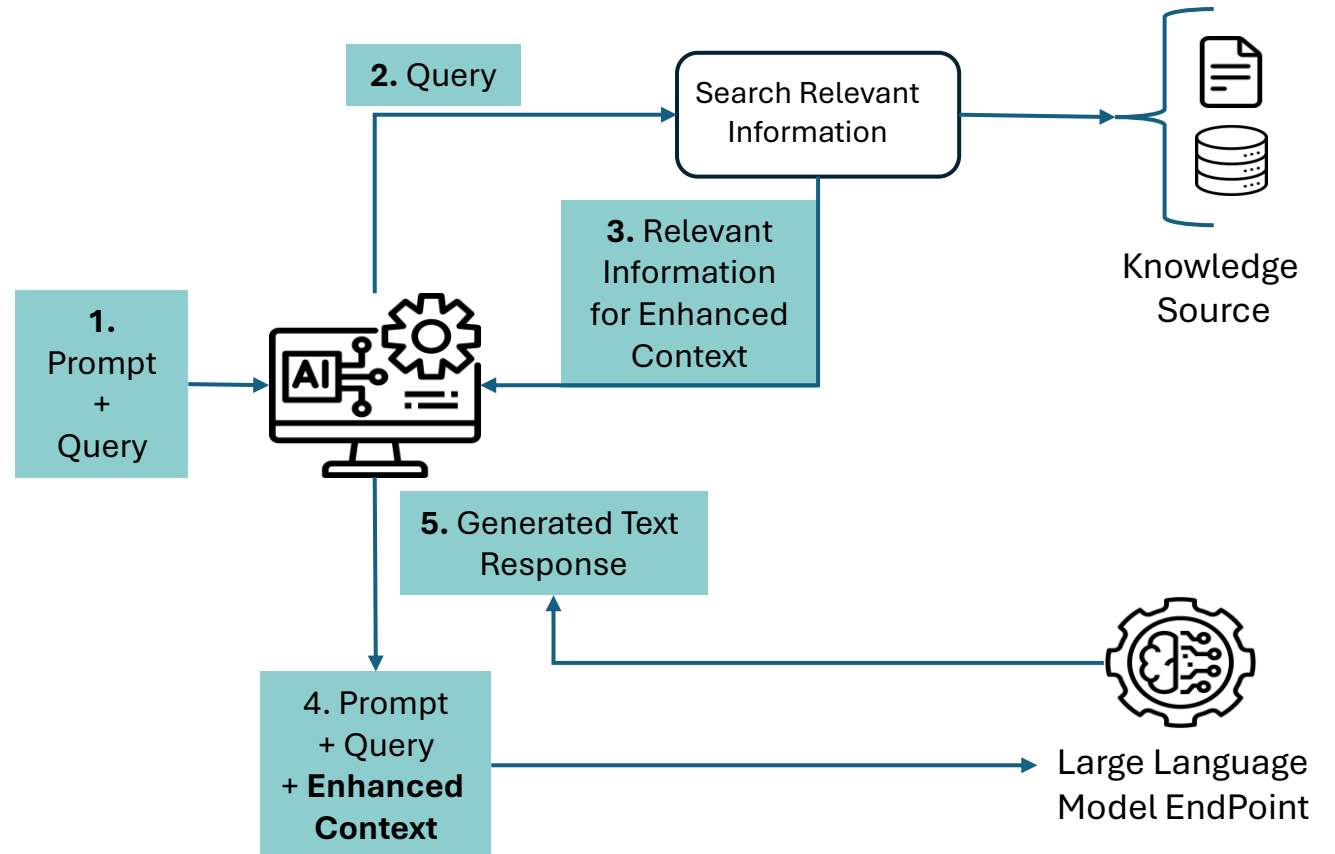
Business Intelligence: For LLMs being used to summarize daily competitor analysis reports and daily updates of internal company performance, RAG can be used to access the updated reports and generate summaries in an instance

Personalized Recommendations: In recommendation systems looking to generate product lists for users, RAG can be used to access user’s search and purchase history, their comments on other products etc. to improve the recommendations and provide personalized results

How does RAG improve an LLM System?

Base LLMs, disconnected from external sources of information, can suffer from

- **Hallucination:** generating false information with confidence
- **Generic Responses:** LLMs provide generic answers to specific questions due to lack of knowledge
- RAG helps update LLM knowledge without re-training the whole model, which provides a cost-effective alternative and faster way of using LLMs for a variety of purposes.



What is MCP?

- ❖ Model Context Protocol (MCP) is an open standard developed by Anthropic that standardizes how applications provide information to LLM models.
- ❖ This enables the building of agents and workflows on top of LLMs without worrying about data transfer formats and focusing only on functionality.
- ❖ MCP is not just restricted to data retrieval but can also expose endpoints for automating functionality in external software through LLM-assisted agentic interventions.

What happened before MCP?

- ❖ Before MCP, all actions in an external service required a unique API and customized logic for implementation.
- ❖ Through MCP integration, AI agents can now send read/write/modify requests to external sources without needing to know the specifics of their implementation.

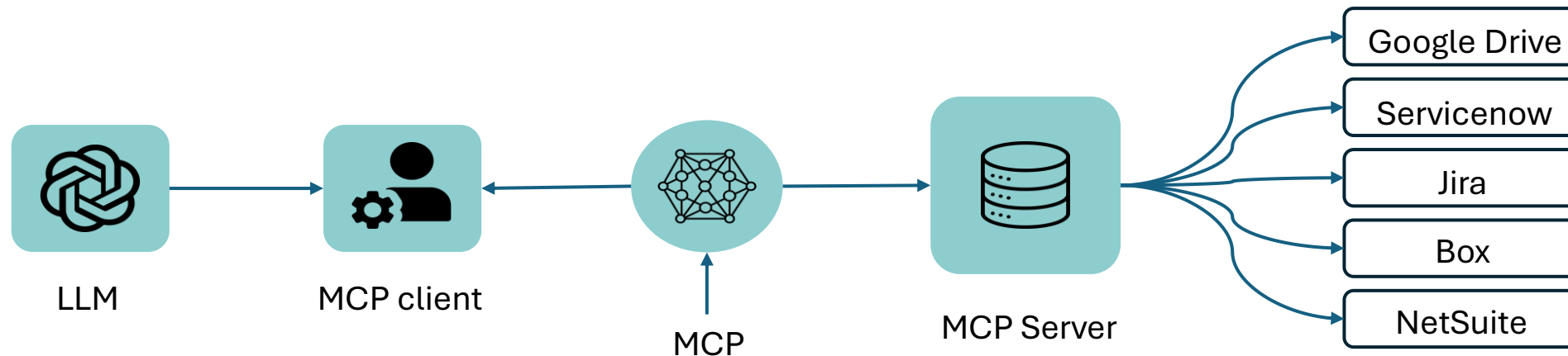
MCP Architecture

An MCP architecture includes

MCP Client: usually residing inside an AI assistant (-like Claude, ChatGPT), the MCP client is used by the AI to communicate with external tools and data sources. When The AI wants to use a tool, it goes through the MCP client to talk to the matching server

MCP Server: an MCP server is the translating entity in the architecture. It takes a text-based request from the MCP client (e.g.: modify the latest excel file with today's data) and translates it into commands that the external tool/data source understands.

MCP Protocol: it is the communication protocol that defines how the client and server communicate. It defines how the messages look like, how actions are described and how results are shown.



RAG vs MCP		
Feature	RAG (Retrieval-Augmented Generation)	MCP (Model Context Protocol)
Purpose	To enhance the knowledge and accuracy of an LLM by providing external data.	To extend the capabilities of an LLM to perform actions in other systems.
Function	Retrieves information from documents, databases, or APIs to inform a response.	Connects to tools and APIs to execute a task or workflow.
Use Case Type	Best for enterprise AI search, chatbots that answer questions, or summarizing documents.	Best for "agentic AI" use cases where the AI needs to take a specific action.
How it works	Retrieves relevant documents, augments the LLM prompt with that data, and then generates an output.	Uses a client-server protocol to access and use "tools" that expose data or functionality from other applications.

In the world of AI, RAG and MCP are not competitors but are often seen as complementary technologies that can work together to create more sophisticated AI systems.⁵ For example, an MCP-powered system could use RAG as a tool to first retrieve information and then use other MCP tools to act on that information.

Agentic AI refers to autonomous AI systems that can act towards predetermined goals without human intervention. They differ from existing ML models in that they are adaptive to different use cases and can exhibit autonomy in actions.

How is Agentic AI different from Gen AI

GenAI refers to recent AI capabilities for generating content. Given a certain input by a human user, GenAI uses LLM models to generate output. It requires human intervention for manual input and cannot function independently. However, an AI agent can act independently and orchestrate with other GenAI platforms to achieve pre-set goals.

Feature	Generative AI	Agentic AI
Core Function	Responds to specific, human-given prompts for generating an output	Autonomously pursues pre-defined goals.
Initiative	Reactive. It waits for instructions.	Proactive. It plans and initiates actions on its own
Process	One-shot execution (Input -> Output)	Continuous loop (Perceive -> Reason -> Act -> Feedback -> Learn)
Example	If a human asks Google Gemini to “generate a meal recipe using tomatoes”, it will generate a recipe in text-format using pre-determined knowledge.	However, if given an AI agent with the goal of managing your food inventory and meal planning, it will check for the available food-stuffs in your inventory and then automatically suggest you a recipe for the day, alongside creating a to-do list of refreshing your food inventory (or maybe even requesting an order on an online food app if it has permission to do so).

AI Agent Characteristics

- **Proactive:** Agents can function on complex inputs and take actions on their own to achieve pre-determined goals. They don't need to be pre-set for different conditions and can take actions **without human approval**.
- **Adaptable:** Agents learn from their experiences and take feedback on its actions and responses, much like traditional ML systems. Multiagent systems can over time increase the scope of their coverage through proper feedback loops
- **Specialized:** While there are hopes of achieving AGI, most agents perform specialized tasks. They are given a restricted set of goals to achieve and the permissions to execute tasks to move toward that goal. Multi-agent architectures can specialize in a range of tasks; however, at their core, they constitute individual agent systems that work toward a defined goal.
- **Acting:** The ability to take actions is the core identity of an agent system. This includes interacting with other systems and/or generating data to achieve a desired outcome.

AI Agents & Automated workflows

- Workflows are rule-based, pre-determined sequential actions that are triggered given a certain stimulus. They follow a restricted set of actions and cannot function outside their programmed domain. For example: receiving a notification whenever your e-mail inbox size exceeds 10,000 is an automated workflow.
- An agent combines the decision-taking capability of workflows with complex logic understanding to take actions that may not have been coded before.
- **For example**, if a workflow is triggered to send a notification when inbox size exceeds 10,000, an agent may further analyze the inbox and segregate the e-mails on basis of relevance, alongside sending you a summary of all the mails.
- Since it is impossible to pre-define workflows for every scenario, AI agents will help in such cases of **open-ended problems**.

When should AI agents be used?

Is the task open-ended and complex?:

Agents increase latency and cost whereas close-ended or simpler tasks can be cheaply automated using rule-based alternatives

Is the cost of being wrong

low?: Autonomous systems can make mistakes, especially while learning. If the cost of the model being wrong is high (such as in financial or healthcare related tasks) it is best to use LLMs in an advisory capacity rather than giving it decision making powers

Is there a clear feedback loop?: There must be a feedback mechanism to improve agent's actions. For example: using AI agents in medical diagnosis can have huge repercussions if the diagnosis is wrong. Having human intervention is best advised in these scenarios. Rather, developing workflows to summarize patient history would be a more suitable use case.

AI agents may be used in cases of documents handling.

For example: maintaining file systems. When a new file is added to a file system, the agent can determine where it should best be stored. In this case, the cost of being wrong is lower and the agent can learn from human intervention of moving the file to a more suitable location.

Increasing complexity: AI Agents are massively complex systems, and designing them involved defining proper goal alignment, otherwise there are risks of the agent going "off the rails". E.g. A vaguely defined goal like "maximize sales" could lead an agent to spam customers with discounts, damaging brand reputation.

Increased Latency time: Since agents interact with LLM systems for context understanding, they can take time in deciding actions and may not be suited for low latency situations.

Importance of Agentic AI for a PM

Modernizing application functionality

- With the growing demand for intelligent functionality in all applications, it is imperative for product managers to understand how AI Agents work and how they can be integrated into their current software. Knowledge of AI agents can help a product manager ideate
- How existing workflows in the customer journey can be automated to improve customer satisfaction and help them explore product functionalities
- Develop interfaces to allow their application to communicate with other 3rd party apps without development of customized protocols for inter-app communication
- For example, Myntra can develop an AI agent that updates a user's buying list residing in another application (like Google Keep) whenever their wishlisted product comes back in stock
- Explore new product categories: AI agents and their smart functionality opens avenues for multi-step, cross-tool tasks without constant user input (e.g.: booking applications that automate travel planning and recommendations through constant updating of user's current journey)

Improving their PM workflow

PMs must communicate with multiple stakeholders every day and interact with a variety of applications to ensure product success. Needless to say, intelligent automation of many such tasks will reduce their workload

For example, AI agents can

- Automatically update their calendars based on new invites and past user interactions with similar events
- Changes made in one document can automatically be updated across all related documents without manual intervention to keep all stakeholders updated
- Agents can provide recommendations on improving product KPIs through updated news articles, company documents etc., providing A/B test recommendations, new product functionality, integration strategies etc.

Concept Note: Low-Code & The AI-Powered Development Stack

Imagine you need to build a house. You could spend years learning to make bricks, mix mortar, and frame walls from scratch.
OR you could use a set of high-quality, pre-made LEGO blocks to build a great-looking house in a single afternoon.

This is the core idea behind Low-Code/No-Code (LCNC) platforms. They provide the building blocks for software, allowing you to create functional applications visually, drastically reducing the need for traditional, line-by-line coding. For a Product Manager, it's a strategic lever for speed.

The New Development Stack: A PM's Toolkit

The following table summarizes the commonly used tools by PMs, each suited for a different job. Choosing which tool to pick is a key skill.

Category	Analogy	Use Case	Key Tools
No-Code	Using a pre-made LEGO kit with instructions	You need to track user feedback. You use Airtable to create a form and a database. Then, you use n8n or Zapier to automatically send a Slack message to your team every time a new piece of feedback is submitted	Bubble, Airtable, n8n, Shopify
Low-Code	Using a LEGO kit, but also having custom 3D-printed pieces for special parts	Your sales team needs a custom dashboard to see which customers use a new feature. You use Retool to build a dashboard that pulls live data from the company's internal database, something a simple spreadsheet can't do	Retool, OutSystems, MS Power Apps
AI-Assisted Pro-Code	Having a master builder robot that builds what you describe	Your engineering team is building a new, complex feature. They use GitHub Copilot , an AI "pair programmer," writing routine code for them.	GitHub Copilot, Replit AI

English is the New Programming Language

The evolution of this trend is the direct use of generative AI, where your instructions in plain English become the code itself.

Conversational Coding (Vibe Coding)

Coined by AI researcher Andrej Karpathy, it describes a workflow where a PM can guide an AI to build an application through a conversation.

How it Works in Practice:

1. **You start with a simple prompt** → "I need a Python script that scrapes Twitter for mentions of our competitor."
2. **The AI (like Claude or ChatGPT) generates the code.**
3. **You test and refine with more English** → "That's good but modify it to analyze the sentiment of each tweet and save the negative ones to a Google Sheet." This iterative, conversational process allows to create powerful tools and prototypes without writing a single line of code.

AI Agent & Workflow Builders

The newest frontier is platforms that allow you to build and deploy autonomous AI agents.

- **Tools in Action** → You could use a platform like **Lovable.dev** or **Bolt.ai** to build a sophisticated customer support agent that can access a knowledge base, interact with APIs to check order statuses, and escalate issues to a human—all configured through a visual, workflow-based interface.

Analysis for using these tools

Adopting these platforms is a strategic decision with significant trade-offs. While they improve productivity, they also have some challenges, as discussed in the table below.

As Andrew Ng states,

"Engineers are 10x faster. Product managers haven't sped up at the same rate. Now they're the bottleneck."

As engineering execution accelerates, the critical constraint shifts to the speed and quality of product strategy and decision-making.

So, a new challenge for PMs is using these AI and LCNC tools to accelerate their workflows, from user research synthesis to prototyping.

Reference:

[Watch Andrew Ng discuss building faster with AI](#)

Advantages	Potential Risks & Limitations
Accelerated Time-to-Market: Drastically reduce development cycles for MVPs, internal tools, and AI-powered features	Vendor Lock-In: Migrating an application off a proprietary platform can be difficult and costly, sometimes requiring a complete rebuild
Reduced Development Costs: Lower reliance on engineering talent for many typical applications and automation	Scalability & Performance Ceilings: Pure no-code platforms may struggle with the demands of large-scale, enterprise-grade applications
Increased Agility & Iteration: PMs can build and modify prototypes and internal tools directly	"Black Box" & Debugging Issues: When AI-generated code fails, debugging can be complex if the team doesn't fully understand the underlying logic
Democratized Innovation: Business users can solve their problems, freeing up core engineering teams	Security & Governance Risks: The proliferation of "shadow IT" built by citizen developers can create security vulnerabilities

The New Currency of Trust

In today’s world, products compete not just on features or price, but on something deeper: **trust**. A decade ago, consumers often accepted trade-offs; convenience at the cost of privacy, or speed at the cost of reliability. Today, **the balance has shifted**. **Users** are more aware, **regulators** are more watchful, and **competitive** landscapes are unforgiving. Security, resilience, and governance are no longer “back-end concerns.” They are the front-line tests of whether a product deserves to exist.

Building Blocks

Security	Resilience	Governance
----------	------------	------------

The PMs Takeaway

Security, resilience, and governance may sound like technical or abstract terms, but at their heart, they are human. They answer the question: *Can I trust this product with my time, my data, my money, or even my beliefs?* For aspiring product managers, especially in interviews, it helps to see these themes not as checkboxes but as lenses. They show how to think about product decisions beyond features - **to ask not just “will it work?” but “will it be safe, will it endure, and will it do right by the user?”**

Security	Resilience	Governance
When users share data, make a payment, or sign in with their identity, they extend trust. Breaches are not just technical failures; they are broken promises. For product managers, thinking about security means asking: <i>“What are we promising our users when they interact with us, and can we keep that promise under stress?”</i> A secure product is not only about encryption or compliance; it is about designing experiences that make users feel safe without friction. The PM’s role is to champion this balance - building in safeguards while preserving usability.	If security is about keeping promises, resilience is about keeping them consistently. In a world of sudden traffic spikes, malicious attacks, and unpredictable failures, resilience becomes the silent foundation of trust. Users rarely notice when systems are resilient - but they immediately feel when they are not. Latency spikes, outages, or fraud attacks don’t just frustrate; they erode credibility. For PMs, resilience means asking: <i>“Can my product absorb shocks and still deliver its core value?”</i> It also means prioritizing fault-tolerance and recovery as much as shiny new features.	As products grow powerful, especially with AI, new questions emerge: <i>“Should we build this? Who could be harmed?”</i> Governance is about guiding products with a moral compass, ensuring decisions don’t just chase growth but respect fairness, transparency, and accountability. This is not a far-removed concern for lawyers or ethicists; it sits squarely with PMs, who decide what gets built, how it is measured, and how it is explained to the world. Good governance is not about saying “no” to innovation; it is about ensuring that innovation sustains trust over time.

As a Product Manager, you won't be expected to write the code, but you will be expected to work with the team that does. **System design** is fundamentally about creating robust, scalable, and reliable applications that can handle millions of users and massive amounts of data.

Imagine you're the product manager for **Zomato**. Your job is to ensure that when millions of people across India open the app, they can browse menus, place orders, and get their food on time, all without the system crashing.

Database - The Digital Storehouse

Think of the database as Zomato's massive storehouse of information, restaurant details, menus, prices, user profiles, ratings, and order history.

Just like warehouses store goods in an organized way, a database stores data so that it can be quickly retrieved and updated. When you search "Biryani," the database is where Zomato pulls all restaurants serving Biryani in your city.

API - The Waiter Who Takes Your Order

The API (Application Programming Interface) is like the waiter who listens to your order and takes it to the kitchen. It's the middleman between the user (you) and the backend system.

When you tap "Order Now," the API takes that request to the correct part of the system and brings the confirmation back to you. We will discuss APIs more in later sections.

Microservices - Specialised Kitchens

Instead of one giant kitchen cooking everything, Zomato has different "kitchens" for specific tasks - payments, restaurant search, order tracking, and delivery partner matching.

Each kitchen (microservice) operates independently, forming the complete service together. The payment microservice processes your UPI transaction while the delivery microservice finds a rider.

Message Queues - The Order Dispatch System

Sometimes, tasks can't be handled immediately and need to be queued. Like a restaurant's dispatch counter holds completed orders until delivery boys pick them up, message queues hold tasks until the next service is ready. Once the payment is processed, the order request sits in a queue until a delivery partner is available.

Security - The ID Check

Before giving access, Zomato checks who you are and what you can do - just like a gated community guard checking IDs.

Key Security Measures include

- **Authentication** → Verifying user identity
- **Authorization** → Checking user permissions
- **Encryption** → HTTPS/TLS for data in transit, encryption for data at rest
- **DDoS Protection** → Preventing malicious traffic overloads
- **Rate Limiting** → Preventing API abuse

Relational Databases (SQL)

- Organize data into tables
- Use SQL (Structured Query Language) for data manipulation
- Known for strong consistency, transactions, and integrity
- Examples: MySQL, PostgreSQL

ACID Properties

- **Atomicity** → Transactions are "all or nothing"; they either complete entirely or don't happen at all
- **Consistency** → Ensures that after a transaction, the database remains in a valid state, enforcing defined rules and constraints
- **Isolation** → Concurrent transactions do not interfere with each other
- **Durability** → Once a transaction is committed, the data is permanently stored and will survive system failures

Non-Relational Databases (NoSQL)

- Designed for flexibility and scalability, often sacrificing some consistency
- Schema-less → Do not enforce rigid schemas or foreign key relationships
- Types include key-value stores (e.g., Redis), document stores (e.g., MongoDB), and graph databases (e.g., Neo4j)
- Ideal for unstructured or semi-structured data and quick iteration

In- Memory Databases

Store data primarily in RAM for fast retrieval, often used for caching and session storage (e.g., Redis, Memcached).

PM Tip

Choosing the right database affects development speed, costs, and user experience.

SQL databases are great for structured data and complex relationships, while NoSQL databases excel at rapid iteration and handling diverse data types.

Scaling and Performance

When an application gains more users, a single server might become overwhelmed. To handle increased demand, systems need to scale.

- **Vertical Scaling (Scaling Up)** → Increase the resources of a single server, such as adding more RAM or upgrading the CPU. It's simple to implement but has limited potential because there's a maximum capacity for a single machine.
- **Horizontal Scaling (Scaling Out)** → Add more servers (replicas) to handle a subset of requests, distributing the load across multiple machines. This approach is more powerful because it allows almost infinite scalability and doesn't require high-end individual machines. It also inherently provides redundancy and fault tolerance, as other servers can continue to fulfil requests if one goes down, eliminating a single point of failure.

Load Balancers - Smart Traffic Managers

Returning to the Zomato analogy, imagine multiple kitchens inside a food court. A load balancer decides which kitchen should take the next order to keep things moving efficiently. If one server is overloaded, the load balancer sends the request to a less busy one.

Load Balancers are essential components (often acting as reverse proxies) that direct incoming requests to the appropriate server in a pool. Their primary goal is to even out traffic, maintain a smooth user experience, and prevent bottlenecks.

Content Delivery Networks (CDNs)

CDNs are a specific type of reverse proxy network consisting of servers distributed worldwide. They are primarily used to serve static files like images, videos, HTML, CSS, and JavaScript.

CDNs copy content from your origin server onto their globally distributed servers. When a user requests content, the CDN delivers it from the nearest CDN server, significantly reducing latency.

Networking Fundamentals

IP Address → A unique identifier assigned to each device on a network, allowing devices to locate each other. IPv4 and IPv6 are different versions of IP addresses.

TCP/IP (Internet Protocol Suite) → These rules define how data is sent over the Internet. Data is broken into packets, sent across the network, and reassembled at the destination.

- **TCP (Transmission Control Protocol)** → Ensures reliable communication. It guarantees the complete and correct delivery of data packets, managing connection setup and retransmitting missing packets. Many other protocols, like HTTP and WebSockets, are built on TCP.
- **UDP (User Datagram Protocol)** → Faster but less reliable than TCP. It doesn't establish a connection or guarantee delivery/order, making it suitable for time-sensitive communications where some data loss is acceptable (e.g., video calls).

DNS (Domain Name System) → Acts as the Internet's "phone book," translating human-friendly domain names (e.g., neetcode.io) into their corresponding IP addresses.

Ports → Within a device, specific processes or services are identified by ports (e.g., port 80 for HTTP).

Firewalls → Monitor and control network traffic to protect networks.

Other Important Terms

- **Availability** → Measures how much time a system is up and running when users need it, often expressed as a percentage (e.g., "five nines" or 99.999% availability).
- **Reliability**: Ensures a system works correctly and consistently.
- **Fault Tolerance** → How a system handles unexpected failures or attacks.
- **Redundancy** → Having backups to ensure continued operation if one part fails.
- **Throughput** → Measures how much data a system can handle over time (e.g., requests per second, queries per second, bytes per second).
- **Latency** → Measures how long it takes to handle a single request (time for a request to get a response).

API Patterns and Protocols

HTTP (Hypertext Transfer Protocol)

Built on TCP/IP, it's the standard for web browsing, following a client-server model.

- **Request/Response** → A client initiates a request (with header and body), and the server sends a response (with header and body)
- **Stateless** → Each HTTP interaction is separate, with no memory of past requests
- **Status Codes** → Provide feedback on the result of a request (e.g., 200 for success, 400 for client error, 500 for server error)
- **Methods** → Common methods include GET (fetching data), POST (creating data), PUT/PATCH (updating), and DELETE (removing)

REST (Representational State Transfer)

A popular API pattern that standardizes HTTP APIs.

- It's stateless and uses standard HTTP methods
- Often uses JSON for data exchange
- Can sometimes lead to over-fetching or under-fetching data

REST APIs are widely understood and easy to debug, making them suitable for most applications.

gRPC (Google Remote Procedure Call)

A framework released by Google, considered an improvement over REST APIs, is mainly used for server-to-server communication.

- It leverages HTTP/2 and Protocol Buffers (a binary format for serializing data)
- Very efficient in terms of bandwidth and resources
- Less human-readable than JSON

WebSockets

They provide a two-way (bi-directional) communication channel over a single, long-lived connection.

- Crucial for real-time updates, unlike HTTP's one-way request/response model
- Commonly used in chat applications, for real-time features like live chat, collaborative editing, or live updates

The Layered View of a Modern Software Product

Layer	Core question	Typical components	Tools /Tech
Experience	How do users interact?	Web & mobile UI, API clients	React/Next.js, Swift/Kotlin
Application logic	What business rules run?	Services, controllers, workflows	Spring Boot, Express/Fastify, Django
Integration	How do parts talk cleanly?	REST/GraphQL/gRPC APIs, gateways	OpenAPI/Swagger, Postman, Kong, Apigee (Swagger , OpenAPI Initiative)
Data	Where is truth stored and read fast?	SQL/NoSQL DBs, caches, search	PostgreSQL, MySQL, MongoDB, Redis, Elasticsearch
Async & real-time	How are spikes and events handled?	Queues, streams, WebSockets	Kafka, RabbitMQ, SQS; Socket.IO
Delivery & runtime	Where does code run and scale?	Containers, functions, autoscale	Docker, Kubernetes, AWS Lambda; CNCF ecosystem (CNCF , Cloud Native Now)
Reliability	How is health measured and protected?	SLOs/SLIs, error budgets, alerts	Google SRE practices, Prometheus, Grafana, Datadog (Google SRE)
Security & access	Who can do what?	AuthN/Z, secrets, WAF, IAM	OAuth 2.0, JWT, Vault, Cloudflare, AWS IAM
Edge & delivery	How is content made fast/globally?	CDN, edge compute, caching	Cloudflare/Akamai, Fastly; Redis

03

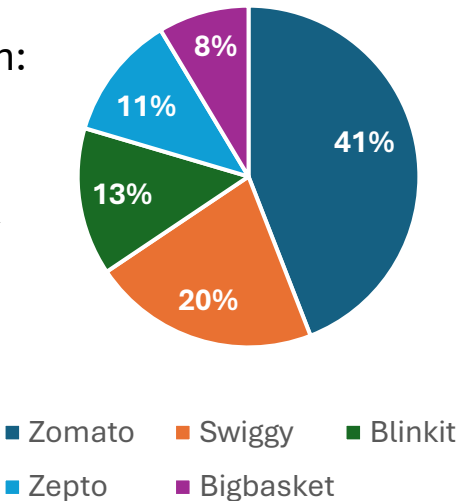
Know Your Sector

Key Players

India: Blinkit (Zomato), Swiggy Instamart, Zepto; (legacy/on-watch: Dunzo)

Global: Gopuff (US; retail media & owned dark stores), Getir (exited US/UK/EU; focusing on Turkey), Flink (Germany; multi-category grocery products)

Reference: [Tracxn](#)



Common Characteristics

Dark stores within 1-3 km of dense demand; 4-8k SKUs; micro-fulfilment pick/pack in 2-6 minutes

Promise windows: 10-20 minute “instant” or 30-45 minute rapid delivery; surge/slotless

Assortment mix: everyday groceries + “urgent” items (ice cream, beverages, snacks), growing into beauty, home, pharma-lite, and seasonal

Use Cases

- **Top-up and Emergencies:** Missed ingredients, baby products, Over-the-Counter (OTC) products
- **Occasion-driven:** Festivals, party supplies, live-sports spikes
- **Segments:** Urban singles/roommates, young families, premium convenience seekers, office micro-kitchens top-up, students in coaching hubs

Reference: <https://redseer.com/articles/quick-commerce-indias-retail-darling-or-profit-mirage/>

Business Model Canvas

Value Proposition:

- **Consumers:** instant convenience, reliable ETAs, curated essentials, late-night availability
- **Brands:** high-visibility placements on platform; rapid trials & promos based on data, benefits from impulse-led sales
- **Last-Mile Delivery Partners:** predictable zones of dense demand

Revenue Streams: fees (platform, delivery, surge), subscription, listed on-site ads, product margins

Costs: Dark store and rentals, technology, rider payouts, pick/pack, spoilage/shrinkage, discounts, marketing and CAC

Channels: Mobile apps, super-app entry points (Zomato → Blinkit; Swiggy → Instamart), web, affiliate links (via offers in Pluxee, Cred), CRM (push notifications), brand collaborations (seasonal campaigns)

User Journey

- **Consumer:** Browse → add to cart → promo/fees visibility → pay → live ETA → doorstep handoff → rate/support
- **Dark Store (Ops):**
 - **Supply-Chain:** Forecast → receive → storage → inventory control and cycle counting → quality control → replenishment
 - **Order Management:** Order prioritization → locate and pick → quality checks and pack → dispatch
- **Delivery Partner:** Accept → navigate → deliver → issue resolution
- **Brand:** Onboard → list SKU data (cataloguing of products) → run promos/ads → track metrics like return on ad spends, price demand elasticities

Key PM Metrics

- **Growth:** Total Orders, Average Order Value (AOV), Monthly Active Users (MAU), retail-media revenue % (ads on app), membership subscriptions/renewals
- **Ops and Quality:** On-time %, speed promise accuracy, fill rate (% orders fulfilled without stockouts), cancellations and returns %
- **Unit Economics:** Contribution margin per order, Customer Acquisition Cost (CAC), payback period (CAC recovery via margins)
- **Trust:** Complaint rate, cold-chain breach rate, fraud/promo abuse %.
- **Investor Lens:** Gross Order Value (GOV) growth vs. adjusted EBITDA

Macro Trends and Opportunities

- **Emerging and Futuristic Trends:** Drones for last-mile delivery, warehouse robotics, eco-friendly alternatives like electric vehicles for delivery
- **New Business Models and Experiences:** Subscription-based ordering, gamified rewards (scratch card, push lever to win cashbacks), better privacy control (hiding sensitive order history with user consent), ONDC apps, nudge economics (opting users for eco-friendly packaging, donation to charities), move towards profitability
- **Category and Geography Expansion / Consolidation:** non-grocery items (electronics, gold), festival specific offerings, cooked meals delivered in 10-15 minutes (Zepto Cafe); Tier-2 and Tier-3 cities expansion in India, Global players retrenching to home markets (Getir → Turkey)
- **AI and Hyper-Personalization:** personalized discounts, data-driven recommendations, customer support
- **Retail Media:** On-site ads and sponsored brand and SKU placements on the platform

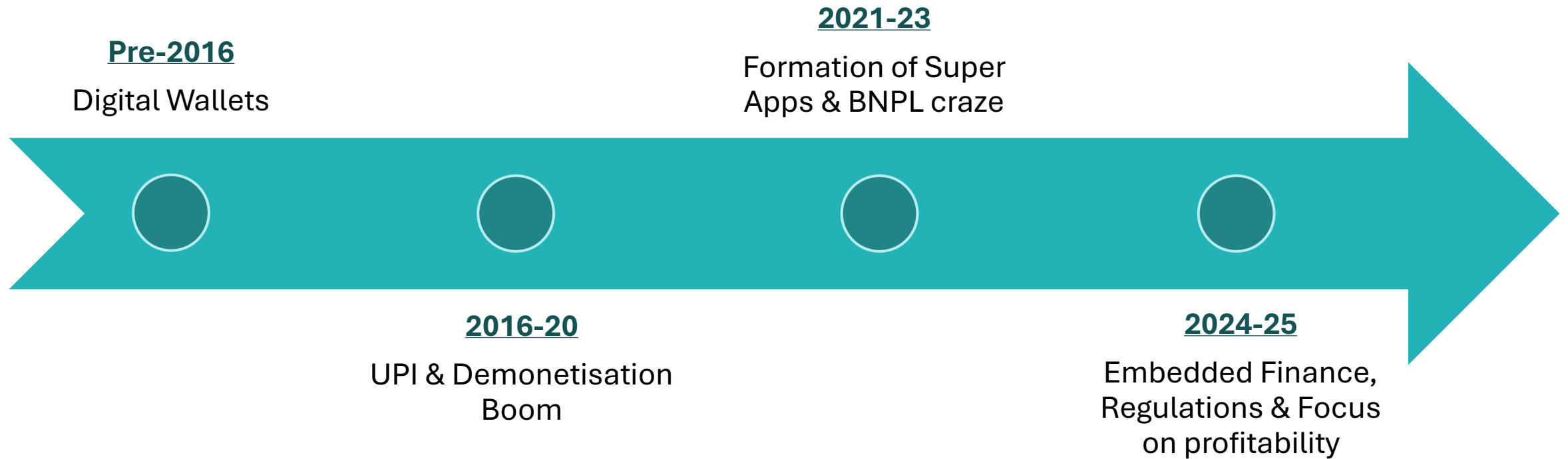
Common Product Pitfalls

- **Unit Economics Myopia:** Deep discounts lead to unsustainable burn
- **Scaling Too Wide, Too Fast:** Expanding into low-density geographies dilutes utilization (orders/hour)
- **Overpromising ETAs:** Optimistic promises → missed SLAs → churn
- **Ops Complexity:** Mismanaged dark store inventory, cold-chain breaks, packing errors, address/entry issues
- **Limited Assortment Trap:** Ignoring adjacent high-margin categories (beauty, home, private label) limits profitability
- **Ad Monetization Pitfalls:** Retail media promises high margins, but poor targeting or cluttered UX can alienate users and reduce purchase intent

Interview Tips

- **Dissect unit economics live (whiteboarding):** Given AOV ₹500, delivery cost ₹70, pick/pack ₹20, discounts 8%, ads revenue ₹5/order - what is the contribution margin?
- **Design for density:** Choose where to open the next five dark stores; trade off coverage vs. cannibalization
- **Raise AOV without killing speed:** Smart bundles (“chips + soda” during cricket matches), free-delivery thresholds, cross-sell at checkout
- **Reliability over raw speed:** Promise accuracy, dynamic SLAs, proactive delay communications
- **Measure what matters:** On-time %, fill rate, delivery utilization, retail-media Return on Ad Spends (ROAS)
- **Ethics and safety:** Cold-chain quality checks, fair fees and wages, transparent surge, customer support availability
Recruiters look for balanced thinking, not just optimization

Sector Evolution Timeline



B2C

Focus on frictionless UX, building engagement loops, and establishing trust.

- **Digital Payments (UPI & Wallets)** → PhonePe, Google Pay, Paytm, CRED UPI. The NPCI is a key enabler with its UPI infrastructure.
- **Wealthtech & Broking** → Zerodha, Groww, Upstox.
- **Lending & Credit (incl. BNPL - Buy Now and Pay Later):** CRED Cash, LendingKart, DMI Finance, Simpl, LazyPay.
- **InsurTech** → PolicyBazaar, Acko, Digit Insurance.
- **Neobanks & Personal Finance** → Digital-only banks that offer modern, user-friendly banking experiences. E.g. Jupiter, Fi, Niyo.

B2B

Focus on robust APIs, developer experience, reliability (SLAs), & compliance.

- **Payment Gateways & Merchant Solutions** → Razorpay, PayU, CCAvenue, Pine Labs
- **SME Lending & Supply Chain Finance** → KredX, M1xchange, CredAble.
- **Banking & Financial Infrastructure (Fintech SaaS)** → They provide APIs and software for tasks like KYC, fraud detection, and more. E.g. Perfios, Zeta, M2P Fintech, Setu

Common Characteristics

- **Frictionless UX** → Fast, intuitive onboarding (under 2-3 mins) to minimize user drop-offs.
- **Trust & Compliance** → Visible security cues and RBI/SEBI-aligned messaging to build user trust.
- **Engagement Loops** → Gamification and rewards to drive DAU/MAU and create habit-forming usage.
- **Tier-2/3 Inclusivity** → Multilingual UX and low-data optimization for the broader Indian market.

- **Integration Speed** → API-first, plug-and-play onboarding for rapid enterprise client integration.
- **SLA & Reliability** → Guaranteed high reliability with Service Level Agreements (SLAs) promising 99.95%+ uptime.
- **Compliance-by-Design** → Built-in modules for KYC, AML, and PCI-DSS for out-of-the-box regulatory compliance.
- **Data Security & Privacy** → Enterprise-grade encryption, comprehensive audit trails, and SOC2 readiness are standard.

Business Model Canvas

Value Propositions → Instant low-cost payments, seamless access to credit, hyper-personalization, 24/7 access, and richer data for underwriting.

Channels → Direct B2C apps (PLG), B2B enterprise sales, embedded finance partnerships (e.g., on e-commerce sites), and API platforms for developers.

Revenue Streams →

- **Payments** → Merchant Discount Rate (MDR), transaction fees.
- **Lending** → Net Interest Margin (NIM), processing fees.
- **Wealth** → Brokerage fees, subscription fees (SaaS).
- **Infrastructure** → API call fees, SaaS licensing.

Key Partnerships → Banks & NBFCs (for credit lines, accounts), card networks (Visa, Mastercard), data providers, and channel partners (e-commerce platforms).

Cost Structure → Dominated by technology & talent costs, followed by Customer Acquisition Cost (CAC), compliance & legal overhead, and fraud losses.

Use Case & Pain Points

Use Case	Pain Point	Opportunity
Retail Consumers (Payments, Lending, Investing)	Complex onboarding/KYC, transaction failures, hidden fees, fear of fraud.	Build a sub-2-minute onboarding flow, provide clear failure-state messaging, and design transparent, trust-building user interfaces.
SMBs / Merchants (Payments, Working Capital)	Slow settlement times (T+2), high integration complexity, managing chargebacks and disputes.	Offer faster settlements (e.g., T+1), build low-code/no-code integration plugins, and provide an automated dispute resolution dashboard.
Developers (using Fintech APIs)	Poor documentation, inconsistent API performance, high costs, complex compliance requirements.	Create best-in-class developer documentation, offer a reliable sandbox environment, and provide compliance-as-a-service features.
NRIs / Expats (Remittances, Global Investing)	High foreign exchange (forex) markups, opaque transfer fees, fear of fraud during cross-border transfers, and complex regulatory hurdles for investing abroad.	Build a secure, low-cost remittance platform with real-time tracking and a simplified, compliant interface for accessing global investment products.

User Journey

Payments (Customer)

App download → Account creation → KYC → Account linking → First transaction → Issue resolution

Account creation → High competition and security fears create user mistrust.

- **Opportunity** → Emphasize security to build immediate trust.

KYC verification → Complex, lengthy KYC causes high user drop-offs.

- **Opportunity** → Aim for a sub-2-minute KYC using India Stack APIs to reduce friction.

Account linking → Technical failures and confusing errors cause frustration.

- **Opportunity** → Provide clear error messages & guided flows to ensure a high success rate.

First transaction → User inertia and no clear incentive to transact.

- **Opportunity** → Drive activation with gamified rewards for the first payment.

Issue resolution → Opaque processes and fear of losing money break user trust.

- **Opportunity** → Design a transparent, self-serve resolution center with real-time tracking

Payments (Merchant)

Account creation → KYC → Technical integration → Configuration → Go-live → Operations

KYC verification → Slow, manual verification delays the merchant’s ability to accept payments.

- **Opportunity** → Automate document analysis & send reminders to speed up activation.

Technical integration → Inconsistent APIs and poor documentation create integration challenges.

- **Opportunity** → Provide best-in-class developer documentation and reliable testing tools.

Platform configuration → Complex dashboards are often confusing for business owners to use.

- **Opportunity** → Create intuitive UI with guided setups and smart defaults.

Go-live → Anxiety about potential failures impacting customers and revenue.

- **Opportunity** → Offer a "Go-Live" checklist and proactive support for a smooth launch.

Compliance & reporting → High overhead of managing chargebacks and complex reports.

- **Opportunity** → Build an automated dispute dashboard and analytics tools.

Key PM Metrics

Growth & Adoption →

- **MAU/WAU** → Monthly/Weekly Active Users
- **TPV/GTV** → Total/Gross Transaction Volume

Unit Economics →

- **LTV/CAC Ratio** → Target >3
- **Contribution Margin** → Revenue per transaction minus variable costs.

Risk & Quality →

- **Authorization Rate** → % of transactions successfully approved. Target >95%.
- **Chargeback Rate** → % of transactions disputed by customers.
- **Platform Uptime** → Target >99.95%.

Engagement & Trust →

- **Onboarding Drop-off Rate** → % of users who start but don't complete KYC.
- **NPS** → Net Promoter Score.

Macro Trends and Opportunities

Credit on UPI → Unifying payments and credit into a single, seamless flow.

- **Opportunity** → Build innovative credit products (e.g., small-ticket loans) delivered instantly at the point of sale via UPI.

Open Finance (Account Aggregator) → Shift towards consent-based data sharing for a holistic financial view.

- **Opportunity** → Create automated personal finance managers or build superior underwriting models using real-time cash flow data.

AI Agents → Moving from simple chatbots to autonomous agents for financial management.

- **Opportunity** → Design AI agents that can monitor finances, detect fraud, and execute investment strategies on behalf of the user.

Embedded Finance → Financial services are becoming invisible infrastructure, forming superapps.

- **Opportunity** → Develop vertical-specific B2B2C solutions, like "Lending-as-a-Service" for SaaS platforms.

Common Product Pitfalls

- **Onboarding Friction** → A lengthy or confusing KYC process leading to high user drop-offs.
- **Ignoring the "Money Flow"** → Not deeply understanding the entire transaction lifecycle, leading to settlement and reconciliation failures.
- **Compliance as an Afterthought** → Treating RBI/SEBI regulations as a final check-box, leading to product rollbacks or fines.
- **Shallow Fraud Detection** → Relying on simple rules instead of a multi-layered fraud defense & data security (AI models, velocity checks, manual review), leading to high losses.
- **Negative Unit Economics** → Burning cash on unsustainable rewards or discounts without a clear path to profitability ($LTV < CAC$).
- **Feature Bloat** → Cluttering the interface of a simple payments app with too many non-core services, harming the primary user experience.

Interview Tips

- **Think Compliance-First** → Start your product design by addressing the regulatory guardrails (RBI, SEBI, NPCI). It shows maturity.
- **Anchor on Trust, Not Just Growth** → For every feature, explain how you would build and measure user trust, even if it means sacrificing short-term metrics.
- **Map the Full Money Flow** → Be able to whiteboard the end-to-end journey of a transaction—from user action to settlement and reconciliation.
- **Know Your Revenue Levers** → Speak the language of finance. Discuss impact in terms of basis points (BPS) on MDR or Net Interest Margin (NIM).
- **Design for Failure** → What is the fallback flow when a UPI payment fails? How do you communicate errors? A great PM plans for exceptions.

Key Players

Online Travel Agencies (OTAs) / Digital Marketplaces

- Global: Booking Holdings (Booking.com, Priceline, Agoda), Expedia Group (Expedia, Hotels.com), Trip.com (Ctrip)
- India: MakeMyTrip, Yatra, Cleartrip (Flipkart-owned), EaseMyTrip

Discovery and Reviews → Google Flights, Skyscanner, Trivago

New Models → Airbnb (home-sharing), Hopper (AI-powered price prediction and multi-category bookings), Zostel (hostels and curated group travel packages), PickYourTrail (personalized Holiday packages)

Common Characteristics

Marketplace / aggregator model → Connects and partners with travel suppliers (airlines, hotels, tours) and consumers

Low-margin, high-volume nature → Monetization via commissions & ancillary (cab pickup, holiday packages, forex card) sales, dynamic pricing

Fragmented supply → Especially hotels in India → aggregation adds value. Heavy reliance on trust, reviews (user generated content), and seamless UX

Seasonal & cyclical demand → Holidays, events

Use Cases

Leisure Travelers → Families, couples, solo trips

Business Travelers → Corporate booking via TMCs (Travel Management Companies)

Budget vs Premium and Experiential → Price-sensitive backpackers or students vs high-end curated stays

Hosts/Hotels → Listing on OTAs for reach

Business Model Canvas

Value Proposition → Convenience, price comparison, discovery, reviews, flexible booking/cancellations, loyalty perks

Revenue Streams →

- Commissions
- Service fees, ads, white-label SaaS (example: ‘powered by Expedia’)
- Fintech add-ons: insurance, buy-now-pay-later, cancellation protection

Costs → Tech and infra, supplier integrations (example: via APIs), customer acquisition (marketing, discounts), support/refunds operations, loyalty & rewards

Channels → Mobile apps, websites, super-app tie-ins (Paytm, Flipkart), affiliate sites, meta-search (Google, Kayak)

User Journey

- **Traveler**
Discover → compare → book → pay → travel → review
- **Supplier (hotel/airline)**
Onboard → manage inventory → set pricing → receive bookings → service traveler → settlement/payout
- **OTA**
Acquire user → aggregate supply → optimize search & pricing → enable booking & payment → upsell ancillaries → resolve support

Key PM Metrics

- **Growth** → Monthly Active Users (MAU), search-to-book conversion %, Average Order Value (AOV) per booking, repeat percentage / stickiness
- **Ops and Quality** → cancellation rates, refund turnaround time (TAT), review ratings
- **Unit Economics** → commissions retained by OTA, customer acquisition cost (CAC), payback period, contribution margin
- **Trust** → fraud rate/chargebacks, fake review detection, on-platform dispute resolution
- **Investor Lens** → Gross Booking Value (GBV) v/s adjusted EBITDA

Macro Trends and Opportunities

- **AI-assisted Bookings and Insights** → Bookings shift from apps to chat-first platforms (Google Gemini), personalized itineraries, consumer trends prediction (example: shift towards colder destinations as a result of climate change)
- **Real-Time Translation** → AI translation removes language barriers, opening remote markets
- **Bleisure and Experiential Travel** → Work trips increasingly extend into leisure; Event Travel (Taylor Swift Eras Tour, Wimbledon)
- **Digital Detox and Well-being Tourism** → Rising demand for wellness retreats and unplugged experiences
- **Self-Guided Tours, Augmented Reality (AR)** → Self-guided audio tours, AR layer for immersive experience (you are standing at the Colosseum → AR shows gladiators fighting)
- **Fintech Layer** → Pay-later, insurance, cancellation protection, FX wallets bundled with travel

Reference: <https://www.statista.com/topics/13141/travel-and-tourism-trends-2025/#topicOverview>

Common Product Pitfalls

- **Commoditization** → OTAs fight in red ocean → differentiation is hard
- **Thin Margins:** Especially for flights; over-reliance on discounts
- **Refund/Support Nightmares** → Delays and cancellations result in poor customer experience if not timely resolved
- **Supplier Conflict** → Hotels/airlines push direct bookings resulting in reduced margins for OTAs
- **Trust Issues** → Fake reviews, hidden fees, bait-and-switch listings
- **Over-expansion** → Launching too many verticals (tours, activities, forex) without focus → diluted UX

Interview Tips

- **Traveler Journey Gaps** → Redesign booking flow to cut drop-offs; think search filters, payment friction, refund transparency
- **Supplier retention** → A hotel chain wants to give fewer OTA commissions - how would you retain them? (Ads, featured placement, dynamic pricing tools)
- **Measure what matters** → Stress on conversion %, measure NPS against industry benchmarks, fraud prevention and loss savings
- **Strategic Trade-offs** → Speed of market expansion v/s deep local supply integration (curated inventory); top skimming v/s mass penetration
- **Ethics angle** → Transparent fees, fair cancellations, reliable refunds handling

Key Players

- **Aggregators:** Swiggy, Zomato, Uber Eats, DoorDash
- **Cloud Kitchens:** Rebel Foods (Faasos, Behrouz Biryani), Swiggy Access, Ghost Kitchens International
- **Restaurant SaaS:** POSist, PetPooja, Toast
- **Meal Kits/D2C:** FreshMenu, HelloFresh, iD Fresh
- **Quick Commerce (adjacent):** Blinkit, Swiggy Instamart, Zepto, BigBasket

Common Characteristics

- **Two-sided or three-sided marketplace:** Restaurants (and/or cloud kitchens) ↔ diners; and at times a third side (couriers)
- **Thin margins, high frequency:** Profit comes from scale plus operational discipline
- **Local density and ETAs:** Unit economics hinge on short distances, batching, reliable prep times
- **Ops and trust heavy:** Food safety, order accuracy, complaint resolution and refund speed matter
- **Demand peaks:** Mealtimes, weekends, rain events → load balancing & incentives

Business Model Canvas

Value Proposition:

- **Diners:** Convenience, variety, reliable ETAs, offers, tracking
- **Restaurants:** Incremental demand, logistics, payments, discovery, insights
- **Last-Mile Delivery Partners:** Predictable zones of dense demand

Revenue Streams: Commissions on orders, delivery and surge fees, subscriptions (example: Swiggy One), **retail-media/ads and sponsored listings**, B2B supply margins (Zomato Hyperpure, Swiggy Access), SaaS/POS fees (POSist, PetPooja), network effects further help onboard more restaurants, riders, and consumers.

Costs: Delivery rider payouts, discounts/offers, restaurant commissions/subsidies, tech and infra, support/refunds, marketing and loyalty spends

Channels: Mobile apps, web, CRM (push/WhatsApp), brand tie-ups, corporate portals (Pluxee)

Use Cases

- **Diners:** Everyday meals, late-night orders, group/family dining, health/diet-specific needs, celebrations
- **Enterprises:** Corporate meal programs, office micro-kitchen supplies, meal vouchers, team events
- **Restaurants:** Independents (reach new customers via platforms), cloud kitchens (online-only brands, virtual menus)
- **Delivery Partners:** Flexible earning, area-based routing, peak-time incentives

User Journey

Diner: Discover → customize → checkout/pay → track → receive → rate → re-order

Restaurant: Onboard → menu/catalogue & pricing → accept → prep → handoff → settlement → analytics/ads

Delivery Partner: Log in → accept/stack batch (multiple orders in one trip, or next order queued before finishing current) → pick up → navigate → deliver → earnings/feedback

Platform: Acquire users/merchants → match & dispatch → collect payments → support/refunds → monetize (commission, fees, ads) → retain (loyalty/CRM)

Macro Trends and Opportunities

Profitability over Blitz-scaling: Tighter discounts, fees rationalization, ops efficiency, ads as margin lever

Retail-media Rise: Brands move ad spend into food apps (sponsored listings, search ads) for high-intent visibility.

Cloud-kitchens: Fewer but better-utilized kitchens, virtual brands based on data

Regulation and Workforce: Transparency on fees, gig-worker standards, hygiene compliance

AI Everywhere: Demand forecasting, dynamic ETAs, menu intelligence, fraud detection, delivery routing

Key PM Metrics

Growth & Mix: Active users, total orders, Average Order Value (AOV), repeat vs new users, membership uptake	Ops and Quality: On-time delivery, accurate ETAs, prep/fulfillment speed, cancellations/returns, order accuracy, delivery efficiency (utilization & batching)	Unit Economics: Commission rate (take rate), delivery cost per order, contribution margin per order, Customer Acquisition Cost (CAC), ad/retail-media share	Trust and Safety: Complaint rate, food safety issues, fraud/promo abuse	Investor Lens: Gross Order Value (GOV) growth, adjusted EBITDA, customer retention
---	---	---	---	--

Common Product Pitfalls

- **Discount Addiction** → Bad unit economics
- **Prep-time Blindness:** Great dispatch but slow kitchens = missed ETAs
- **Low Density Expansion:** Long trips, poor batching, high cost/order
- **Menu/Catalog Debt:** Bad images, tags, or availability → drop-offs & cancellations
- **Support Black Holes:** Slow refunds and canned replies kill trust
- **Ad Overload:** Retail-media that hurts discovery → conversion drops
- **Quality/safety lapses:** Food incidents outweigh months of growth

Interview Tips

Dissect unit economics live (whiteboarding): Given AOV ₹400, commission 20%, delivery ₹65, payment 1.5%, ads + ₹5/order, discounts 8% - what's contribution? Which levers move fastest (AOV, batching, distance, ads, fees)?

Design for density: Choose next 5 zones; trade off demand hotspots vs cannibalization; target orders/hr and average trip distance

Speed vs reliability: Dynamic ETAs, prep-time SLAs with restaurants, proactive delay credits

Raise AOV without slowing: Smart bundles, fee-free thresholds, time-of-day upsell, occasion packs

Reduce cancellations: Real-time inventory sync, auto-reroute to nearest branch, smart throttling

Measure what matters: On-time %, prep time p95, cancellation/return %, refund TAT, cohort retention, ads ROAS.

Key Players

Global



- **Alibaba**
- **Amazon**
- **Pinduoduo**
- **JD.com**
- **Coupang**
- **Rakuten**

India



- **Flipkart-Myntra (48%)**
- **Amazon (31%)**
- **Meesho (8%)**
- **Jiomart-Ajio (8%)**

Common Characteristics

- **Supply chain:** centralized warehouses (hub and spoke model); logistics follows Inbound to FC → Mid Mile (Line haul, Sortation, Cross-dock) → Last mile hub → reverse logistics
- **Promise windows:** 1-3 days/ scheduled slots/ same day delivery timelines; faster delivery for premium plan subscribers (e.g. Amazon Prime); free reverse logistics
- **Assortment mix:** 50K-100M SKUs covering most categories
- **Trust Signals:** product reviews and ratings driving customer confidence

Business Model Canvas

Value Proposition:

- **Consumers:** Wide product assortment, reliable delivery ETAs, personalized recommendations, attractive discounts & cashbacks, hassle-free refunds/returns.
- **Brands/Sellers:** National/global reach; high-visibility ad placements; targeted recommendations to buyers; data-driven trials and promotions; uplift from impulse-led sales; cost-efficient last-mile delivery.

Key Partners: Payment gateways (stripe, razorpay), Delivery partners (Delhivery, Ecom Express, DHL)

Revenue Streams: Marketplace commissions, advertisement charges, premium subscriptions, fulfilment/storage fees, and product margins (private labels)

Business Models: Inventory-led, Marketplace

Delivery Models: Platform fulfilled, Merchant fulfilled, Drop shipping: Warehouse lease, platform maintenance & upgrade, discounts, delivery partner payouts, unsold inventory, customer support, payment gateway fees, packaging, returns handling.

Use Cases

Urban mass-market consumers: convenience driven shoppers seeking competitive prices and home delivery, brand-loyal consumers.

Deal/Discount driven: Shoppers responding to festive offers, trending products, or seasonal events (e.g., Prime Day, Big Billion Day).

Tier 2/Emerging city shoppers: price sensitive consumers, first-time users, prioritizing reliable delivery

Marketplace sellers: D2C, SMBs seeking wider market reach.

User Journey

Consumer: browse → add to cart → promo/fees visibility → pay → track order → doorstep delivery → rate/support

Delivery partner: Assigned to route → pick up from last mile hub → deliver to customer → report issues / handle exceptions

Brand: onboard → list SKUs → update online catalogues → run promos/ads → fulfill orders → track performance metrics (return on ad spends, gross monthly sales, return rates)

Key PM Metrics

- **Growth & Mix:** Gross merchandise value (GMV), Average order value (AOV), Cart abandonment rate (CAR), Category mix
Ops & Quality: OTD rate, Perfect order rate, Order picking accuracy, Return rate, Cancellation rate, Sell-Through rate
- **Unit Economics:** Contribution margin per order, Take rate, Customer lifetime value (CLV), Average cost per order
Ad Performance: Click Through Rate (CTR), Cost per Click (CPC), Return on Ad Spends (ROAS), Ad conversion rate
- **Trust:** Complaint rate, Fraud/promo abuse %, Counterfeit complaint rate
- **Investor Lens:** Marketplace penetration, Adjusted EBITDA

Macro Trends and Opportunities

Futuristic technology trends:

- Drones for last-mile delivery
- Warehouse robotics
- Augmented reality for product visualization and trial
- Voice shopping using Alexa/Google Assistant.

New Consumer Experiences:

- Social commerce (Integration of shopping directly within social media platforms. Ex - Instagram, TikTok)
- Livestream shopping (Real-time product demonstrations by influencers/hosts with instant buying)
- Buy now pay later (Flexible credit at checkout, allowing deferred or split payments)
- Micro subscriptions (Small recurring payments for curated/niche products)

Emerging Business Models:

- Vertical marketplaces (Nykaa)
- Super apps (Grab, Tata Neu, GoTo)
- Cross-border sales (Amazon Global Sales)

Common Product Pitfalls

- **Listing Quality Gap:** Misclassified categories, inconsistent images, and broken variation mapping reduce product discoverability and conversions.
- **Price Blindspots:** Inefficient competitor tracking leads to uncompetitive pricing and lost transactions.
- **Trust Gaps:** Counterfeit/restricted/unsafe products and fake reviews erode customer confidence and lead to compliance issues.
- **Ad Waste:** Poor targeting and cluttered placements reduce ad ROI and purchase intent.
- **Supply Chain Inefficiencies:** Forecasting errors, incorrect stock displays, suboptimal routes, and inefficient order aggregation increase costs and delays

Interview Tips

- **Conversion Drop-offs:** If you diagnose a 30% cart abandonment in beauty- is it due to poor quality of product display page, trust gap, checkout friction, or poor recommendations? Propose fixes to lift conversion.
- **Returns at Scale:** Redesign the returns flow for apparel (E.g. 40% return rate). Balance customer satisfaction with lower reverse logistics cost.
- **Counterfeit Detection:** Build a proactive system to catch fake items before they reach customers - think workflows + tech levers.
- **Ad Slot Prioritization:** During peak events (Prime Day, Black Friday), design a system that allocates limited ad slots to maximize ROI without hurting UX.

Ground your answers in customer experience, articulate trade-offs clearly and show your awareness of unit economics.

Key Players

Global



- Netflix (300M+ subscribers)
- Amazon Prime Video (200M+)
- Disney+ (125M+)
- Tencent Video (115M+)
- iQIYI (101M+)

India



National

- JioHotstar
- Amazon Prime
- Netflix
- MX Player Zee5
- SonyLiv

Regional

- Sun NXT
- Hoichoi
- aha

Common Characteristics

Digital supply chain: Content creation/Acquisition → Capture & Transcoding → Metadata enrichment (titles, descriptions, tags, thumbnails) → Digital rights management (encrypting content, licensing, access controls) → Ingest (uploading content to platform) → Content distribution

Promise windows: Accessibility across devices, personalized recommendations, access to global content, buffer-free streaming, multi-language audio

Assortment mix: On-demand content, live streaming, regional content, originals, sports, and music

Business Model Canvas

Value Proposition:

- **Consumers:** Content diversity, control over viewing experience, synchronization across devices, customized watchlists, price differentiation.
- **Content owners:** Low distribution costs, global audiences, viewer analytics insights, experimental content funding, flexible revenue options.

Revenue Streams: Subscription revenue, advertising revenue, rental/pay-per-view revenue, merchandise, licensing content

Business Models:

- SVOD (subscription video on demand | example - Netflix)
- AVOD (advertising video on demand | example - MX player)
- TVOD (transactional video on demand | example - Apple TV)
- Freemium (YouTube, Spotify)
- Others include: Studios & creators (licensing fees), app stores (platform fees), Smart TV OEMs (integration), payment gateways, cloud & CDN providers

Use Cases

Urban digital natives: Young professionals and students with demand for global content. Typically looking for single screen options.

Family households: Looking for bundled plans for the family. Favours regional content and popular local shows.

Sports fans: Key growth driver in the OTT segment. Subscribes to OTTs based on streaming rights of IPL/EPL/Olympics/World cup.

Content creators & owners: Sell licenses of movies, shows, streaming rights of sports for wider audience reach at low distribution costs. E.g. TVF, BCCI.

Stakeholder Journey

Growth & Mix: Subscriber acquisition rate, churn rate, monthly recurring revenue (MRR), content/genre mix

Engagement: Monthly active users (MAU), watch time, session duration, content completion rate, binge rate

Technical Performance: Buffering rate, video startup time (VST), streaming performance index (SPI), bitrate, latency & load time

Unit Economics: Average revenue per user (ARPU), Content spend efficiency, Content Cost per Hour/Title

Key PM Metrics

Consumer: subscription → content discovery → add to watchlist → view content → pause/replay → feedback/rating → renew subscription

Content creator or owner: Pitch for acquisition or production of originals → contract on licensing rights → content delivery → content promotion → analyze engagement data → monetization → contract renewal

Macro Trends and Opportunities

Emerging Content Formats:

Live watching sports/games (e.g Twitch), Short form videos, Non-linear storytelling (e.g. Black Mirror: Bandersnatch)

Futuristic Technology Trends:

Conversational search, Spatial Audio/Adaptive Sound (Netflix-Sennheiser partnership) , AI-powered personalization and recommendation systems, VR watch parties, AR overlays

New Business Models:

Shoppable streaming/commerce-enabled OTT (Shop with Max), OTT bundling with telecom plans , FAST Channels, Gamified content models (Hotstar's cricket prediction contests)

Common Product Pitfalls

Content Discovery Gap: Poor metadata tagging and weak recommendations reduce engagement, while narrow personalization traps users in repetitive viewing loops, leading to content fatigue.

Subpar Streaming Quality: Poor video resolution, frequent buffering, latency issues in livestreaming and insufficient infrastructure to handle traffic spikes degrade the viewing experience.

Security Vulnerabilities: Poor DRM (Digital Rights Management) and anti-piracy measures can lead to content theft and revenue loss.

Ad Experience Breakdown: Irrelevant targeting, repetitive spots, and intrusive formats reduce user satisfaction and ad ROI.

Fragmented User Journeys: Lack of device sync and limited offline access disrupt continuity and lower customer satisfaction.

Interview Tips

In this sector, you can look out to the following:

Engagement Drop-offs: If new users stream <1 hour after week one, it signals churn risk. Diagnose causes and propose levers to boost early engagement.

Personalization Gaps: If users scroll endlessly but don't watch, refine recommendation engines to drive higher play conversion and watch time.

Ad Experience Design: For peak shows/events, optimize limited ad slots for yield while preserving immersion through smart placement and timing.

Content Navigation Design: A cluttered homepage reduces discovery. Redesign layout and categorization (genres, moods, trending, personalized rows) to improve relevance without overwhelming users.

Ground your answers in real user experiences and showcase your analytical strengths.

Key Players

Global



- **Uber (30M rides daily),**
- **Didi Chuxing (30M+ rides)**
- **Grab (9M rides)**
- **Ola (4M)**
- **Lyft (2M)**

India



B2C

- **Ola**
- **Uber**
- **Namma Yatri**
- **Rapido**

B2B

- **Uber for Business**
- **Ola Corporate**
- **BluSmart for Business**
- **Move in Sync**

Common Characteristics

Ride Workflow: Driver onboarding (verification, vehicle checks)--> User onboarding -> Ride request -> Matching & dynamic pricing -> Navigation (route optimization) -> Payment & settlements (in-app payments, driver payouts) -> Feedback loop (ratings, complaints)

Corporate Ride-Hailing Workflow: Corporate onboarding -> Employee onboarding (registration with company credentials) -> Ride scheduling (on-demand or pre-scheduled) -> Route optimization (multi-stop pickups/drops) -> Ride execution -> Payment & settlements -> Feedback & analytics

Promise windows (passengers): Proximity-based matching, low wait times, transparent fares, safety features (SOS, ride tracking, verified drivers), personalization (saved location, preferred vehicles), well-maintained vehicles

Promise windows (drivers): Efficient vehicle utilization, incentive structures, use of real-time GPS data for navigation

Assortment mix: Multi-modal options (auto, bike, car, EV), carpooling, premium/luxury (e.g. Uber Black), scheduled rides, outstation trips, multi-stop trips, Airport transfers

Business Model Canvas

Value Proposition:

- **Passengers:** Seamless travel through easy booking, reliable availability, fair pricing, and trusted drivers
- **Drivers:** Higher earning potential through efficient matching, surge pricing benefits, seamless payment settlements

Revenue Streams: Trip-based commission, subscription fees, corporate contracts, in-app advertising, ancillary services (rentals)

Cost Structure: Driver payouts, platform development, customer support, driver incentives (e.g. underserved location, peak hour service), vehicle financing, insurance, mapping/navigation software purchase

Business Models: Commission-based model, Subscription/Membership model, B2B model (corporate partnerships)

Key Partnerships: Payment Gateways, map and technology providers, charging station networks, insurance providers (passenger & driver safety coverage)

Use Cases

Urban commuters: Office-goers, students, and city dwellers needing frequent/daily rides for commutes and social outings. Value cost efficiency, quick pickups, and minimal cancellations.

Tourists & leisure travellers: Travelers seeking airport transfers, sightseeing, and outstation rides. Prefer reliable service, comfortable or luxury cars, and transparent pricing.

Corporate users: Companies and business travelers using rides for employee commutes, client meetings, and airport transfers. Expect time-efficient aggregation, billing integration, and strong customer support.

Vehicle/Fleet owners: Drivers and operators leveraging the platform to maximize earnings with reduced idle time and fuel waste. Look for analytics-driven insights to optimize vehicle placement and quick payment cycles.

Stakeholder Journey

Rider: Registration → ride search/request → driver matching & ETA → ride execution (pickup → trip → drop) → payment (in-app/cash) → feedback/rating → repeat usage

Driver/Fleet Partner: Onboarding & verification → vehicle check → receive ride request → navigate & complete ride → receive payment/payout → performance feedback → continued engagement / retention

Corporate Client: Company onboarding → policy setup (ride limits, billing rules) → employee onboarding → ride scheduling & tracking → consolidated billing & reporting → contract renewal

Key PM Metrics

Growth & Mix: Gross booking value (GBV), ride volume, active users, average daily rides, geographic coverage

Tech & Operations: Ride completion rate, ride cancellation rate, match time, driver utilization rate, driver acceptance rate, ETA accuracy

Trust: Safety incidents per million rides

Unit Economics: Contribution margin per trip, average revenue per user (ARPU), Take rate, Driver earnings per active hour, cost per ride

Macro Trends and Opportunities

Emerging and Futuristic Trends:

- Autonomous ride-hailing (e.g., Waymo, Baidu Apollo Go)- Self-driving fleets enabling cost-efficient and round-the-clock ride services.
- Sustainable mobility (Ola electric, Uber green) - EV adoption and green ride options reducing carbon footprint
- AI driven optimization (Didi's Gaia platform) - AI-based systems improving route planning, demand forecasting, and fleet utilization

New Business Models and Experiences:

- Mobility as a service (MaaS) - Integrated transport solutions combining ride-hailing with public and shared mobility
- Subscription rides (Lyft Pink, Uber One) - Membership models offering discounted fares and loyalty benefits
- Event & Tourism mobility (Uber Events) - Tailored rides for concerts, weddings, and tourist hotspots
- Micro-mobility (Rapido, Bounce) - Affordable short-distance travel via bikes, scooters, auto-rickshaws and self-drive options
- In-car Advertisement (GrabAds) - Showing targeted ads to passengers during rides
- Hyperlocalization (Namma Yatri in Bengaluru) - Platforms adapting to local cultures and pricing, often driver-cooperative owned

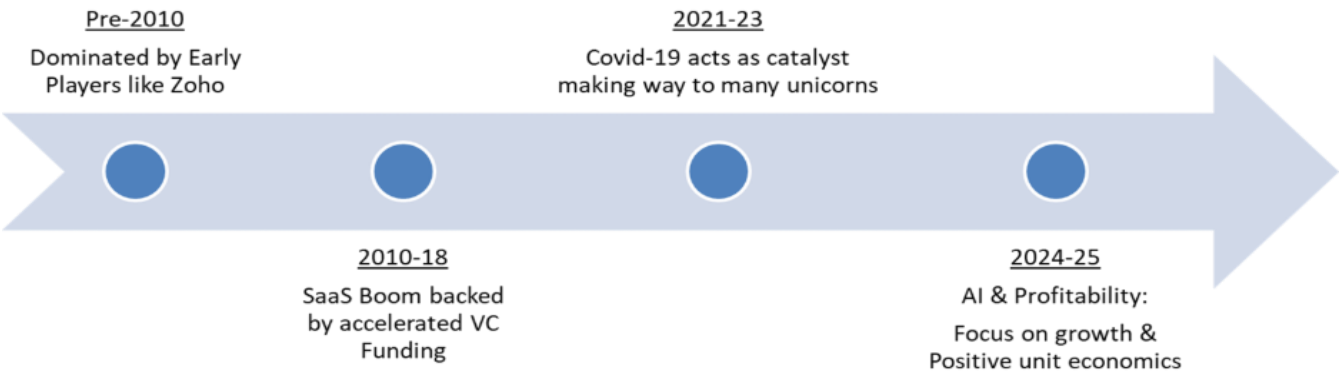
Common Product Pitfalls

- Real-time Matching Failure:** Latency in the matching algorithm can result in slow driver-rider connections, leading to increased wait times and missed ride opportunities.
- Inefficient Driver Dispatch:** Weak forecasting and routing create driver shortages in some areas and oversupply in others, increasing wait times and reducing earnings.
- Navigation & Routing Gaps:** Inaccurate GPS, outdated maps, or weak route optimization increase ETAs, cause detours, and impact rider experience.
- Driver Quality & Safety Concerns:** Inconsistent driver onboarding, lack of background verification, or weak in-ride safety features erode user confidence.
- Pricing Opacity & Surge Shock:** Opaque surge pricing calculations erodes trust and reduces user trust and increases rider churn .
- Cancellation Handling Gaps:** Unrestricted driver cancellations reduce reliability, delay trips, and frustrate passengers.

Interview Tips

- Demand-Supply Imbalance:** Rapid growth near Bangalore airport has increased rider demand, but driver reluctance to serve the area creates long wait times. Diagnose root causes and design levers. Think targeted incentives or dynamic pricing or pooled rides .
 - Driver Multi-App Switching:** Surge-zone cancellations rise as drivers shift to rival apps offering better fares. Build loyalty mechanisms, earning parity, and reduce idle-time inefficiency to retain supply.
 - No-Show Costs:** In corporate fleets, 20-30% employee no-shows inflate per-head trip cost and reduce utilization. Redesign with real-time confirmations, waitlist allocation, and automated rerouting.
 - Passenger Safety:** Low SOS adoption and delayed response weaken trust. Reimagine the safety workflow with proactive nudges, live tracking, and faster escalation.
- Keep your answers rooted in real rider and driver issues. Interviewers want to see clarity in identifying problems and actionable solutions.*

Sector Evolution Timeline



The SaaS Stack & Key Players

L1: Horizontal SaaS (Function-Specific)	L2: Vertical SaaS (Industry-Specific)	L3: DevTools & Infrastructure SaaS
Focus on best-in-class user experience, deep integrations, and clear differentiation in crowded markets. Key Players: Freshworks (Customer Support), Zoho (Business Suite), Darwinbox (HRTech), WebEngage (MarTech), Chargebee (Billing).	Focus on deep domain expertise, industry-specific workflows, and becoming the "system of record" for a particular vertical. Key Players: Lentra (BFSI), Innovaccer (Healthcare), Zenoti (Wellness), UrbanPiper (Restaurants).	Focus on superior developer experience (DX), robust documentation, performance, and building a strong user community. Key Players: Postman (API Platform), BrowserStack (Testing), Hasura (GraphQL).

Common Features

- **PLG (Product-Led-Growth) / Self-Serve Focused**
 - Frictionless Onboarding: Quick, self-serve sign-up and in-app guided tours to drive user activation.
 - Freemium/Trial Models: Offering a free tier or a time-bound trial is the default to lower the barrier to entry.
 - Community & Docs: Public documentation and a strong user community are key for support and engagement.
- **B2B / Enterprise-Focused**
 - Security & Compliance: SOC2, GDPR, and ISO certifications are non-negotiable table stakes.
 - Enterprise-Grade Features: Single Sign-On (SSO), role-based access control (RBAC), and audit logs are mandatory.
 - Integrations & APIs: A robust integration marketplace and well-documented APIs are critical for fitting into a customer's existing tech stack.

Business Model Canvas

- **Value Propositions:** Lower Total Cost of Ownership (TCO), faster time-to-value, scalability on demand, and continuous, automatic updates.
- **Channels:** Product-Led Growth (PLG) through freemium/trials, Sales-Led Growth (SLG) with inside and field sales teams, and content marketing.
- **Revenue Streams:**
 - Recurring Revenue: Annual/Monthly Recurring Revenue (ARR/MRR) through tiered subscriptions (based on features, users, or usage).
 - Other: One-time implementation fees and premium support packages.
- **Key Partnerships:** Cloud providers (AWS, Azure, GCP), technology partners for integrations, and channel resellers.
- **Cost Structure:** Dominated by Talent Costs (R&D) and Customer Acquisition Cost (S&M), followed by cloud infrastructure expenses.

User Cases and Pain Points

Use Case	Pain Point	Opportunity
SMBs (Self-Serve Users)	High costs of traditional software, lack of technical expertise, need for simple, all-in-one tools.	Offer affordable, easy-to-use products with a freemium model and excellent self-serve support.
Enterprise Buyers (Decision-Makers)	Complex procurement cycles, data security & compliance risks, unclear ROI, and integration challenges with legacy systems.	Provide clear ROI calculators, enterprise-grade security features, a robust integration marketplace, and dedicated support.
Developers (Using SaaS APIs)	Poor documentation, unreliable API performance, opaque pricing, and restrictive rate limits.	Create a superior developer experience (DX) with clear documentation, a reliable sandbox, and transparent, usage-based pricing.

User Journey

PLG User Journey

Discovery → Free Trial/Sign-up → Activation → Conversion → Expansion

- **Activation:** Steep learning curve and poor first-time user experience. → Design a seamless onboarding flow to a quick "win."
- **Conversion:** Unclear value proposition of paid plans. → Clearly gate high-value features and demonstrate ROI to encourage upgrades.
- **Expansion:** Product lacks features for growing needs. → Build modular features that users can adopt as they scale.

Enterprise Buyer Journey

Problem Awareness → Evaluation → Procurement & Legal → Implementation → Renewal

- **Evaluation:** Inability to properly test the product for security and integration. → Offer guided proof-of-concept (POC) trials and a robust sandbox.
- **Procurement & Legal:** Long negotiation cycles and inflexible contract terms. → Standardize contracts and empower sales teams with clear guidelines.
- **Implementation:** Difficult data migration and a lack of user training. → Provide dedicated implementation support and comprehensive onboarding for teams.

Key PM Metrics

Growth:

- ARR/MRR: Annual/Monthly Recurring Revenue

Unit Economics:

- LTV/CAC Ratio: Target > 3
- Payback Period: Target < 12-18 months

Retention:

- Net Revenue Retention (NRR): Target > 120% for best-in-class
- Gross Revenue Retention: Target > 90%

Engagement & Value:

- Activation Rate: % of users completing a key action
- NPS: Net Promoter Score

Macro Trends and Opportunities

AI as a Co-pilot: Generative AI is being embedded to automate tasks and act as an intelligent assistant within workflows.

- *Opportunity:* Move beyond simple features to create AI-native products that fundamentally change how work is done.

Vertical SaaS Deepening: Companies are building deep, industry-specific platforms that become a business's central operating system.

- *Opportunity:* Identify an underserved niche and build a "system of record" with tailored workflows and domain expertise.

Focus on Net Revenue Retention (NRR): The most efficient growth comes from the existing customer base.

- *Opportunity:* Design products for modular adoption, enabling clear paths for upselling and cross-selling.

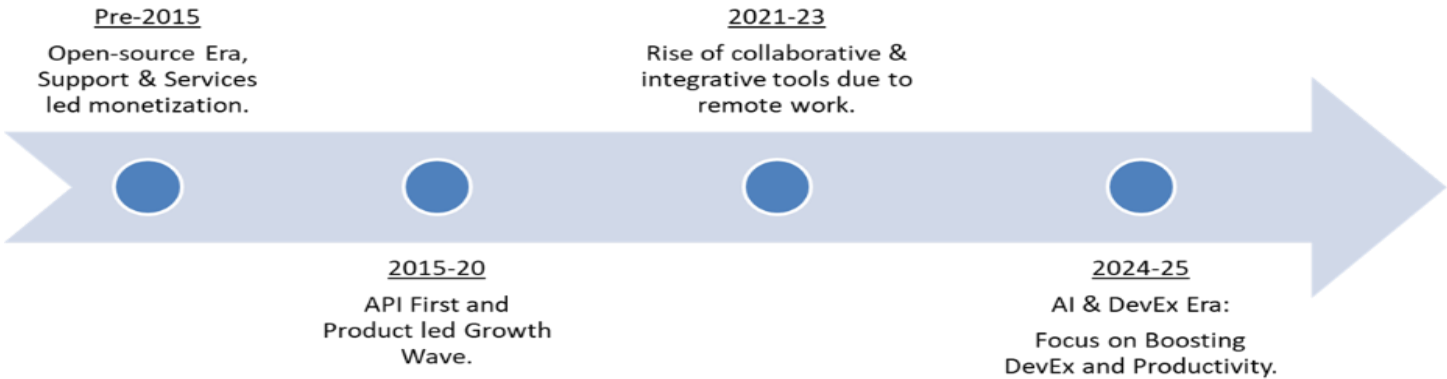
Common Product Pitfalls

- **Building a "Vitamin, not a Painkiller":** Creating a nice-to-have product that doesn't solve a critical business problem, leading to high churn.
- **Ignoring Go-To-Market (GTM) Fit:** Building a complex product but expecting a simple self-serve model to work, or vice versa.
- **Underestimating Enterprise Needs:** Trying to sell to large companies without mandatory features like SSO, audit logs, and SOC2 compliance.
- **Technical Debt:** Prioritizing new features over platform stability, resulting in a slow, buggy product.
- **Poor User Onboarding:** Failing to guide new users to the "aha!" moment quickly, leading to low activation and abandonment.

Interview Tips

- **Speak the Language of SaaS:** Always frame your answers using core metrics like ARR, NRR, LTV/CAC, and Churn.
- **Clarify the GTM Motion:** Start by asking, "Is this a Product-Led Growth (PLG) or Sales-Led Growth (SLG) model?"
- **Distinguish Between Personas:** Clarify if you are solving for the Buyer (focused on ROI) or the End User (focused on workflow efficiency).
- **Think in Workflows, Not Just Features:** Frame your ideas around improving a user's end-to-end workflow.
- **Anchor on Expansion Revenue:** When asked how to grow a product, focus on increasing Net Revenue Retention (NRR) through upselling and cross-selling.

Sector Evolution Timeline



The DevTools Stack & Key Players

L1: Code & Collaboration	L2: Build, Test & Deploy	L3: API Platforms & Infra	L4: Observability & Monitor
Focus on performance, seamless collaboration features, and deep integrations into the developer's editor (IDE). Key Players: GitHub, GitLab, Atlassian (Bitbucket), JetBrains.	Focus on speed, reliability, and providing insightful analytics to debug failures quickly. Key Players (India-born): BrowserStack, LambdaTest. Global Players: Jenkins (Open Source), CircleCI, Harness, Vercel.	Focus on best-in-class documentation, API reliability, and a simple, predictable pricing model. Key Players (India-born): Postman, Hasura. Global Players: Stripe, Twilio, Docker, HashiCorp.	Focus on reducing noise, providing actionable alerts, and helping developers correlate data from different sources (logs, metrics, traces). Key Players: Datadog, New Relic, Grafana (Open Source), Dynatrace.

Common Features

Developer Experience (DevEx) Focused

- **CLI & API Access:** A powerful Command-Line Interface (CLI) and a well-documented API are non-negotiable.
- **Stellar Documentation:** Documentation is treated as a core part of the product, not an afterthought.
- **Community is Key:** A thriving community (on Discord, Slack, etc.) for support and feedback is a critical feature.

Team & Enterprise Focused

- **Collaboration Features:** Real-time sharing, commenting, and multiplayer modes are becoming standard.
- **Governance & Security:** Role-based access control (RBAC), audit logs, and security features (DevSecOps) are mandatory for enterprise adoption.
- **Performance Analytics:** Dashboards showing build times, deployment frequency, and failure rates (DORA metrics) are crucial for engineering leaders.

Business Model Canvas

Value Propositions: Increased developer productivity, faster time-to-market for new features, improved software quality and reliability, and enhanced collaboration.

Channels: Product-Led Growth (PLG) driven by individual developers, open-source adoption, content marketing (technical blogs, tutorials), and a sales-led motion for enterprise accounts.

Revenue Streams:

- **Freemium & PLG:** A generous free tier for individuals and small teams is standard.
- **Usage-Based Pricing:** Charging based on consumption (e.g., build minutes, API calls, data storage).
- **Per-Seat Pricing:** Tiered subscriptions based on the number of developer seats.

Key Partnerships: Cloud providers (AWS, Azure, GCP), IDE makers (Microsoft for VS Code), and open-source foundations.

Cost Structure: Dominated by **Talent Costs (R&D)** for highly skilled engineers, followed by community management and cloud infrastructure costs.

User Cases and Pain Points		
Use Case	Pain Point	Opportunity
Individual Developers	Time wasted on setting up local environments, repetitive boilerplate tasks, and a steep learning curve for new tools.	Offer a zero-config setup, build intelligent code-generation tools, and provide interactive tutorials.
Engineering Teams (Team Leads & Managers)	Slow CI/CD pipelines, inconsistent development environments across the team, and difficulty in reviewing and debugging complex code.	Build high-performance testing platforms, offer cloud-based development environments, and create powerful code review and collaboration tools.
Enterprise Buyers (VPs of Eng, CTOs)	Lack of visibility into developer productivity, ensuring security and compliance across the SDLC, and managing tool sprawl.	Provide comprehensive analytics dashboards (DORA metrics), build DevSecOps features, and offer an integrated platform solution.

User Journey
PLG Developer Journey Discover (e.g., on Twitter/Blog) → Sign-up (often via GitHub) → Activation → Community Engagement → Expansion • Activation: <i>The tool is complex to set up and the "aha!" moment is unclear.</i> → Provide a "Getting Started" guide that leads to a meaningful outcome in under 5 minutes. • Community Engagement: <i>The developer gets stuck and has no one to ask for help.</i> → Build an active Discord/Slack community for peer-to-peer support. • Expansion: <i>The developer loves the tool but can't convince their team to adopt it.</i> → Build collaborative features that are easy to share and demonstrate value to the entire team.
Enterprise Team Journey Bottom-up Adoption → Team Evaluation (POC) → Security & Compliance Review → Procurement → Team-wide Rollout • Evaluation (POC): <i>Difficult to prove the tool's value and ROI over existing solutions.</i> → Offer a guided Proof-of-Concept with clear success metrics and dedicated support. • Security & Compliance: <i>The tool doesn't meet the company's strict security standards.</i> → Achieve SOC2 compliance and build enterprise-grade features like SSO and audit logs. • Rollout: <i>Lack of training and integration makes team-wide adoption a challenge.</i> → Provide detailed migration guides and offer onboarding sessions for the entire team.

Key PM Metrics

Adoption & Growth:

- **Weekly Active Users/Teams:** Measures recurring usage.
- **Stars/Forks (for Open Source):** Indicates community interest.

Unit Economics:

- **LTV/CAC Ratio:** Target > 3, often higher due to low-touch sales.

Retention:

- **Net Revenue Retention (NRR):** Target > 130% for best-in-class DevTools.

Engagement & Value:

- **Activation Rate:** % of users who successfully complete a core workflow (e.g., run their first build).
- **Time to Value (TTV):** How quickly a new user finds the "aha!" moment.

Macro Trends and Opportunities

AI in the SDLC: AI is moving from being a code assistant to an autonomous agent that can write, test, and even deploy code.

- *Opportunity:* Build AI-native tools that automate entire workflows, not just individual tasks.

Platform Engineering: Enterprises are building internal developer platforms (IDPs) to standardize tools and improve DevEx.

- *Opportunity:* Create modular, API-first tools that can be easily integrated into these internal platforms.

DevSecOps is Mainstream: Security is shifting left and becoming an integral part of the developer's workflow.

- *Opportunity:* Build tools that embed security checks and vulnerability scanning directly into the CI/CD pipeline and the developer's IDE.

Common Product Pitfalls

- **Poor Developer Experience (DX):** A slow, buggy, or unintuitive tool will be quickly abandoned by developers.
- **Ignoring the Open-Source Community:** Treating the community as a marketing channel instead of a core part of the product development process.
- **Failing to Bridge the PLG-to-Enterprise Gap:** Not building the necessary security, compliance, and governance features required to sell to large companies.
- **"Leaky Bucket" Funnel:** Focusing on top-of-funnel sign-ups without a clear strategy for activating and retaining those users.
- **Solving a Niche Problem:** Building a tool for a workflow that only a very small subset of developers has, limiting the total addressable market.

Interview Tips

- **Be the User:** Demonstrate genuine empathy and a deep understanding of developer pain points, workflows, and the tools they use daily.
- **Think in Ecosystems:** Frame your answer around how your product would integrate with the broader DevTools ecosystem (e.g., GitHub, VS Code, Slack).
- **Community-First Mindset:** When asked about GTM, a great answer is to start with building a strong open-source or user community.
- **Developer Experience is Everything:** Anchor your product decisions on what will create the most seamless and delightful experience for the developer.
- **Articulate the PLG Flywheel:** Be able to explain how a great product with a strong community can lead to a powerful, low-cost acquisition and expansion loop.

Key Players

Global



Consultation - Teladoc Health, Amwell, Doctor on Demand
Hospital SaaS & EMR - Epic Systems, Oracle Cerner, Meditech
Fitness Tracking - MyFitnessPal, Strava, Fitbit

India



Consultation - Practo, Tata 1mg, MFine
Hospital SaaS & EMR - Practo Ray, HealthPlix, MediBuddy
Fitness Tracking - Cult.fit, HealthifyMe, Fittr

Common Characteristics

Promise windows (Patients/Users)

- Verified doctors, nutritionists and fitness coaches
- Remote consultations, workouts, home diagnostics
- Continuous health monitoring and progress tracking

Promise windows (Providers)

- Smart scheduling with integrated insurance, billing
 - Wider patient pool through digital platforms
 - Analytical insights from digitized health records

Assortment mix: Telehealth, enterprise solutions (EHR, hospital management), diagnostics and monitoring, medicine & nutrition delivery, wellness, lifestyle platforms

Key Bottlenecks

- Regulatory and compliance hurdles (HIPAA, GDPR, NDHM)
- Strict oversight of health claims & outcomes
- Tight controls on e-prescriptions & online medicine sales

Business Model Canvas

Value Proposition

- **Patients/Users:** Accessible healthcare & fitness, supported by expert consultants, healthcare services
- **Providers:** Simplified appointment, follow-up management, integrated patient/user records, expanded reach through discovery, add-on services

Revenue Streams

Consultation commissions, Diagnostic fees, Subscription plans (chronic care, wellness), SaaS licensing (EMR/EHR for hospitals/doctors), Corporate employee packages

Business Models

Pay-per-service (Practo, Tata 1mg), Freemium (HealthifyMe), Enterprise solutions (Practo Ray), Subscription models

Key Partnerships

Doctors, hospital networks, pharmaceutical companies, diagnostic labs, technology and cloud providers

Use Cases / Customer Segments

- **Care seekers:** Looking for quick and reliable access to doctors, medicines, and diagnostics. Value convenience, trust, and affordability.
- **Health & fitness enthusiasts:** Focused on fitness, diet, sleep, and stress management. Engage with trackers, workout videos, and habit-building programs.
- **Chronic condition patients:** Looking for continuous monitoring, specialist support, and personalized care plans.
- **Enterprise-linked users:** Access healthcare through corporate or insurance tie-ups. Seek preventive care, mental wellness, and subsidized consultations.
- **Providers:** Healthcare/wellness providers and hospitals adopting digital tools for practice management, teleconsultation, EMR/EHR, and patient engagement.

Stakeholder Journey

Patient: User onboarding → identify need → discover provider → book consultation/test → pay → consult/diagnosis → medicine delivery → follow-up → review

Provider: Registration and KYC → list offerings → setup profile and slots → deliver services (consult/therapy/training) → share plan/prescription/report → track performance/analytics

Healthcare SaaS subscribers: Organization onboarding → workflow setup (services, slots, billing) → record digitization and data management → billing & settlements → analytics & reporting → subscription renewal/upgrade

Chronic Care/Fitness: User onboarding → profile setup → need assessment → program discovery → subscription/payment → periodic consult/coaching → continuous monitoring → progress review

Metrics and KPIs

Growth & Mix: New user acquisition rate, Churn rate, Conversion rate (free to paid), CLV, CAC payback, GMV, ARPU

Ops & Engagement: Monthly active users (MAU), session frequency and duration, appointment wait time, appointment no show rate, diagnostic turnaround time

Functional Performance: Care plan adherence rate, clinical outcomes improvement, consultation completion rate

Technical Performance: Feature adoption rate, user feedback score, System downtime

Macro Trends

Consumer Experience:

- AI powered personalized health journeys
- VR/AR enabled healthcare training (e.g. Osso VR)
- Wearables + Healthcare Integration (e.g. Apple Health - aggregates health data with option to share with healthcare provider)

Futuristic Technology Trends:

- Gen AI in Diagnostics
- Blockchain for securing health data (e.g. BurstIQ)
- Remote patient monitoring (e.g. Medtronic)

New Business Models:

- Subscription based consultation
- Health super apps (GrabHealth)
- Niche wellness platforms (Flo, Wysa)

Common Product Pitfalls

- **Subpar Device Connectivity:** Video drops, buffering, device pairing issues, and slow data updates undermine remote care and real-time monitoring.
- **Poor UX/UI Design** - Complex navigation, confusing workflows, and lack of intuitive design reduce user adoption and engagement, especially among elderly or less tech-savvy patients.
- **Privacy, Security & Compliance Risks:** Inadequate data protection, access control issues, and missing consent tracking put patient trust and regulatory compliance at risk.
- **Engagement Drop-off and Retention Issues:** Many users download wellness apps but show high churn after initial use due to lack of sustained motivation or personalization.

Interview Prep Tips

In this sector, you can look out to the following:

- **Engagement Drop-offs:** Wellness apps see steep usage decline after the first month. Diagnose friction points (complex flows, fatigue) and propose product levers. Think gamification and habit-building nudges.
- **Monetization Barriers:** High consult volumes on Practo aren't converting into medicine orders. Diagnose drop-off points and design levers to capture downstream revenue.
- **Elderly Adoption Gap:** Senior citizens are most in need of digital health support but often can't use the apps. Solve with simplified UI, voice support, and caregiver-linked accounts.
- **Data Fragmentation & Interoperability:** Patients juggle multiple apps (consults, fitness, chronic care) while providers manage siloed systems (labs, wearables, EHRs), leading to broken journeys and app fatigue. Diagnose integration gaps and propose unified health hub with interoperable standards to deliver seamless, continuous care.

Prioritize accessibility, reduce friction, and connect fragmented experiences to scale health tech products beyond a simple aggregator model.

Sector Evolution Timeline



Key Players

Global

coursera



1. Coursera
2. Udemy
3. Duolingo



India



upGrad



1. PhysicsWallah (PW)
2. upGrad
3. Unacademy
4. Vedantu
5. Simplilearn




Common Features and Product Norms

- **Pricing Models:** Freemium, Subscription, One-time Cohort Pricing
- **Delivery Format: Hybrid** Learning is the norm (Live Classes + Recorded Content + Offline Centers).
- **Engagement Levers:** Gamification (leaderboards, points), Community (peer groups), and 24/7 Doubt-Solving (AI + Human TAs).
- **Tech Enablers:** AI Tutors, Adaptive Testing, Vernacular-First UI/UX and Scalable architecture.
- **Product Focus:** A shift from content delivery to **measurable learning outcomes** and career success

Business Model Canvas

- **Value Propositions:** Affordability for Tier-2/3, tangible career outcomes, AI-powered personalization, and trust via hybrid models.
- **Channels:** Digital marketing (SEO, YouTube), direct sales teams, offline centers, and organic referrals.
- **Revenue Streams:** B2C subscriptions & course fees, B2B SaaS licenses, and outcome-linked models (Income Share Agreements - students pay as a %age of their future income after they get a job).
- **Key Resources:** Brand & star educators, proprietary tech, content library, and physical offline network.
- **Key Partnerships:** Universities (for accreditation), corporates (for upskilling), and schools (for distribution).
- **Cost Structure:** Dominated by Customer Acquisition Cost (CAC), followed by employee salaries, content development, and tech infrastructure.

Key Use Cases

- **K-12 Supplementary Learning:** After-school tutoring and concept clarification.
- **Test Preparation:** Coaching for competitive exams (JEE, NEET, UPSC).
- **Professional Upskilling:** Acquiring new skills for career growth (e.g., Data Science, Digital Marketing).
- **Higher Education:** Pursuing accredited online degrees and certifications.

User Journey and Pain Points		
User Journey: 1. Discover → 2. Evaluate → 3. Learn → 4. Complete → 5. Outcome		
Stage	Pain Point	Opportunity
Discover	Information overload, lack of trust, confusing choices.	Build trust through reviews, referrals, testimonials, and quality content.
Evaluate	Unclear pricing, weak demos, refund concerns.	Offer free trials, transparent pricing, and interactive demos.
Learn	Low motivation, digital fatigue, slow doubt resolution.	Improve engagement with gamification, AI tutors, and community learning.
Complete	Course drop-offs and inconsistent learning habits.	Use reminders, personalized nudges, and milestone rewards.
Outcome	Limited career support and unclear learning outcomes.	Provide certificates, placement assistance, and progress dashboards.

Key Metrics
Acquisition - CAC - Conversion Rate
Engagement - DAU/MAU - Session Time - Course Completion %
Retention - D7/D30 Retention - Churn
Business - LTV/CAC - ARPU - NPS

Macro Trends & Opportunities (2025)

- **AI-First Learning:** Personalized tutors, adaptive learning, and automated content.
 - *Opportunity:* Build AI-native learning experiences.
- **Outcome-Based Education:** Focus shifting from content consumption to measurable career outcomes.
 - *Opportunity:* Improve placements, certifications, and skill validation.
- **Vernacular & Tier-2/3 Growth:** Next wave of learners demand affordable, localized education.
 - *Opportunity:* Build vernacular-first, mobile-first products.
- **Hybrid Learning:** Blending self-paced, live, offline, and community learning.
 - *Opportunity:* Create seamless omnichannel experiences.
- **Lifelong Learning & Upskilling:** Continuous learning replacing one-time education.
 - *Opportunity:* Offer modular courses and micro-credentials.
- **Enterprise Learning:** Rising demand for corporate training and workforce upskilling.
 - *Opportunity:* Build B2B learning platforms with analytics and ROI tracking.

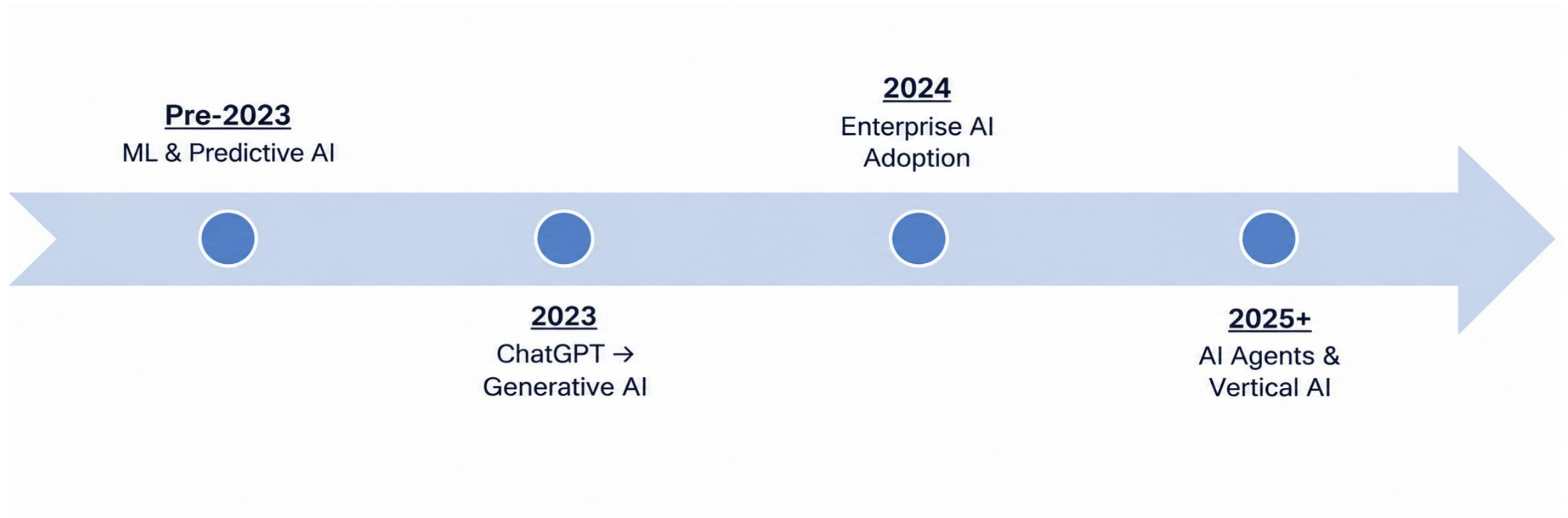
Common Product Pitfalls (with Examples)

- High CAC, weak retention
- Low course completion
- Ignoring learner motivation
- Poor onboarding
- Weak trust & transparency

Typical Interview Questions

- Improve Duolingo retention
- Design AI tutor
- Increase course completion
- Build learning for Tier-2 India
- Improve Coursera monetization

Sector Evolution Timeline, Drivers & Challenges



The AI Technology Stack & Key Players

L1: Infrastructure

Compute GPUs and Cloud.

PM Lens: Cost, Latency & Scalability

Key Players:

Chipmakers - NVIDIA, AMD, Google, Intel

Cloud Providers - AWS, Azure, GCP, Shakti Cloud (Indian)

L2: Foundational Models

The general-purpose LLMs and multimodal models.

PM Lens: Build vs Buy • Closed vs Open • Fine-tune

Key Players:

Global - OpenAI, Gemini, Meta, Anthropic, Mistral, DeepSeek

Indian - Sarvam, Krutrim (Ola), Project Indus (Tech Mahindra)

L3: AI Platform

RAG, vector DBs, orchestration & evaluation

PM Lens: Where trade-offs (latency, cost) are made.

Key Players:

LangChain, LlamaIndex, Pinecone, Weaviate, Databricks

L4: Applications

The end-user AI products (Haptik, Vertical SaaS).

PM Lens: Focus on UX, workflow, integration & adoption.

Key Players:

ChatGPT, GitHub Copilot, Notion AI, Perplexity, Cursor

L5: Enterprise & Services

Deployment, integrations & AI transformation

PM Lens: Focus on GTM partnerships to scale enterprise sales.

Security, governance, GTM.

Key Players:

Indian IT Services Giants: TCS, Infosys, Wipro, HCLTech.

Global Consulting Firms: Accenture, Deloitte, PwC, EY.

Common Features & Product Norms

- **Delivery:** APIs & SDKs are the standard interface for AI capabilities.
- **Interaction:** Natural language chat, copilots, and AI assistants are the dominant UX patterns.
- **Context:** RAG, memory, and context injection improve response quality and personalization. Human
- **Oversight:** Feedback loops, approvals, and fallback workflows are common in enterprise AI.
- **Trust & Safety:** Citations, guardrails, moderation, and evaluations are becoming table stakes.
- **Pricing:** Usage-based pricing (per token/API call) with SaaS subscriptions for enterprise products.

Use Cases & Pain Points

Use Case	Typical User	Pain Point	Opportunity
Writing	Students, marketers, content creators	Blank page, slow drafting	AI-assisted content generation
Search & Research	Knowledge workers, analysts	Information overload	AI answer engine with citations
Coding	Developers, startups	Slow development & debugging	AI coding copilot
Customer Support	SMEs, e-commerce, enterprises	Long response times, high support costs	AI support agents
Data Analysis	PMs, business teams, executives	Manual reporting & dashboarding	Automated insights & summaries

Key AI Metrics

- **Model Performance: Groundedness / Hallucination Rate**, p95 Latency, Accuracy, F1 Score, Eval Coverage.
- **User Engagement:** Task Success Rate, Feature Adoption Rate.
- **Business Metrics:** Cost-Per-Resolved-Task, API Uptime, ROI for customers.

AI Regulation & Ethics

Area	Why it Matters
Privacy	Protect user data and ensure consent for AI usage.
Bias & Fairness	Prevent discriminatory or unfair AI outputs.
Transparency	Clearly disclose AI-generated content and explain decisions where possible.
Security	Safeguard models and sensitive enterprise data.
Compliance	Adhere to regulations like the EU AI Act , GDPR , and emerging national AI policies.

Macro Trends

Macro Trend	Opportunity
AI Agents	Build autonomous workflows that can plan, reason, and execute tasks.
Multimodal AI	Deliver richer experiences using text, image, audio, and video together.
Vertical AI	Create domain-specific AI solutions for industries like healthcare, finance, and legal.
Enterprise AI	Enable secure AI adoption with governance, compliance, and workflow integration.
On-device AI	Provide faster, private, and offline AI experiences on user devices.
Open-source Models	Build cost-effective, customizable AI products with greater control.

AI Trade Offs

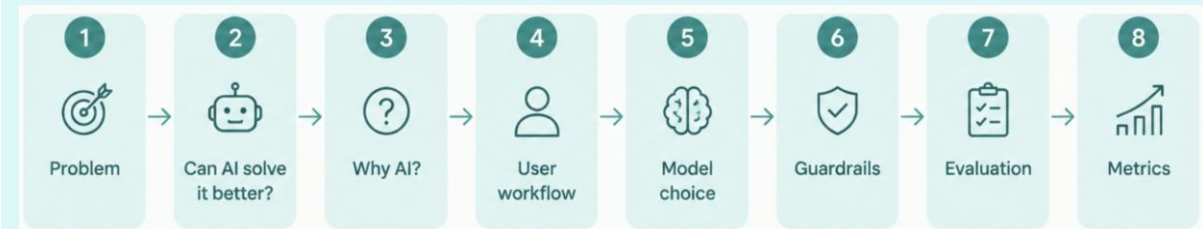
Trade-off	Example
Accuracy vs Latency	Better model vs faster response
Cost vs Quality	GPT-4 vs smaller model
Personalization vs Privacy	Better UX vs user trust

AI Product Pitfalls

- AI without a user problem.
- Hallucinations without safeguards.
- Poor evaluation metrics.
- High inference costs.
- Ignoring human workflows.
- Privacy & compliance gaps

AI PM Framework

When should AI be used?



AI PM Interview Tips

- Start with the user problem - not the model.
- Define success before discussing AI.
- Explain why AI is necessary.
- Discuss guardrails and fallback flows.
- Mention trade-offs (cost, latency, accuracy).
- Define evaluation metrics.

04

Tackling Cases

Introduction

In **Product Management interviews**, guesstimates are used to assess a candidate's skills in....

➤ **Problem Solving**

➤ **Reasoning**

This is vital because, **in real-world scenarios**, precise data on

- market sizes
- consumer preferences
- adoption rates

....is often unavailable.

What should a strong PM do then?

A strong PM must...

Make quick, logical estimates to seize opportunities

Allocate resources

Validate business cases



....Thus, enabling faster, data-driven decisions even amid **uncertainty.**

Framework for Solving Guesstimates

1

Clarify the Objective

Confirm what needs to be estimated -metric, geography, timeframe, and customer segment.

2

Ask Relevant Questions

Reduce ambiguity by clarifying assumptions, constraints, and definitions before starting.

3

Choose the Right Framework

Use the simplest approach: **Top-down, Bottom-up, Supply-side, or Demand-side** - based on the problem.

4

State Assumptions Clearly

Use reasonable assumptions and communicate them before making calculations.

5

Break into Segments

Split the problem into logical buckets (e.g., geography, age, income, customer type, usage) to simplify estimation.

6

Use Round Numbers

Prefer simple numbers for faster calculations and fewer arithmetic errors.

7

Sanity Check Your Answer

Validate the estimate using another method or compare it with real-world benchmarks.

A structured approach and logical assumptions matter more than arriving at the exact number.

Key Reference Data for Guesstimates (Specific to India only)



You can refer to the below figures that are updated rough estimates which are rounded for ease of solving calculations during guesstimates:

Population and Area

- Total Population - 1.45 billion
- Total Area - 3.3 Mn sq. km
- Avg. people/ household - 4
- Smartphone Users - **~800-900 Million**
- Internet Users - **~900 Million**

Gender Split

- Male - 50%
- Female - 50%

Rural Urban Split

- Rural - 60%
- Urban - 40%

Age Split

- 0-20 - 40%
- 21-40 - 30%
- 41-60 - 20%
- 60+ - 10%

Income Split

- Low - 50%
- Middle - 30%
- Upper Middle - 15%
- High - 5%

Major Cities (Population)

- Delhi - 2%
- Mumbai - 1.5%
- Bangalore - 0.9%
- Kolkata - 1%
- Hyderabad - 0.9%

Approach Tips	
Product Design interviews are generally asked in two ways....	
➤ Designing a new product	➤ Improving an existing product

Both of these questions can be answered using the **CIRCLES framework**. CIRCLES is a very powerful way to structure your approach. **Do not mention it explicitly, but rather direct the interview accordingly.**

Importance of a Customer Journey		
✓ Always ensure to come up with the Customer Journey and verify with the interviewer.	✓ This helps structure the interview better , but it requires some user POV thinking.	✓ Be very minute in the steps that the user is taking and then come up with a Journey.

The CIRCLES Framework

C

Context

- ❖ **The first should be to understand more about the product, the company, what their objectives are etc.**
- ❖ **Ask 3-4 questions to the interviewer.**
Also, if the product is well known, give a brief about your understanding of the product and then ask for clarification from the interviewer.

I

Identify User Personas

- ❖ **Identify 2-3 user personas which could use your product.** You should try to make the persona as relatable as possible. Talk about the persona's demographics, their psychographics etc.
- ❖ **Give the persona a name as well.** For example, Aditya, a Sports enthusiast, looks for new sports gear online and wants the best quality products.
- ❖ **Select a persona** you would like to build your product for. Give your reasons for selecting this persona to the interviewer.

The CIRCLES Framework

R

Report Customer Needs

- ❖ You need to identify the needs and pain points for your selected persona.
- ❖ This is the step where you would **build the user journey** for the selected user persona. Talk about the exact points in the journey where we see maximum friction/issues for the customer.
- ❖ In the end, you should have **3 crisp problems** that you see for the user.

C

Cut Through Prioritization

- ❖ You would likely find a lot of needs and pain points across the user journey.
- ❖ **Prioritize a few needs and pain points you would like to solve right now.**
- ❖ Give your reasoning to the interviewer. You can **evaluate the pain points** in the lens of impact (business or customer experience), resources required to build and the complexity of solution.

The CIRCLES Framework

L

List Solutions

- ❖ **Try to be detailed with your solution.** The more specific you are, the more relatable the solution becomes.
- ❖ Do not focus on the technical details too much. **Focus on the product story** and how the solution fixes issues for the customer. Clearly articulate each use case that would be solved.
- ❖ **Come up with 2-3 solutions for the selected problem.** Always specify the most obvious solution and then talk about fancier options. Do not over engineer at this stage.

E

Evaluate Tradeoffs

- ❖ Talk about **why a certain solution is better than all others.** You can use similar lens as Cut Through Prioritization to substantiate your solution.

S

Summarize using KPIs

- ❖ **Come up with metrics to measure your solution.** There should be one north star metric to measure the overall objective of the product and a few metrics to measure the success of your features. You can also evaluate trade-offs by thinking of metrics that might go down due to this and whether the new feature is still worth it.

The CIRCLES Framework

Summarizing the CIRCLES Framework

1. **Comprehend situation** (What? Who? Why? And How?)
2. **Identify customer** (different personas)
3. **Report customer needs** (as a customer, I want to..., so that I can ...)
4. **Cut through prioritization**
5. **List solutions**
6. **Evaluate trade-offs**
7. **Summarize recommendations**

The Do's in Product Design

1. Structure your approach

- Outline your approach before jumping into solutions.
- Walk the interviewer through each step.
- Keep your thinking logical and organized.

2. Build around the user

- Define clear personas.
- Map the customer journey.
- Capture pain points using user stories (*As a..., I want..., so that...*).

3. Prioritize before solving

- Prioritize problems based on impact and feasibility.
- Brainstorm multiple solution options.
- Explain why you chose the final solution.

4. Measure success

- Define a North Star Metric.
- Add 2-3 supporting metrics.
- Mention possible trade-offs and risks.

The Don'ts in Product Design

1. Don't jump to features

- Understand the problem before ideating.
- Avoid proposing solutions without validating user needs.

2. Don't design for everyone out the framework

- Avoid generic personas.
- Focus on one target user and their key pain points.

3. Don't over-engineer

- Start with simple, practical solutions.
- Use AI/ML only when it clearly adds value.
- Don't ignore obvious solutions.

4. Don't ignore trade-offs

- Don't defend only one idea.
- Discuss alternatives, limitations, and success metrics.
- Avoid ending without a recommendation.

What is GTM?

- ❖ GTM strategies are action plans for a company to launch a new product/functionality in areas where the product is currently not present. These may be regions where the company is already operating with a different set of products, or it might be an untapped region for the company.
- ❖ GTM strategies entail that the product reaches the intended audience at the right time through the proper channels and solves the right use cases to fulfill the company expectations for that product.
- ❖ The aim of a GTM case is to ensure success for a product launched by a company. It requires devising plans to support customer awareness, acquisition and retention both before and after the product is launched.
- ❖ Customers should be made clear of the value proposition of the product, and the right marketing should be done to capture the intended target audience.

Why is GTM important?

- ❖ As Product Managers, you are responsible for the success of your product. This includes creating the strategy for product targeting and communication, ensuring that the intended target audience is attracted to your product and will be retained as users, ensuring product success.
- ❖ GTM cases may especially be asked in PMM (Product Marketing Manager) roles, where the job responsibility has a large overlap with the kind of strategic thinking required for GTM cases. GTM may also be asked as follow-ups to Product Design questions, testing whether you have developed a comprehensive outlook over your product functionalities and whether it will be useful to the target audience.

Approaching GTM Questions

Define the WHAT

Define the Product

- ❖ To start a GTM case, you can ask clarifying questions regarding the product to be launched. More specifically, gather information about...

1

Product functionalities

- What does the product do and how does it deliver value to customers?
- Any features that need to be highlighted?
- Is there a unique value proposition that separates us from competitors?

2

Product History

- Has the product been launched in other regions?
- Where does the product sit in the company portfolio?

Define the Company

- ❖ Get an overview of the company, what it specializes in and what are its aims from the product launch
- Are there other products launched in the same market?
- Are there any possible product synergies from other products?

Define the Market Landscape

- Are there other competitors operating in the same space we are trying to enter?
- What kind of a market is this? Is this a D2C market or a B2B space?
- What is the market size and distribution of market share by competitors?

Approaching GTM Questions

Define the WHY

Why is the company launching this product? What are their expectations from the product launch?

Define the WHO

Understand the customers

Understanding the customer persona is **extremely important** for product cases.

Spend a good amount of time clarifying the customer pain points, the market gaps and exploring potential opportunities to create a value differentiation for customers.

Customer persona can be understood through exploring these dimensions.

- Who is our target segment that we want to sell the product to?
 - What are the dimensions of their demographic data (age, gender, working/students etc.)
 - What are the major pain points in the product marketplace that we can possibly solve?
- What value proposition is delivered to customers through the usage of our product
- What are the customer sentiments associated with the company's image and products?

Approaching GTM Questions

Define the HOW

Deliver a Clear Value Proposition for our Customers

On understanding the customer pain points and the features of the product, clearly strategize what product features you would like to highlight when promoting the product. These features can be:

- **Novel offerings not present in the market that solve a unique customer pain point**
- **An improvement over similar functionalities delivered by existing products in the market**

A common way of delivering a value proposition follows the below format:

“For (target customer) who (statement of need or opportunity), our (product name) is a (product category) that (key benefit, compelling reason to buy). Unlike (primary competitive alternative), our product (key differentiator).”

Approaching GTM Questions

Define the HOW

Develop a Pricing Strategy

Design a pricing model to attract customers and help the company turn a profit. Common pricing strategies include:

1. Value-Based Pricing

This is commonly used when offering a novel functionality or solving a previously unaddressed pain point. Price should be set close to the perceived value of the functionality for customers. Value-Based Pricing can be calculated as:

Value of the utility that customers gained by using the product + %age premium

OR

Cost paid by customers to solve the problem through alternate means + %age premium

Approaching GTM Questions

Define the HOW

Develop a Pricing Strategy

2. Competitor Based Pricing

Used in price-sensitive markets, competitive pricing is used to gain market share while balancing profitability for the company. This is used when our product is similar to other competitive products in the markets in the eyes of a customer.

To calculate a competitor-based price:

- 1. Calculate the average of price of similar competitors in the market***
- 2. Price a %age above or below the average price, depending on whether you want to prioritize market share or margins***

Approaching GTM Questions

Define the HOW

Develop a Pricing Strategy

3. Cost-Plus Pricing

Usually used when you are trying to offload a product in the market that is not selling well.

Cost-plus pricing should generally be avoided in GTM cases.

To calculate a cost-plus price:

cost-plus price = Total cost of product development*(1+markup %age)

Approaching GTM Questions

Define the HOW

Develop a Pricing Strategy

4. Dynamic Pricing

Dynamic Pricing is primarily used for services offered online where the prices are declared on customer request.

It balances customer demand and competitor pricing to maximize profit for the company.

It uses technologies like machine learning to find the optimal price for a given situation.

Approaching GTM Questions

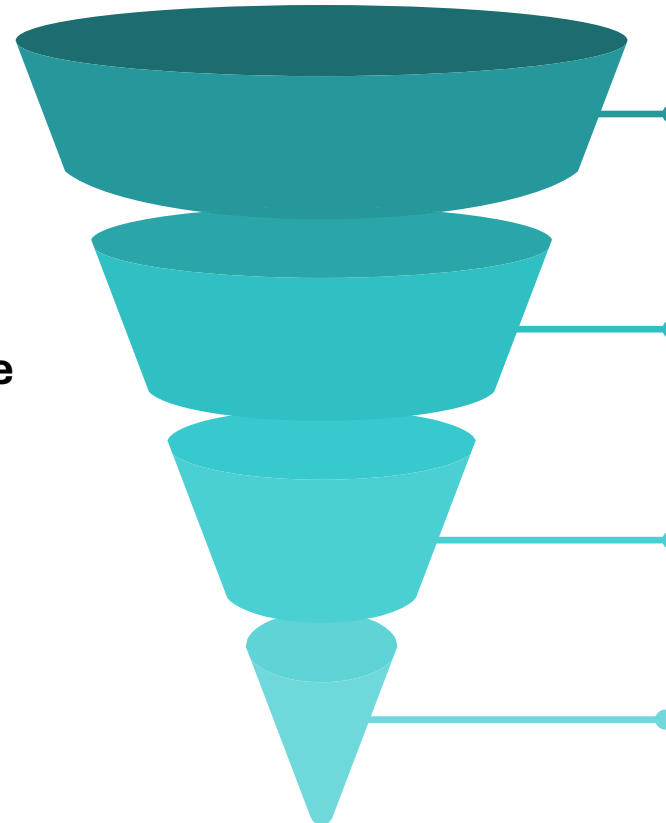
Define the HOW

Develop a Marketing Strategy

Discuss marketing tactics that expose the intended target audience to the appropriate product functionalities.

This involved choosing the marketing themes and choosing the appropriate channels **where the concentration of our target audience is high.**

Appropriate marketing strategies can be developed by choosing the right channels at each stage of the **marketing funnel.**



Awareness: Publicize the product to reach high customer awareness. **Eg:** paid ads, social media marketing etc

Interest: Develop strategies to build customer incentive in using the product and increasing first time users. **Eg:** e-mail campaigns

Decision: use insights to target likely buyers. **Eg:** sales enablement tools

Adoption: Develop after-sales service, onboarding tutorials etc to ensure customer satisfaction

Approaching GTM Questions			
Define the HOW			
Develop a Launch Strategy			
Pre-Launch	Choose the right Metrics		
- Develop a pre-launch strategy, choosing the appropriate pricing and marketing options to ensure customer exposure to the launched product. - Your main goal here should be to increase customer awareness in the target audience at the lowest cost possible, by utilizing the right marketing channels	Adopt the right set of metrics that are aligned with the product launch business goals. For example:		
	Customer satisfaction score	Product Adoption Rates	Daily Active Users
			Customer Acquisition Costs
Post-Launch	Evaluate Risk Factors		
- After customer adoption and product launch, focus on developing strategies to ensure customer retention and customer satisfaction. - Develop the appropriate strategy to ensure customer redressals and reduce drop out rates.	Address the risks that may be associated with the product launch and adoption. These risks may be:		
	External Risk Factors	Use PESTEL	Product Related Risks
			Switching costs, Exit costs, etc.

"If you can't measure it, you can't improve it." - Peter Drucker

Why Product Metrics?

Metrics are the objective truth that transform opinions into facts, guiding every decision you make from feature prioritization to strategic pivots. From a recruiter's perspective, metrics test a candidate's skill at

- Connecting product changes to quantitative metrics
- Understanding the impact that metrics have on one another
- Prioritizing based on data.

❖ **Start here:** You can watch this video (<https://www.youtube.com/watch?v=wtDFgx41Rho>) to get familiar with some common metrics.

The Measurement Hierarchy: OKRs, KPIs, and Metrics

It's easy to get lost in the jargon. We can break these down, as if you were a taxi driver.

OKRs (Objectives & Key Results)

- **Objective:** Drive from Mumbai to Goa for a vacation.
- **Key Results:**
 - Arrive before 8 PM.
 - Keep fuel costs under ₹5,000.
 - Get a 5-star rating from your passengers.

KPIs (Key Performance Indicators):

These are the critical dials on your dashboard you must watch to ensure you reach your destination successfully.

Examples: Fuel Level, Average Speed

Metrics:

These are all the quantifiable, measurable data points available to you.

Examples: Current RPM, tire pressure, outside temperature, odometer reading. They provide context but aren't all key indicators for this specific trip's success.

As a PM, you set ambitious OKRs, track your most critical KPIs, and monitor a broader set of metrics to understand the full picture.

Finding Your North Star Metric (NSM)

A PM may track many metrics, but the North Star Metric is the single most important one. It best captures the **core value** your product delivers. An ideal North Star metric should:

- Lead to revenue
- Reflect customer value
- Measure progress.

Examples - Spotify (“Time Spent Listening”), Airbnb (“Nights Booked”)

Must Reads

[NSMs used by growth-stage companies](#) - Real-world NSM examples across industries.

[Shreyas Doshi on Basic Product Metrics](#) - Fundamental metrics every PM should know.

How to Ace Any Product Metrics Interview Question

The chances of an interviewer asking a metrics-based question are low. However, knowing the process can help you to improve your product design, RCA skills. That being said, a structural way to tackle a product metric question could be:

When asked to define success metrics for a product (e.g., "What are the success metrics for Instagram Reels?")

1. Clarify the Goal & Context

First, ask clarifying questions.

"What is the primary objective of this product? Are we focused on growth, engagement, or revenue? Who are the target users?"

2. Map the User Journey

- Briefly outline the key user actions from discovery to becoming a power user.
- Use frameworks like AARRR here.

3. Analyze Metrics for Each Stage

List potential metrics for each step of the journey (Acquisition, Activation, etc.).

4. Prioritize & Define the North Star

- Pick the 3-4 most critical KPIs.
- Commit to 1-2 North Star Metrics that best capture the core value of the product.
- Justify your choice.

5. Discuss Counter-Metrics: Trade-off Awareness

*E.g. "As a product manager at YouTube, you may want to increase the number of ads shown to drive **Revenue** (KPI), but you must monitor the Churn Rate (counter-metric) to ensure that it's not hurting the user experience and long-term **Retention**."*

As a PM, your goal is to move your thinking up the strategic ladder. The three core strategies represent an evolution in strategic thinking.

Level 1: Cost-Plus Pricing (The Foundation)	Level 2: Competitor-Based Pricing (The Market)	Level 3: Value-Based Pricing (The Goal)
• The "How": You calculate your costs (development, marketing, support) and add a markup. • The PM's Reality: This is your floor. • You must know your costs to ensure profitability. But pricing based only on your internal costs is lazy. It completely ignores the customer's perspective and the market dynamics. • Never stop here.	• The "How": You anchor your price to what your competitors are charging. • The PM's Reality: This is table stakes. • You must know your competitors' pricing to position your product effectively. However, this assumes your competitors have priced correctly and that your product is directly comparable. Relying on this is a reactive strategy that leads to "me-too" pricing and price wars.	• The "How": You price based on the quantifiable economic value your product delivers to a specific customer segment. • The PM's Reality: This is your north star. It forces you to answer : "How much better, faster, or cheaper does my product make my customer's life?" • It decouples your price from your costs and your competitors, anchoring it instead to the value you create. This is the hardest strategy to execute, but the most profitable and defensible. Hence, we see companies moving towards Dynamic Pricing.

Section	Key Questions	Why It Matters for a PM
Business Objective	Why are we pricing this? Are we aiming for market share, profit maximization, or user acquisition?	This sets the strategic direction for every subsequent decision.
Target Customer Segment	Who is the specific group of users we are creating the most value for? What are their characteristics?	You cannot determine value until you know who you're delivering it to. Be ruthlessly specific.
Quantifiable Value	What is the next best alternative for our customer? What is our unique differentiation? What is the dollar value of that differentiation?	This is the heart of value-based pricing. You must translate features into dollars (e.g., "\$X saved per month," "Y% increase in revenue").
Willingness to Pay (WTP)	What is the customer's perceived value and ability to pay? How does this change based on their urgency or the opportunity cost of doing nothing?	This is the psychological ceiling. Your price must be less than or equal to the customer's WTP to drive a purchase.
Pricing Model & Tactic	How will we charge? (Subscription, Usage-Based, etc.) What tactics will we use? (Freemium, Tiered, etc.)	This is the implementation. The model should align with how your customer gets value over time.

Pricing Tactics

Model	When a PM Should Use This	Key PM Considerations & Risks
Freemium	When your product has low marginal costs and benefits from network effects or a "try-before-you-buy" experience.	The Conversion Trap: The free version must be good enough to attract users but "painful" enough to drive upgrades. You risk cannibalizing your paid product if the free offering is too generous.
Tiered (Feature Bundling)	When you have distinct customer segments with different needs and WTP (e.g., Individual, Team, Enterprise). This can also be used for decoy pricing. E.g. introduce a third option to make one of your other options look much more attractive.	The "Right" Features: The biggest challenge is correctly assigning features to tiers. Your goal is to make each tier a compelling upgrade, not a frustrating limitation. Too many choices can overwhelm users and lead to no decision at all.
Usage-Based	When a customer's consumption of your product directly correlates with the value they receive (e.g., cloud computing, SMS APIs).	Predictability vs. Value: Customers love the fairness but hate unpredictable bills. Consider hybrid models (a base subscription + overage charges) to offer the best of both worlds.

Pricing Tactics		
Model	When a PM Should Use This	Key PM Considerations & Risks
Per-User	The standard SaaS model. It's simple to understand and forecast.	The Collaboration Penalty: This model can discourage adoption within a company (e.g., "I don't want to add my colleague because it will cost more"). Consider "active user" pricing or team-based bundles.
Market Skimming (Based on Product Lifecycle)	When launching a highly anticipated, innovative product with brand equity. The strategy is to set a high initial price to "skim" the maximum revenue from early adopters, and then gradually lower the price over time to attract more price-sensitive customer segments.	Brand & Differentiation are Crucial: This only works if your product is perceived as highly desirable and unique. Without strong brand loyalty, customers will simply wait for a cheaper alternative. The Alienation Risk: Early adopters who paid a premium can feel cheated if the price falls too quickly, potentially damaging brand loyalty for future launches.

Step 1: Frame the Problem & Validate the Data (Ask Clarifying Questions)	Step 2: Investigate Internal Factors	Step 3: Investigate External Factors
Before RCA, Know the Product: Understand what the product does 1. Reiterate & Define the Metric - Clarify what the metric means and how it’s calculated. - Check if calculation or reporting methods have changed recently. - Verify data integrity - confirm BI tools and data pipelines are working correctly. 2. Isolate the “Blast Radius” Time Period: Identify if it’s a sudden drop or gradual decline, check for seasonality, and pinpoint when it started. Geography: Is the impact global or specific to certain regions? User Segment: Determine if all users are affected or specific personas - e.g., new vs. existing users, or by platform (Android vs. iOS).	1. Trace the User Journey - Map the whole user flow in the app. - Identify the exact funnel stage where the drop/change occurs (e.g., sign-up, checkout, key feature use). 2. Product & Technology Changes - Check for recent UI/UX releases that could cause confusion or friction. - Investigate technical issues - bugs, glitches, latency, crashes. 3. Business & Offering Changes Demand-Side: Review changes to pricing, subscription tiers, and free trials. Supply-Side: Check for salary shifts, service quality, or cannibalizing products pulling users away. 4. Qualitative User Feedback - Analyze app store reviews, NPS scores, and support tickets. - Look for patterns or complaints that align with the timing of the change.	<i>(If internal causes are ruled out, widen the lens)</i> 1. Competitive Landscape - Identify new entrants or competitive moves. - Check for price cuts, promotions, or better offers from competitors. - Look for cheaper substitutes entering the market. 2. Market & Environment - Review regulatory or legal changes affecting the product. - Consider environmental events (pandemics, disasters) that could shift user behavior. - Spot shifts in consumer psychology or buying patterns. 3. Public Perception - Scan PR coverage and social media sentiment. - Investigate any trust or brand reputation issues.

Tips for Aspiring PMs (Interview Execution)

1. *Think Out Loud, Skip the Jargon*

Walk through your logic step-by-step.

Example: say “*First, I’d clarify the metric...*” instead of naming the framework.

2. *State Assumptions Clearly*

Acknowledge data gaps.

Example: “*I’ll assume this affects all platforms equally, but I’d verify with the data analyst.*”

3. *Be a Collaborative Partner*

Frame your investigation as **questions + hypotheses**.

Let the interviewer guide where needed.

4. *Link to Product Lifecycle*

Adapt your approach based on stage (Launch, Growth, Maturity).

Example: “*Since we’re in Maturity, I’d focus on competitor actions and retention.*”

5. *Prepare for the “What’s Next?”*

After RCA, propose a solution.
Outline **how to test it** and **how to measure success**.

05

Practice Cases

Section 5.1

Guesstimates

Estimate the number of Instagram uploads occurring in India.

A few clarifying questions first. Should I estimate uploads per day or per year, and should "uploads" include posts, stories, or both?

Consider only posts and estimate uploads per day.

Should I consider both personal and business accounts?

Personal accounts only for now.

Can I assume this estimate is for a typical day, not a festival or a holiday?

You can assume a typical day and start working on your approach.

I'll break the problem down as:

Instagram uploads per day = active personal Instagram users × average posts per user per day.

First, active personal users:

active users = population × % with smartphone and internet × % on Instagram × % monthly active

India's population is about 1.4 billion. Assuming 70% smartphone-and-internet penetration:

1.4 billion × 70% = 980 million smartphone users with internet.

Should I take Instagram penetration across all smartphone users, or segment by app usage?

What do you think?

Instagram is among the most downloaded apps on the Play Store, with over a billion downloads, so I'll assume around 50% of smartphone-internet users have it.

Sure, good point. For simplicity, assume 50% penetration among smartphone users and feel free to make reasonable assumptions.

Thanks, assuming 50% of these people use Instagram:

$980\text{M} \times 50\% = 490\text{M}$ accounts

I'll assume that 80% of these accounts are active personal accounts:

$490\text{M} \times 80\% = 400\text{M}$

So, we have around 400 M active personal users.

Sounds reasonable.

Now, I'll estimate the average number of uploads per person per day. We can divide the monthly active users into three categories, with rough estimates for the percentage of users in each category from my own experience on the app as follows:

1. Light users: 60% of users, posting one post every month = $0.6 \times 400 = 240$ M users
2. Moderate users: 35% of users, posting one post every week = $0.35 \times 400 = 140$ M users
3. Heavy users: 5% of users, posting one post every 2 days = $0.5 \times 400 = 20$ M users

Therefore, the number of posts per day = $240\text{M} \times (1/30) + 140\text{M} \times (1/7) + 20\text{M} \times (1/2)$
 $= 8\text{M} + 20\text{M} + 10\text{M}$
 $= 38$ M

That sounds good. Anything else? I think Instagram has around 100 million daily posts.

Then India contributing around 38 million close to a third of the global total is in a plausible range, given India's large user base and the fact that Instagram is blocked in China.

Great, thank you. We can close this interview.

Estimate the revenue that YouTube makes in a day.

Sure! I'll focus on YouTube's daily ad revenue. YouTube has several revenue streams ads, Premium, Music, TV and Super Chats but ads contribute the majority, so I'll concentrate on those. Should I estimate globally or for a specific market?

Let's estimate YouTube's daily ad revenue specifically in the United States.

Perfect. Here's the approach I'll take:

YouTube daily ad revenue = ad views × revenue per ad view, where

- ad views = total video views × % of videos carrying ads
- revenue per ad view = CPM ÷ 1,000

I'll estimate total views from the demand side how many videos people watch per day rather than the supply side, since YouTube can scale its infrastructure to meet demand, so capacity isn't the constraint. Does that sound fair?

Alright, that does sound like a plan. Let's go ahead.

I'll take a US population of about 330 million, rounding to 300 million for cleaner segmentation, and estimate daily viewing behaviour by age group:

Age Group	% of Population	Viewership rate	Daily Users (Million)	Avg. Videos per day	Daily Views (Million)
0-5	10%	0%	0	0	0
6-12	10%	50%	15	5	75
13-24	20%	90%	54	20	1080
24-44	20%	85%	51	10	510
45-64	20%	60%	36	5	180
65+	20%	40%	24	3	72

Total daily views ≈ 1,917 million ≈ 2 billion.

Not all views are monetized:

1. Some users have YouTube Premium (assume ~20% in US)
2. Some creators don't monetize
3. Some content has skippable or no ads

So, I'll assume 50% of views are monetized with ads.

Ad Views per Day ≈ 2B × 50% = 1B ad views

That sounds good, lets move forward.

YouTube charges advertisers via CPM (Cost per 1000 Impressions).

In the US, CPM varies by content and targeting. For this case, lets assume a CPM of \$20/1000 impressions.

Revenue per ad view = \$0.02/ad view.

Hence, daily revenue = 0.02 * 1B = \$20M.

For the year, we have \$20M * 365 = \$7.3 B

Anything else you would like to add?

Google's annual ad revenue = \$200B. Assuming YT gives 10-15% of that, we have global YouTube revenues to be around \$20-30B. Hence, assuming USA gives \$7.3B is a fair estimate.

Okay, we can close the interview.

Estimate Zoom's daily server usage

I'd like to ask some clarifying questions. My understanding is that Zoom's server usage would comprise the following:

1. Storing information about its users and their call logs
2. Streaming the data (audio, video and text) to and from participants in a zoom call
3. Miscellaneous processing tasks (backend)

Since most usage spikes are likely tied to live calls, especially video streams, I'll focus on streaming for now, the most significant driver of infra load. Shall I proceed with this assumption?

Yes, that sounds reasonable.

Do you want me to account for global usage or be constrained to a particular geography, such as India?

Consider the usage for India.

There are three broad categories of users for video conferencing.

1. Personal
2. Student
3. Business

Business accounts should be the most prominent, given higher average usage time, many post-COVID roles carry an important remote-working component, so a greater share of office time is spent in video conferences. Shall I focus on business accounts?

You can go ahead with business accounts for now.

Among business accounts, should I account for event streaming, professional calls between employees or both? If we were to include webinars or events, the calculation would change, as the data pattern is one-to-many, which is a different kind of server load.

We can proceed with professional calls for now.

I will break down this problem as follows:

$\text{Zoom's daily server usage (on a weekday)} = \text{number of people using video conferencing (VC) in India} \times \text{Zoom's market share} \times \text{average time spent on VC} \times \text{data required per hour}$

To estimate the number of people using video conferencing, I will take the share of the working population in India (predominantly aged between 20-60 years), which is 60%. I assume that VC is mostly used by white collared employees in India, whose share in the working population is around 10%.

To estimate the proportion of the white-collar workforce using VC, I will break down the jobs into client-facing and traditional roles:

1. About 40% are in tech, consulting, or client-facing roles where VC is essential (90% adoption).
2. The other 60% are in support of traditional roles with less need (let's say 40% adoption).

This gives us a blended average of $(0.4 \times 0.9) + (0.6 \times 0.4) = 0.36 + 0.24 = 60\%$. The number of people using VC in India comes out as:

$\text{India's population} \times \text{Share of working population} \times \text{Share of white collared employees} \times \text{Share of employees requiring VC services}$

$= 1.4 \text{ billion} \times 60\% \times 10\% \times 60\% \approx 50 \text{ million}$

$= 5 \text{ Cr (approx.)}$

Sounds reasonable. Please go ahead.

In the video conferencing market, there are three major players that I can think of: Zoom, Google Meet and Microsoft Teams. Exact market share might fluctuate by region, but I'll take 50% as a working estimate for India. To estimate the average time spent on VC services, I will break down the users as follows:

1. Heavy users (VC usage of more than 4 hours): 20% - This set will comprise executives, managers
2. Medium users (VC usage of around 2 hours): 40% - This set will comprise engineering managers, marketing and brand teams
3. Light users (VC usage of around 30 mins): 40% - This set will comprise core engineering teams

The average usage comes out as: $20\% \times 4\text{hrs} + 40\% \times 2\text{hrs} + 40\% \times 0.5\text{hrs} = 1.8\text{hrs}$

Further, I will assume that the data requirement of a VC is 1Mbps, which makes it 3600 Mb per hour or 450 MB. Zoom's daily server usage = $50 \text{ million users} \times 50\% \text{ market share} \times 1.8 \text{ hours} \times 450 \text{ MB/hour} \approx 20,000 \text{ TB/day}$.

Note: The data requirement can also be a weighted average based on video quality and usage. However, for simplicity it has been taken as 1 Mbps

Before we end, I have one more question for you. If Zoom observes rising VC usage in Tier 2/3 cities or heavy mobile participation, what should it do?

Users from those segments are less likely to have high-quality, reliable internet. Zoom should consider a "lite" version that lowers the data requirement below the current 1 Mbps, so the experience stays usable on constrained connections.

Okay, we can end here, thank you.

Estimate the number of users for a food delivery app in a South American country.

Sure, I have a couple of clarifying questions before I start the estimation. What is the total population of the country, and should I assume this is a product in its early stage or already launched?

The country has a population of 100 million. You can assume the platform is already launched and is looking to scale. Go ahead with the assumptions you need.

I'll assume the app has achieved some market penetration and is targeting urban consumers first, as they have better access to infrastructure, smartphones and restaurants. I'll also assume a significant share of users are urban working professionals or students, who drive demand for food delivery.

Seems logical. You can proceed with this assumption.

Thank you. Before I begin with the calculation, I'd like to outline my segmentation framework:

1. **Urban vs rural:** given infrastructure and internet constraints in rural areas, adoption will concentrate in cities.
 2. **Age:** those aged 18-40 are the most likely users, given lifestyle and disposable income.
 3. **Income and digital access:** smartphone ownership, internet and digital payments as enablers.
 4. **Food-delivery propensity:** the share of the digitally-enabled group likely to use delivery.
- Are these factors exhaustive?

That approach works. Assume the slicing percentages as you see fit.

Great. I'll proceed with these assumptions:

1. **Urban population:** Assume 70% of 100M = 70 million
2. **Age Distribution** in urban areas:
 - i. 0-17: 25%
 - ii. 18-40: 50%
 - iii. 41+: 25%

So, 18-40 urban population = 50% of 70M = **35 million**

(I'm treating 18-40 as the core users; the 41+ group does order delivery too, at a lower rate, so this funnel is slightly conservative.)

3. **Income and Digital Access** (smartphone + internet + digital payments): Among 18-40 urban population: Assume 70% have sufficient income and access → $0.7 \times 35\text{M} = 24.5\text{ million}$

4. **Food Delivery Propensity:** Of the digitally active population, estimate 40% actively use food delivery platforms (based on competitive landscape and market maturity) → $0.4 \times 24.5\text{M} = 9.8\text{ million}$

Final Estimate: **~9.8 million active users**

That figure seems reasonable. Can you explain your rationale for the 40% adoption assumption?

Certainly. I based this on a few considerations:

1. **Market maturity:** Since the app is already launched and we're looking at a South American country with urban penetration, early adopters would already be on the platform.
2. **Competition:** Multiple food delivery platforms may exist, so active usage may be distributed across providers, reducing the app-specific share.
3. **User behavior:** While many users have access to the service, not all order frequently-some prefer cooking or dining out.

I chose 40% as a mid-point assumption considering these factors.

Makes sense. We can close the discussion.

How many dark stores would a quick-commerce player need in Ahmedabad to guarantee 10-minute delivery across the city?

Three clarifications first. Is this the city, or the full district?

City plus immediate suburbs - the area a player would realistically serve.

A single player, or everyone combined?

A single dominant player, covering the area on its own.

And does the 10 minutes include pick and pack, or is it pure travel?

Full order-to-doorstep window.

Then I will solve this twice.

One limit is coverage - how much ground a single store can reach in 10 minutes. The other is throughput - how many orders that store can handle in its busiest hour.

I need to satisfy both, so the answer is the larger number. May I start with coverage?

Please go ahead.

Path 1 - Coverage

Of the 10 minutes, 3 go to pick, pack and rider handoff. That leaves 7 minutes of travel. For serviceable area I will use 250 sq km.

Ahmedabad municipal limits are considerably larger than that. Justify the number.

Fair. Quick commerce only works where order density supports a store.

That means the denser residential and commercial belt - the western side around SG Highway, Satellite, Bodakdev and Vastrapur, plus the denser central and eastern areas. Not the industrial fringe.

It is also my most load-bearing assumption here, and the coverage answer scales linearly with it. Full municipal coverage would roughly double this number

Accepted. Give me the calculation.

Path 2 - Throughput

Now store capacity, built from the floor up. I will assume 8 pickers on the peak shift, at 15 orders per hour each.

That is 120 orders in the peak hour.

The obvious next step is wrong. Multiplying by a 16-hour day gives 1,920 orders.

But that assumes flat demand, and grocery delivery has a sharp evening rush. The busiest hour carries roughly 15% of the day.

Coverage maths	Value
Travel time inside the promise	7 min
Two-wheeler speed in mixed traffic	20 km/h → 2.3 km road distance
÷ circuitry factor 1.3 (roads bend)	1.8 km straight-line radius
Area per store (πr^2)	≈ 10 sq km
250 ÷ 10, plus ~10% because circles do not tile	≈ 28 stores

Correct - most candidates multiply by opening hours and stop there. Go on

Staffing does not flex minute to minute, so the peak hour is the ceiling.

Effective daily capacity = 120 ÷ 0.15 ≈ 800 orders per day.

Now the demand side.

Demand maths	Value
Ahmedabad urban population ÷ household size 4	~23 lakh households
× 17% active quick-commerce penetration	~3.9 lakh active households
× 0.3 orders per active household per day	~1.2 lakh orders/day, all players
× 35% share for a category leader	~42,000 orders/day for our operator
÷ 800 orders per store per day	≈ 53 stores

Hold on. 0.3 orders per household per day is 9 a month. Defend that for Ahmedabad.

Fair challenge. "Active household" is a wide net - anyone who has ordered recently, including someone topping up twice a month.
Heavy users in SG Highway or Satellite order three or four times a week. But they are a minority. Most of Ahmedabad still does the main grocery run at a kirana.
Blended across the whole active base, 0.3 is defensible. It is also the number I am least confident in, so let me show you what it costs me if I am wrong

Please do

Orders per active household/day	Operator orders/day	Stores needed
0.2 (6 a month)	~27,000	≈ 34
0.3 (9 a month) - base case	~42,000	≈ 53
0.4 (12 a month)	~55,000	≈ 68

A one-third error in that number swings the store count by about 35%.
Penetration, frequency and market share are all multiplicative, so they behave the same way. Store capacity is the only input I built from something observable, so it is the one I would defend hardest.

Fine. Keep going with 0.3.

Reconciling

Your two numbers disagree. 28 against 53. Which one is wrong?

Neither. They measure different constraints, and the gap is the most useful thing here. I have to satisfy both, so the answer is the larger. Demand binds, not geometry. That has a real consequence. The operator is not adding stores to reach farther neighbourhoods; 28 would do that.
It is adding 25 more to stop stores overflowing at peak. So those extra stores cluster in the dense pockets rather than spreading evenly across the map.

Sanity-check 53 for me before we go further.

53 stores across 250 sq km is one every 2.2 km, and one per 1.75 lakh residents. Each does about 790 orders a day, which matches the 800 capacity I built. The two paths reconcile.

Recommendation

So would you build 53?

No. 53 is the theoretical minimum, and I would not build to it.
At 53 stores the network handles 42,000 orders against 42,400 of capacity. That is 99% utilisation with no headroom. One rainy evening or one festival weekend and the peak hour breaks.
Sizing for 80% peak utilisation instead gives 42,000 ÷ 640, or about 66 stores.
So I would plan for roughly 65, and phase them. Open the 28 that give geometric coverage first. Prove store-level economics there. Then add density in the high-order pockets as demand appears, rather than funding 25 half-empty stores up front.

Section 5.2

New Product Design

Design a mobile application for an office space sharing business for Airbnb.

PM Tip: The first step for the candidate is to clarify any ambiguities in the problem statement.

Before moving further, I would like to understand a bit more about the problem at hand. What is the exact business model of this office sharing app? For whom would we be designing the app, the owner or the renter, or both? And lastly, what is the objective of this app.

Good questions. The whole business idea is that people with extra office space can rent out to start-ups or freelancers, Airbnb will take the empty space and remodel it, but the owner will take care of the operations. We need to design the app for both owner and the renter. As of now, the objective is to get as many customers as possible on board.

That's helpful. To avoid ambiguity I'll use two terms consistently: the "host" is the party that lists and supplies the space, and the "renter" is the start-up or freelancer who occupies it. I'll assume monetisation for Airbnb is out of scope for now (to be revisited later), though I'll still handle the payment flow between host and renter. My approach: identify the user segments, find their needs, write user stories, propose solutions, and assemble an MVP.

PM Tip: After understanding the problem statement, the Interviewee clears all his/her assumptions beforehand. Also, he/she gives an overall flow of how he/she is attacking the problem. This helps to keep the interviewer intact with the broad view and aligned with the Interviewee's thoughts. It's always good to first give a high-level view and then go on to the details.

There are 3 users for this application:

1. Owner
2. The person renting it
3. Airbnb

Which one should I focus on?

You may decide that.

For any multi-sided platform, it is important to solve the chicken-egg problem. Taking the example of Uber who first focused on on-boarding the drivers to create supply, I will also focus on owners. Since we are focusing on the owner, we must first define their persona. Mostly, the owner will be a person or a company who have extra space in their building which can be monetized. They will be comfortable in sharing their space to a stranger. If it is a company, this will be an additional source of

money.

Few of the needs of the owners would be:

1. Finding reliable people to rent it to, also the ability to approve or decline requests
2. Furnish and maintain the space with office supplies
3. Estimated rent and information on rental periods and owner's rules
4. Payment and customer support
5. Security of the place

The user needs can be analyzed by navigating through the steps the owner would need to go through the whole process.

So, the needs of the user should be identified at each stage of the process



PM Tip: It's always good to break down the whole process into intermediary stages and figure out the customer in these stages, this helps get the exhaustive list of user needs and avoids missing any important part.

Fair enough. But how will you prioritize among the different needs of the user?

Since we are focusing on MVP, I will look at the most needed features which are 2, 3, and 4. So my user stories as an owner are:

- Keep a check on the office supplies and other furniture, replenish them whenever necessary.
- I want to know the rent I will get so that I can decide whether it is meaningful for me or not.
- A way to interact with the system to confirm/cancel bookings and settlements

ProTip: Prioritizing the requirements is one of the key roles of a product manager. Since it's a boundary-spanning role, product managers are bombarded with multiple tasks of the product, from the business team, from the engineering team, from the marketing team, etc.. Still, the product managers must make hard calls to prioritize the requirements. Some use the effort-impact matrix to do this; however, multiple other frameworks exist.

For an MVP, I will address all the stories individually first.

Solution to story 1:

- Display the list of all the supplies and furniture available in the location.
- Provide the capability to the place orders through the app itself, Airbnb can provide support.
- An ability to display to the renters what all furniture and equipment is present in the office.

Solution to story 2:

- Option for the owner to send the rent to the person renting it
- Rent estimation based on
 - Space occupied
 - No. of days
 - Special requests/Customization
 - Number of people/employees

Solution to Story 3:

- A tab to manage request for renting.
- Option to confirm/cancel or chat with an interested person.
- Have a payment gateway to pay full rent in advance.
- Provide a weekly settlement into the owner's account.
- Cancellations/Refunds will be handled by Airbnb in between settlements

Would you go ahead with all of these?

No, I will evaluate the solutions in each story based on customer impact and its ease of implementation. After the evaluation, I will select 1,2 in S1, 2 in S2, and 1,2 in S3 for the MVP. Do you want me to go into detail about these?

No, this is good. Thanks for your time and all the best.

PM Tip: The Interviewee could have explained here a bit more about how they selected the features for the MVP; however, since the interviewer confirmed, it's fine. It's always better to back up your arguments with data and logic; interviewers are looking for the critical thinking of the Interviewee's ability.

Perugia's voting turnouts have stagnated at 55% in the last few years. The Government of Perugia has hired you to design an application to improve voter turnout. What are some of the features you would recommend for such an application?

Okay. Can I start with a few clarifying questions about the problem?

Sure, go ahead.

Why has turnout stagnated? Are the people skipping from a specific demographic? What are the current voting methods, and is the app meant to be a voting channel itself or only an engagement tool?

Those who skip cite a lack of knowledge of the candidates, low interest in the process, confusion about registration and voting, a sense that their vote doesn't count for much, and being busy on voting day.

The government's awareness campaigns reach only a few. The people skipping are mainly new voters aged 18-30. Voting remains a paper-ballot system and the government doesn't intend to change that. The app should attract more voters and engage them through the election season so they turn out.

Since new voters are the most affected, I'll focus the app on them. The reasons cluster into three problems I can design for: a lack of information across the process, low interest, and the belief that one's vote doesn't matter. The third is important to call out separately, because information and engagement features alone won't shift it.

The remaining reason being busy on voting day I can only mitigate, not solve: voting stays a paper ballot (out of scope), so the app handles it through reminders and helping voters plan their booth visit rather than enabling remote voting. May I take a moment to think through the solution?

Sure, go ahead.

Alright. Keeping the needs in mind, I have come up with the following features in the app:
Addressing lack of information -

- **Candidate Scorecard:** This would be curated using the information from their previous work, their previous election performance, their headlines in the news, criminal and other charges against them. Since we are working with the government, some of this information can be directly sourced from the candidates, and the rest can be gathered from external sources.
- **App-based registration:** By digitizing the voter registration through the app, the voter would be able to register from anywhere using the app, reducing the time and money taken to set up and use an extensive offline registration system.
- **News section:** Capture the headlines to inform the user of the current issues in the news, which would be localized to their voting region.
- **Timeline of the voting process:** What is my booth and navigation to the voting booth.

Addressing lack of interest:

- **Gamification of the voters:** Running an app-based daily/weekly quiz to find the most informed voters. There may be perks associated with that, like preferential voting etc., depending on the rules and laws of the land.
- **Notification center:** Using notifications specifically to keep the voter engaged about the happenings of the process and the critical dates.
- **Connect with the candidates:** The candidates can leverage the app to communicate directly with their voters specific to the region. On the other side, the voters could ask questions and upvote the questions that they find relevant. It would be in the candidates' interest to answer more of the questions asked since it would only increase their reach, and the voters would be more engaged.
- **Non-transferrable badges for voters:** Perks like badges, QR to your exclusive photo at the booth etc., could be incorporated into the app to encourage voting.

Do these look fine, or should I explore more features?

These should do. Could you elaborate on one of the user workflows that you see the user having on the application?

A week before the election, when the app holds the most information: the user opens the app to their profile and gamified score (a percentile of how informed they are), the next quiz, and local news. They take the quiz, then open the candidate tab and select a re-contesting incumbent. The app shows the scorecard, recent work in office, latest headlines, and any legal charges.

The user opens the “work” tab, which details recent parliamentary participation and bills presented, reads it, and closes the app a flow where the user engages and consumes information.

Yes, that is good enough. Can you quickly provide some metrics that you would track for this application?

The north-star metric is turnout lift turnout among app users versus non-users, and overall turnout against the 55% baseline since that's what the government actually hired for.

Around it I'd track engagement (DAU/WAU, session duration, notification open rate), awareness (candidate-profile views per user, articles read, quiz completion), and conversion (voters registered through the app).

Thank you, that will be all.

You are a PM at Disney. You have been asked to design an application to improve the customer experience at Disney theme parks. What would you do?

A few questions first, to confirm the park's offerings. Parks generally offer rides and shows, have eateries, and some have an on-site hotel. Is that right?

Assume the park has all of those except hotels.

Got it. I will start with identifying the primary user of the application and then listing down the actions they take while visiting the theme park.

The most common visitors of theme parks would be families where the adults are accompanying the children. The decision taker would be one of the parents, so they would potentially be the primary user of the application.

The users perform the following activities:

1. Book the entry ticket to the park
2. Decide on the attraction to engage with
3. Navigate to the selected attraction
4. Make the payment if the attraction has an associated fee
5. Visit eateries for meals or refreshments

Based on the activities I listed down, the users will have the following needs:

1. Ticket Booking: I would want to explore the different tiers of tickets available and their offerings to find the tier which best suits my budget and interests.
2. Picking the amusement ride/show: During this phase, the customers would have the following needs:
 - i. understand the experience of the attraction I am planning to visit to ensure that the attraction would be interesting to me.
 - ii. understand the restrictions (e.g., age, height) of the ride/show of my choice to validate that I am eligible to visit it.
3. I would be visiting the park for a fixed amount of time and want to make the best use of my time.

4. I want to visit an attraction when it is not too crowded so that I don't have to wait in line.
5. Navigate to selected attractions: I need help figuring out the way to the ride or the show I want to visit.
6. Eateries: I want to pick an eatery that would provide me with the food I like.

The pain point assessment seems fair. Assume that you had the budget to solve only one of these problems. Which one would you select, and what feature would you build for it?

I would prioritize the solutions based on the following parameters:

1. Effectiveness of Existing Solutions
2. Value customer would derive from the problem being solved

My selection rule: choose the pain point with the weakest existing solution and the highest customer value.

Pain Point	Existing Solutions	Solution Effectiveness	Value for customer
Ticket Booking	Third party booking platforms	High	Medium
Ride Picking	List of attractions on the park website or brochure	Low - solution is tedious	High
Navigation	Google maps, physical maps, directions in park	Medium	Medium
Eateries	Park Website, Brochures	Medium	Low

Chosen pain point: Ride Picking the existing solution is weakest (tedious) and the customer value is highest.

Based on the selected parameters, I would choose 'Ride Picking' as the pain point to be solved.

Solution:

The goal of the features to be built would be to help the customers optimize the time spent in the park. I propose the following 2 features for the Ride Picking functionality:

- 1. Attraction Catalogue:** explore rides and shows with filters and rich detail.
 - a. Filters:** age and height (eligibility); attraction type (shows, rollercoasters, games, audio-visual).
 - b. Detail per attraction:** rules and regulations, a ride video, timings with available slots (for scheduled shows), expected wait time, and customer reviews.
 - c.** Option to pre-book scheduled shows and rides.
- 2. Ride Sequencer:** given the visit duration, party size and each visitor's ride preferences, it produces an ordered schedule of attractions that fits the most-liked rides into the available time, using expected wait times and show slots. The customer can edit it manually.

Scope note: the Sequencer decides which attraction to visit next and when it deliberately stops short of turn-by-turn navigation, which is the separate Navigation pain point we chose not to fund here.

Sounds good. We can close the case now

India's startup ecosystem has grown immensely in the past decade. As a PM at LinkedIn, I suggest features to leverage that growth.

LinkedIn's mission is to connect professionals to make them more productive and successful. Given its 1B+ user base, may I assume the major startup stakeholders, founders and investors are already on the platform?

Yeah, sure. Please go ahead.

With acquisition largely done, new features should drive engagement, banking on LinkedIn's strengths: its large base, domain experts, access to professional data (past employers, education), and LinkedIn Learning. I'll map the needs of the key stakeholders, founders (who form the core business idea) and investors (who provide capital for equity).

Before the needs, which stakeholder would you prioritise?

As the current business goal is engagement, I'll prioritise by engagement potential. Investors are usually backed by a team, do their own primary research, and have limited need for added services.

Founders are often resource-constrained and actively seeking buzz and awareness, giving LinkedIn many avenues to engage them. So I'll focus on founders.

Looks good to me.

A founder's early journey breaks down as:

1. **Ideation and validation** needs: market research, guidance for conceptualisation, idea validation.
2. **Building and training** needs: prototyping, building the core team, training the team, building an MVP.
3. **Scaling** needs: marketing, product feedback, engagement-driven metrics, raising funds.

Prioritising on fit with LinkedIn's mission and strengths, customer value, and the lack of existing solutions, the priority needs are market research, mentorship, recruitment, training, and product feedback. One feature each:

1. **AI Trend Miner (market research)**: scrapes industry conversations, hashtags and job-post keywords to surface emerging pain points in real time for a given industry.
2. **Mentor Connect (mentorship)**: founders earn mentor credit-hours and a tiered badge reflecting engagement on shared learnings and hours spent reviewing ideas.
3. **Startup Talent & Co-founder Match (recruitment)**: startup-specific talent pools, co-founder matching by complementary skills and equity expectations, and “founding-team” hiring tools using LinkedIn's professional graph, its strongest right to win.
4. **In-Post Beta Surveys & Polls (product feedback)**: founders tag product posts with a “Beta” badge so commenters auto-receive a lightweight NPS poll; LinkedIn Polls and Events run micro-surveys in founder groups, segmented by firmographic filters.
5. **LinkedIn Learning Team Pass (training)**: three free licences per founding team, with more seats unlocked via milestone achievements.

Ola has identified the travel needs of school children as a business opportunity. Design the experience on Ola to meet their needs.

Would this be integrated into the existing Ola app or a new one? And what’s the region of operation?

Integrated into the existing Ola environment in India.

The service is for children aged 4-17, but the user and the buyer are different people. The child is the rider. The parent is the decision maker and the payer, and is the person the service actually has to convince. The school is a third stakeholder - it controls gate timings and pick-up rules, and at scale it is the cheapest channel to reach parents. The driver is the supply side.

So I’ll design for the parent’s decision and the child’s experience, and treat the school as a partner rather than a customer. I’ll segment on the distance between home and school - closer students may walk or cycle, while farther students need comfortable transport and on the child’s current commute mode, since that is what the service has to displace.

Sounds good. Which segment should the service target?

Looking at the four modes in use today:

Current mode	Why	Target?
School-run bus	School-provided, route-fixed and cheap. Trusted by default and available to families across income levels.	No - hard to displace
Private car or family driver	The commute is already solved and the parent controls it.	No - low intent to switch
Informal shared auto or van	Unverified driver, no accountability, no receipts, frequent overcrowding. Largest group.	Yes - launch here
Walk or cycle	Distance is too short for the service to add value.	No

This is where the pain is sharpest and where a verified, trackable alternative is worth paying for. Once density and utilisation improve, a lower-priced tier can extend the service to more price-sensitive families.

Fair. What other factors will you consider?

The mode of transport, and because this is a child-safety service, I’ll choose modes by safety and age-appropriateness first, not just cost:

Mode	Suitability for child transport
Bus	Secure and disciplined, but high establishment cost to add - excluded at launch
Auto	Informal and less safe; acceptable only for older students (e.g. 13+) with safeguards
Van	Enclosed, relatively safe, flexible timings - well-suited; primary mode
Cab	Enclosed and comfortable but pricier; suited to longer distances
Bike	Unsafe for children and infeasible in monsoon - excluded

As the current Ola service already includes travel via autos, cabs, and bikes, we can target all the service segments excluding buses, as including buses could result in a huge establishment cost.

That’s alright. What are the user pain points the service should solve?

Three buckets:

- 1. Driver / vehicle selection:**
 - poor vehicle condition
 - few driver/vehicle options
 - price unrelated to service quality
- 2. Commute:**
 - driver indiscipline and irregular leaves
 - information asymmetry on driving skill
 - missing licences/paperwork; overcrowding
- 3. Payments:**
 - irregular price hikes
 - no receipts or official paperwork

These are all must-solves to earn parents' trust and keep children safe, and the incremental effort to add each is low, so the MVP should solve all of them.

Okay, now let's move on to the solutions.

The solution is an end-to-end school pick-up and drop service:

- **Ola-verified drivers:** driving tests and background checks, with always-on GPS tracking to monitor driving quality.
- **Free trial rides: parents** experience the service, meet the driver and check the vehicle before subscribing; feedback and a callback if they're unhappy.
- **Real-time location updates:** notifications for ride start, driver 10 minutes away, arrival at home and at school.
- **Backup arrangements:** a no-extra-charge backup if the regular vehicle breaks down or the driver is unavailable.
- **Capacity caps (overcrowding):** a fixed seat allocation per child, so vehicles are never overcrowded.
- **Transparent payments:** fixed subscription pricing (no surge or ad-hoc hikes) and in-app receipts, directly addressing the payments pain points.

Every feature here answers an anxiety the parent has, because the parent is the one who signs up and renews.

The child's side of the design is narrower - a fixed seat, a familiar driver and a ride that arrives on time.

Alright. Can you walk-me through the proposed MVP?

Four steps:

1. Sign in:

- create a profile (name, age, photograph)
- select school and home address

2. Select service and driver:

- view price estimates per service
- see driver and vehicle details
- schedule a test ride, finalise the driver

3. Payment and agreement:

- choose a subscription (monthly/quarterly/yearly)
- and sign the online agreement

4. Avail the service:

- confirm the weekly schedule (driver notified of minor tweaks),
- track live location,
- and select a backup if needed

That sums up the case nicely. Thank you for your time.

You are a PM for Microsoft Surface and Delta asked you to design the perfect airline entertainment system. You decide the features, scenarios, and users. What would you do?

A few clarifying questions first.

Sure, go ahead.

Are we improving the existing system, or building from scratch? And what problem are we solving?

Improve the existing software. The hardware can change, given the new Microsoft-Delta relationship. Microsoft wants to become the leading provider of airline entertainment systems. Delta wants a better experience. Usage of the current system is low, especially on short flights.

Are there any constraints?

Six to eight months. Budget is not a constraint.

Then time is the real constraint, not money. I will keep that in mind when I choose what to build.

Users

Three personas stand out.

1. The business traveller. Flying on company time. Carries their own laptop and buys Wi-Fi.
2. The economy traveller on a short domestic flight. Paying for themselves. Rarely buys Wi-Fi.
3. The international leisure traveller. Long flight, plenty of time, already well served.

I would build for the second one.

- The business traveller already has a better device in their bag. We will not beat their laptop.
- The international traveller has eight hours and a full content library. The system already works for them.
- The short-haul economy traveller is where usage is lowest. That matches the problem you described.

Agreed. Go deeper on that persona.

The problem

Their flight is 90 minutes. Roughly 30 of those minutes are taxi, take-off and landing.

So they have about an hour to fill, and three things go wrong:

1. The good content sits behind a paywall. They will not pay for one hour.
2. The free content is thin and the interface is slow.
3. They give up and use their phone, which they downloaded to before the flight.

The system loses them in the first two minutes. Everything after that is a lost hour.

Features

Six ideas, grouped by the need they serve.

Feature	What it solves
Link your own OTT account	They already pay for content. Let them continue their watchlist.
Pre-flight download	Pick content on your phone before boarding. It is ready at the seat.
Casual games and trivia	Fills short gaps. No commitment. Rewards with a drink coupon.
Free news feed via MSN	Something to read without paying.
Faster navigation, refreshed UI	Removes the friction that loses them in minute one.
Pair your own device to the screen	Uses the seatback as a bigger display for their phone.

Would you build all six?

No. Six to eight months is short, and anything touching hardware will eat that time.

I will score them on impact for this persona against effort, and cut below a line.

Feature	Impact	Effort	Build?
Faster navigation, refreshed UI	High	Low	Yes
Link your own OTT account	High	Medium	Yes
Casual games and trivia	Medium	Low	Yes
Free news feed via MSN	Medium	Low	Yes
Pre-flight download	High	High	Later
Pair your own device	Medium	High	Later

The two I am cutting both need hardware or airline IT changes. Pre-flight download needs the Delta app and the seatback to sync. Device pairing needs new ports or wireless casting at every seat. Neither fits eight months.

The four I am keeping are all software on the existing screen.

You had Microsoft 365 apps on your original list and dropped them. Microsoft is paying for this. Would you really tell them no? [Pushback]

Yes, for this persona.

Nobody writes a document on a seatback touchscreen during a 90-minute flight. On the business traveller's route it might work, but that user has a laptop.

If Microsoft wants the ecosystem visible, the better version is device pairing - their own Surface, our bigger screen. That is on my Later list, not cut.

Fair. How would you measure success?

Metrics

One north star, three inputs, and two guardrails.

Type	Metric
North star	Share of available flight time a passenger spends engaged with the system
Input	Percentage of passengers who open the system at all
Input	Percentage who are still using it after five minutes
Input	OTT account link rate among those who open it
Guardrail	Paid content revenue per flight - free content must not cannibalise it
Guardrail	Crew support requests per flight - a new UI must not create cabin work

I chose share of flight time rather than total usage minutes. A 90-minute flight and an 8-hour flight are not comparable on raw minutes.

The five-minute mark matters because that is where we lose them today.

Assumptions

- The existing seatback hardware can take a software update.
- Content licensing is out of scope. We are not buying new titles.
- Revenue is not the goal here. Usage is.
The paid tier is protected by a guardrail, not grown.

You are a PM at Spotify. You are asked to come with solutions to improve Spotify as a podcast app

Spotify is a music and podcast streaming platform operating on a freemium model. It gives listeners access to millions of songs and creator content from around the world.

That's correct.

Before diving in, I have a few clarifying questions:

1. Are we improving the experience for listeners or creators, or both?
2. Are we targeting a specific user demographic?
3. Are there any budget or timeline constraints I should be aware of?

We are focusing only on listeners. For simplicity, assume no specific demographic, budget, or timeline constraints.

Thank you for clarifying. I'd like to map the user journey to identify where we can have the most impact. The key stages are: Acquisition, Activation, Engagement, Retention, and Referral. For this problem, I'll focus on Engagement and Retention, since improving engagement is the most direct lever for driving long-term retention. Does that work for you?

Sure, Go ahead.

I'd like to identify the main user personas for Spotify's podcast listeners. I see three distinct groups:

1. Students (school and college)
2. Working Professionals
3. Retired Citizens

I will define the needs of each segment and then prioritize the one with the highest impact. Does that sound okay?

That sounds okay.

Students

- Primarily interested in music, but listen to podcasts occasionally.
- Often struggle to find topics that sustain their interest and tend to drop off mid-episode.
- Distracted by the variety of entertainment options on the platform.

Working Professionals

- Listen to both music and podcasts, with a stronger focus on career-relevant content.
- Despite higher intent, they get distracted during long episodes.
- Struggle to find new podcasts aligned with their interests due to overwhelming content volume and lack of engaging previews.
- No built-in community space to discuss episodes - pushed to third-party platforms like Reddit or Discord.

Retired Citizens

- Have more free time but struggle with content discovery.
- Overwhelmed by the variety of recommendations on the home page.
- Prefer a curated, trusted listening experience.

Which segment would you like to prioritize?

I would like to focus on working professionals, as they are the most consistent podcast listeners and represent the highest-value engagement segment for Spotify's growth.

That's a good understanding. You can go ahead with your line of thought.

The key pain points for working professionals are:

- Difficulty discovering new podcasts aligned with their interests, due to overwhelming content volume and lack of meaningful previews.
- No built-in community space to ask questions, comment, or discuss episodes - pushing users to third-party platforms like Reddit or Discord.

That sounds fair. What product features would you propose to address these pain points?

I'd suggest two features:

- **Podcast Snippets:** A short-clip discovery feed where users can browse 60-90 second excerpts from various podcasts. Tapping a snippet opens the full episode, helping users sample content before committing. This lowers discovery friction and exposes users to genres they might not have explored otherwise.
- **Podcast Community Feed:** Once a podcast crosses a follower threshold, creators can unlock a community space where listeners can post notes, ask questions, and join creator-hosted Q&A sessions - keeping engagement within the app.

Those are interesting suggestions. Can you also define success metrics for these features?

Sure. I'd track the following:

Podcast Snippets:

- Monthly increase in users who save and listen to full episodes after engaging with a snippet.
- Total unique users saving snippets.
- Average number of snippets saved per user.

Podcast Community Feed:

- Increase in user engagement (likes, comments, Q&A participation) on podcast community feeds.
- Growth in followers for podcasts with active community feeds.

That's correct. That wraps up the case nicely. Thank you for your time.

Can you walk me through how you'd improve a ride-hailing app like Uber or Ola?

Before jumping to solutions, I'd like to clarify a few things to better scope the problem:

- 1. What is the state of the company? - Is it early-stage or mature?
- 2. What is our primary goal? - Increasing bookings, improving retention, enhancing driver satisfaction, or expanding to new segments?
- 3. Who is our primary user? - The rider or the driver?

Let's assume the company is mature with a significant urban user base. Competition is intense, and we want to improve rider experience and increase loyalty.

That's helpful. I'll assume we are targeting urban Indian users - both existing and new - and our primary user is the rider. With retention and satisfaction as our north stars, I'll focus on reducing friction and building rider trust.

Good question. Let's prioritize riders for now. Improving their experience is our current focus.

For urban Indian riders, the key pain points are:

- 1. **Long wait times** - during peak hours
- 2. **Unpredictable price surges** - with little advance notice
- 3. **Safety concerns** - especially for female users traveling at night
- 4. **Poor ETA accuracy** - and low visibility into driver location
- 5. **Unreliable driver cancellations** - leading to frustrating rebooking cycles

Sounds fair. What would you build to address this?

I'd propose four features:

1. Smart Pickup Suggestions

Use real-time location data, traffic patterns, and geofencing to recommend optimized pickup points nearby. This reduces confusion, failed pickups, and wait times - especially in areas with complex road layouts or traffic restrictions.

2. Trust Mode for Night Rides

An opt-in safety feature aimed primarily at female users. Riders get matched with high-rated drivers or female drivers (where available), with enhanced live tracking and optional emergency sharing to a trusted contact.

3. Priority Matching for Loyal Users

Frequent users receive faster ride assignments and reduced cancellation risk. Additional perks like a loyalty badge and early access to new features help drive habitual usage and reduce churn.

4. Predictive Surge Alerts

Notify users in advance when surge pricing is likely in their area, giving them a window to book early or adjust plans. This reduces frustration and builds trust through transparency.

How would you prioritize amongst these features?

As a mature ride-hailing company, our key product goals are retention, user satisfaction, and competitive differentiation. Here's how I'd evaluate the features:

Feature	Retention	User satisfaction	Competitive differentiation
Smart Pickup Suggestions	Reduces frustration, improves reliability	Lower pickup delays, better experience	Industry-standard
Trust Mode for Night Rides	Builds long-term trust with a key segment	Directly addresses safety concerns	Strong
Priority Matching for Loyal Users	Rewards frequent use	Speeds up access, builds habit	Encourages switching cost
Predictive Surge Alerts	Neutral to low	Manages expectations	Easy to copy, not sticky

I'd launch Smart Pickup Suggestions first - it directly addresses a daily pain point, scores well on two of our three goals, and is relatively straightforward to implement. Next, I'd prioritize Trust Mode for Night Rides. While it targets a specific segment, it addresses a critical safety gap and positions us as a more thoughtful, safety-first brand - a strong differentiator. Priority Matching would follow as a retention and loyalty play, once the core experience is stable. Predictive Surge Alerts, while useful, are lower priority since they're easier for competitors to copy and have limited retention impact.

Sounds good. Let's say you launch Smart Pickup Suggestions. How would you measure success?

- Failed Pickup Rate - Did cancellations or missed pickups decrease after launch?
- Pickup Time Accuracy - Is the gap between estimated and actual driver arrival smaller?
- CSAT on Pickup Experience - Specifically, ratings and comments related to pickup ease and clarity.

Nice. Would you roll this out all at once?

No - I'd recommend a phased rollout. I'd start with a soft launch in two or three high-density zones in cities like Bengaluru or Delhi, where pickup confusion is most common due to traffic complexity and infrastructure gaps.

This lets us validate assumptions, catch edge cases, and iterate before a broader release.

Great answer. We can end the discussion here.

Develop a health and fitness application for smartphone users.

Before jumping in, a few clarifying questions:

- 1. Geography - Which market are we targeting?
- 2. Company stage - Are we a startup building this from scratch, or an established player with existing products?
- 3. User focus - Are we building for casual users, serious athletes, or a mix?

Let's focus on the urban Indian market. You are a startup building this app for the first time.

Ok. Do we have a target set of customers in mind we want to sell our product to?

How about you recommend the same? What would be the best target for our application?

Got it. Since we are a startup, growth and adoption will be our primary objectives. Given that, I'd recommend targeting urban users aged 18-30 - they have higher experimentation rates, are increasingly fitness-conscious, and have sufficient disposable income to try new products. Does that work?

Sounds good. We can move forward.

Urban young adults in India lead busy lives - most are working full-time and have limited time to dedicate to health and fitness. Many visit gyms occasionally, play recreational sports like cricket or badminton, or run on weekends. Diet is also an emerging concern, with more users becoming aware of calorie and nutrition tracking.

What features would you propose for this app?

I'd suggest four core features:

1. Workout Scheduling

A weekly planner that helps users schedule workouts, set targets per body part, and track training intensity. Given that time is the biggest constraint for our target users, a simple scheduling tool that fits fitness into their week is the most immediately useful feature.

2. Health Analytics Tracking

Track workouts, calories burnt, heart rate, and activity history. Build a user profile over time and surface personalised recommendations based on workout history. This is table-stakes for any fitness app and a key driver of daily habit formation.

3. Personalised Workout Plans

Generate workout plans based on a user's fitness goals and history. We can also onboard certified trainers who sell their programs through the app - creating a marketplace model that drives both engagement and revenue. This should be a later-stage feature given the content and onboarding effort required.

4. Diet Tracking

A calorie counter with meal logging and food-plan recommendations based on the user's fitness goals. Building an accurate Indian food database will require significant effort, so this is best positioned as a Phase 2 feature after core engagement is established.

These sound good. How would you prioritize building them?

Since we are an early-stage startup, I'd prioritize based on three factors: Reach (how many users benefit), Cost (development complexity), and Revenue potential. Given our stage, Reach and Cost should take priority over Revenue.

Feature	Reach	Cost	Revenue Potential
Workout Scheduling	High - universal need for all fitness users	Low - simple calendar logic, no hardware needed	Indirect - drives retention
Health Analytics Tracking	High - core expectation in any fitness app	Medium - requires device sensor integration	Indirect - enables premium upsell
Personalised Workout Plans	Medium - appeals to motivated users	High - trainer onboarding and content ops	Direct - trainer marketplace revenue
Diet Tracking	Medium - niche among casual users	High - Indian food database required	Indirect - premium feature

Based on this, my recommended build order is:

1. **Workout Scheduling** - high reach, low cost, and directly solves the time-constraint pain point of our target users. Strong fit for MVP.
2. **Health Analytics Tracking** - core expectation in any fitness app - users will compare us against incumbents like HealthifyMe. Medium cost due to sensor integration, but essential for retention.
3. **Personalised Workout Plans** - high revenue potential through a trainer marketplace, but requires content ops and onboarding investment. Best suited for post-MVP once we have an active user base.
4. **Diet Tracking** - useful but appeals to a narrower segment of motivated users. The Indian food database requirement also makes this costly to build accurately. Recommended for a later phase.

Sounds good. Any other recommendations?

Yes - I'd suggest two key success metrics to track, given that growth and adoption are our north stars:

- **New User Adoption Rate** - the percentage of new users acquired over a given period. Paired with geographic segmentation, this helps us identify which cities or clusters to prioritise for growth marketing.
- **Daily Active Users (DAU)** - measures how frequently existing users engage with the app, and is a strong proxy for the value our features are delivering. A healthy DAU-to-MAU ratio indicates habit formation.

Together, these two metrics give us a clear view of both acquisition momentum and feature stickiness - the two things that matter most at this stage.

That sounds great. We can close the case now. Thank you for your time!

You are a Product manager of XYZ company. Design a web portal that will serve as an internal knowledge base of this company.

Sure, I have a couple of preliminary questions. What is the scope of the product we are looking to design - in terms of users, the types of knowledge shared, and whether the portal should focus on just consumption or include contribution and collaboration as well?

Good question. We are looking at a company-wide internal portal. It will serve as a single source for resources like SOPs, playbooks, and internal decks (e.g., Stage 0 decks). Currently, there's no formal structure; documents are scattered. You may decide whether collaboration and contribution are in scope for the MVP.

Understood. One more question - do we have any constraints in terms of geography, scale of organization, or existing tools in use (e.g., SharePoint, OneDrive)? And is this a revamp of an existing tool or a fresh build?

Assume it's a fresh build for a mid-to-large organization, spread across functions and locations in India. Knowledge is currently scattered in drives and email threads.

Got it. Based on this, I would define the goal of the product as: To create a centralized, searchable, and structured portal for employees to access and engage with key organizational knowledge such as SOPs, playbooks, and internal decks. Would you like me to proceed with structuring the problem now? I'll define the user personas, outline their key use cases, and then propose MVP features and success metrics.

That sounds good. Let's go through each of the sections one by one.

- Great. Let me start with the user personas. We would primarily have two segments of users:
1. Consumers (Knowledge Seekers): Employees who are looking for information - SOPs, playbooks, strategy decks, etc.
 2. Contributors (Knowledge Owners): SMEs or team leads responsible for uploading and maintaining documents.

Alright. Now walk me through the use-cases for each of the user personas.

Yes, I'll now proceed with defining some use-cases for each user.

For the consumers, the following use-cases may be considered along with their priority order, which is defined in terms of usefulness to the customers for the MVP.

Use Case	Priority
I want to search for and view relevant documents (SOPs, Playbooks, etc.)	P1
I want to see who created a document and when it was last updated	P1
I want to download documents for offline access	P1
I want to ask questions or leave comments on a document	P1
I want to bookmark frequently used documents	P2

For the contributors, the following use-cases may be considered along with their priority order.

Use Case	Priority
I want to upload knowledge documents with proper tags and metadata	P1
I want to receive and respond to comments on my uploaded resources	P1
I want to be reminded or able to update outdated documents	P2

Would you like me to move on to the proposed solutions for the P1 use cases?

Yes, let's proceed with the proposed solutions

Before diving into the specific features, I recommend building this as a responsive web portal for the MVP. This allows:

- Easy access across desktops and mobile browsers
- Lower prototyping cost
- No app store dependencies or versioning issues

Now, based on the PI use cases, I will proceed with the solutions targeted towards the Consumers and I have used the RICE framework for feature prioritization:

Feature	Reach	Impact	Confidence	Effort	Priority
Global search with filters to view relevant documents (SOPs, Playbooks, etc.)	High	High	Medium	Low	High
Display metadata on each document page	Medium	High	Medium	Low	High
Download button with version timestamp and file type shown	High	High	High	Low	High
Threaded comment section under each document to ask questions or leave comments on a document	Medium	Medium	Medium	Medium	Medium

I will now proceed with the solutions targeted towards the Contributors in the similar manner:

Feature	Reach	Impact	Confidence	Effort	Priority
Upload form with fields (title, tags, category, contact info) to upload knowledge documents with proper tags and metadata	High	High	High	Low	High
Email notifications + comment response dashboard	Medium	High	Medium	Medium	Medium

The proposed solutions are comprehensive. Why don't you quickly list down the solutions that should be prioritized first for the MVP?

Sure. Based on the above analysis, I would propose the features in the following priority order for the MVP for both Consumers and the Contributors:

Consumers:

- Global Search with Filters - It is central to discovery and while slightly more complex, it delivers high value to most users.
- Metadata Display (Author, Timestamps, etc.) - It enhances document credibility and transparency and is easy to implement with immediate benefit.
- Download Button with Version Info - It has high reach, impact, and confidence with low effort which enables offline access and builds trust with clear versioning.
- Threaded Commenting on Documents - This introduces collaboration and clarifications at a moderate cost and is useful for feedback loops.

Contributors:

- Upload Form with Metadata Fields - This is a critical enabler for structured knowledge contribution which is low effort and high impact for long-term content growth.
- Email Notifications to Contributors - This is useful for engagement and can be added after core workflows are functional.

Fair analysis. Can you quickly define some success metrics before we wrap up the case?

Sure. I'd group the success metrics across three dimensions:

1. Adoption Metrics
 - % of employees using the portal weekly/monthly
 - No. of unique documents uploaded per month
 2. Engagement Metrics
 - Avg. time spent per session
 - No. of downloads made
 - No. of comments or feedback submitted
 - No. of searches performed
 3. Retention Metrics:
 - Churn rate (percentage of users who stop using the platform).
- These metrics will help gauge both the utility and sustainability of the portal.

Good analysis. We can close the case now.

You are a Product manager of ABC company. Design a book recommendation platform that helps users discover and find the best book recommendations suited to their interests.

Sure, a few clarifying questions before I begin:

- 1. Target users - Who are we building for - casual readers, avid readers, or both?
- 2. Geography - Which market are we targeting initially?
- 3. Scope - Are we focused only on book discovery, or does the platform also handle purchasing and post-read engagement?
- 4. Company context - Is this a new initiative or does ABC have an existing product in the books space?

We're building a mobile-first AI-powered book recommendation platform where users can get personalised suggestions, compare prices across marketplaces, and read or write reviews. We're targeting India initially, and this is a fresh startup with no existing product in the books space.

Understood. I'd define the product goal as: help users discover, evaluate, and purchase books through an intuitive AI-first mobile experience. I'll structure my response across three areas: user personas and use cases, proposed features with prioritization, and success metrics. Does that work?

Yes, please go ahead.

I see three primary user personas for this platform:

- **Engaged Reader** - a student or working professional who reads regularly and looks for high-quality, personalised recommendations. Wants to browse, evaluate, and purchase books quickly on mobile.
- **Review Contributor** - a reader who enjoys sharing opinions and influencing the reading community through detailed reviews and ratings.
- **Deal Seeker** - price-conscious buyer who compares costs across platforms and prioritises fast, affordable delivery before committing to a purchase.

Based on these personas, here are the core use cases prioritised for MVP:

Engaged Reader

Use Case	Priority
Get AI-powered book suggestions based on genre, mood, or specific themes	P1
Ask targeted queries like 'best books on productivity' or 'top thrillers of 2023'	P1
Scroll through suggestions and reviews easily on mobile	P1
Save or revisit previously viewed books through a History view	P2
Bookmark books for later consideration	P2

Review Contributor

Use Case	Priority
Write and submit detailed book reviews with star ratings and genre tags	P2
Read community reviews and respond to comments	P2
View and manage a list of books they've reviewed	P2
Receive notifications when someone reacts to their reviews	P3

Deal Seeker

Use Case	Priority
View price comparisons across Amazon, Flipkart, and other online marketplaces	P1
Quickly access external purchase links with minimal steps	P1
Receive alerts or notifications on price drops or new offers	P2
Filter books by price, format (paperback/hardcover), and delivery speed	P2

Yes, let's proceed with the proposed solutions.

Given the mobile-first nature of the product, the design should prioritise minimal typing, high discoverability, and quick access flows. I propose four core features:

- Sidebar Navigation - a persistent sidebar giving users quick access to the AI Assistant, browsing History, and review Notifications. This is the structural backbone of the app and low effort to implement.
- Book Detail Screen - scrollable tabs covering summary, reviews, and metadata (author, genre, ratings). This is the primary evaluation surface and the key step between discovery and purchase.
- Price Comparison View - live price comparisons across Amazon, Flipkart, and other marketplaces, with direct external buy links. Drives the Deal Seeker journey and supports monetisation through affiliate links.
- Conversational AI Assistant - the core discovery engine. Users enter natural language queries or tap pre-built prompts (e.g. 'top thrillers', 'books like Atomic Habits'). High reach and impact, but higher development effort - best rolled out iteratively after core navigation is validated.

Using the RICE framework to prioritise:

Feature	Reach	Impact	Confidence	Effort	Priority
Conversational AI Assistant	High	High	Medium	High	Medium
Book Detail Screen	High	High	High	Low	High
Price Comparison View	High	High	Medium	Medium	Medium
Side Bar Navigation	Medium	Medium	Medium	Low	High

The proposed solutions are comprehensive. Which features should be prioritised first for MVP?

Based on the RICE analysis, I'd recommend the following MVP build order:

1. Book Detail Screen - high reach and impact with low effort. This is the core evaluation surface that enables the full discovery-to-purchase journey and should ship first.
2. Sidebar Navigation - low effort and critical for app structure and user flow continuity. Should be built alongside or immediately after the Detail Screen.

3. Price Comparison View - drives purchase decisions and opens affiliate monetisation. Should follow once core discovery is in place.
4. Conversational AI Assistant - the highest-impact feature long-term, but requires more development effort. Best introduced as an iterative upgrade after validating the core user journey.

Fair analysis. Can you quickly define some success metrics before we wrap up the case?

I'd group success metrics across three dimensions:

Adoption

- Percentage of users who initiate an AI query within their first session
- Daily Active Users (DAU) growth over time

Engagement

- Average number of books viewed per session
- Click-through rate on marketplace purchase links

Retention

- Percentage of users who return within 7 days of first use
- Number of reviews posted per active user

Together, these metrics cover the full funnel - from first-time activation through to habitual use and community contribution.

Good analysis. We can close the case now.

You're working at Samsung on the product team for Galaxy Buds. Using listening data, Samsung wants to explore new approaches to improve the listening experience - in particular, how this data might be made social. How would you approach building something valuable here?

Before jumping to solutions, a few clarifying questions:

- 1. Goal - Is the primary objective improving user engagement, building brand loyalty, or creating monetisation opportunities?
- 2. Existing app - Galaxy Buds users already have the Wearable app installed post-purchase - is usage dropping off after setup? Should we build within it or create something new?
- 3. Scope - Are we open to building a standalone social product, or should this live within the existing Samsung ecosystem?

Great clarifying questions. Engagement is the priority. You're right that Wearable app usage drops off after setup. We're open to both approaches - I'm interested in what you think makes the most sense from a product standpoint.

Given that the Wearable app is already installed on the majority of Galaxy Buds users' devices, I'd recommend enhancing it rather than building a new standalone app. A new app faces a cold-start problem - discoverability and install friction - whereas the Wearable app already has distribution. We can layer social features on top of an existing trusted surface.

Sure. I'm with you on the approach. Please go ahead.

I'll start by identifying and prioritising user segments most likely to benefit from a social listening feature. I'll evaluate them on four dimensions: engagement potential, social affinity, product affinity, and strategic priority.

Segment	Age	Location	Description	Engagement	Social Affinity	Product Affinity	Priority
Casual Urban Listeners	25-35	Urban	Use earbuds for commute, work focus, and light music. High daily usage frequency.	High	Medium	High	1

Segment	Age	Location	Description	Engagement	Social Affinity	Product Affinity	Priority
Students / Young Adults	16-25	Mostly Urban	Stream heavily; love social sharing. Listen during study sessions or commutes.	High	High	Medium-High	1
Semi-urban Music Enthusiasts	25-35	Semi-Urban	Mix of offline and streaming. Interested in community-based music discovery.	Medium	Medium	Medium	2
Fitness-first Users	20-40	Urban / Rural	Music is secondary to fitness tracking. Low social sharing intent.	Medium	Low	Medium	3
Audiophiles / Tinkerers	20-40	Urban	Highly engaged with sound customisation but less interested in social features.	High	Low-Medium	High	3
Rural Listeners	20-40	Rural	Prefer local/offline music. Low app engagement and limited digital sharing.	Low	Low	Low	4

Based on this framework, I'd prioritise two segments for the initial build:

- 1. Casual Urban Listeners (25-35) - large base, high daily usage, and strong product affinity. Even moderate social engagement from this group drives scale.
- 2. Students / Young Adults (16-25) - inherently social, heavy streamers, and highly motivated by self-expression through music. The natural early adopters of any social feature.

I'm also adding Audiophiles / Tinkerers as a segment I considered but deprioritised - they have high engagement with sound customisation but low social sharing intent, making them a poor fit for a social-first product at this stage.

That's a great catch. Please proceed with the analysis.

Based on the top two segments, here are the core user needs driving this product:

User Need	Why It Matters
Understand and track listening habits over time	Users want visibility into their music routines - genres, duration, trends - to feel more intentional and connected to their listening identity.
Share music highlights and preferences with others	Sharing creates social currency and self-expression. For younger users especially, what you listen to is a part of how you present yourself.
Discover new content or sound setups through peers	Peer recommendations are more trusted than algorithmic ones. Social discovery surfaces niche content that formal recommendation engines miss.
Feel part of a like-minded audio community	Community gives the product emotional stickiness. It transforms a solitary habit into a shared one, increasing both engagement and retention.

I think you've defined the needs well. Please proceed to the solutions.

I'd propose three features that map directly to these needs:

Feature	What It Is
Genre Clubs	In-app social spaces organised around music genres or moods (e.g., 'Chill Lo-fi', '90s Bollywood', 'Focus Beats'). Users can join, share weekly digests, view others' stats, discuss music, and discover new tracks.
Weekly Listening Digest	A personalised, visual summary of weekly highlights - top songs, favourite genres, total listening time, and mood tags (e.g., 'Chill', 'Energetic'). Shareable with friends or postable to community groups.
Peer Sound Preset Exchange	A tool for users to create and share custom EQ/sound settings. Others can try, save, and upvote presets (e.g., 'Bass Boost for EDM', 'Crisp Vocals for Podcasts'). Presets can surface within Genre Clubs.

Using the RICE framework to prioritise (scores out of 5):

Feature	Reach	Impact	Confidence	Effort	Score	Priority
Genre Clubs	4	4	4	4	16	1
Weekly Listening Digest	4	3	3	3	12	2
Peer Sound Preset Exchange	3	3	3	3	9	3

I'd prioritise Genre Clubs as the MVP feature because:

- 1. Most aligned with the core goal - building sustained social interaction around Galaxy Buds usage, not just one-off sharing.
- 2. Drives ongoing participation - polls, recommendations, and shared digests create reasons to return regularly.
- 3. Unlocks future feature surfaces - badges, leaderboards, and community challenges can be layered on top once the clubs are established.

Weekly Listening Digest is a strong complement - it gives users something to share into their clubs - and can ship shortly after. Preset Exchange is a Phase 2 feature best introduced once the community layer is active.

That was a thoughtful walk-through. Can you also define success metrics for the Genre Clubs feature?

I'd track success across four stages of the user journey - Discovery, Adoption, Engagement, and Retention:

Stage	Metric
Discovery	% of app users who tap to explore club previews (genre tiles, trending clubs)
Adoption	1.% of users who join at least one club within 7 days of first discovery 2.% who complete club onboarding flow
Engagement	3.Average club visits per user per week 4.Average posts/replies per user per week (shared digests, comments) 5.% of users who react or like content within a club
Retention	6.% of users who return to a club within 7 and 30 days of first visit 7.% of users who post or comment in a club more than 3 times in a month

A healthy product would show strong Discovery-to-Adoption conversion in the first 7 days, and growing Engagement and Retention numbers by Week 4. If club visits per user per week plateau early, that's a signal to investigate content freshness or notification strategy.

Design an AI copilot that helps human agents in a customer-support team. Walk me through your approach.

Before I start, I'd like to understand the context. Could I clarify a couple of things - who exactly is the copilot for: our own support agents, or end customers directly? And what channels are we talking about: chat, email, voice?

It's an internal tool for our own agents. They handle chat and email tickets for a B2B SaaS product - the customer never talks to the copilot directly.

Got it. And what's the primary outcome the team cares about - faster resolution, higher quality, lower cost, or quicker agent ramp-up? I'd like to optimise the design for the main one.

Cut handling time and improve resolution quality. But a human has to stay accountable for every reply that goes out - we're not automating the customer away.

Understood - the copilot assists, the agent approves. I'll go: the user and where they struggle, then what to build for an MVP, then the safeguards, then metrics. The primary user is the support agent; the customer feels the outcome and the support lead owns SLAs and quality. Does that structure work for you?

Yes, that works. Start with the agent.

I think an agent's workflow today broadly looks like this:

Receive ticket → Understand customer context → Search documentation → Decide next action → Draft reply → Resolve or escalate.

Looking at this journey, I see four major pain points.

1. Context-gathering is slow: the agent reads account history and the ticket before they can even start.
2. Finding the right answer is hard: knowledge is scattered across help-centre articles and past tickets.
3. Reply quality is inconsistent, and new agents ramp slowly.
4. Agents are unsure when to escalate, so risky cases are handled inconsistently.

Before finalizing the MVP, I'd also validate that these are indeed the biggest bottlenecks by looking at support analytics like handle-time breakdown, search frequency, and the percentage of repetitive tickets. That helps ensure we're solving the highest-impact problems first.

That's a fair list. You can't build all of it for v1 - where would you start?

Context-gathering and reply drafting dominate handle time, so I'd build those first and treat next-best-action and escalation as fast-follows.

I am prioritizing these because they improve every ticket irrespective of issue type, making them high-reach, high-impact features with relatively lower implementation complexity compared to automated actions.

Shall I detail the MVP around those two?

Please do.

Here's what the copilot does:

1. **Context:** summarise the customer's account, history and the open ticket so the agent is oriented in seconds.
2. **Draft:** suggest a reply grounded in our help-centre and past resolved tickets (RAG), with the source shown.
3. **Act (fast-follow):** surface the next-best action - the right macro, refund eligibility, or the team to route to.
4. **Escalate (fast-follow):** detect sensitive cases (legal, churn risk, safety, abusive tone) and hand them to a senior human.

You said the agent stays accountable. Concretely, how do you keep a human in the loop - and what happens when the model isn't sure?

Four safeguards handle exactly that:

1. Suggest, don't send: the agent edits and approves every reply - we trade a little speed for accountability.
2. Each suggestion shows its grounding source and a confidence signal; low-confidence drafts are flagged.
3. A hard guardrail routes regulated or high-risk topics to a human regardless of model confidence.
4. When retrieval finds no grounded answer, the copilot returns "no suggestion" rather than guessing.

PM Tip: Naming the human-in-the-loop early signals you treat an AI support tool as decision-support, not an autonomous agent - the distinction interviewers probe in AI design rounds.

Good. Now - how would you measure whether this copilot is actually working?

I'd separate two questions: is the agent more effective, and is the model itself any good? So two metric sets:

- **Product outcomes:** average handle time, first-contact resolution, suggestion acceptance rate, customer CSAT.
- **Model quality:** grounded-answer rate, wrong-answer (hallucination) rate on a labelled sample, and escalation accuracy - precision/recall on the "should this be escalated" decision.

If you could ship with only one launch gate, which metric would it be?

The wrong-answer rate on a labelled sample. A confident-but-incorrect suggestion erodes trust faster than a missing one, so I'd cap it before optimising for speed - then expand to next-best-action and escalation once trust is proven.

Agreed. That's a solid v1!

Our quick commerce platform works with thousands of vendors. Today, purchase orders, shipment updates and invoices flow through a mix of EDI integrations, emails and spreadsheets, making procurement inefficient. As the Product Manager, design a Vendor Management Platform that streamlines supplier collaboration and standardizes these workflows. Walk me through your approach.

Before I begin, I'd like to understand the problem a little better.

Can I clarify a few things?

- Will this platform be used only by our internal procurement teams, or will vendors also have access?
- Are we replacing existing procurement systems or building a layer that integrates with existing ERPs and EDI systems?
- What is the primary business goal - reducing operational effort, improving inventory availability, or improving vendor collaboration?

Good questions. The platform will be used by both procurement teams and vendors. We're not replacing ERPs like SAP. Instead, we're building a collaboration layer that integrates with existing ERP and EDI systems. The primary goal is to reduce manual operations, improve vendor communication, and ensure products reach our dark stores on time.

Understood. I'll first identify the users and their pain points, then map the procurement workflow, identify opportunities, prioritize an MVP, and finally discuss success metrics. Does that approach sound good?

Yes, please go ahead.

I see four primary stakeholders.

Procurement Managers

- Create purchase orders
- Track deliveries
- Compare vendor performance
- Resolve supply issues

Vendors

- Receive purchase orders
- Confirm quantities

- Send shipment updates
- Submit invoices
- Track payments

Warehouse Teams

- Receive shipments
- Verify quantities
- Report shortages and damages

Finance Teams

- Match invoices against purchase orders
- Process payments
- Handle disputes

I'd prioritize Procurement Managers and Vendors because they directly influence procurement efficiency and inventory availability.

Makes sense. What problems are they facing today?

From the problem statement, I see a few major challenges.

For procurement teams:

- Vendor communication happens across emails, phone calls and multiple EDI channels.
- Shipment visibility is poor.
- Exceptions like delayed deliveries require constant follow-ups.
- Vendor performance is difficult to track.

For vendors:

- Different retailers require different EDI formats.
- It's difficult to know whether a purchase order was received successfully.
- Payment and invoice status isn't transparent.
- Disputes take a long time to resolve.

Overall, everyone spends significant effort coordinating instead of making procurement decisions.

That's fair. What would you build?

Before jumping into features, I'd prioritize them using the RICE framework, balancing business impact with implementation effort.

Using a simple RICE lens:

- **Purchase Order Hub:** Highest reach since every procurement transaction starts here, with high business impact and relatively low implementation complexity.
- **EDI Integration:** High impact but lower reach initially, as not all vendors use EDI.
- **Shipment Visibility:** Depends on purchase orders already flowing through the platform.
- **Vendor Scorecards:** Valuable, but requires historical transaction data, making it better suited after adoption.

So my launch order would remain:

1. Purchase Order Hub
2. EDI Integration & Monitoring
3. Shipment Visibility
4. Vendor Performance Dashboard

For the MVP, I'd focus on reducing manual coordination and improving visibility.

The platform would include four core capabilities.

Feature	What it is
Unified Purchase Order Hub	Procurement teams can create purchase orders through the ERP, while vendors receive them through a single portal regardless of whether they connect via EDI or manually. The platform tracks every purchase order through its lifecycle: from creation and acknowledgement to shipment, invoice status and payment status.
EDI Integration & Monitoring	The system integrates with common EDI standards and automatically validates incoming and outgoing documents. If an EDI transaction fails or contains incorrect data, both sides receive alerts instead of discovering the issue later. This reduces manual troubleshooting significantly.
Shipment Visibility Dashboard	Vendors can acknowledge purchase orders, submit Advance Ship Notices (ASN), update shipment status and expected delivery dates. Procurement teams receive real-time visibility into shipment progress.

Feature	What it is
Vendor Performance Dashboard	Every vendor receives a scorecard showing: • On-time delivery rate • Fill rate • PO acknowledgement time • Invoice accuracy • EDI transaction success rate This helps procurement teams make better sourcing decisions.

These all sound useful. Which one would you launch first?

I'd prioritize:

1. Purchase Order Hub
2. EDI Integration & Monitoring
3. Shipment Visibility
4. Vendor Performance Dashboard

Purchase orders form the foundation of the procurement process. Once they're flowing reliably through the platform, shipment tracking and analytics become much easier to build.

What would you intentionally leave out of the MVP?

I'd postpone:

- AI demand forecasting
- Automated vendor recommendations
- Dynamic procurement negotiations
- Contract lifecycle management
- Predictive stock replenishment

These features become much more valuable once we've collected reliable procurement and vendor performance data.

Good. How would you measure success?

I'd divide metrics into four buckets.

Metric Bucket	Metric
Adoption	• % of vendors onboarded to the platform • % of purchase orders processed through the platform • % of vendors using EDI integrations • Weekly active procurement managers • Weekly active vendor users
Operational Efficiency	• Average purchase order processing time • Average PO acknowledgement time • Average time to resolve EDI failures • Number of manual follow-ups per purchase order • % of procurement workflows completed without manual intervention
Supply Reliability	• On-time delivery rate • Fill rate • Shipment delay rate • ASN submission compliance • Invoice matching success rate
Business Impact	• Inventory availability at dark stores • Procurement cost per purchase order • Vendor satisfaction score • Procurement team productivity • Stockout rate

If you had to choose one North Star Metric, what would it be?

I'd choose **Procurement Cycle Time**, measured as the average time from purchase order creation to goods being received at the warehouse.
Reducing this directly improves procurement efficiency, inventory availability, vendor collaboration, and ultimately reduces stockouts for customers.

Sounds good. We can conclude the discussion here.

Design a new bicycle renting app to be used in a college's campus.

So, we have to design a bicycle renting application for the people staying in large college campuses. I want to confirm that the goal of this product will be to provide an easy mode of commute for the college's residents? Also, will these bicycles be allowed to be driven outside campus?

Yes, this is the goal. The bicycle can be driven only inside the campus.

Then this can be a mobile application, thus, giving users the flexibility to plan the commute from anywhere. Moving forward, I will first identify the customers for the product, then list down and prioritize their needs. Then, I will brainstorm the solutions and prioritize among them for the MVP. Lastly, I will define the success metrics for the product. Does this plan look good?

Yes, please continue.

There can be following type of users:

Residents:

- College students: They would be young (generally, 18 yrs to 30 yrs) and tech savvy. However, they will have lower willingness to pay. They might use the bicycles for faster commuting between hostels and academic complexes, and for leisure campus rides.
- Faculty members: The younger section of faculty members might be interested in utilizing the bicycles for commuting within campus. Their willingness to pay would be higher than that of students.
- College staff: They would comprise guards, mess workers and workers in food outlets. Fewer of these will be tech savvy and have higher willingness to pay than students. Their main goal would be to travel from one location to another faster and carry goods.

Visitor: They may visit the campus during fests, conferences and competitions. Their willingness to pay might be higher when compared to resident students. Their goal would be to travel from dorms to event locations faster and sightseeing.

Whom shall I focus on for this interview?

Let us focus on students.

Sure. I will first list down the needs of a student as per their user journey.

1. A student would want to find the nearest available bicycle so that she is aware of where the bicycle can be picked from. Furthermore, she would block the bicycle for some time so that no

one else takes it by the time she reaches there.

2. She might need directions to reach the nearest bicycle stand.
3. After reaching the location, she would have to identify her bicycle and unlock it to start the ride.
4. New students might want to be navigated to the destination. Students will also require an emergency contact number whom they can reach out in case the bicycle breaks in the route.
5. She will end the ride after reaching the destination so that she can stop the meter from charging her anymore.
6. She would want to make the payment through the apps she commonly uses so that she doesn't have to maintain a new wallet.
7. She would want to generate recurring bicycle booking requests so that she can have a mode to commute for each of her classes.
8. She might want to synchronize her academic calendar with bicycle booking so that a bicycle can be tentatively blocked for her.

For MVP, we would initially focus on fulfilling the following needs so that students rent the bicycle for easy commuting: 1, 2, 3, 5, 6.

Brainstorming few features for MVP of mobile application and also measuring their impact (*shown in the table at the end of the case*). Based on this, I will prioritize the following features (from the table): 1, 2, 3, 8, 10, 11, 12, 13, 16.

To measure the success of the product, I would track the following metrics:

- No. of app downloads per month
- No. of new registrations per month
- No. of rides per month
- Average duration of a ride
- Average distance of a ride
- Percentage of pickups from each point
- Percentage of drops from each points

Would you like me to do something more?

This is good. Thank you!

S.No.	Features	User Impact/Value	Cost of building impact/Effort
1	GPS on college campus map for finding the nearest pickup point with available bicycle	High	Low
2	Maintaining state of bicycle: Available, Not Available, Soon Available	High	Low
3	Blocking the bicycle and making it unavailable for other users for a time duration	High	Low
4	Measuring estimated time to reach destination and making it soon available at that point	High	High
5	If a bicycle is not currently available, user should get to know an estimated availability time	High	High
6	Whenever a user leaves hostel, provide him with a push notification if a bicycle is available near them	Low	High
7	The user should be able to navigate to the available bicycle	Low	Low
8	The user can scan a barcode on bicycle through app to unlock it and start the ride	High	Medium
9	The user enters the destination and be navigated to the nearest drop point from there	Medium	Low
10	The user should be able to end his ride through the mobile app	High	Low
11	When the user ends the ride, the bicycle should be locked automatically	High	Medium
12	The user should be allowed to end a ride if he is not at a drop location	High	Low
13	The user should be able to link UPI and wallets to the app for ease of payments	High	Medium
14	The user should be able to see the ride duration and distance travelled for past rides	Low	Medium
15	Gamification can be implemented by maintaining a leaderboard on the distance travelled and also providing users with badges for motivation	Medium	High
16	An alert should be shown if user takes the bicycle outside the campus	High	Low

Table: Features for MVP and corresponding impact

Build a Product to buy and sell antiques

Sure, I have a couple of preliminary questions. What is the scope of the product that we are looking to design, in terms of users, the nature of antiques, and the orchestration of the solution?

Good question. We are looking at an online marketplace where users can buy and sell antiques. By antiques, we imply goods that are older than 100 years.

Understood. What is the geography that we are targeting, and does the client have experience in delivering similar products or marketplaces in the past?

Let's assume the geographical target to be pan-India. Also assume that it is a startup venture that the client is looking to enter, with no prior experience.

Right. Given this information, I would define the goal of such a product to be an online marketplace to enable sellers to list their antiques and sell them to prospects, and enable buyers to search, view and buy antiques. Is the goal sufficiently defined, or would you like me to consider any other aspects? Also, I take it that the antique-based market is presently unorganized in India - is this a fair assumption?

This is fine.

I'll like to structure the design into three parts. First, I'll define the user personas for buyers and sellers and outline some use cases for each of them. Second, against each of the use cases, I'll present some potential solutions. Lastly, I would define some success metrics that can be employed for these proposed solutions. Is it a fair structure to follow, or would you like me to look into a specific aspect of the solution.

Looks good. Let's go through each of the segments one by one.

Sure. given the goal of our product, we would primarily have two segments of users - the buyers and sellers.

Among the buyers, we would have two sub-categorizations:

1. Purchase for self: Users who want to purchase antiques for themselves
2. Purchase for re-sale: Users who want to purchase antiques for re-sale in another market or potential buyers.

For the sellers, we can again have two broad sub-categorizations:

1. Direct sellers: Users who directly sell their personal antiques
2. Merchants: Users who own a business to sell antiques.

Let us only consider direct customers for the buy-side.

Understood. I'll now proceed with defining some use-cases for each user. For the seller, the following use-cases may be considered along with their priority order, which is defined in terms of usefulness to the customers for the MVP:

Use Case	Priority
I want to display and get maximum reach for my antiques	P1
I want to have a QnA platform to connect with the potential buyers and answer their queries	P2
I want to safely sell my product to the interested buyers	P1
I want to know about the customer feedback and ratings for the products sold in the past	P2

Use Cases for Direct Buyers (Self-purchase):

Use Case	Priority
I want to view the antiques that are available for sale	P1
I want to verify the authenticity of the antique	P1
I want to know the past reviews and ratings of the sellers that have been added by past buyers	P2
I want to safely carry out the purchase transaction	P1

In the initial phase, I would like to prioritize only the P1 features and propose solutions against them. Is this a fair approach? Do let me know in case there is any specific use-case that the client wants to incorporate or any re-prioritization that may be required by the client.

The use-cases are comprehensive. Let us proceed with some proposed solutions.

Understood. Before delving into the specific interface, I would also like to outline the interface for the product. Given the need for an online marketplace, I would propose the solution to be hosted as a reactive website at least in the initial phase, with further interfaces being added in the later stages of the product life cycle. The choice for web platform is based on two factors:

1. Lower cost of prototyping
2. Minimal issues with versioning and compatibility

The specific features for each of the PI use cases are as under:

User	Use Case	Potential Solutions	Business Impact	Cost
Seller	I want to display and get maximum reach for my antiques	Standard form fields for listing of products for sale. Additional option for sponsored listing for bumping-up the listing on the listing page.	H	L
		Analytics dashboard to track product metrics such as impressions on temporal basis.	H	M
		Integration with social media APIs to enable direct sharing of products on them.	H	L
	I want to safely sell my product to the interested buyers	Have a 3rd-party payment gateway integration (eg Razorpay, PayU).	H	M
		Order management dashboard to update status of dispatch and delivery and track current orders.	H	M
Buyer	I want to view the antiques that are available for sale	Multifilter functionality basis location of seller, nature of antique, pricing, seller, etc.	H	L
		Statistical ML model to use past purchase history/profile details to bump up relevant listings on the search feed	H	M

Buyer	I want to verify the authenticity of the antique	Add mandatory attribute for “certificate of authenticity” for each listing.	M	L
	I want to safely carry out the purchase transaction	Have a 3rd-party payment gateway integration (eg Razorpay, PayU)	H	M
		Live tracking of order status (Payment->Dispatch->Delivery)	M	L

The proposed solutions are comprehensive. Why don't you quickly list down the solutions that should be prioritized first.

Sure. I would like to prioritize the solutions basis two factors - Business Impact and Cost. By Business Impact, I aim to incorporate the importance of the said solution in developing a MVP and the first level go-live, and through cost, I aim to take into consideration the cost of development on a financial level.

Using the above two factors, the key solutions that should be targeted first are as under.

1. Standard form fields for listing of products for sale
2. Integration with social media APIs to enable direct sharing of products on them.
3. Multifilter functionality basis location of seller, nature of antique, pricing, seller, etc.
4. Add mandatory attribute for “certificate of authenticity” for each listing.
5. Order management dashboard to update status of dispatch, delivery, and track current orders.
6. Have a 3rd-party payment gateway integration.

While the first four features are prioritized basis their high business impact and low cost, the last two are prioritized since payments and order status form a crucial part of the journey of a marketplace solution

Fair analysis. Can you quickly define some success criteria before we wrap up this case?

Sure, some of the key success metrics for this solution can be as thus:

- I. Adoption Metrics:**
Number of new signups - both buyers and sellers - per month

2. Engagement Metrics:

- i. Average session duration
- ii. Number of listings viewed
- iii. Number of listings shared on social media
- iv. Purchase conversion rate

3. Retention Metrics:

Churn rate (percentage of users who stop using the platform)

Good analysis, we can close the case now.

Section 5.3

GTM Strategies

Our company wants to launch a social fitness product. How would you develop its go-to-market strategy?

Before I begin, I'd like to ask a few clarifying questions to understand the product, the problem it solves, and the context of this launch:

- 1. Product type - Is this a tangible product like a smartwatch, or a smartphone app?
- 2. Core value proposition - What does the product do - is it primarily about fitness tracking, social motivation, or a combination?
- 3. Third-party integrations - Does the product connect to existing social platforms or wearable devices?
- 4. Target market - Who is the intended user, and which geography are we launching in first?
- 5. Launch context - Is this a brand-new product, or has it been piloted? What is the goal - awareness, adoption, or revenue?

It's a smartphone app where users can share workout updates, diet plans, and progress with each other. They can collaborate through in-app video calls and compete in group or individual fitness challenges. The goal is to encourage health and fitness through peer motivation. Users can also share updates to Facebook and Instagram, and the app integrates with wearables like Apple Watch, Fitbit, and Garmin for automatic health data syncing. We're targeting 20-60 year-olds in urban and metropolitan India. The MVP launch and beta testing were successful and we're now ready to launch the full version. The goal is to build awareness, drive adoption, and generate revenue.

Thank you - that's very helpful. A few follow-up observations before I structure the GTM:

- Company context - You mentioned this is a sports equipment and apparel company diversifying into apps. That's an important asset - existing brand equity, retail touchpoints, and a customer base of fitness-oriented buyers can be activated for distribution and credibility.
- Competitive landscape - The app competes with MyFitnessPal, HealthifyMe, cult.fit, and FTTR. The key differentiator here is the social and challenge layer - most incumbents are primarily tracking tools. That's the wedge.

- Target segment - Given the beta success, I'd narrow the initial launch focus to urban users aged 20-35 who are already gym-goers or active on fitness-adjacent platforms. They have higher social sharing intent and are more likely to drive organic word-of-mouth.

I'll structure the GTM across three phases - Pre-Launch, Launch, and Post-Launch - and cover Pricing, Promotion, and Distribution for each:

Phase	Pricing & Product	Promotion	Distribution
Pre-Launch	Freemium model: free tier with core features; premium tier (₹199-₹299/month) with advanced analytics, unlimited challenges, and ad-free experience	- Seed 50-100 fitness influencers and personal trainers with early access for organic content creation - Release teaser content on Instagram and YouTube 4-6 weeks before launch - Waitlist campaign with referral incentives (early access, free premium month)	- Identify gym chains, corporate wellness programmes, and running clubs as beta partners - Collect structured feedback from beta cohort to inform launch priorities
Launch	- Exclusive early-adopter offer: 3 months premium free for first 10,000 sign-ups - No paid tier friction at onboarding - users discover value before paywall	- Performance marketing on Meta and Google targeting urban fitness-interested users aged 20-40 - Gym and wellness centre partnerships for in-venue promotion and QR-code installs - App Store optimisation (ASO) and SEO for organic discovery	- App Store (iOS) and Google Play Store - primary distribution - Gym partner installs and corporate wellness tie-ups as secondary channel

Phase	Pricing & Product	Promotion	Distribution
Post-Launch	- Introduce group and corporate wellness plans based on adoption data - Test cashback and reward integrations (e.g. Healtheon, fitness vouchers) for premium conversion	- Re-engagement campaigns for churned users - Feature-led PR around community milestones (e.g. 'IM workouts logged') - Referral programme with fitness merchandise rewards	- Expand to Tier 2 cities via targeted digital campaigns - Explore smartwatch / wearable integration (Apple Watch, Fitbit, Garmin) to increase stickiness - Plan international expansion based on India performance

Pricing Rationale

The freemium model is the right call for this stage because:

- Fitness apps have a high discovery-to-churn risk. Users need to experience the social motivation loop before they'll pay for it.
- A free tier maximises top-of-funnel reach, which is critical when building network effects - the product gets more valuable as more users join.
- The ₹199-₹299/month premium tier is benchmarked below HealthifyMe Pro (₹999/month) and cult.fit, making it accessible to price-sensitive urban users while maintaining a credible value proposition.

Success Metrics

I'd track performance across the full funnel, from acquisition through to revenue:

Stage	Metric	Why It Matters
Acquisition	# of downloads; Cost per install (CPI); Conversion rate from waitlist	Measures paid and organic reach efficiency
Activation	% of users completing onboarding; % who log a workout or post within Day 1	Identifies if users find immediate value - the critical first-session moment
Engagement	Avg. session time; # of posts/updates per user per week; feature usage breakdown	Shows which features drive habit; informs v2 priorities
Retention	Day 7, Day 30 retention rates; Churn rate by cohort	Core health indicator - social apps live or die on whether users come back
Revenue	Free-to-premium conversion rate; ARPU; MRR growth	Validates the freemium model and pricing assumptions
Satisfaction	NPS score; App Store ratings; qualitative review themes	Surfaces product-market fit signals and brand perception

The most important early signal will be Day 7 retention and activation rate. If users are not logging a workout or engaging socially within their first session, the onboarding flow needs to be revisited before scaling acquisition spend.

Post-Launch: V2 Planning

Based on engagement data and user feedback from the full launch, I'd plan v2 around three themes:

- Community depth - Group leaderboards, club-level challenges, and streaks to deepen the social habit.
- Corporate wellness - B2B channel targeting HR teams at large enterprises - a high-value acquisition path with lower CAC than consumer ads.
- Rewards and monetisation - Cashback integrations, fitness vouchers, and brand partnerships (sports equipment, nutrition) to drive premium conversion and ARPU growth.

Sounds good. We can close the case now.

Swiggy has decided to enter the table reservation category in restaurants. Formulate a launch strategy for the same.

Before structuring the GTM, a few clarifying questions:

- 1. Build approach - Should this be a new standalone app or a feature within the existing Swiggy app?
- 2. Stage of development - Is the product defined, or are we still in the conceptualisation phase?
- 3. Target geography - Which cities are we launching in first?
- 4. Goal - Is the primary objective user acquisition, restaurant partner growth, or revenue?

The product is still in the idea conceptualisation phase - I was hoping you could help with that too. It should be a feature within the existing Swiggy app.

Good. Building within the existing Swiggy app is the right call. It avoids the cold-start problem of a new app - Swiggy already has 90M+ users and strong restaurant relationships. We can distribute instantly without any install friction.

Before moving to the GTM, let me map the stakeholders and their pain points - this will ground the feature design and the launch strategy in real user needs:

Stakeholder	Pain Points	What the Feature Solves
Customers	• Don't know wait times, cuisines, or table availability before arriving • Dislike queuing - especially for planned occasions • Want parking and timing information upfront	• Browse restaurants, check real-time availability, and book a table in advance • Pre-order dishes during booking to reduce wait after arrival • Arrive with confidence - no queue, no uncertainty
Restaurants	• No-shows and late arrivals block tables and cost revenue • Unpredictable demand makes staffing and prep difficult • Waiting customers are lost if competitors are nearby	• Estimated arrival times help kitchens prepare orders ahead of time • Buffer time policy protects against table-blocking without penalising punctual guests • Better demand visibility enables staffing and inventory planning

Stakeholder	Pain Points	What the Feature Solves
Swiggy	• Currently absent from the dine-in occasion - losing engagement to Zomato • Existing user base not monetised beyond food delivery	• Expands Swiggy's presence into the dine-in journey • Increases DAU and session frequency through a new use case • New revenue stream via booking fees or premium restaurant listings

But doesn't Zomato already provide table reservation features?

Yes - and that's an important competitive signal. But the key differentiator here is pre-ordering. Zomato's reservation feature lets users book a table, but the order is still placed at the restaurant.

Swiggy's feature can allow customers to pre-order dishes during the booking flow, so the kitchen starts preparing before they arrive. This reduces wait time significantly and turns the reservation into a full-service, seamless dine-in experience - something Zomato doesn't currently offer at scale.

This also addresses the restaurant's biggest fear about reservations: blocking a table for a customer who might not show up or arrives late. To manage this, we can introduce a configurable buffer window (e.g. 15 minutes) - the restaurant holds the table for the duration of the buffer.

If the customer arrives after the buffer expires and the restaurant is full, they join the walk-in queue. This protects restaurant revenue without penalising punctual guests.

Considering this to be what the application does, how would you go about launching it?

I'll structure the launch across three phases - Pre-Launch, Launch, and Post-Launch - covering Product & Pricing, Promotion, and Distribution for each:

Phase	Product & Pricing	Promotion	Distribution
Pre-Launch	- Launch as a feature within the existing Swiggy app - no new install required for existing users - No booking fee initially - free for customers and restaurants to lower adoption friction	- In-app banners and push notifications to existing Swiggy user base - Teaser campaign on Instagram and YouTube targeting urban millennials - Outreach to 200-300 premium restaurants in Mumbai, Bengaluru, and Delhi for early onboarding	- Leverage existing Swiggy app distribution - 90M+ users already installed - Restaurant partner sign-up via dedicated onboarding team and self-serve portal
Launch	- Introduce referral incentive: existing users get a discount voucher for every new user who makes their first reservation through a shared link - Exclusive launch offers - flat discounts or complimentary desserts - negotiated with partner restaurants for early adopters	- Smart nudges in the app: surface the reservation feature when users search for a restaurant (not just food delivery) - Social sharing of reservations to drive organic awareness - Targeted ads in dining-heavy localities and near premium restaurant clusters	- App Store and Google Play via existing Swiggy app update - Restaurant partners promoted as 'Swiggy Reserve' outlets - visible badge in search and discovery

Phase	Product & Pricing	Promotion	Distribution
Post-Launch	- Introduce convenience fee (₹20-₹50 per booking) once the feature has critical mass - Premium restaurant listing - paid placement for restaurants wanting higher visibility - Group booking and corporate dining packages	- Re-engagement campaigns for users who browsed but didn't book - Feature-led PR: 'X million tables reserved on Swiggy' milestone content - Partner with food bloggers and review platforms for discovery	- Expand to Tier 2 cities (Pune, Hyderabad, Ahmedabad) based on adoption data - Integrate with Google Maps and Apple Maps for restaurant discovery outside the Swiggy app - Explore corporate tie-ups for team dining and client entertainment use cases

Success Metrics

I'd track performance across six dimensions - with reliability metrics being the most critical, since a feature that fails to hold reservations destroys trust faster than it builds it:

Stage	Metric	Why It Matters
Acquisition	# of reservations made; Cost per first booking; Conversion rate from feature page views	Measures whether the feature is discoverable and compelling enough to drive action
Engagement	# of reservations per active user per month; % of reservations with pre-orders; Bounce rate on reservation page	Pre-order attachment rate is the key differentiator vs Zomato - it validates the unique value proposition
Revenue	Booking fee revenue (post-launch); Premium listing revenue; ARPU from reservation users	Validates the path to monetisation once the feature has proven user and restaurant value

Stage	Metric	Why It Matters
Supply Quality	# of restaurant partners onboarded; % of listed restaurants with real-time availability	A thin restaurant supply kills the feature - this tracks the health of the two-sided marketplace
Reliability	% of customers who arrived within the buffer window; No-show rate; Restaurant cancellation rate	Trust is the core product - if reservations don't hold, the feature fails regardless of acquisition numbers
Retention	% of users who make a second reservation within 30 days; Feature DAU/MAU ratio	Repeat booking is the signal that the product has created a new habit, not just a one-time trial

The most important early signal will be the repeat booking rate within 30 days. A single reservation is a trial. A second reservation is a habit. If repeat bookings are low, it signals either a supply problem (not enough restaurants) or a reliability problem (reservations not being honoured) - both require different interventions.

Expansion Roadmap

Based on performance data from the initial metro launch, I'd plan expansion in three directions:

- Geographic - roll out to Tier 2 cities (Pune, Hyderabad, Ahmedabad, Chandigarh) where dining out is growing but table reservation tools are absent.
- Vertical - extend to cafes, cloud kitchens with dine-in sections, and corporate dining - broadening the addressable restaurant base.
- Monetisation - introduce a per-booking convenience fee (₹20-₹50) once NPS and reliability metrics confirm the feature is trusted. Layer in premium restaurant listings and corporate dining packages.

Sounds good. We can close the case now.

A B2B SaaS company has built an AI Sales Assistant. It joins sales calls, creates meeting summaries, captures action items, and updates CRM fields automatically. The company wants to launch this product for mid-market sales teams in India. How would you design the go-to-market strategy?

I'd first clarify the product and the launch objective. Is this a standalone product or an add-on to an existing product?

It is an add-on. The company already sells a CRM workflow tool to mid-market companies.

Got it. Since it is an add-on, existing customers may be the best starting point. I'd also clarify the main objective of the launch. Are we trying to drive adoption first, revenue first, or mainly learn from early users?

The immediate goal is adoption among existing customers. Revenue expansion comes next.

Then I'd structure the GTM around five parts: target customer, value proposition, launch plan, pricing, and success metrics.

First, I'd define the customer. Since this is a B2B product, I would separate the **user**, **buyer**, and **influencer**.

The users are sales reps who spend time writing notes and updating CRM after calls. The buyer is likely the Head of Sales or Sales Operations Head. Other teams like IT and legal may also influence the decision because call data is sensitive.

For the first launch, I'd target existing mid-market customers with large sales teams, high call volume, and regular CRM usage. These customers will feel the problem more clearly and can show value faster.

What problem would you focus on?

The main problem is that sales reps spend too much time on post-call admin work, while managers still do not get clean and updated CRM data.

So the value proposition would be:

"Help sales teams reduce manual post-call work and keep CRM data updated by automatically capturing summaries, action items, and next steps from sales calls."

I would not position it as just an "AI tool." The message should focus on time saved, cleaner CRM data, and better follow-ups.

How would you launch it?

I would start with a pilot-led launch for existing customers.

I'd pick 5-10 current customers and run a 6-8 week pilot. The pilot should be narrow: one sales team, one CRM workflow, and three core use cases - meeting summaries, action items, and CRM updates.

The goal of the pilot would be to check three things:

1. Are sales reps using it regularly?
2. Is it saving time or improving CRM data?
3. Are customers willing to pay after seeing the value?

Why start with existing customers?

Because trust and integration are already easier with existing customers. In B2B, adoption is not only about the product. It also depends on internal approval, data security, workflow fit, and proof of value.

Existing customers will help us learn faster and create early success stories.

How would you price it?

I'd start with a free pilot and then convert it into a paid add-on.

The pricing can be simple:

- Free 6-8 week pilot
- Paid monthly add-on per sales user
- Team-level pricing for larger teams
- Custom pricing for companies needing extra security or integrations

I would avoid usage-based pricing at the start because buyers may worry about unpredictable costs.

Which channels would you use?

For the first phase, I'd use a sales-led approach.

The main channels would be:

- Account managers reaching out to existing customers
- Product demos for Sales Heads and Sales Operations teams
- Webinars around sales productivity
- Case studies from successful pilots
- In-product nudges for existing admins and managers

For new customers later, I'd use LinkedIn campaigns, CRM partner channels, and ROI-based sales outreach.

What metrics would you track?

I'd track three types of metrics.

For adoption:

- Number of pilot teams onboarded
- Weekly active sales reps
- Calls captured per rep
- Summary acceptance or edit rate

For value:

- Time saved per rep
- Improvement in CRM data completeness
- Reduction in missed follow-ups
- Rep and manager satisfaction

For business:

- Pilot-to-paid conversion

- Add-on revenue
- Expansion within existing accounts

The most important early metric would be weekly active usage by sales reps. If reps do not use it regularly, the product will not create value for the buyer.

What risks do you see?

There are four key risks.

First, sales reps may feel the tool is monitoring them. So, the positioning should be productivity-first, not surveillance-first.

Second, users may not trust the summaries. So, they should be able to review and edit them before CRM updates are pushed.

Third, data security and consent can slow adoption. So, we need clear recording consent, access controls, and data retention settings.

Fourth, CRM integration must be smooth. If the tool creates extra work, adoption will suffer.

What is your final recommendation?

I would launch the AI Sales Assistant through a focused pilot-led GTM for existing mid-market customers. The first target should be sales teams with high call volume and regular CRM usage.

The product should be positioned around reducing manual post-call work and keeping CRM data updated, not just around AI. I'd run 5-10 pilots, prove usage and value, convert successful pilots into paid add-ons, and then use those success stories to expand into new customers.

PM Tip: In B2B GTM cases, always separate the user, buyer, and influencer. The person using the product may not be the person paying for it. A strong answer should clearly cover the target customer, pain point, value proposition, launch plan, pricing, success metrics, and risks that could block adoption.

The tata group wants to incentivize their customers and want to increase their loyalty towards the brand. We need your help to prepare a loyalty program for the conglomerate.

To make sure I understand the problem correctly, we're designing a single loyalty program across all Tata brands-such as Taj, Titan, Tanishq and automobiles.

Yes, that is correct.

Many Tata brands already have their own loyalty programs. What happens to those?

You can assume that all the existing loyalty programs will be replaced by the new loyalty program that you would suggest.

One concern: customers have already earned reward points. Invalidating them would create a poor customer experience.

Good that you mentioned it. In your solution, ensure that there is a smooth transition for the existing reward system.

Sure. One last clarification before I begin-will brands have flexibility in rewarding customers, or will Tata Group control everything centrally?

We would like you to think how this Loyalty program should be structured. Also, comment on its technical feasibility.

Sure. May I take a couple of minutes to structure my thoughts?

Sure.

I'd divide the solution into three parts:

Reward structure

Technical implementation

Customer experience

For the reward structure, I think there should be three levels:

Tata Group Points - Common rewards across all Tata brands.

Brand Points - Individual brands can run their own reward programs.

Store Points - Local stores can run city/store-specific promotions.

A customer can earn one or more of these depending on the campaign.

Makes sense, carry on.

This hierarchy gives flexibility while keeping the experience unified.

- Tata Group manages ecosystem-wide rewards.
- Brands control brand-specific campaigns.
- Stores can run local offers when needed.

Usually when the campaign runs, the points are provided in a particular city. How will you ensure that?

Instead of asking every store to configure campaigns:

- Configure offers at the brand or Tata level.
- Use the Store ID to determine eligibility.
- Stores automatically inherit the applicable campaigns.

This keeps campaign management scalable.

That seems like a good plan. Can you explain the implementation plan?

I'd use the customer's mobile number as the primary identifier because it's:

- Easy to remember
- Already available across brands
- Suitable for creating a unified customer profile

One limitation is that customer identity will exist at the Tata level instead of the brand level. To solve this, every brand should register customers against the common Tata ID.

Can you suggest other things to consider for a good loyalty program?

The real value of loyalty comes from redemption.

I'd enable:

Redemption within the same brand.

Cross-brand redemption wherever possible.

For example, points earned at Titan could also be redeemed at Starbucks.

Interesting, anything else you'd like to add?

One last point, we discussed the existing reward points at the beginning.

I'd ensure a smooth migration by:

- Mapping all existing reward points to the new customer ID.
- Migrating historical customer data.
- Running a one-time data migration before launch.

This ensures customers don't lose their rewards while transitioning to the new system.

Thank you for the insights.

Section 5.4

Metrics

You are a product manager at Instagram working on the Creator Commerce product tagging feature. How would you go about measuring its success?

Before I begin, let me confirm my understanding.

Creator Commerce allows creators to tag products from partner brands in their posts, reels, and stories. Users can tap the tag, view product details, and complete the purchase on the merchant's website.

Is that accurate?

The description is accurate. Please proceed with the rest of the solution.

Great. Since this feature serves multiple stakeholders, I'd first map their journeys before defining metrics.

The three stakeholders are:

- Creator
- Buyer
- Brand partner

If that's okay, I'd first cover the creator and buyer journeys.

The approach seems good. Let's keep the focus on the creators and buyers. You can skip the brand-side metrics for now. Please go on with the journeys.

Sure.

Creator Journey

- Discover the Creator Commerce program.
- Verify eligibility and enable product tagging.
- Tag products in posts, reels and stories.
- Track performance through Instagram Insights.
- Optimize content based on results.

Buyer Journey

- Discover tagged products while browsing Instagram.
- Open the product detail page.
- Visit the merchant website.
- Complete the purchase.
- Return to Instagram for future engagement.

You've captured the journeys accurately. Please go ahead with the metrics.

I'll define metrics separately for creators and buyers.

Creator Metrics

Awareness

- % eligible creators aware of the program
- Invite CTR

Engagement

- % creators enabling product tagging
- Tagged posts/reels per creator
- Weekly active tagging creators

Conversion

- Outbound clicks to merchant sites
- GMV attributed to creator content
- % creators driving purchases

That's a solid set. What about metrics for buyers?

I'd use a similar funnel.

Buyer Metrics

Awareness

- % users exposed to tagged products
- Product tag CTR

Engagement

- Product detail views
- Time spent on product pages

Conversion

- Merchant website CTR
- Purchase conversion rate
- Repeat outbound clicks

Do you think we can consider any other metrics to track for this program?

Yes. I'd also monitor operational and guardrail metrics

Operational Metrics

- Outbound link success rate
- Merchant page load time
- Attribution accuracy

Guardrail Metrics

- Creator satisfaction
- Abuse/fraud reports
- Buyer complaints about checkout or product quality

Makes sense. How would you go about defining a North Star and tracking all these metrics?

Since Instagram doesn't own checkout, the North Star should reflect successful commerce driven by creators.

North Star : Creator-attributed Purchase GMV

(or Outbound Purchase Conversion from Creator Content)

To measure this, we'd instrument events across the funnel:

- Product tag impressions
- Product tag clicks
- Product detail views
- Outbound merchant clicks
- Purchases attributed using creator and campaign IDs

I'd also capture creator and buyer feedback through lightweight post-purchase surveys.

Thanks for the great discussion. We can end the interview.

You are the PM for Facebook Marketplace. What metrics would you track to measure the health and success of the feature?

Before jumping into metrics, I'd like to confirm my understanding.

Facebook Marketplace lets buyers and sellers discover, buy, and sell products within their local communities. Sellers can list both new and second-hand products, while buyers discover listings through Marketplace or Facebook groups.

I'll assume the primary business goal is to increase user engagement on the platform.

You are right. Please go ahead.

Before defining metrics, I'd like to map the journeys of both stakeholders so the metrics align with the user actions.

Seller Journey

- Visit Marketplace
- Create a listing
- Add product details & price
- Receive buyer interest
- Accept/reject an offer
- Receive payment
- Handover the product

The last two steps happen outside Facebook's ecosystem.

Buyer Journey

- Visit Marketplace
- Search/discover a product
- Contact the seller
- Make payment
- Receive the product

There is also another entry point where buyers discover listings through Facebook Groups before reaching Marketplace.

Your understanding of the journey of the stakeholders is fair. Let's move ahead to the metrics

I'd group the metrics into four buckets: **Awareness, Activation, Conversion and Retention.**

1. Awareness

Measures whether users are discovering Marketplace.

- Users visiting Marketplace
- Marketplace tab clicks/day

2. Activation

Measures active participation on the platform.

- Products listed/week
- Bounce rate
- Product listing clicks/week

3. Conversion

Measures successful buyer-seller interactions.

- Unique interactions/week
- Purchases/week

4. Retention

Measures repeat usage by buyers and sellers.

- Repeat purchase rate
- Repeat listing rate

Alright, out of all these metrics mentioned, which one should be prioritized by us as the top metric?

I'd segment Activation and Conversion metrics by product category to uncover category-specific trends.

Since the objective is increasing engagement, my **North Star Metric** would be:

Unique Interactions per Week

This captures meaningful buyer-seller activity on the platform.

As a counter metric, I'd track:

Purchases per Week

This ensures we're not optimizing engagement at the cost of successful transactions or marketplace quality.

You are the Digital Product Manager of Big Basket. What would be the metrics you would like to follow?

I'd first like to confirm my understanding of BigBasket.

BigBasket is an online grocery platform where users can order groceries, fruits & vegetables, packaged foods, household essentials, and personal care products through its mobile app or website.

Yes, you may move forward.

Before defining metrics, could you share the business goal we'd like to measure?

Assume that the goal of the business is to increase the value of orders placed via the BigBasket mobile app.

Understood. I'll first break down the business goal into measurable drivers and then map the user journey to identify the right metrics.

Should I cover both the website and the mobile app?

You can define the metrics for only the mobile app for now.

Sure.

The business goal can be represented as:

- **Total Order Value = Number of Orders × Average Order Value (AOV)**
- **Number of Orders = Visitors × Add-to-Cart Rate × Purchase Conversion Rate**
- **Visitors = New Visitors + Returning Visitors**

I'll now map the user journey.

The mobile app journey would broadly be:

- Install the app
- Open the app
- Browse products via home page, categories or search
- Add products to cart or save them to a list
- Log in / Sign up (if required)

- Review cart, choose address & delivery slot, apply coupons or opt for BBStar
- Complete payment
- Receive order confirmation
- Track, modify or cancel the order through the app
- Receive delivery and rate the experience

Based on this, I'd divide the metrics into five buckets.

Awareness

Measures whether users are discovering and installing the app.

- Number of app installs per month
- % installs from organic sources (Search, Website)
- % installs from paid campaigns (Ads, Influencers)
- App Store ranking in Shopping/Grocery
- Click-to-install conversion rate
- Monthly uninstall rate
- Average App Store rating

Engagement

Measures how actively users explore products before purchasing.

- Bounce rate by product/category page
- Average session duration
- Average products viewed per session
- Average quantity added to cart
- Number of custom lists/wishlists created
- % users using in-app search
- % logged-in users
- % new registrations
- Daily Active Users (DAU)
- Monthly Active Users (MAU)
- Monthly value of returned orders

Conversion

Measures how efficiently browsing converts into completed purchases.

- Average number of items per order
- Average basket value
- Cart abandonment rate
- Order cancellation rate
- Average Order Value (AOV)
- % users accepting product recommendations during checkout
- % users applying coupons/vouchers
- Average voucher value redeemed

Retention

Measures whether users continue purchasing through the app.

% users subscribing to BBStar

- Repeat purchase rate
- Purchase frequency (orders/user/month)
- Average time between two purchases

Referral

Measures growth driven by existing customers.

- % users sharing referral codes
- Referral conversion rate
- Revenue generated through referrals
- Net Promoter Score (NPS)

This is great. Thank you!

We just shipped an AI feature that auto-summarizes long email threads at the top of our inbox product. How would you measure its success?

Let me first make sure I understand the feature. When a user opens a long thread, we show a generated Too Long ; Didn't Read at the top, is that right? And is the goal mainly to save reading time, or also to drive more replies and actions?

That's right. The main goal is saving reading time and helping users act faster. Trust matters too, we can't have it inventing things.

Then I'd measure two things separately: whether the feature is solving the user's problem, and whether the model itself is good. Should I start with the product side?

Yes - but before you list metrics, how would you decide where to look?

I'd anchor to the user journey: the user opens a long thread, sees the summary, reads it, then acts - replies or archives - and may still expand the full thread. I'll hang the metrics off that flow.

Good. Go ahead with the product metrics.

I'd structure them as a North Star metric with supporting and guardrail metrics:

- **North Star:** share of long threads where the user acts - replies or archives - without opening the full thread.
- **Supporting - adoption:** percentage of eligible users who view a summary; summaries viewed per active user per week.
- **Supporting - value:** time-to-first-action on a thread, comparing summary users against a holdout group that never sees summaries.
- **Guardrail - trust:** thumbs-down rate and the rate at which users expand to the full thread anyway - a high expansion rate signals low trust in the summary.

I like the trust guardrail. But product metrics won't tell me if the summaries are actually any good - what about the model itself?

Exactly - that's the second set: metrics that measure quality independent of how much the feature is used.

- **Faithfulness:** percentage of summaries fully grounded in the thread with no invented facts

(hallucination rate), measured on a labeled sample.

- **Coverage:** a human or LLM-judge rating of whether key decisions and action items were captured.
- **Performance:** p95 latency to render the summary, and cost per summary - both influence whether users wait for it.
- **Safety and fairness:** error and complaint rates checked across languages, thread lengths, and user segments, to catch bias.

Suppose you can only gate the launch on one metric. Which one, and why?

The hallucination rate on the labeled sample. A summary that invents a commitment or a number is actively harmful in an inbox, so I'd set a hard ceiling on that metric and treat adoption and time saved as things to optimize only once trust clears that bar.

PM Tip: Separate “is the feature valuable” from “is the model good,” and gate the launch on a trust metric rather than an engagement metric. This is the core move in any AI-metrics question: for a feature that can fabricate information, a false positive is more costly than a missed opportunity.

Agreed - that's the right instinct for an AI feature.

You are a product manager in the Google Pay team. You have been given the task of defining the metrics for tie-in promotions on Google Pay. How will you go about it?

Before I begin, let me confirm my understanding of the feature.

Google Pay rewards users with scratch cards after eligible payments. These rewards may be cashback or coupons from partner brands like Lenskart, Mamaearth, Zomato, etc. These partner coupons are tie-in promotions.

Is that accurate?

Your description of tie-in promotions is accurate. Essentially tie-in promotions involve a company partnership with another company for promotions. You can proceed further with your analysis.

Great.

The metrics should align with the business goals.

- Users should find the promotions useful enough to continue using Google Pay.
- Promotion partners should benefit through customer acquisition and revenue.

Accordingly, I'd define metrics from the perspective of **users** and **promotion partners**.

The business goals seem accurate. Please proceed with the metrics.

Great.

The metrics should align with the business goals.

- Users should find the promotions useful enough to continue using Google Pay.
- Promotion partners should benefit through customer acquisition and revenue.

Accordingly, I'd define metrics from the perspective of **users** and **promotion partners**.

The business goals seem accurate. Please proceed with the metrics.

User Metrics :

The user metrics should measure the frequency, utilization and effectiveness of promotions.

Success Formula :

No. of successful uses of promotions = No. of payments $\times$ % of tie-in promotion scratch cards $\times$ % of tie-in promotions used

Key metrics:

- Number of payments / payment frequency
- % of scratch cards that are tie-in promotions
- % of promotions that expire unused
- Average number of brands/promotions shown per user
- % of repeat promotions from the same brand/type

Promotion Partner Metrics :

From a partner's perspective, success depends on how effectively Google Pay drives customer engagement.

Success Formula :

No. of successful uses of a specific promotion = No. of payments $\times$ % of promotional coupons for a given partner $\times$ % of that specific coupon $\times$ % of users successfully using the promotional coupon

Key metrics:

- Number of tie-in promotion partners
- Partner retention rate
- Partner churn rate
- Average coupons shown per partner
- % of coupons expiring unused (overall and partner-wise)
- Average coupon recipients per campaign
- % of users redeeming a partner's promotional coupon

These metrics help identify which promotions are performing well and which need to be redesigned or discontinued.

I think this covers most of the important metrics for tie-in promotions.

How will you measure the success of WhatsApp's in-app video playing feature?

First, I would like to describe the feature and my understanding of its objective. WhatsApp has introduced an in-app video playing feature, whereby users can play YouTube videos from within the chat window itself. I think the objective of the feature is to keep the users within the app context, so that the time spent on WhatsApp can be increased. There are three types of interactions that can be done when this feature is in use:

1. Expand the video to full screen
2. Close the video
3. Watch the video on YouTube by clicking on the YouTube icon

That sounds good. Please proceed.

The user's journey for using this feature would look like this:

- Activation: The user receives a link to a YouTube video on WhatsApp
- Engagement:
 - They click on the link, and the video starts playing within the app
 - They can expand the video to full screen or stop viewing by clicking on the close button.
- Alternatively, they can click on the YouTube button to be taken to the YouTube app. If they choose this option, they can either continue browsing on YouTube (Leakage) or return to WhatsApp.
- Post-Engagement: After video interaction, the user may resume chatting, share the video, or initiate a call within WhatsApp

Since this feature would be available only on phones with an Android version that supports in-app windows and an updated version of the app, all the metrics should account for that.

Activation:

- % of users who play a link they are sent
- Relative activation rate: % of users with this feature available that play a video v/s % of users without the feature who play a video (which opens the YouTube app instead)

Engagement:

- % of users who played the link and watched the video (either partly or wholly)
- Relative return rate: % of users who stayed in WhatsApp after they stopped watching the video v/s % of users without the feature who return to WhatsApp after watching the video
- % of users who share the video link within WhatsApp after watching
- Time spent on WhatsApp by users with this feature v/s without this feature

Leakage:

- % of users with the feature opting to view the video on YouTube
- % of users who opted to view the video on YouTube that returned to WhatsApp

Okay, and how would you use these metrics to evaluate the feature's success?

The engagement metrics will show whether the feature has boosted time spent on WhatsApp and whether it enables users to engage with YouTube content more frequently. The leakage metrics will help identify if the feature successfully limits app-switching or if significant drop-offs still occur.

Great, can you please summarize in 30 seconds?

To summarize, the in-app video playing feature is an effort by WhatsApp to boost time spent by users within WhatsApp and enable more media consumption for users. Primarily, the relative activation and retention rates will help evaluate these feature value propositions.

Section 5.5

Pricing

Assume you are the new product manager in our Amazon Prime business and are deciding pricing. The vice president would like to lower the price from \$99.99 per year to \$89.99 per year. Making your own assumptions, develop the financial projections for this decision.

A lower price should increase Prime adoption. At the same time, it will reduce subscription revenue from existing members and increase shipping costs. I'll estimate the impact separately for existing and new Prime customers.

Let's assume there are **200 million Prime members** today.

1. Existing Prime Customers

Reducing the membership fee by \$10 leads to a direct revenue loss of:

$$\text{\$10} \times \text{200M} = \text{\$2 billion}$$

Net impact: -\$2B

2. New Prime Customers

Next, I'll estimate the impact from new customers acquired due to the price reduction.

Let's assume Prime membership grows by **20%**, adding **40 million** new members.

Membership Revenue

Incremental membership revenue:

$$\text{40M} \times \text{\$90} = \text{\$3.6B}$$

Incremental Shipping Cost

Assume each new Prime customer increases shipping costs by **\$20/year**.

$$\text{40M} \times \text{\$20} = \text{\$0.8B}$$

Incremental Commerce Revenue

Prime members tend to purchase more because of free 2-day shipping.

Assume each new Prime member spends an additional **\$40/year**.

At a **25% gross margin**, that's **\$10 profit/customer**.

$$\text{40M} \times \text{\$10} = \text{\$0.4B}$$

To recap the annual impact:

Existing Prime customers: -\$2.0B

New Prime customers:

- **+\$3.6B** Membership revenue
- **-\$0.8B** Incremental shipping cost
- **+\$0.4B** Incremental gross margin

Net impact: +\$3.2B

So, what's the overall change in revenue, and what's your recommendation?

Overall annual impact:

$$\text{\$-2.0B} + \text{\$3.2B} = \text{\$1.2B}$$

Based on these assumptions, reducing the annual membership fee by **\$10** results in an estimated **\$1.2B net annual upside**.

Recommendation: Proceed with the price reduction.

JioSaavn is a music streaming service that wants to launch a family music plan for its users. How would you go about it?

From what I understand, JioSaavn wants to introduce a new subscription plan targeted at families. I'll assume the objective is to come up with an appropriate pricing strategy.

Yes, go ahead.

Before recommending a price, I'd like to clarify the business goal. I'll assume the goal is to increase paid subscriptions by attracting households, especially members who don't usually purchase individual plans, such as children or elderly family members.

Fair enough. How do you go about pricing such a plan?

Given how competitive music streaming is in India, I'd primarily use competitor-based pricing. Before benchmarking prices, I'd identify JioSaavn's target segment. Based on its positioning, I'd assume it focuses more on regional music listeners, particularly in Tier 2 and Tier 3 cities.

Okay, go ahead.

The closest competitors would be Spotify and YouTube Music. I'd compare their family plans before deciding our price.

Could you share the latest pricing for these platforms?

Of course. Here are the current approximate monthly plans:

- JioSaavn: Individual - ₹99/month; Duo (2 users) - ₹129/month
- Spotify: ₹179/month for up to 6 users
- YouTube Music: ₹189/month for up to 6 users (via YouTube Premium)

Looking at these benchmarks, I'd recommend pricing the JioSaavn Family Plan at ₹149/month for up to 6 users.

Here's why:

- It is meaningfully cheaper than Spotify and YouTube Music.
- It complements JioSaavn's strength in regional content.
- It aligns well with price-sensitive households in Tier 2 and Tier 3 cities.
- It creates a strong value proposition without triggering an aggressive price war.

We should also validate the pricing through small-scale experiments to understand willingness to pay and price sensitivity.

I'd compare our paid subscriber base with competitors as well. If we're significantly behind, a more aggressive introductory price could help accelerate adoption.

Overall, the idea is to keep the plan affordable, leverage JioSaavn's strengths, and price it to maximize household adoption rather than short-term revenue.

Great! This sounds good.

The Indian OTT market is intensely competitive and price-sensitive. Netflix has already implemented price cuts and a mobile-only plan. Now, with the global rollout of its 'paid sharing' policy to curb password sharing, you, as a Product Manager at Netflix, need to refine the pricing and product strategy for India to continue growing the subscriber base and revenue.

I understand that the objective here is twofold: continue to grow our subscriber base while also increasing revenue, specifically by converting password-sharers into paying customers.

Yes, that sounds fair. Go ahead.

To structure my approach, I'll use the 5 Cs framework: Company, Customers, Competitors, Collaborators, and Context. I'll begin with what I know and then ask for a few details.

Sure, you may take your time and go ahead.

- 1. Company (Netflix):** Our goals are shifting from pure subscriber growth to profitable growth, meaning Average Revenue Per User (ARPU) is now a key metric. The "paid sharing" feature is the critical global initiative to support this. Our core competencies remain our content engine and personalization algorithm.
- 2. Customers:** The Indian market isn't monolithic. I see three key segments:
 - **Mobile-First Youth:** Highly price-sensitive, needs a low-cost, mobile-only option.
 - **Family Household:** Values multiple screens and 4K quality for Smart TVs. They are less price-sensitive and prioritize convenience.
 - **Converted Sharer:** This is a new, important segment. They are being forced into a purchase decision and need an affordable, low-friction way to keep their profile and viewing history.
- 3. Competitors:** I know the landscape is defined by bundling and low prices from major players like Amazon Prime Video, Jio Hotstar. To ground my recommendations in the current market, could you provide the latest pricing tiers for us and our key competitors?

Of course. Here are the current approximate monthly plans:

- Netflix: Mobile (₹149), Basic (₹199), Standard (₹499), Premium (₹649). Netflix doesn't offer annual plans.

- Amazon Prime: Included with Prime membership at ₹299/month (or ₹1499/year), which also includes shopping and music.
- Jio Hotstar: Has multiple tiers, with its premium ad-free plan at ₹299/month (or ₹1499/year).

This data confirms my hypothesis: our premium plans are priced significantly higher than competitors, who leverage bundles (Amazon) and live sports (Jio Hotstar) as key value propositions. Our core value is our original content library. Competing with them on prices will neither justify our costs nor our value proposition. Moving on to cost-based pricing, it doesn't make sense to analyze it from a cost perspective, as the variable cost involved in serving every additional subscriber is close to null. This industry primarily comprises high fixed costs of obtaining content licenses, content production, IT infrastructure, etc., and hence shouldn't be looked at from a cost-plus pricing angle.

- 4. Collaborators:** Telco operators like Jio and Airtel are key acquisition channels through bundling. OEM partnerships for pre-installation on devices also reduce friction.
- 5. Context:** The password-sharing crackdown is the most significant trend. Additionally, the 5G rollout makes HD/4K plans more appealing, and the ubiquity of UPI simplifies payments.

That's a thorough analysis, especially highlighting the new "Converted Sharer" segment and the competitive pressures from bundled services. Based on this, what are your specific recommendations on the "paid sharing feature"?

Based on that analysis, my recommendations are focused on successfully implementing paid sharing while refining our offerings for key segments.

- 1. Optimize the "Paid Sharing" Feature for India:** This is our biggest opportunity to boost ARPU. The "Profile Transfer" feature must be flawless. I propose pricing the "add an extra member" slot at ₹99/month.
 - a. Rationale:** Our ad-free Basic plan is ₹199. At ₹99, the fee is less than half the cost of a new plan, making it a low-friction "yes" for someone forced to pay for the first time.

Excerpt: While most pricing questions cater to setting the appropriate price for new products or services, negotiating vendor contracts is another kind of pricing problem that a manager might encounter

Hi, you are a product manager at a popular cloud services company. Your main product is a platform that provides on-demand GPU instances for AI startups and researchers to train their models. You source your capacity from a third-party GPU server vendor. This vendor is dissatisfied with the current pricing contract and is asking for the payment of significant overage charges. How would you proceed?

That's an interesting question. To begin with I would like to clarify some aspects of the problem.

Go ahead.

I would like to understand what kind of pricing structure is currently in place, and what is the reason for which the vendor is asking for the renegotiation?

Currently, we have a pay-per-use payment model. Here, the amount to be paid per user depends on the band in which the lump sum requirements of the GPU server's capacity lie. Recently, a major new open-source AI model was released, and there has been a massive, unexpected surge in demand from our customers wanting to use it.

Alright. So, this surge in usage could be from more customers signing up to use the new model, or our existing customers running much larger or more frequent training jobs. Or both. Which one has been the case with us?

Good question. So, we have had a spike in the number of users and in addition, the average time for which a user engages with the platform has increased too.

Also, to understand the business requirements better, I would like to know more about the business model in place. Typically, such platforms would either have a pure pay-as-you-go model or a subscription-based model with a certain number of credits. If our model is purely pay-as-you-go, we should be in a better position, because the increase in usage would translate directly to higher revenue, offsetting the vendor's overage charges. If we have fixed-price subscription customers, the existing fee would have been decided considering the break-even point with the ongoing GPU costs - hence, the loss of profits would be higher.

Fair analysis. Assume for simplicity that there are no pay-as-you-go revenues, we are simply using a subscription fee-based model.

Great, so we can approach the issue in the following way. There are two counter forces to shape the response to the situation - Server costing & Product pricing.

GPUs can either be our own or from a third-party. The third-party server additionally can be existing or new. The existing one can have the ongoing costing model, or it can be renegotiated.

Since the ongoing costing model is clearly not working, we would exclude that from our analysis.

On the pricing end. We can either continue with the current pricing strategy or shift to a new one. The new strategy can, in turn, be subscription-based or usage-based. Is there something else I should consider?

No, this looks perfect. What parameters would you evaluate these options on?

We could evaluate the options on the revenues (a proxy of pricing success), cost (a proxy of costing success), and risk (a sanity check on whether the option makes sense for the business).

Great. Now you can present your analysis and recommendations?

H=High, M=Medium, L=Low.

S. No.	Alternative	Rev. Impact	Cost Impact	Business Risk	Evaluation
1	Build Own Data Center	L	L	H	Although this option would reduce reliance on third-party vendors, it involves extremely high capital costs and a long timeline. It carries risks of migration downtime and, critically, the surge in demand for this specific AI model might cool off, leading to unused capacity. Since the process is long, revenue realization might be lower than the potential.
2	Diversify (Add a New GPU Server Vendor)	M	M	M	Costs are potentially low due to competitive pricing. However, there is a medium risk due to the time and effort required for technical integration with our platform, quality assurance, and the added operational complexity of managing a second vendor relationship.
3	Renegotiate with Existing Vendor	H	M	M	The business risk is moderate as this is the least disruptive option, but our vendor has significant bargaining power due to our dependency. Therefore, the impact on cost may only be moderate. Revenue is not directly impacted due to no transition losses. Also, no capital or fixed payments would be required.

4	Revise Subscription Pricing (New Customers Only)	M	M	L	This would lead to higher revenue from new customers, improving future profitability. However, it doesn't solve the immediate cost problem, as existing subscribers would continue to cause losses. The risk of churn is low since it doesn't affect current customers.
5	Introduce usage-based tiers for all new subscribers	M	M	L	Improves future profitability without upsetting the existing customer base. Doesn't solve the immediate loss from current customers.
6	Launch usage-based pricing tiers for all users (fair-use)	H	H	H	This would lead to the highest revenue by fully monetizing the usage surge. However, the risk is extremely high. Forcing existing fixed-price subscribers into a usage-based model would likely cause significant customer churn, negative PR, and brand damage. Also, technical changes might be required in the platform.

Based on this analysis, I would like to recommend opting for option 5 immediately. The reason is that it's least disruptive to our customer base and addresses both our cost structure and future revenue without taking on excessive risk.

On the other hand, we need to explore option 2 as a strategic initiative. Building relationships with a second vendor will protect us from being in this position again and will give us bargaining power for future negotiations.

We could evaluate the options on the revenues (a proxy of pricing success), cost (a proxy of costing success), and risk (a sanity check on whether the option makes sense for the business).

That'd be all, thanks!

Section 5.6

RCA

We're seeing a drop in engagement for Duolingo over the last month. How would you analyse this?

Thanks. I'll work through it in four steps: clarify the problem and the metric; set a framework for the root cause; drill down to isolate it; then propose solutions and a validation plan. To confirm the product: Duolingo is a freemium language-learning app that gamifies learning through points, levels and streaks to drive retention. Correct?

Yes, that's accurate.

And as for 'engagement', which metric exactly? DAU/MAU, average session time, or lessons completed per user per week?

For this discussion, engagement is the weekly average number of users who complete at least one lesson. It's declined about 15% over the past month.

A 15% drop is significant. I'll structure the analysis as external factors first, then internal.

- External: outside our control - seasonality, competitor actions, major news.
- Internal: within our control - product changes, technical bugs, marketing.

That sounds logical. Please proceed.

External factors. Has the 15% drop been sudden or gradual, and do we see a similar dip year-on-year, which would point to seasonality?

Gradual over four weeks. We do see a slight seasonal dip around now, but this 15% is far steeper than the usual 3-5%.

So seasonality is a minor contributor at most, something more dominant is at work. Any major competitor moves, a viral feature, or negative PR?

No unusual competitor activity and no significant negative press.

Then the cause is unlikely to be purely external. I'll move to internal factors, in three buckets: user segments (is the drop uniform or concentrated?), the user-journey funnel (where is the drop-off?), and technical / product changes. First, let's segment by geography, platform, user type and language.

The drop is most pronounced among returning users, on iOS, in the United States, across all the major languages they learn.

That narrows it a lot.

Now the funnel for a returning user:

opens the app → lands on the learning path → starts a lesson → works through the exercises → completes the lesson. The metric is 'completes at least one lesson', so I'd expect a sharp drop at one step - are users failing to **start** a lesson, or failing to **finish** it?

Start-rate (home → start lesson) is stable, but lesson-completion rate has dropped - users start but don't finish.

So the problem is inside the lesson. What changed there in the last month for returning iOS users in the US? Three candidates: a technical bug (e.g. an iOS update causing crashes or assets failing to load), a performance regression (slower lessons), or a product/content change - I'd check the release log and feature-flag dashboard.

Assume bugs and performance are ruled out. We did roll out a change via a feature flag to a segment of US iOS users: we increased the proportion of speaking exercises to improve retention.

That's a strong lead. My hypothesis: the higher frequency of mandatory speaking exercises is pushing users to abandon mid-lesson, because they're often somewhere they can't or won't speak aloud - commuting, in a quiet office, or at night.

Before I lock that in, I'd run a few checks:

- Drop-off by exercise type: which exercise screen are users on when they exit? A spike on speaking exercises would strongly support the hypothesis.
- Time of day: are exits concentrated around commute and work hours, when users are more likely to be in public?
- Completion of an alternative lesson: since the metric is '≥1 lesson per week', I'd confirm that blocked users aren't simply completing a different lesson instead - otherwise the speaking exercise explains mid-lesson abandonment but not the weekly-metric drop.

Say the data confirms it, users drop off the instant a speaking exercise appears. What would you propose?

A well-meant feature creating a poor experience in certain contexts. I'd brainstorm and prioritise on impact versus effort:

1. Quick fix - 'Can't speak now' button: on speaking exercises, let users skip or swap to a typing/listening exercise. It solves the contextual problem without disabling the feature.
2. More control - exercise preferences: in settings, let users set how often they get speaking exercises, or turn them off.
3. Smarter system - context-aware serving: use device signals (headphones connected, mic denied, time of day) to decide when to serve a speaking exercise. Best experience, most effort.

For immediate rollout I'd champion solution 1 - most direct, lowest effort, stops the bleeding. I'd ship it as an A/B test: control keeps the current experience; the variant gets the 'Can't speak now' button. The primary success metric is lesson-completion rate (recovering towards the pre-drop baseline), with long-term retention as a guardrail so we don't fix completion at its expense. If successful, roll to 100% of the affected users, then explore solutions 2 and 3 on the roadmap.

Excellent and thorough - problem identified, root cause isolated, and a concrete, data-driven plan. Thank you.

Instagram recently launched a feature letting users buy merchandise of the clothes shown in Reels. Initial adoption is very low. Walk me through how you'd investigate.

Let me confirm the product first. Instagram is a visual, content social platform; its core value is letting users express themselves through photos and short, form video, connect with a global community, and discover content. Right?

Yes.

I'll start by narrowing the scope. Is the low adoption specific to a geography, platform or demographic?

It's low across all geographies.

Is it a recent issue, or has it been low since launch?

It's a recent issue.

Any technical or performance problems reported with the feature?

No technical issues.

It is low across all Reels, or concentrated in some content categories?

Across all content.

Thanks. I'll look at external then internal factors. Externally, any shift in the social, media landscape, a competitor launching something similar, or new regulation?

No significant external changes.

Then I'll map the internal user journey. A user opens the home feed, opens a Reel, scrolls vertically through recommended Reels, and can like, comment, share or follow the creator. Where in this flow does the new feature sit, and where are users dropping off?

Good. The feature lives on the creator's profile page, the user taps the creator's profile image to reach a page to buy that creator's merchandise. Metrics in the old flow are unchanged.

So, the new flow is:

home feed → Reel → (if they like the Reel and the creator) open the creator's profile → tap the profile image → buy merchandise.

Since the old, flow metrics are unchanged and the problem spans all content, let me follow the new steps. What's the rate of users tapping the profile image, and then the merchandise link?

Profile, image tap rate is unchanged. The merchandise, link click rate is minimal.

Everything upstream is normal and the fall, off is right at the merchandise entry point. Two explanations fit: users don't know the feature exists, or they don't discover the entry point because it's hidden behind a nonobvious gesture (tapping a profile image, several steps removed from the Reel). Both are likely contributing.

You're right, users largely didn't know the feature existed. Only those who'd read or heard about the launch used it. How would you improve awareness?

I'd run two tracks. Awareness, to fix the immediate gap; and placement, to fix the structural one, because awareness alone won't help if the buy action stays buried.

Awareness:

- In-app announcements/notifications to all users (or targeted segments) when they open the app.
- A temporary 'Shop from Reels' spotlight as the first item in the feed to announce the feature.
- Short explainer ad, Reels showing how the feature works.

Placement (the deeper fix):

- Add an in, Reel shoppable entry point, a tag or 'Shop this look' chip on the Reel itself, at the moment the item is on screen and intent is highest, rather than relying only on a profile, image tap two steps later.

Thank you. We can close the interview.

We're facing a sudden spike in ride cancellations on the Uber app, and customers are getting poor service as a result. Walk me through how you'd find the root cause.

To start, can you clarify what the app does here and which service the cancellations affect?

The app connects riders with drivers for transport. The issue is ride cancellations.

Helpful. One clarification on the metric: are these rider-initiated or driver-initiated cancellations - or both? That single split will narrow the search a lot, because the causes are quite different.

Good question - they're mostly driver-initiated.

Then I'll focus on driver behaviour. Over what period has this spike been seen?

Recent, mainly the last couple of weeks.

Any geographic concentration?

More prevalent in metros, but not limited to one city.

Which customers are most affected - new, existing, or both?

Both, but more complaints from frequent riders.

Which platform - iOS, Android, both?

Both, so it isn't platform-specific

Any change to how the cancellation metric is calculated or reported?

No change to calculation or reporting.

Now the journey: a rider opens the app, enters a destination, requests a ride, and waits for a driver to accept; they're then shown the driver's details, price and ETA. Since the cancellations are driver-side, the relevant question is what changed for drivers. Any recent release affecting the driver app or experience?

Yes - we rolled out an update letting drivers see the drop-off location before they accept a ride.

At which point in the flow are the cancellations happening?

Drivers usually cancel after accepting but before reaching the pickup.

Any technical glitches reported with the new feature?

No technical glitches.

Any other recent change - pricing, driver incentives, service quality, new offerings that could shift driver behaviour?

No, the main change is drivers seeing the drop-off location.

And externally - new regulations, environmental events, competitor promotions, or PR/social sentiment affecting drivers?

None that we're aware of.

Then the evidence points clearly to the drop-off-visibility feature. By accepting and then seeing or by seeing then cancelling just after accept, drivers are declining trips with undesirable drop-offs (long dead-heads back, low-demand areas, traffic), which matches the after-accept, before-pickup timing and the driver complaints. I'd address it by rebalancing driver preferences with rider needs: incentives or surge for less-desirable drop-offs, a cap or penalty on post-acceptance cancellations, and possibly showing drop-off only **after** a commitment window. I'd validate by correlating cancellation rate with drop-off characteristics and confirming the spike tracks the feature's rollout.

That makes sense. Thank you for walking through it so thoroughly.

Netflix subscriptions have fallen 8% month-on-month. How would you identify the root cause?

To confirm the product: Netflix is a streaming service offering TV shows, films and documentaries to subscribers for a monthly fee. Correct?

Yes.

Is the 8% decline only in the last month, or a trend over several?

Just the last month.

Is it India-only or global?

Specific to India.

Across all customer segments, or worse for new versus long-term subscribers?

Both new and existing subscribers are affected.

Noted. Is this change in subscription rates sudden, or has it been gradual over the month?

It's been quite sudden.

That's helpful to know. Could you clarify how the subscription metric is calculated? Is it the number of active subscriptions, revenue, or something else?

It's based on the number of active subscriptions.

Any change in how it's calculated or reported, and is the BI tool healthy?

No change to the method, and the BI tool is fine.

Now internal factors, via the journey: a user logs in, manages payment preferences and pays, sets language/genre preferences, then searches or watches recommended content. Any recent tech change, UI release, glitch or non-intuitive UX?

Yes. We added an instant-messaging reminder to customers about their scheduled auto-debit, and a new, easily accessible auto-debit-cancellation button in the payment section of the app and site.

Insightful. Before I attribute the drop to that, an important distinction: cancelling auto-debit stops **future** auto-renewals - it doesn't unsubscribe a user immediately. So, a sudden one-month 8% fall in **active** subscriptions implies either many renewal dates fell within this window, or the metric is counting failed/declined auto-renewals as churn. Which mechanism are we seeing?

Largely the renewals - users opting out of auto-debit in significant numbers, who then lapse at renewal.

Now external factors. Any new regulation or law beyond the reminder requirement?

Yes, the Government of India has introduced consumer-protection changes: stricter data-privacy rules and tighter rules on auto-renewal across digital services. The updates we made were to comply.

Any competitor moves, or PR/sentiment shifts?

Some competitor discounts, nothing unusual. No major PR issue, though the regulation is a talking point. Behaviourally, users are opting out of auto-debit in large numbers.

So the chain is clear: the new regulation forced the reminder and the easy-cancel button; together they made users far more aware of, and able to stop, automatic renewals; opt-outs then convert to lapses at renewal, producing the active-subscription decline. The fix isn't to hide the button, it's to strengthen the reason to stay: surface the value users would lose (watchlist, recommendations, upcoming releases) at the renewal moment, offer a loyalty or annual-commitment incentive, and win back lapsed users with targeted offers. I'd measure renewal opt-out rate, renewal-stage retention, and win-back conversion.

Sounds good, thank you!

You're the PM for our mobile banking app. Android Daily Active Users (DAU) fell 40% in the last two weeks while iOS DAU stayed steady. Walk me through how you'd diagnose the root cause.

Let me confirm the issue: Android DAU down 40% over two weeks, iOS flat. Correct?

Yes, that's correct.

A few clarifiers. Is DAU unique users who opened the app at least once that day? Is the drop across both new and existing users? And is it global or localised?

Yes, unique daily users. Both new and existing are hit. It's global but more pronounced in North America.

First, metric reliability - any change to analytics tooling or event definitions, or any processing delay or mis-aggregation?

No changes, and no delays or aggregation errors.

Given accurate metrics, external factors: any industry shift hitting Android specifically, or competitor promotions targeting Android users?

No significant external events or competitor promotions.

Ruling those out, let me segment deeper. Is it consistent across all Android versions, or specific to one? And across all manufacturers, or specific ones?

It's primarily Android 12 and above, regardless of manufacturer.

Now internal changes - sudden drops usually coincide with a release. Did we ship a new Android version around when this started, with UX, backend, security or permissions changes?

Yes - a major Android update two weeks ago, specifically for security and permissions. No equivalent iOS update.

That aligns with the timeline. Release quality next: did crash reports or complaints spike post-release, and did the Play Store rating drop?

Yes - crash reports surged, specifically at app startup, and negative ratings mentioning crashes rose sharply.

Then the crash logs: a consistent error across devices, correlated with the new permissions logic?

The logs show repeated crashes tied to permission checks during launch, specifically on Android 12+.

Before I conclude, one loose end. If the bug is purely Android-12+ permission logic, the drop should be roughly uniform wherever Android 12+ is common - yet it's more pronounced in North America.

The most likely reconciliation is a staged rollout (the update reaching North America first) or higher Android-12+ penetration there. Can we confirm the rollout sequence and the regional OS-version mix?

Yes - the update was rolled out to North America first.

To summarise the path: metrics verified; external factors and competitors ruled out; segment isolated to Android 12+ globally, surfacing first in North America via a staged rollout; cause narrowed to our recent security/permissions update crashing the app at startup on Android 12+.

Solutions:

Short term: roll back to the last stable version; if rollback isn't feasible, ship a hot-fix correcting the permissions logic and deploy fast.

User communication: tell affected users, give clear update instructions once fixed, and consider a goodwill gesture to rebuild trust.

Long term: strengthen QA for Android OS upgrades, permissions and startup paths; add automated alerting on DAU and crash-rate anomalies; and phase future rollouts so a regional canary catches this earlier.

Agreed. Thank you - we can close the interview.

There's a roughly 10% decline in cart additions (Add to Bag) over the last three days on Myntra. Can you help investigate?

To align on the problem: cart addition means adding an item via the 'Add to Bag' button, and I need the root cause of its 10% decline. Correct?

Yes, that is correct.

Myntra has Android and iOS apps and a website. Is the 10% drop across all platforms, or specific to one?

It's on the Myntra Android app.

Across all product categories or specific ones? And I'll assume this is abrupt, not a month-end/start seasonal effect.

Across all categories. Yes, you may assume that.

Specific to certain user segments - new, premium, a demographic?

No, broadly similar across segments.

Can I assume the analytics pipeline is healthy - i.e. the reporting tool itself isn't the problem?

Yes, we've checked the logs multiple times.

Useful - but I'd separate two things that 'the analytics tool is fine' can hide: the reporting pipeline being healthy is not the same as the new app build still firing the Add-to-Bag event. A drop in **recorded** adds can come from a tracking regression even when the pipeline is perfectly fine. I'll keep both 'real drop' and 'measurement drop' on the table.

Let me frame external and internal factors.

1. **External:** competitor launches or announcements in the last three days that could divert or delay purchases; any macroeconomic news or seasonality.
2. **Internal:** new Myntra campaigns; design changes or app releases; glitches in the steps before cart addition; and, importantly, whether the Add-to-Bag **event** is still being captured.

One thing stands out before we start: the drop is uniform across all categories and all segments and abrupt. A uniform, abrupt drop with no demand-side concentration is the classic signature of a technical or instrumentation regression, not a demand shift - so I'll weight the release/tracking path heavily. Does the approach look good?

Yes, that looks great. Please continue.

Externally - any sale or announcement by Myntra's online or offline competitors in the last three days?

No, nothing unusual; they're running regular campaigns.

Safe to assume no macroeconomic change affecting behaviour?

Yes, it is safe to assume.

Then internal. Any new Myntra campaign that might have users waiting for an event, or shifting between categories?

No - the decline is spread across all categories and we're running our usual campaigns.

Has the Android app shipped any recent update?

Yes, a few updates to improve the user experience.

Do any of them touch the path to cart addition, specifically the 'Add to Bag' feature?

Yes - design changes to 'Add to Bag'. Analyse further: what factors could be causing the decline?

Given a release touching exactly that button, three things in priority order:

- Is the Add-to-Bag event still firing? Verify the tap actually logs an event in the new build. If the build stopped emitting the event, adds may still be happening but going unrecorded - a measurement drop, not a real one. This is my first check given the signature.
- Add-to-Bag API success rate over the last three days - is the call itself failing for some users (a real drop)?

- Forced vs optional update: if not forced, some users are on an older build the backend may no longer fully support; check whether a rollback is possible to mitigate.

Exactly - when a user taps 'Add to Bag', the action isn't being captured by the API. It was a deployment mistake. Thanks for resolving it.

Do you want me to go further through the customer journey?

Happy to, but if the event simply isn't being captured, that's the root cause - a tracking regression from the release, not a demand problem. I'd fix the event, back-fill or annotate the three-day gap so the metric isn't misread and add a release check that fails the build if a core event stops firing.

No, that will be all.

You're the PM for our CRM's sales module. Usage data shows that only 40% of reps actively use the CRM daily, even though it's mandated by their sales ops team.

Leadership wants to know why, and what you'd do about it. Where do you start?

Before I hypothesize, I want to pin down what "actively use" means here and where the drop-off happens. Is 40% daily login the real problem, or is login healthy but reps aren't updating records once they're in? Those point to very different root causes.

Fair question, let me clarify. Login is actually closer to 70%. The 40% refers to reps who log in but don't meaningfully engage: they don't update opportunity stages or log activities, so pipeline data is stale.

That reframes it well. This isn't an access or awareness problem, it's a behavior problem inside the product.

Before I hypothesize on causes, I'd want to segment the 40% who aren't updating: does it cluster by team, region, tenure, or deal size? If it's concentrated in one segment, that points to a local cause - a specific team's process or a regional workflow gap rather than something systemic in the product.

Good instinct. It's fairly evenly spread across regions and teams, but it skews toward reps with more than five years of tenure. Newer reps update records far more consistently.

That's a useful signal. It suggests the issue isn't team-specific process, and it's less likely to be pure friction, since tenured reps are generally more efficient in any tool once they've learned it. It points more toward habit and trust: tenured reps may have workflows built up on spreadsheets or personal systems from before the CRM was mandated, and they may have less faith that the data pays off for them specifically.

I'd still want to break the underlying causes into three hypothesis buckets before picking one: the workflow is too costly relative to its perceived value, the data reps would enter doesn't come back to help them, or there's a trust issue with the data already in the system.

Walk me through each of those.

- **High friction, low perceived value:** updating a stage requires many clicks or mandatory fields that don't reflect how the rep actually sells, so reps see it as data entry for management, not a tool for themselves.
- **No visible payback:** reps who do log activities don't see anything come back to them - no smarter follow-up reminders, no forecasting help, no faster deal support - so the effort feels one-directional.
- **Low trust in existing data:** if the CRM already contains duplicate accounts, outdated contacts, or stale stages from other reps, a rep reasonably concludes that keeping their own data current won't fix a system that's already unreliable.

Which of those would you investigate first, and how?

I'd start cheap and diagnostic before expensive and interview-based. First, I'd pull product analytics: time-on-task for an opportunity-stage update, the number of required fields, and where in the flow reps abandon the update. That tells me directly whether friction is the culprit, and it's data we already have. In parallel, I'd look at data quality metrics like duplicate rate, percentage of stale records, last-modified age - since that's also measurable without talking to anyone. Only after that would I run 8 to 10 rep interviews, weighted toward the tenured segment we just identified, because self-reported reasons for not doing something are the least reliable signal and the most expensive to collect.

Suppose the data shows both: the update flow takes 12 clicks and involves fields reps call irrelevant, and duplicate account rate is 18%. You can only fix one this quarter. How do you decide?

I'd prioritize the flow, not the data cleanup and I'd use a simple frame to justify it: which fix changes rep behavior fastest, and which fix is a prerequisite for the other paying off. A faster update flow is something reps feel immediately, every time they use it, and it's fully within the product team's control to ship in a quarter. Fixing the 18% duplicate rate is valuable, but it's a one-time data hygiene project, likely owned partly by RevOps, and it doesn't change the reason reps avoid the flow in the first place, a clean database with the same 12-click flow still won't get updated. I'd sequence it as: reduce the flow to 3 to 4 fields now, then run a deduplication and data-quality initiative next, once reps are actually updating records worth keeping clean.

How would you know the fix actually worked, and not just that reps got used to fewer fields?

I'd track a leading and a lagging metric.

Leading: percentage of opportunities with a stage update in the last 7 days, segmented by the tenured-rep group specifically, since that's where the gap was concentrated, it should move within two to three weeks of shipping.

Lagging: forecast accuracy, the gap between CRM-predicted pipeline and actual closed revenue. Because that's the metric leadership actually cares about, and it should improve over one to two quarters as the underlying data gets fresher. If update frequency rises but forecast accuracy doesn't, that tells me reps are updating the field but not accurately, which would point back to a trust or incentive problem rather than a friction problem.

That's a good approach.

PM Tip

Before jumping to hypotheses, segment the affected population by team, region, tenure, or deal size. A problem concentrated in one segment points to a local cause; a problem spread evenly points to something systemic in the product. In enterprise B2B tools, low adoption is rarely a single root cause, it's usually friction, missing payback, and data trust compounding on each other. Diagnose with data you already have (analytics, data-quality metrics) before spending interview budget on self-reported reasons, and prioritize the fix that is both fastest to ship and a prerequisite for the others to matter.

There is an AI chatbot assistant in our consumer app. Over the last month, users have increasingly been abandoning the conversation mid-way. How would you investigate?

I will first clarify the metric. What exactly do we mean by "abandoning", do you mean users close the chat or users stop responding?

Yes. Users abandon the conversation mid-way without completing the task.

Okay, got it. Next, I'd like to know the extent and pattern of the issue. What is the current abandonment rate, what was the baseline, and did it occur abruptly or gradually?

It's a gradual increase over the last four weeks.

Then I will try to localise the problem before forming hypotheses. I will segment users based on:

- Platform: iOS, Android, web
- Geography and language
- New users or returning users
- Intent/task type
- App version and chatbot/model version
- Conversation stage: first response, mid-point or end of conversation

I would also want to check if there was any infra incident, release of app, change in UI, pricing/policy update, or even model update recently.

There were no infra incidents recently. However, we have rolled out a new version of the model five weeks back. The latency has been the same.

Then the model rollout is a strong hypothesis, but let me still verify it. I will compare the abandonment rates among new model cohort and any older model cohort, as well as whether the increase is specific to certain intents/user segment.

Before I diagnose the problem, I will classify abandoned conversations into failure buckets:

1. Accuracy failure: the bot gives an incorrect, wrong or irrelevant answer.
2. Capability failure: the user asked something valid, but the bot could not complete the task.

3. UX/flow failure: the conversation has a lot of iterations, is not clear with the next steps or doesn't have options to escalate.

4. Tone failure: the bot sounds robotic, repetitive, confusing or non-helpful.

5. Hand off failure: the user wanted to connect to a human, but couldn't

How will you identify which one is causing the abandonment?

I will use both quantitative and qualitative approach.

For the quantitative part, I will check:

- Abandonment rate by model version and intent
- Average number of turns before abandoning
- Rate of negative feedback: thumbs down rating or complaint keywords or click on "Talk to Human"
- Rate of repeat question: the user has to ask the same question multiple times since he didn't get useful answer
- Rate of escalation and failed escalation
- Completion rate by task type

From the qualitative side, I will sample some abandoned conversations and manually label them according to the buckets above. A couple of hundred conversations should already give me some idea whether the cause of the abandonment is specific bucket.

Let's assume that most of the abandoned conversations occurred after wrong or out-of-scope answer and it is worse for the new-model cohort.

Then the probable root cause is an accuracy and capability issue due to the new model rollout. Let me check it further by seeing if:

1. Affected intents performed well before the change
2. There are hallucinations by the new model
3. Retrieval or grounding has changed in some way
4. The bot answers the low-confidence queries instead of asking clarification questions

How will you fix the issue?

I will address it in three layers.

Immediate fixes:

- Add stricter fallback for the low-confidence queries.
- Route the high-risk or unsupported intents to human directly instead of giving the answer.
- Add guardrails for the worst affected intents.
- If the problem is very serious, roll back the new model for the affected intents.

Medium term fixes:

- Improve the retrieval and grounding for the intents where the bot gives wrong answers.
- Expand capabilities of the bot for the high volume out-of-scope tasks.
- Make the conversation flow such that users receive clear instructions and quicker escalations.

Long-term prevention:

- Build the offline evaluation set using failed conversations.
- Monitor abandonment rate, hallucination rate, negative feedback and completion rate for each model version.
- Use staged rollouts and A/B tests for any future changes in models.
- Set up alert thresholds to detect the issue.

Okay, can you make your final diagnosis?

Abandonment increase seems to be caused by the new model rollout. This happened due to the higher rates of wrong answers and out-of-scope responses for some intents. The fix is to mitigate it through fallback and escalation for immediate solution, improving the retrieval and grounding and adding model version level monitoring to catch any future problems.

PM Tip: Root cause analysis should follow the standard RCA flow of metric clarification, sizing, segmentation, recent changes, hypotheses, data validation and solutions. However, there is the AI-specific component - classification of the failures into accuracy, capability, tone, UX and expectation mismatch buckets and measurement of those metrics through logs, feedback, conversation reviews and model comparison.

Section 5.7

App Improvement

Facebook wants to improve its Reactions feature. How would you go about suggesting actions it can take?

A few clarifications first. By 'Reactions' do we mean the feature where users react to their own or others' posts via emojis? Is there a specific user persona to focus on? And is there a trigger for this - negative feedback, or something else?

You're right about the feature. There are multiple personas, but restrict yourself to individual users. Assume Facebook wants to drive higher engagement and is exploring how Reactions can help.

Understood. My structure: identify the personas using the feature; find their pain points; propose solutions for the highest-impact ones; and assess trade-offs and risks. Fair approach?

This seems comprehensive. We can proceed.

Within individual users, I'll bucket by engagement behaviour:

1. Passive viewers: don't react to posts.
2. Nominal engagers: react with emojis and/or the like button.
3. Active engagers: engage deeply, through emojis and comments.

To prioritise, I'll use two criteria: depth of unmet need and network effect - how much a segment pulls others into engaging. Active engagers have the highest unmet need for expression, and they act as the pull that draws passive and nominal users up the ladder, expanding the active base over time. They're a smaller segment by count, so I'm deliberately not using raw reach (TAM) as the deciding criterion here - the lever is the pull effect, not segment size. So, I'll focus on active engagers. Shall I move to pain points?

Yes, we can proceed.

For active engagers, whose goal is deep expression, the major pain points are:

1. Too few emojis to express themselves, and each emoji captures a single feeling - so complex emotions can't be expressed.
2. Emojis are static, limiting how engaging and expressive a reaction can be.
3. Only one reaction is allowed per post, inhibiting multiple simultaneous reactions.
4. Limited counter-reaction options to existing reactions, which cuts exchanges short.

Good observations. Pick one to prioritise, and propose solutions.

Since the objective is engagement, I'd prioritise pain point 2 - richer, audio-visual reactions that go beyond static emojis - because it opens the most new expression and interaction.

Solutions:

1. Avatar-based reactions via facial-recognition integration, making reactions personalised.
2. Short video reactions - record, or pick from a standard gallery, a brief clip used as a visually expressive reaction.
3. UX cues - a catchy, highlighted icon that draws attention to these new reaction types.

These are good. Now think about success measures, and mitigations for any challenges.

Key challenges: content moderation/censorship of user-generated video, and higher storage requirements.

On measurement, I'd distinguish adoption from the objective. The objective is higher overall engagement, so my north-star is engagement, with adoption as a supporting diagnostic:

- North-star (engagement): reactions per active user per week, and the share of posts that receive any reaction - plus downstream session time and return visits, to confirm the feature lifts engagement rather than just shifting it.
- Supporting (adoption): number of users using video reactions, and the share of reactions that are video.

Mitigations for the challenges:

1. Moderation: an AI model screens video reactions for risky/uncensored content, flags it to the user, and blocks it from posting.
2. Storage: cap clip length (e.g. five seconds) and fix the resolution to prevent storage overruns from long, high-resolution content.

Nicely elaborated. We can close the discussion.

BookMyShow wants to improve engagement and user satisfaction through new experiential features. How would you go about suggesting actions it can take?

A few clarifications. Are we targeting the whole flow from discovery to post-booking, or one phase? Are we improving engagement across all users, or a priority persona? And is there a trigger - a drop in repeat usage, or a need for more content creation?

Assume we're increasing engagement and stickiness across the lifecycle. Focus on individual users who use BookMyShow regularly to book movies and events.

Understood. My structure: define personas by engagement behaviour; identify key pain points across the journey; propose solutions for the highest-impact ones; and cover trade-offs and mitigations. Sound good?

That works. Go ahead.

By engagement behaviour:

1. **Occasional users:** only for major releases or popular events.
2. **Transactional users:** book regularly but don't engage beyond the core booking flow.
3. **Engaged users:** book often and also explore events, write reviews and refer others.

I'd prioritise transactional users. They already trust and transact with the platform but are under-leveraged on engagement; nudging their mid-journey and post-booking interaction can move them towards the engaged segment and lift overall stickiness. Shall I move to pain points?

Yes, please proceed.

For transactional users, the key pain points:

1. Static location: travelling users don't get relevant suggestions for the city they're in.
2. Passive discovery: no intelligent or voice-based help for finding events.
3. No reward for reviews: users contribute ratings but get no incentive.
4. Reviews don't drive visibility: consistent reviewers aren't acknowledged.

I'd prioritise pain points 2 and 3, as they tie most directly to habit formation and engagement loops.

Prioritised: pain points 2 (passive discovery) and 3 (no review reward). Pain point 1 (static location) is deferred to a later release, and pain point 4 is folded into the rewards design.

Makes sense. What solutions would you propose to address those?

Two solutions, mapped to the two prioritised pain points:

1. AI assistant with prompt-based search (voice supported) for passive discovery. A smart assistant on the home page handles prompts like 'top movies near me', 'sport this weekend' or 'comedy shows in Mumbai'. Users type or speak, and it returns a pre-filtered set, cutting cognitive effort and encouraging proactive exploration.
2. Reviewer rewards programme for the review-incentive gap. Verified reviewers earn points for each review written after attending an event or film, redeemable for wallet credit, partner discounts or early access. This improves community content and drives post-transaction re-engagement, and a reviewer badge/leaderboard addresses pain point 4's visibility gap.

These are thoughtful. What challenges or trade-offs do you foresee?

Three:

1. The AI assistant needs strong natural-language understanding to avoid frustrating misinterpretation; it needs a fallback (e.g. pre-filtered cards).
2. Risk of low-quality or spam reviews written just to earn points, diluting trust and adding moderation overhead.
3. If we later add auto-location, it raises privacy concerns for users who don't want tracking.

Good. Now propose success measures and mitigations for those challenges.

KPIs, tied to the engagement objective:

- Share of users using the AI assistant (text or voice) in discovery.
- Share of first-time reviewers who become repeat reviewers.
- Average reviews per 1,000 bookings.
- Uplift in session-to-booking conversion from faster discovery.
- Opt-in rate for location-based recommendations (once that v2 feature ships).

Mitigations:

1. Use pre-built AI intents (e.g. Dialogflow) for the assistant, scoped to defined genres/events at launch.

2. Apply content filters and quality thresholds before awarding points (minimum word count, flagged-word checks).
3. Make any future auto-location opt-in, with its benefit clearly communicated upfront.

Well structured and nicely articulated. We can close the discussion now.

06

Collaborators

COLLABORATORS



Subhash S

PGP-2



Meghna

PGP-2



Jai Kumar

PGP-2



Shalini Sheetal

PGP-2



Sarthak Kumar

PGP-2



Shounak Ray

PGPX

COLLABORATORS



Piyusha Patil

PGP-2



Abhishek Aditya

PGP-2



Kiran Ambokar

PGP-2



Sai Uddiep Gonapa

PGP-1



Harsh Kumar Badhai

PGP-1



Keerat Kaur Jassal

PGP-1

COLLABORATORS



Dhansree Kumari Meena

PGP-1



Adesh Yadav

PGP-2



Shubham Nehete

PGP-2